Content of Non-Exam Assessment Programing Project (Component 03)

Tom Obungu

Candidate Number: 4877

Centre Number: 56635

Contents

Analysis

Introduction

I want to undergo the development of an adaptation that is targeted to Windows but not specifically exclusively to Windows, of Chris Danford's 2001 arcade style rhythm/dance game: "StepMania". StepMania can be referred to as certain sub-genre of rhythm game(s), commonly referred to as Vertical Scrolling Rhythm Game (VSRG). In StepMania's gameplay, the difficulty selection system of maps (maps can be interchangeably referred to as charts or "beatmaps") has no difficulty calculation system and it is up to the map creator to determine and input the map difficulty itself during the map creation (I will discuss the features of map creation in my essential features later). This is problematic as the difficulty is subjective and prone to human error which can lead to incorrect perceptions of a map's difficulty. My adaptation intends to add a difficulty calculation system and to improve the problem of chart difficulty calculation of most modern VSRGs through an algorithm to determine arrow patterns prone to being miscalculated and introduce a reduction/promotion system of the difficulty calculation based on these factors. I will do this whilst maintaining the regular core aspects of gameplay. This will be beneficial as it will add accurate difficulty calculation and allow players to indicate their true ability in their gameplay without their scoreboard performance being inflated or deflated. This will also prevent mis-ranked players from taking advantage of maps with miscalculated difficulties (this is often referred to as "farming").

Stakeholders

As VSRGs have grown in popularity since their birth in 1998, there is a varying age range and demographic of players who play them. For my game there will be two main audience members (stakeholders).

Early VSRGs from the late 1990s and early 2000s such as StepMania and Konami's "Dance Dance Revolution" (DDR) (more examples will be included in research), typically have a much older age range since most players were in their teens, i.e. 13-18 years old, when they first played them during the era they were released and have either continued to play them as they have got older or have returned to playing them when they have matured. This means the age group of i.e., 21-30+ years old, will be included as part of my Stakeholders. The reason being is that the much older generations return to VSRGs due to feelings of reminiscence and/or the nostalgia of the gameplay. Another factor is that during the 1990s and early 2000s, most VSRGs like Konami's "Beatmania" Series were only playable in arcade dance machines and the computers themselves that were required to run these games were not a common household item due the expensive of owning a computer. Often the older age range continue to play the much older VSRGs as a hobby or a way to pass time and shy away from long hours of play. This is due to older VSRGs not integrating online features and therefore not having competitive scoreboard rankings of the much younger modern VSRGs. This is also since the older age group have less free time due to increased responsibilities such as working for a job and providing for their families (more on this later).

However more modern VSRGs such as osu!mania that were released in 2010 and onwards (osu!mania was released in 2007), have a much younger age range as since most players were born the time they were released and have picked them up as they became teenagers, during the 2020s modern era. This means the teenage age group of i.e., 12-18 years old will also be part of my Stakeholders. From the teenage perspective, there is more free time due to decreased responsibility. This means the younger teenage demographic is more likely to play VSRGs for longer periods. This is evident in the fact that many content creators, a famous and popular example being "jkzu123" on the platform "YouTube" (Channel link may not be feasible, however at the time of writing this, the link was active: https://www.youtube.com/channel/UCeL9uQhZ8WsXFxGvQCDzrYw), have arisen in number and gained a large audience due to consistently having the time to play and upload content on VSRGs. jkzu23 even stated he began playing VSRGs "since I was 12 years old" which further solidifies my belief. Furthermore, due to the teenage population being more socially interactive due to things such as education and increased time, teenagers are more likely to share VSRGs with their peers, thus leading to an increase in the younger

audience playing them. All this evidence suggests that there is more of an audience for the younger generation of VSRG players.

Therefore, I intend to adapt my game to be inclusive for both younger and older audiences. My adaptation will try to cover all the age ranges of VSRGs and appeal to both audiences. This includes the much older age range of 21-30+s of older VSRGs from the 1990s and 2000s and the much younger age range of 12-21 from 2010s and onwards. I will do this by including the nostalgic and retrospective arcade aesthetic of the much older VSRGs i.e., Dance Dance Revolution but also adding the modern-day functionality and mechanics that are appealing to the younger audience that play VSRGs i.e., osu!mania. I also intend to keep my focus on the fact that older VSRGs typically have features (as mentioned and will discuss later) that the much larger younger generation consider to be "flaws." This is due to modern technology improving from the time of the older generation and VSRGs themselves improving also.

To give an insight into the benefit of my solution, I will represent and outline real-life stakeholders from the stated audiences and demographics and explain the certain features that will benefit them and features that will fit their needs as shown in the list below. I will also interview the stakeholders and include the results of their perspective on VSRGs:

- Nathaniel Binuhe is a peer in my computer science class, and he picked up playing the modern VSRG: osu!mania during the pandemic and has continued to play them ever since. He enjoys playing the game and often plays it in his spare time (as mentioned earlier that the younger generation have more spare time). He represents the much younger audience that play VSRGs in their teens. However, he also states that the "grading (difficulty calculation of charts) system" of osu!mania and other VSRGs is "inconsistent" and is not satisfied (as mentioned earlier). Therefore, my solution of an adaptation using an improved algorithm to create a better difficulty calculation system for charts will benefit him and the rest of the younger audience as part of my stakeholders as a consistent and accurate difficulty calculation system will cause less dissatisfaction and improve their experience playing the VSRG.
- Maria Sheehy is representative of the upper bound of the older generation as she recalls playing the original VSRGs such as Dance Dance Revolution from the 1990s. The most notable aspect of it is that she played them when she was 28 years old when they first came out, meaning that from the time they came out, to the time of writing this, she has significantly aged and mostly reminiscent of the game. Maria also stated, "I used to play those games with my daughter" and "my daughter still

plays those games with her kids as well." This gives an insight into the point that the upper bound of my game audience will be 30+ years old. Maria also states that she only plays VSRGs like Dance Dance Revolution, "when we go the arcade" and "which is a couple of times a year." This further signifies my earlier claim of the older generation only playing these games to "live in the past" and reminisce and do not play these games often for prolonged periods albeit to the younger generation. This is due to Maria having more responsibilities such as caring for her family (as mentioned earlier). Therefore, it will benefit the older generation of my stakeholders if I retain the aspects of the arcade style graphics to provoke feelings of nostalgia in my audience. However, a major drawback to my adaptation is that it will not be able to support dance pads (more on this lay) which is fundamentally one of the core reasons of feelings of nostalgia, therefore keeping the style of the arcade style user interfaces.

- Jean Obungu also grew up playing rhythm games when she was a teenager and thoroughly enjoyed them. She recalled that you had to "hit the notes to the beat really fast." She also recalls that she played it with her older brother and said, "I remember having a lot of fun." Since then, she has now aged and relinquished from playing the game and feels "nostalgic" about them and further iterates "it was fun at that time." She further represents the older generation of the stakeholders who are reminiscent of the nostalgic 2001 arcade-style graphics from early VSRGs. Therefore, keeping the aspects of the original Stepmania/Dance Dance Revolution game screen and menu interfaces will benefit the audience of the older generation and fit their needs.
- Samuel Smedley is also a fellow peer of mine who I personally played VSRGs with during the pandemic era of 2021-2022. He initially picked up the popular VSRG "Friday Night Funkin'" at its peak of popularity in late 2021 and advanced onto osu!mania as he was further interested in playing VSRGs. Samuel Smedley recalls finding osu!mania "hard but really fun" and said he "likes the cool graphics and skins" (Which I will further explain in detail in the similar systems section of research later). He is representative of the younger teenage generation who picked up VSRGS in their spare time and enjoyed the much more modern implementation of VSRGs. Because of this it will be beneficial for my adaptation to retain the modern useability aspects of rhythm games such as Quaver and osu!mania (which I will point out in the research section).
- Louis White is also representative of the younger generation who has had experience playing arcade style games, as well as experience playing the VSRG: Harmonix's "Fortnite Festival" and is interested in rhythm games and contemporary games in general. Louis represents the more casual group of the younger audience

who play for shorter periods of time. When asked what Louis's intention of playing games was. Louis responded with "Just to pass time, whenever I have free time." This further shows my point of the younger generation playing due to increased free time.

- Harrisson Jarvis has had experience in playing rhythm games such as Beat Saber and Harmonix's "Fortnite Festival" (The same game studio that made the VSRG Guitar Hero). Harrison is representative of the younger generation that finds rhythm games very exciting. Harrison even stated that he found rhythm games like Beat Saber, "very immersive."
- Hamish Lindsay has also had experience playing VSRGs such as Friday Night Funkin.' He stated it was "fun and challenging" and had experience playing it when it first came out. However, he pointed out that there are certain aspects that he did not like about the game and even other aspects of VSRGs. Hamish represents the VSRG players who have had experience with VSRGs and recognized their flaws.

Research

Ever since their origination in 1987, dance/rhythm games, modern examples being Ubisoft's "Just Dance" to Harmonix's Guitar Hero (more on this in my research section) have been prominent in today's modern game industry. A VSRG is a type of rhythm game in which the user must yield input and "hit" icons (commonly in form of arrows) "on time to the beat," on par with a set of stationary arrows (commonly referred to receptors), synchronous to music playing in the background.

In VSRGs, sequences of arrows (interchangeably referred to as notes) can be arranged to form patterns. These sequences of arrows are reflective of the music's tone and beats per minute (BPM). For example, a fast-paced song's "beat drop" will have multiple arrows sequences that must be hit quickly, whilst a slow based song's break will have fewer arrows and are hit slowly. As the arrows scroll, there are normally a stationary set of target arrows. Once the arrows meet the stationary set of arrows, the user must hit the corresponding arrow via their input device. As music progresses, arrow sequences may vary in pattern and build up to form maps of arrows. As a user plays a map, the user is evaluated on their accuracy of how on time they hit the note coordinated with the beat of music. Each time a note is hit, it is given a judgment of accuracy based on how accurately they hit the note, or if they even hit the note at all. The user is then given a performance "grade" based on the average accuracy of how on time they hit the notes. This forms the basis of most VSRGs. Different songs can have different maps. Within these maps the arrow sequences can vary, some being harder to hit than others. A song can have multiple maps with varying arrow sequences. This forms maps with their own corresponding

difficulties. As VSRGs have progressed and allowed multiple songs with maps with multiple different difficulties, these difficulties are given a value and ranked quantitively, e.g., Difficulty 1, 5, 11, or qualitatively. easy, medium, hard, expert.

Various research resources of VSRGs, of which two examples being on Dean Herbert's "mania" game mode of his game "osu!" (Referred to as "osu!mania") and on Swan's "Quaver" (I will discuss and analyze modern examples of modern VSRGs in my research in more detail later), have become online and have increasingly large online communities. This has led to VSRGs becoming competitive with scoreboard rankings based on performance and the difficulty of maps achieved. As a result, typically in modern VSRG's such as osu!mania, the value the difficulties calculated is based on the average density of the sequences arrows in a map. This means the quantity of arrows that are to be hit in a certain amount of time (typically per second).

However, this approach is problematic as it has caused incorrect estimates of the chart difficulty. Incorrect estimations of chart difficulty can make maps seem harder or easier than they are. This has led to inflated or deflated scoreboard ranking; Thus, players being mis-ranked on online competitive scoreboards. Often mis-ranked players take advantage of maps with mis-calculated difficulties by specifically playing those maps only. This further inflates their scoreboard ranking, giving an incorrect measure of their ability. This leads to dissatisfaction and anger in the online communities for VSRGs and even leads to players quitting the VSRGs they play. (Evidence of this will be covered later)

Therefore, this is why it is a sensible approach to develop my adaptation to be better suited for map difficulty calculation by introducing a

VSRGs and rhythm games are currently amassing a large amount of traction and revenue during the modern era of gaming due to their intricate action-based game style. This can be shown according to Wikipedia's analysis (which can be found by in the follow link: https://en.wikipedia.org/wiki/Friday_Night_Funkin%27) of Ninjamuffin99's popular VSRG "Friday Night Funkin'", revealing that the game's Kickstarter funding was able to "reach[ed] its goal of $60,000 within hours." and in the end the "Kickstarter ultimately raised over $2 million". This then led to an article report from IGN (which can also be found in the following link: https://www.ign.com/articles/kickstarter-record-number-games-2021) stating, "that Friday Night Funkin': was one of the most funded Kickstarter projects of 2021". Another example of the popularity and market revenue of VSRGs can be seen in another Wikipedia analysis of Dance Dance Revolution (https://en.wikipedia.org/wiki/Dance_Dance_Revolution) states that "In 2004, Dance Dance Revolution became an official sporting event in Norway" and that through recent years Konami (the brand behind the game) dedicated "their own competitive tournament,

[:] the Konami Arcade Championship"" allowing "different regions around the world to sign up and play in specific online events to earn a spot in the grand finals, typically held in Tokyo, Japan." The article also states that this led to countries like "Korea, Taiwan, and other Asian countries" were "allowed to enter," then in "February 11, 2017", competitors from the "United States were eligible" and in 2020, "eligibility for players in Australia and New Zealand" was available. The article then states that competitors have moved on to win the "global tournament." This signifies the international outreach of VSRGs alike and considering that StepMania is based off Dance Dance Revolution itself, a modern-day adaptation will undeniably have the capability of amassing popularity. Furthermore, the article states that "In March 2023, the first ever *upbeat* tournament was held at Round1 in Denver, Colorado" with a "$10,000 prize pool". This further shows the resolution of popularity and revenue that VSRGs have the potential to create. This further justifies my reason for making an adaptation.

A list of the similar systems that I will research, some already mentioned, include Friday Night Funkin,' osu!mania, Konami's Dance Dance Revolution, Roxor Game's "In The Groove," and Guitar Hero. I have chosen these systems as all these games still retain the core functionality of a VSRG, but each has their own characteristic and customized "spin." Despite this, each has their fundamental flaws within their games. To provide evidence of what features of my adaptation are to be prioritized and what features are to be ignored. To this I gained insight from the interview results of my stakeholders and described their thoughts on the different similar systems.

Before I delve into the research of similar systems, it is fundamental to highlight the core aspects of StepMania and the reasons for a newer adaptation.

StepMania

StepMania consists of basic VSRG gameplay and is tailored to both the arcade and keyboard users. In the gameplay there are customizable settings, navigation systems to play maps and an area to play songs. The game features multiple game modes that are selectable and multiple modes to play with more than one play cooperatively.

To gain insight into the experience of the gameplay, I interviewed Hamish Lindsay. Hamish played a song on StepMania and immediately noticed a difference in gameplay. Hamish stated, "The game(StepMania) felt "unresponsive" and "I feel like there should be more visuals to audio feedback". He then noted that "It is a lot different to the newer games (VSRGs) I played." This is because StepMania's gameplay was developed before the onset of modern VSRGs and as a result, contrasts with the modern features included in newer VSRGs. This shows evidence of the immediate differences between older and modern day

VSRGs and how much older VSRGs may be considered to have fundamental flaws to their gameplay.

Of these fundamental flaws is the map editor system. As mentioned earlier, although there the system allows for users to create customized maps, the difficulty calculation of the map is entirely up to the map editor/creator to input in themselves which leads to problems mentioned earlier of incorrect estimations of the difficulty. Figure 1 shows a screenshot of what entering the difficulty of a map scenario would look like.

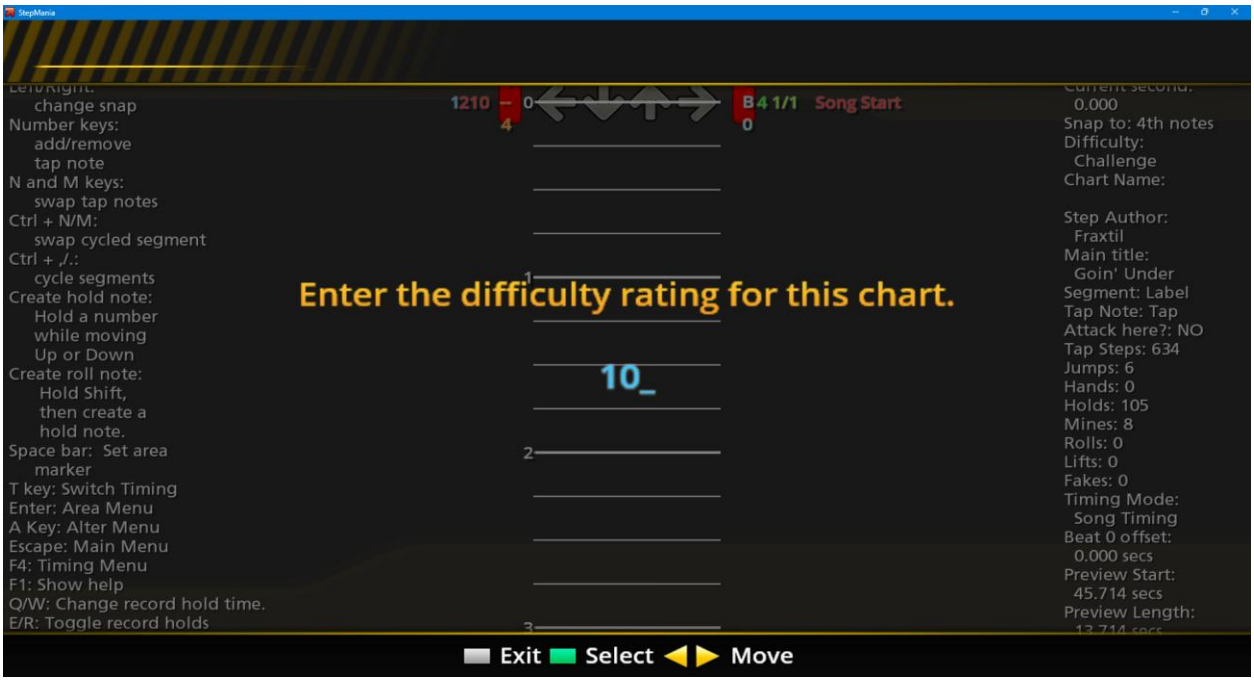


Figure 1 – StepMania's difficulty calculation system requiring the user to input the map difficulty.

Furthermore, I showed Hamish the system of map editing/map creation and asked him the following question:

- What do you think about the map editing system?

Hamish responded by saying "It can lead to human error" and "alongside human error, there is also bias" "A lot of experienced players are more likely to play a higher rated map, for example rated 10 rather than 1, which leads map creators to inflate the difficulty to play them." This shows the fundamental flaws of being able to input the difficulty and clearly identifies the issues of bias and misjudgment. The is due to every map creator has had different learning curves whilst playing the game, therefore perception of difficulty is relative; Somone else's perception of being difficult may be different to another. This suggests that in my newer adaptation of StepMania there must be constant, quantitative measures to determine difficulty to avoid misjudgment and bias.

Osu!mania

As mentioned earlier, Dean Herbert's osu!mania is a popular online VSRG with a competitive scoreboard ranking. This is evident in Osu!'s official country rankings website (https://osu.ppy.sh/rankings/osu/country) that show, at the time of writing this, that there are currently 981,025 active users in the United States alone and a total of 24,202,202 total registered users, as seen on their home page (https://osu.ppy.sh/). On top of this there is a specific scoreboard ranking for osu!mania as seen in the osu!mania rankings page (https://osu.ppy.sh/rankings/mania/performance), showing that the higher your performance score is, the higher your scoreboard ranking. Thus, giving osu!mania the most competitive nature of modern VSRGs. This is a crucial piece of information as the score of your performance, or as it is termed in Osu! as "performance points" (commonly abbreviated as "pp") you gain is proportional to the difficulty of the charts you play. This is evidently shown through Osu!'s official FAQ (https://osu.ppy.sh/wiki/en/FAQ#scoring) on performance points, which states "The easiest way to improve it is to score high on difficult songs and playing more songs." Often higher-ranking players must continually play difficult maps to maintain their status due to this very reason. Furthermore, in Osu!'s Wiki[1], it describes a "weightage system" to "prevent the rapid and repeated gaining of lower pp scores" on "easy beatmaps" by "reducing the amount of pp that is gained." This means that a percentage of pp is lost the lower the rank the map is, meaning a pp score that ranks second place in best plays will be lose Further suggesting that the only way to increase your "pp" is to play maps continually and progressively with increasing difficulty.

1 https://osu.ppy.sh/wiki/en/Performance_points/Weighting_system

All this evidence suggests that the difficulty of the maps a user plays is a key factor. Therefore, the calculation system to determine the difficulty is substantial.

Before the map difficulty must be calculated, the map must first be made in the map editor system. Osu!mania contains a section of the game in which audio files can be dragged into the game. Upon doing this, a screen allowing the user to enter the song details and customize the metadata of the song appears. Afterwards the user is taken to an area where they can place notes and navigate through the different sections of the song in a customizable manner. Afterwards, once the user is satisfied with the way they have arranged the notes for song, they can exit the map creation system.

This means that my adaptation must contain a system to create and adapt maps in real-time. My adaptation should also include a map selection system to select maps to play and an audio file management system. Much like osu!mania's system, my adaptation must also contain the ability to edit maps metadata and background images. Furthermore, Osu!mania's map creation system does not contain an autosave system meaning you

must save maps manually. Therefore, my adaptation may add a system to automatically save maps whilst editing them. In osu!mania, once the map has been saved (often manually), The difficulty of the sections of the map is calculated.

As mentioned earlier, osu!mania uses density system to do this. The vertical column with pink and white rectangles indicates the sequences of arrows that are within the song as it scrolls. The vertical bar with yellow bars indicates the density of the patterns (notes per second). The taller the bars the denser the pattern. If the bar turns pink this signifies an extremely dense
sequence of arrows.

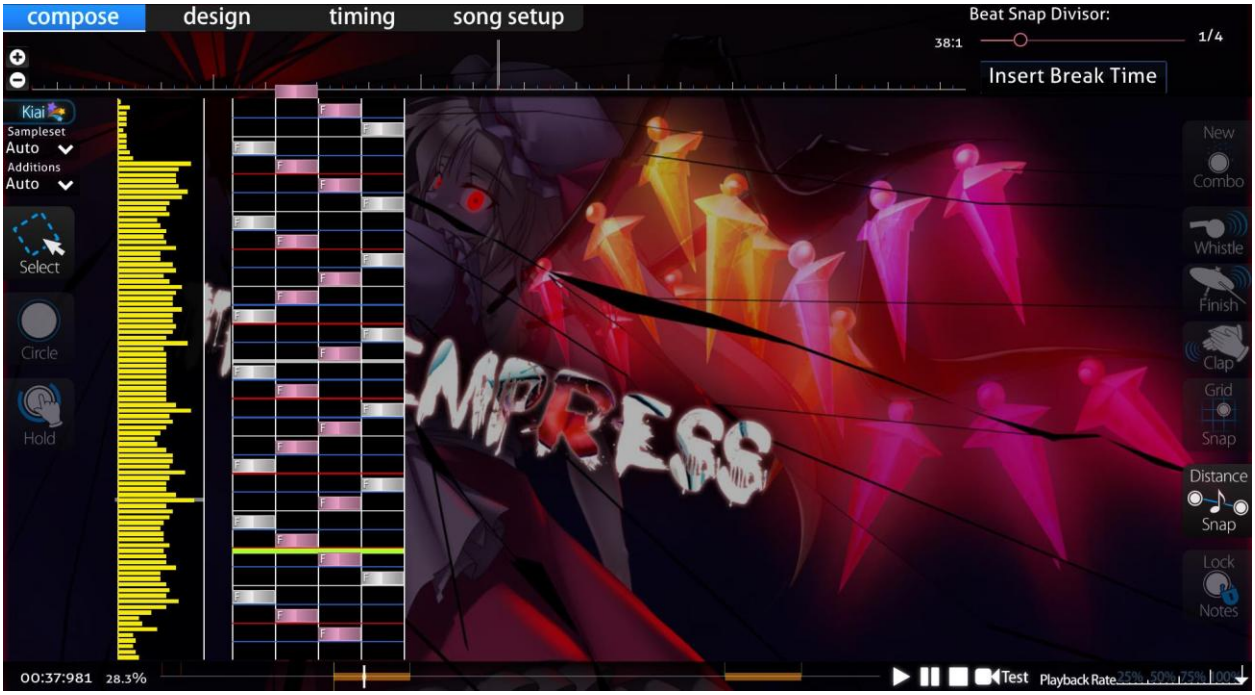


Figure 1 - An osu!mania beatmap in map editing. This reveals the inner workings of the map and the density of the notes. Yellow bars show density.

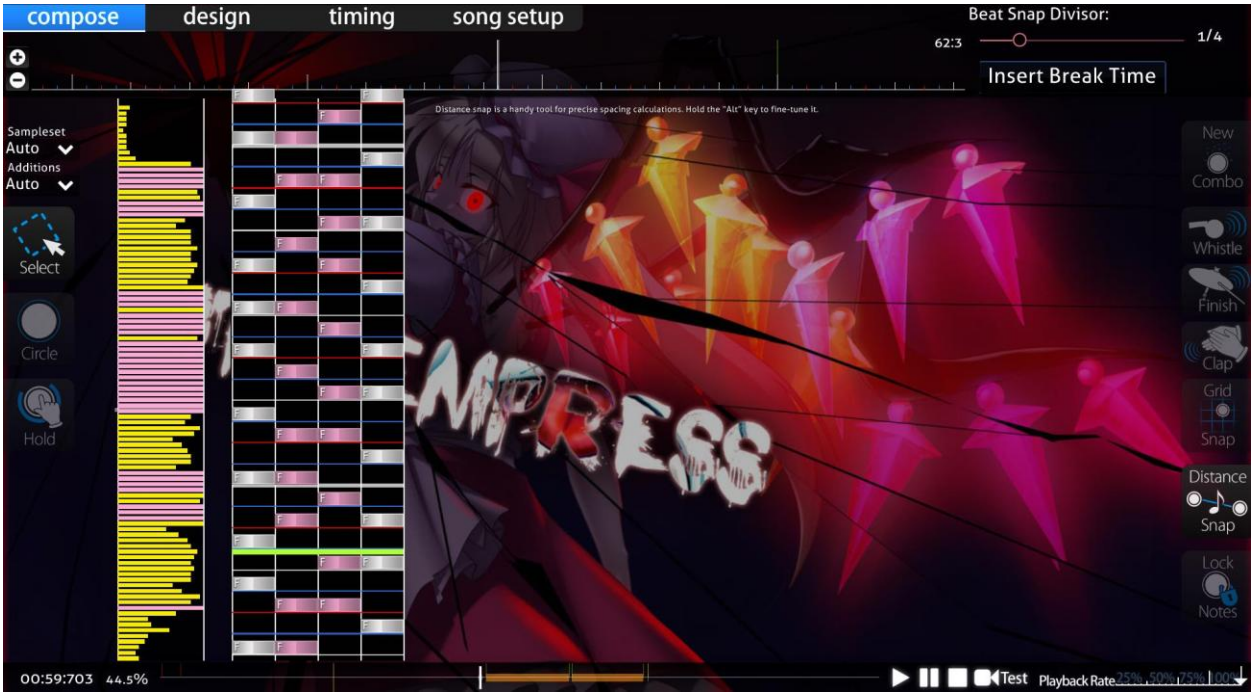


Figure 2 - The same beatmap but a higher difficulty. The pink bars show the extemely dense nature of patterns

Figure 3 shows the nature of maps with high difficulty requiring more dense patterns for longer periods of time. This is shown as most of the map is “pink” with extremely dense patterns.

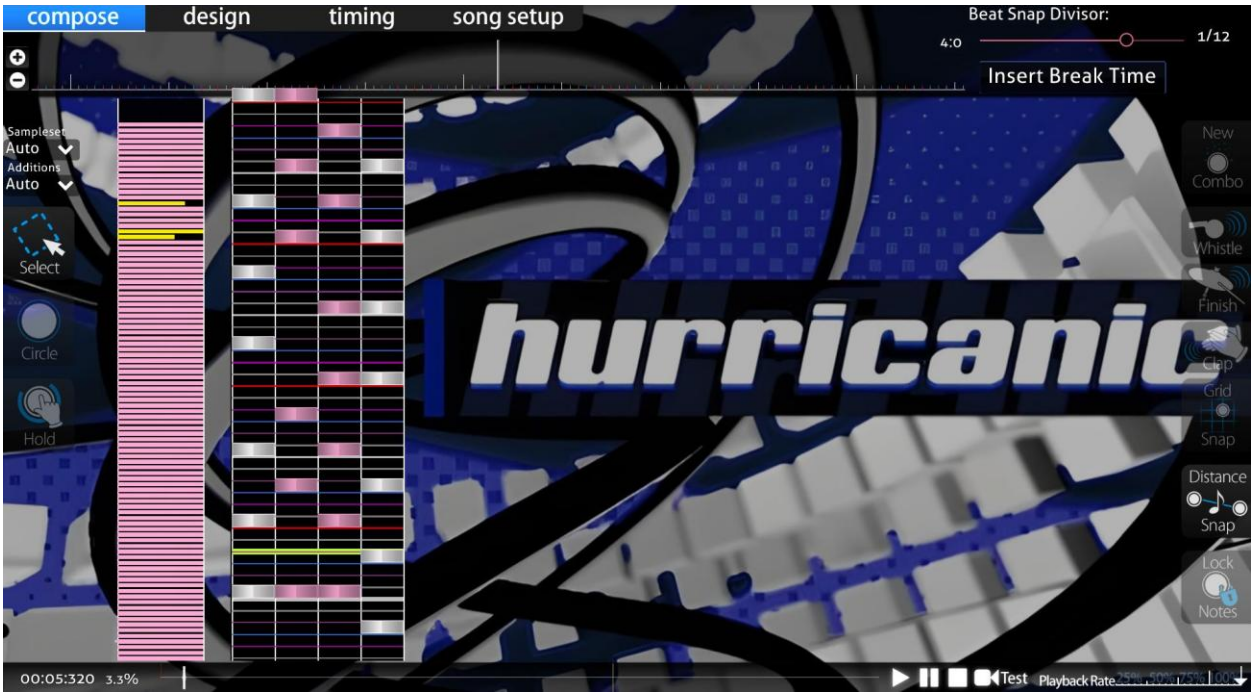


Figure 3 - The pink density graphs of beatmap with 5-star difficulty rating in osu!mania.

There are flaws to this system. This is evident in the fact that Osu! have tried to improve their difficulty calculation system by releasing an update to the difficulty calculation as seen in this official article ：https://osu.ppy.sh/home/news/2022-10-09-changes-to-osu-mania-sr-and-pp. This already gives an insight into the issues that are faced with difficulty calculating. Despite these changes, the difficulty calculation flaws have remained and are still prominent to this day.

To describe the problem with this system and show evidence of the issue of difficulty calculation, I interviewed Nathaniel Binuhe (as mentioned earlier). Nathaniel agreed to play a couple different beatmaps on osu!mania to discuss the problem of incorrect difficulty calculations. To start with, Nathaniel played a song that rated 3.88 stars but with a high BPM of 270 and with technical patterns (see earlier sections). The star rating, BPM, artist name, source (record label), beatmap creator, overall difficulty, long note count (will discuss this later), key count, etc. Can all be seen in the top left corner of the map selection as shown in Figure 4 and 5.

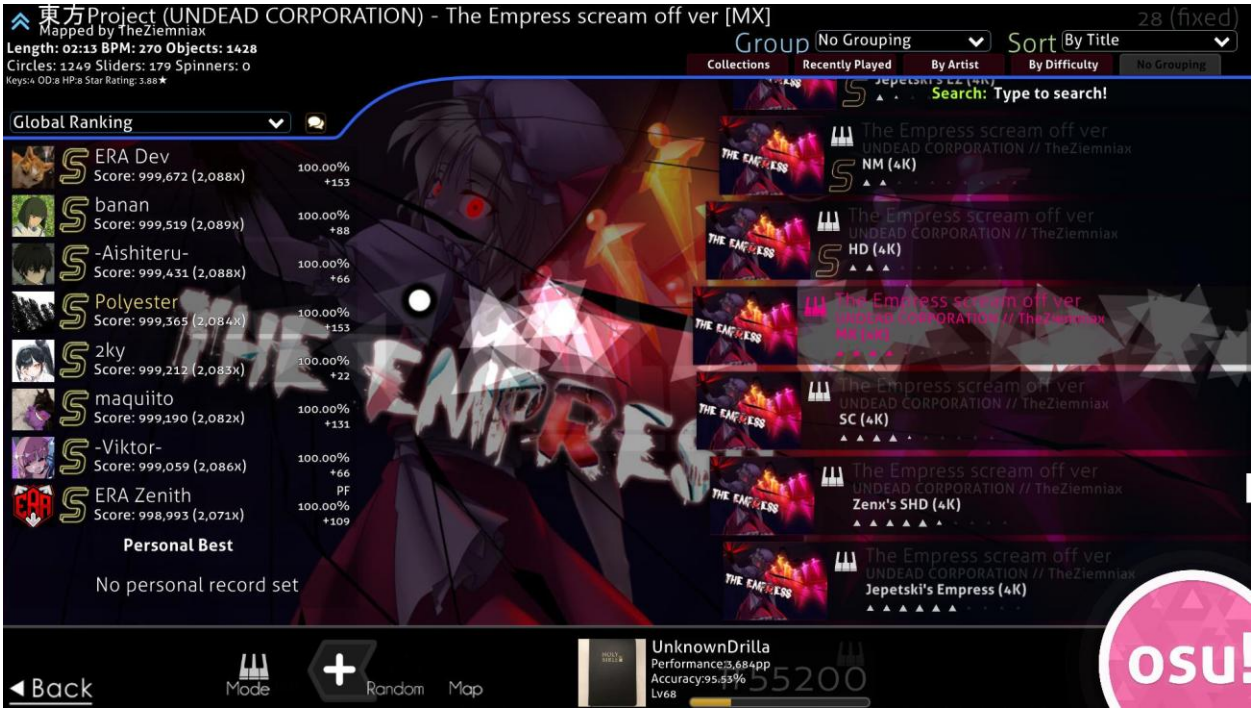


Figure 4 - Map selection screen of osu!mania. Top right shows details and difficulty calculation.

Figure 5 - A zoomed in photo of the beatmap details located in the top right corner of the map selection.

Nathaniel then played a song rated 4.27 stars with a lower BPM of 140 and full of patterns that are prone to being miscalculated. He played the first map and could not complete it. He stated it was "much harder" because of the "higher bpm" and the "insanely fast and hard to read technical patterns." He also stated that "the first map should be harder (have higher difficulty calculation) than the second map," even stating that the first map "should not be 3.88 stars." This signifies the misleading nature of osu!mania's difficulty calculation and the need for improvement on difficulty calculation. On the other hand, the second map, although having a burst of technical patterns that are prone to being miscalculated, Nathaniel stated, "it is much easier" and "slower due to the lower BPM." He was surprised stating "the second map is harder is than first map," further stating that "It should not be harder than the first map." Overall, Nathaniel stated that the main issue with the game is the "misleading ratings." Despite the fundamental flaw of the difficulty calculation, Nathaniel still stated that the game is "still fun to play" and commends the "nice modern graphics" and the ability to "add skins."

A discussion on Osu!'s official GitHub (https://github.com/ppy/osu/discussions/12980) can be found to show a user named "BrokenGale's" disclosure of the need for "osu!mania star difficulty improvements", proposing a new and "augmented star rating for mania".

Thus, it is evident to see the potential issues of mis-judged rankings that can arise, especially if there is a malformed difficulty calculation system to determine the difficulty of maps. Therefore, It is justifiable to identify my adaptation's approach of a new difficulty calculation system.

Friday Night Funkin'

Ninjamuffin99's Friday Night Funkin' is one the most popular and prevalent VSRGs of the modern-day era. It is widely renowned for its unique cartoon style graphics and its upbeat and catchy video game style music. This can be shown in two reviews on Metacritic (https://www.metacritic.com/game/friday-night-funkin/) by a user under the alias of "izack" states that the game has "banger songs" and "the sprites are fire". EndlessFlame928 also states that "The mods are keeping the game alive" and "the songs are a banger". This shows that the more modern and younger audience prefers a more modern, retrospective style of interface. This contrasts with older VSRGs such as Dance Dance Revolution and Stepmania which include a more perspective 3D style of interface. To get an insight into the modern graphics that are common in VSRGs I interviewed Samuel Smedley by asking him a series of questions:

- Do you like cartoon art and graphics of the game?

- Do you like music and character sprites?
- Did you think that the difficulty of maps was harder/easier than they should be?
- Did you feel like it lacked better difficulty calculation/assumption of charts?

Samuel Smedley then responded, “I like [the] cartoon art because it has a unique style that’s extremely vibrant and exciting.” This shows the evidence of the younger generation having more of an appeal for more modern and smooth graphics. Samuel also disclosed his taste for the music of the game by stating “The music is very high energy which fits the setting of each “map”.”

Therefore, it is evident that an approach of keeping the more modern useability and functionality of the user interface but at the same time maintaining the older arcade style 3D renditions of older VSRGs is an identified and justifiable approach to my adaptation. 3D re

Samuel also responded with criticism of “The character sprites I think lacked variety in how it was the same for every map and situation which left more to be desired.” Samuel’s criticism justifies the need for an ability to integrate customized design in the creation of maps. This will allow variation and not leave users with

Alongside the graphics, Samuel responded by stating “the very high energy… doesn’t always translate to the charts as the notes were not always synchronized.” This adds evidence to my previous point of VSRGs sharing a widespread problem of difficulty judgement and as Nathaniel mentioned earlier, “inconsistent grading.” Samuel also expressed “For a beginner player, the difficulty is a good standard but quite quickly after a few hours the hardest difficulty is too easy.” This led Samuel into suggesting “There should be a setting which progressively makes the encounters more difficult” . This suggestion shows the effect of underestimation of difficulty calculation and further signifies why a quantitative measure of difficulty calculation system must be approached. This is evident in the difficulty selection screen for songs as shown in figure 6. The difficulty system has no calculation and is based upon the game developers and mod developers to determine the difficulty. Another factor is that there is no quantitative measure. As mentioned earlier. the difficulty system of Friday Night Funkin’ is based on qualitative measurements e.g. “Hard”, “Medium” and “Easy”, that are relative to perspective and somewhat abstract. These measurements are prone to being misjudged generally and sometimes may give an incorrect indication of the map. Thus, it leads to inconsistencies. This means that my adaptation must implement a quantitative representation of the difficulty.

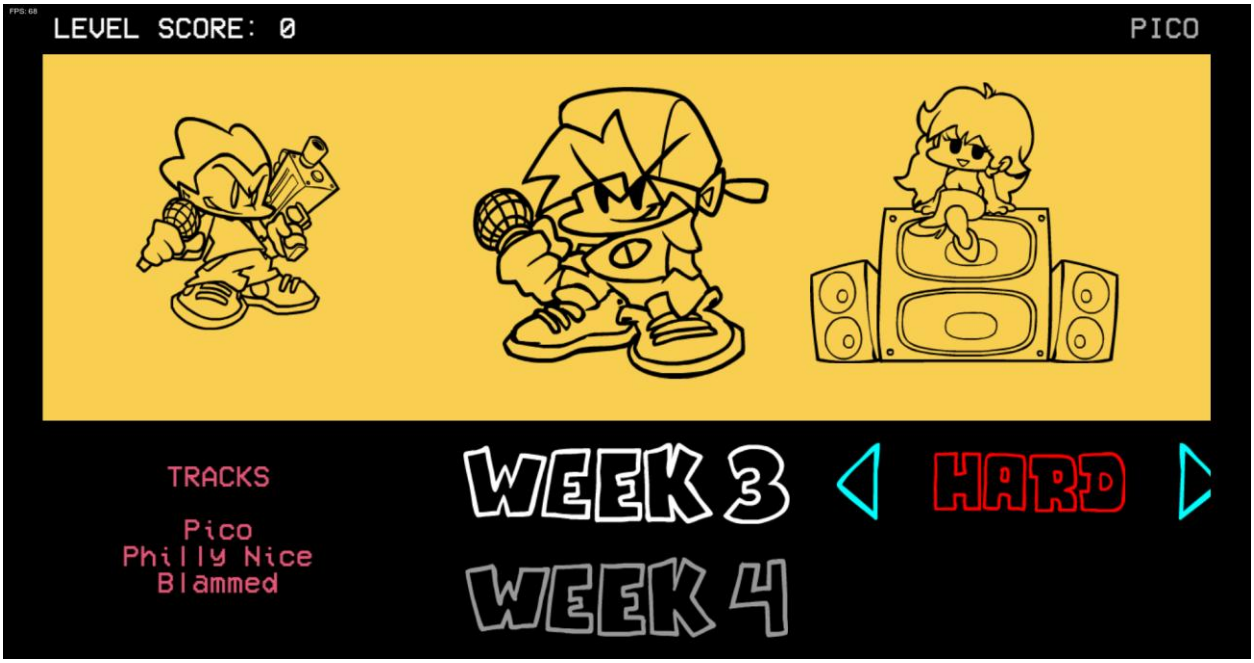


Figure 6 - Abstract difficulty calculation system of Friday Night Funkin'

Dance Dance Revolution

The significance of the success of DDR can be seen in Wikipedia's "Similar Games" section in their article about DDR (https://en.wikipedia.org/wiki/Dance_Dance_Revolution), stating "Due to the success of the Dance Dance Revolution franchise, many other games with similar or identical gameplay have been created." This evidence shows that the onset of multiple VSRGs is a result of inspiration from DDR and therefore the basis of VSRGs after DDR took inspiration from it. This is also evident as the article mentions "Fan-made versions of DDR have also been created... The most popular of these is StepMania." This indicates that core aspects of early VSRGs such as StepMania originated from Dance Dance Revolution as they were either fangames or spin-offs of the game.

As a result of my interview with Maria Sheehy, stated that she is "reminiscent" of the game and particularly the art style of the older VSRGs. Figure 7 shows an insight into the style of the graphics on Dance Dance Revolution. It consists of less cartoon stylized graphics as seen in Friday Night Funkin' and more solid 3D text and a realistic perspective. Therefore, for my adaptation to maintain the older arcade style of older VSRGs, 3D aspects of text and design shall be an implementation of my adaptation's user interface. To implement these 3D aspects, I must program and utilize a 3D rendering system into my game engine that will allow rendering of 3D and allow transformations in the z-plane.

As well as the user interface, DDR is notoriously known for its dance machines that are required to play the game. However, I do not intend to add this functionality as the younger audience of my stakeholders do not wish to have this functionality and prefer simpler forms of playing VSRGs. This is evident in my interview with Louis White. Although Louis stated, “I enjoy going to the arcade,” he feels “not everyone has access to an arcade in their local area” and that “the cost of going to an arcade may not be worth it.” He then stated, “playing at home with a computer is much easier and saves time.” To further elaborate on Louis’ point, an article from Kineticist[1] states that the average cost of a DDR machine is “$3,000-$20,000+” and a new machine is upwards of “$9,000-$20,000+”. This shows if the younger generation would want to play for longer durations personally, they would have to deal with the grievous expenses of buying a dance machine. This is evidence to suggest that integrating features of dance machine hardware into my game would not benefit my stakeholders’ needs.

1 https://www.kineticist.com/post/buy-a-ddr-arcade-machine#:~:text=TL%3BDR%20on%20how%20much%20a%20DDR%20machine%20will,a%20game%20room%20retailer%20like%20Game%20Room%20Guys.

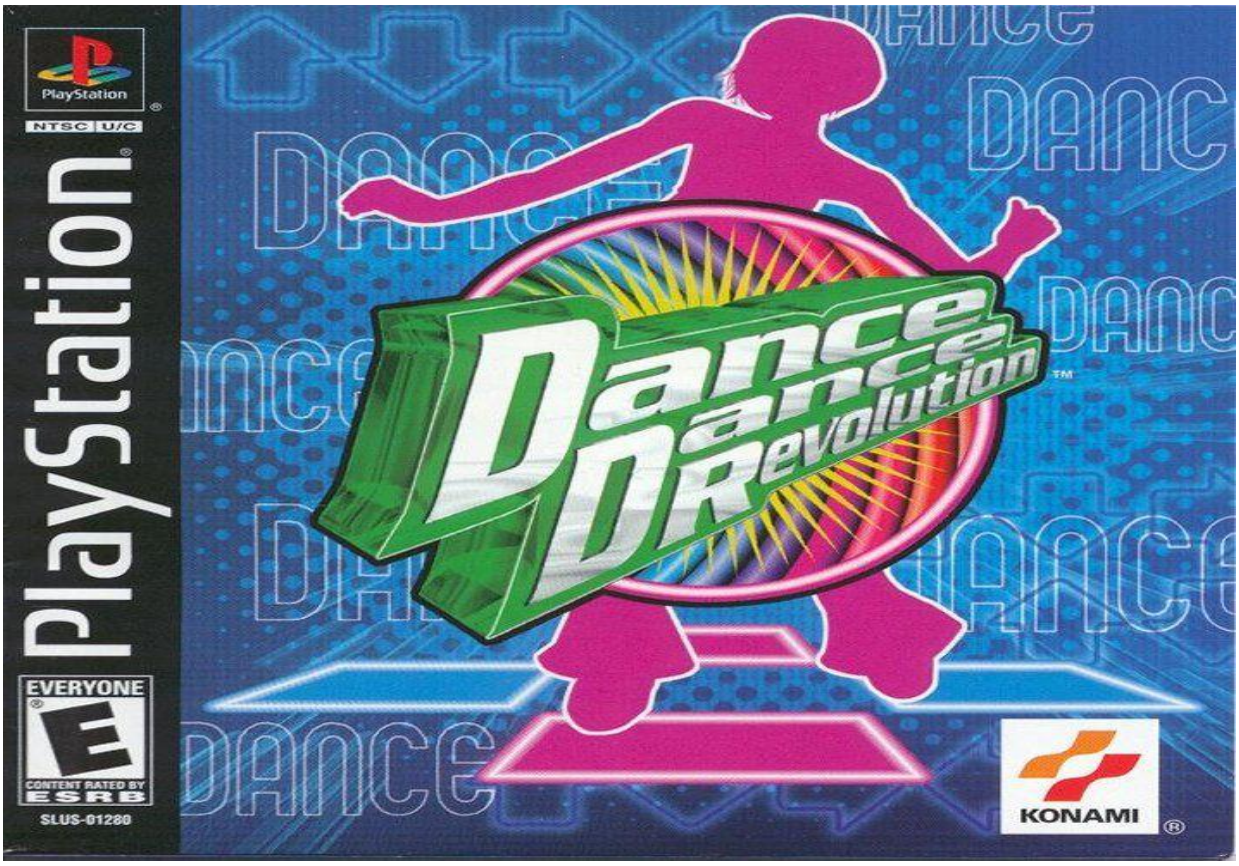


Figure 7 - PlayStation Release of Dance Dance Revolution's game cover.

In The Groove

To delve deeper into the graphical design of older VSRGs, I researched Roxor Game's In The Groove, commonly known as ITG on online forums. ITG is another PlayStation 2 clone of Dance Dance Revolution and was released shortly after. ITG's use of 3D aspects in their logo design, gameplay and menu give the game its nostalgic acquisition of the general style of arcade games of the early 2000s. I believe this is thoroughly shown thrown ITG's PlayStation 2 game cover, which involves a 3D representation of the "arrow" notes (commonly found in the main gameplay of Dance Dance Revolution and other VSRGs) at a different perspective angle and with a 3D depth to its design. This really amplifies the early 2000s feel of the game and gives it a nice touch that is distinguishable from other modern VSRG's such as Friday Night Funkin'. Therefore, in my adaptation's system, including forms of perspective shifts, as seen in the game cover of ITG and Dance Dance Revolution alike, will add to the general feel of mid 2000s VSRGs. These perspectives shifts may be included in micro interactions and parts of user interface that require animation and movement.

Harrison Jarvis has experience with the immersive 3D aspects of rhythm games such as Beat Saber. When asked about his take on the 3D immersion and perspective of ITG. Harrison stated that "It is not too crazy" and "I like the simple design of the logo and game as it makes the game style and gameplay more fluid." He stated, "I like the retro style of ITG and "It shows you what it is from the cover… you can tell it's a dance game." He concludes with "The "big arrow" reminds of the classic dance games of the arcades." Harrison's perspective further suggests that the simple retrospective 3D designs that use clear features relating to VSRGs i.e. arrows, people dancing, musical terms, etc. are what make arcade VSRGs so iconic and appealing. This is especially true for the older audiences as such distinct features are what causes feelings of reminiscence.

Figure 8 - PlayStation 2 Release of In the Groove's game cover.

Another aspect of ITG's style is its main gameplay's arrangement of arrows using different depths in the interface as well as effects of outlining, shadows and in game lighting, that

further enhances the retrospective feel of an early 2000s VSRG.

Figure 9 - In The Groove's main gameplay.

To achieve the desired 3D aspects and designs of older VSRGs, my adaptation must contain a 3D environment to render and perform the different perspectives shifts Therefore, I must use different computational 3D rendering techniques such as model and view matrices and matrix transformations with the use of perspective projection. This should be done programmatically, and the 3D aspects shall come from coding the transformations instead of it entirely being based on design.

Computational Methods

Thinking abstractly

Areas of my adaptation will need abstraction to make the solution to complex problems clearer and easier to solve. Using abstraction will help in focusing on what a part of my code does and its function (what it will output), rather than how it works. This is through hiding the actual implementation of functions after writing them and then reusing them within larger sub-routines.

These abstract functions will then help in piecing together solutions without needing to care so much about how each individual function works. This will benefit my process of development as solutions and methods shall be compounded and based only on the relevant information required for the specific problem. This means that during solving

problems, I will only require knowledge of what data needs to be input and what function is needed to process the data that gives an expected outcome.

This will benefit during debugging problems as I will have only the data, I need to solve the problem and do not have to dissect other irrelevant information. It will also greatly speed up the process of writing code itself and allows a focus on conceptualization of solutions to problems and other computational methods.

Abstraction will also benefit in areas such as making program code more readable and maintainable through the reuse of abstracted sub-routines. It will also benefit the users themselves as they will not need to understand the complicated processors of my adaptation's inner workings. Examples of how abstraction is used in my program include :

1) The use of external libraries, such as glm, to perform complex mathematical operations in accordance with development. I will need to use mathematical constructs such as:
    a) matrix transformations
    b) matrix multiplications
    c) vector addition
    d) vector subtraction
    e) matrix addition
    f) matrix subtraction
    g) multiplying matrices by a scaler
    h) multiplying matrices by a vector
    i) use of trigonometric functions
    j) compute the magnitude of a vector

All functions provided by the library will hide the unnecessary details of how to compute these mathematical concepts and purely focus on performing their intended tasks. Another factor is that library functions have been optimized and thoroughly tested by their developers. This will especially benefit debugging as it will indicate that the error is mostly likely to do with my own source code and how I used the function and not the function itself. Furthermore, the library functions are less likely to have large computational complexity. For example, in a standard matrix multiplication algorithm, to multiply two $n \times n$ matrices, it will require n^3 new multiplications of scalars and $n^3 - n^2$ new additions to compute its product. This means a standard matrix multiplication algorithm has an average time asymptotic (time) complexity of $O(n^3)$. However better matrix multiplication algorithms that are likely to be included in the library functions, have a lower asymptotic complexity of $O(n^{2.3751552})$, thus saving time and being less intensive on the CPU/GPU.

2) Navigation of the user interface - the user does not need to know the details of how the menu navigation system works, just the ability to know what inputs are required to move from one section of the game to another. I will require functions that integrate user interface controls that abstract the process of drawing/rendering triangles through the graphics pipeline and loading of textures/images. These include:
    a) Functions/subroutines that abstract the updating of uniforms variables within shaders and GLSL files to be part of the 3D rendering process.
    b) Have function/subroutines embedded within rendering/drawing GUI classes to abstract the view/projection matrix and remove the need to attribute each matrix transformation when updating source code.
    c) Have reusable sub-routines to simplify and abstract the process of drawing GUI elements. For example, the entire process of rendering a triangle, texturing it, converting it from local space coordinates to view space coordinates and then implementing accessibility to the texturized quad made of triangles to form a "GUI element", can all be simplified into one draw function e.g. `drawGUI()`
3) The use of external window rendering libraries such as SDL2 will abstract the process of rendering a window on the screen. These libraries will simplify window creation and hide all the details that happen in the process of forming a window. Instead, I will only focus on a singular window creation function and the aspects of my window. SDL2 will also provide an OpenGL context for me to render in. Rendering in the OpenGL context is the focus of my adaptation as it is where every element of the user interface shall be rendered. Window libraries, such as SDL2, will abstract features such as
    a) Providing an OpenGL context – The library does not require me to implement me a 3D rendering context/environment myself
4) The use of an OpenGL toolkit/API such as glad that will abstract the process of implementing the specification functions/subroutine calls to the drivers that the graphics card supports. OpenGL is only a standard/specification and there are many different versions of OpenGL drivers, therefore the location functions/subroutines to use OpenGL is not known at compile-time. This means it is up to the user to retrieve the location of the specification functions/subroutines (typically in pointers) and store them for later use. This process is cumbersome as you may need to retrieve the location for every function that has not been declared. However, these toolkits/APIs have abstracted the need to do this. Examples of the abstractions due to toolkits/APIs:
    a) Not having to focus on the entire process of retrieving the graphics specification function/subroutine location within the drivers and instead focus on the actual use of the subroutine. For example, the subroutine: `glGenBuffers()`, would not have to be manually retrieved and instead I can focus on the use of it. This eliminates the unnecessary process of having to retrieve it beforehand.

b) The use of the toolkit/API means that the subroutine calls are updated and thoroughly tested. This provides access to the most up-to-date function/subroutine calls within the drivers and allows me to not worry about the function itself and instead focus my adaptation's code.

Thinking Ahead

Before designing my solution, I must think about the sub-routines required to form a working GUI navigation system as the majority of my adaptation's functionality and accessibility shall be through a GUI. Furthermore, I must think about being able to integrate the 3D aspects within a 2D GUI, to provide a multi-scene perspective. For this, I must ensure that my navigation system is fully responsive to button presses such as WASD and the Left/Right/Up/Down arrows keys.

I must think about the different reusable classes needed in forming a GUI system that is maintainable, reusable and quick to form new sub classes such as different menu screens and game states like the editing/creation maps section. Features of the GUI class/system include:

1. Universal functions that are inherited in all derived classes are required to render the GUIs onto the screen. These functions will be reusable all throughout the GUI system and will simplify the process of making an interactable GUI. These functions include:
    a. A draw function to draw the GUI element on screen
        i. Within this function it will have:
            1. Position of GUI
                1) Converting local space coordinates to screen space (NDC)
            2. Rendition of triangles and quads for the element
            3. Width and height of GUI element
            4. Percentage of fill of the texture
            5. Texture filtering type (more on this later)
        ii. Within this function a function to load images for the texture
    b. A rotation function that integrates matrix transformations to be able to rotate the GUI without rewriting code
    c. A scale function that integrates the matrix transformations to scale GUIs accordingly
    d. A function to adjust the color, hue and alpha values of the textures and GUI elements without re-texturization.

    e. A function to draw untextured polygons and fill them with color. This will benefit in making some UI elements for menus
2. An animation derived sub-class that packages and abstracts the transformations into more feasible general subroutine. These include:
    a. Flip the GUI element vertically and horizontally and on its axis
    b. Animate these transformations as transitions by implementing them as animation functions
    c. A movement/transform animation that integrates a matrix transformation with iteration to give a "moving" animation on the GUI element. This will be useful while implementing gameplay.
3. Accessibility within the GUI system to ensure the usability of the GUI elements once it's been created. This includes:
    a. Checking if mouse corresponds to the range of the coordinates of the GUIs "size"
        i. Converting from local space coordinates to view space coordinates
            1. Vector and matrix multiplication
        ii. Collision detection algorithm

I must also think about the audio aspects of my game. To do this I must reuse a core audio class. This will also involve outputting audio through the user's sound system accurately and synchronously to the different aspects of gameplay. Features of audio and timing can include:

1) Starting and stopping the map's audio whilst playing a map on time to the input
2) Adding a preview of the map's soundtrack during map selection#
3) Ability to upload audio files for map creation
4) Ability to load different audio files for maps during the map selection in real time
5) Ability to preview sections of audio files for a short period of time during map selection.

Alongside audio, I must pair this with input timing at the same time. Audio and timing go hand in hand and are crucial aspects to the main gameplay. I must be able to utilize an input timing class that is able to correctly time the users input and give a valid response to their timing in real-time without any inconsistencies. This is so that users can receive accurate judgment based on their timing when hitting notes in gameplay. For the timing I must:

1) Form a system that will not have inconsistent input registration and be responsive to the different timings of input. This will be evident during the main VSRG gameplay.

a. The system must be able to handle multiple keyboard input simultaneously and expect accurate output.
b. Must be able to calculate the timing of the input to an accurate degree and compare the timings of the data within a map. This will involve real-time processing and will be used during the gameplay when analyzing hit time results. This will be crucial in determining hit accuracy (as discussed earlier).
c. The system must be able to give the timing to a suitable degree of accuracy and have a universal accuracy gauge to determine the timing.
d. An average timing system to determine the g

As well as classes and subroutines, I must think about the different data structures that I will use to implement the different aspects of my game. The data structures must allow me to access data efficiently and store large amounts of data without affecting performance and main memory. Some of these data structures include:

1. Arrays – Needed to store vertex buffer data and objects to the data of the coordinates and vertices of the triangles that will represent interfaces and icons
2. Vectors/Lists/Dynamic arrays – During the process of storing the map files and comparing them to the input timing
3. Hash Tables – For providing fast access to maps whilst searching for them in

Thinking Procedurally

To form a stepwise approach to my problem, I must break my problem into more manageable sub-programs. Each sub-program is broken down into their own retrospective elements to the solution of the problems. Examples of the sub-programs include:

1) Enhanced difficulty calculation system to determine note patterns prone to being miscalculated
    a. Data processing algorithms and pattern recognition algorithms to determine certain patterns within map files.
        i. Multiples that contribute to difficulty calculation algorithms such as strain, readability, playability and speed, not just density alone.
            1. Process and read map data
                a. Have a suitable data format for processing e.g. text files.
                    i. Organize that data to be processed in a fashion such as:
                        1. Note timings
                        2. Column placement

3. Beat snap

b. Have predetermined data to signify certain patterns e.g. four continuous notes in the same column are considered as a “Jackhammer” pattern (This is the terminology used to describe repetitive notes in one column in VSRGs) and therefore is a strenuous pattern and should be a factor to increase the difficulty of the map.
    i. Count notes in adjacent and previous columns
    ii. Count notes in adjacent and previous rows
    iii. Measure notes timings
    iv. Count quantity of notes in each time frame
        1. Determine average quantity of notes throughout entire time frame (mean)

2. Calculate the difficulty based on the data
    a. Add discrete factors such as readability and playability of the pattern and multiply it by continuous data such as strain and speed to give a final calculation
        i. Calculate averages
        ii. Calculate the magnitude of continuous intervals of data
    b. Add a deduction system for patterns that are prone to inflate difficulty
        i. A tier/ranking/hierarchy class system of all the patterns and their different presidencies of playability.
        ii. Contribute the averages of all these factors to prevent short bursts of difficulty spikes overinflating maps.

b. Processing of difficulty efficiently and quickly to factor in large quantities of maps.
    i. Efficient use of data structures
        1. Arrays
        2. Hash tables
        3. Linked list
    ii. Efficient use of searching and sorting algorithms

2) Adding 3D renditions to the adaptation’s user interface and gameplay
    a. Use of perspective projection and matrix transformations

i. Use of model, view and projection matrix in perspective projection
    1. Constructing an identity matrix (A $n \times n$ matrix filled with 1s diagonally)
    2. Applying matrix multiplication using the math library functions
        a. Apply multiplication to model, view and projection matrix
ii. Apply to the model, view and projection matrix uniforms within shaders
iii. Allow multiple renditions of this process on different elements for reusability.

b. Integrate this into the GUI elements
    i. Have the updating of the projection matrix as part of the GUI class process.
        1. Potential inheritance or class association of the GUI and the graphics pipeline class

Thinking Logically

I will require logical thinking in the difficulty calculation system of my adaptation, especially during the forming of algorithms when determining patterns. I will also need logical thinking for handling mathematical concepts and constructs throughout the code. Examples of logical thinking in the development of my adaptation include:

1) Determining the note patterns based on conditions such as position. My algorithms must be able to determine and factor in the positioning of the notes in their respective columns and rows.
   Here is just one example of a note pattern that can be determined using logical thinking. These are not all the examples however it does show the logical thinking involved with determining the algorithms.
   a) The “trill” note pattern
      i) The use of if-else statements or switch/case chains to check If notes are alternating between columns on adjacent rows
         (1) “Trills”, where the first note begins on column *n with a column length of l*, where *n=1, l=4*
            (a) If the $n+1^{th}$ note from note n, on the $n+1^{th}$ column is on the $n+1^{th}$ row.
               (i) Check if note *n+2* is in the same column as note n and repeat checking of condition (i) until note n+k is not in the same column as note n.
      ii) To do this I must use while loops to count the notes until the condition is broken.

iii) Once the condition is broken the count of the loop is I must set the strain factor to the count of the notes and total time taken of the map, *t,* from note *n,* to note n+*k*.
(1) The use of multiplication

2) Determining whether the user is clicking/interacting with the user interface
   a) The use of if else statements in coordinate calculations to determine if they are overlapping
      i) Determine whether the user's mouse is colliding with the GUI element
         (1) Check if the mouse coordinates lie in range with the GUI element's "box size"
            (a) Calculating the difference between the GUI element's center and the mouse position.
               (i) Integrate magnitude checking using if statements
               (ii) Translating local space coordinates to screen space coordinates
                  1. Multiply local space coordinates by model matrix to get world space coordinates
                  2. Multiply world space coordinates by view matrix to get view space coordinates
                  3. Multiply view space coordinates by projection matrix clip space coordinates
                  4. Pass in the result into the uniform variables in the shader files
                  5. Lasty set the position of the coordinate within the shader files
               (iii) Determining the difference between the GUI element's top left and the center of the element by finding the midpoint of the coordinates.

3) The use of switch/case statements to determine whether the inputs are within the different ranges of accuracy measurement. An example of this can include:
   a) Checking if the timing lies within millisecond intervals
      i) For example, If the judgment window to achieve "Flawless" hit accuracy is 22.5 milliseconds and the input timing is 16ms then use an if statement to compare then
         (1) Use switch/case chains to compare if the input accuracy is equal to the judgment in real time.
            (a) Use of an array or hash table data structure to provide instant access to timing data to minimize delay

Backtracking

I will need backtracking for various parts of my algorithms in where I will need to sequentially go back to previous the note *n* from note *n+k* to determine the duration of the and pattern sequence and the time taken to hit the sequence.

I will also need backtracking to go back over the user's gameplay data in their gameplay replay to determine an accurate "grade" based on the average count of accuracy measurements.

Heuristics

I will need Heuristics for forming new algorithms that are like each other. These algorithms will generally have the same format except the position of the columns are shifted. For example, the VSRG note pattern commonly referred to as a "jumptrill" is much like the note pattern "trill" except it consists of notes alternating in two adjacent columns instead of one adjacent column (An example of this would be two notes in the first and second column and on the next row, two notes on the third and fourth columns). The algorithm to determine whether the note pattern is a "jumptrill" will be very similar to the algorithm to determine whether the note is a "trill." Therefore, taking a heuristic approach of going back and reusing the structure of the "trill" note pattern algorithm shall be beneficial to approaching the structure of the algorithm.

Divide and Conquer

I could potentially use divide and conquer in the process of searching during map selection. However, the use of data structures like hash tables may not require me to search for maps using divide and conquer due to their constant access time of O(1) and their ability to be accessed via a key.

Visualization

I will use Visualization during the map creation process. As mentioned earlier, many VSRGs such as StepMania and osu!mania use a visual representation of the notes to allow the editing of the map file. Despite the data itself being in text file, the sequences of notes and their patterns are represented as adjacent icons in a horizontal row of vertical columns. The user can then place or delete notes by clicking on them. The user can also adjust the beat time of the maps and make the visual representation of the gaps between the notes smaller. This thoroughly helps in the map editing/creation process as it abstracts/simplifies the need to edit and place the map data in a file one note at a time.

Another key concept of visualization is based within the concept of VSRGs themselves. VSRGs and rhythm games alike typically do not give a judgment on whether the visual

artifacts are overlapping but give it based on the accuracy of the user's input timing . In cases like StepMania and other VSRGs, notes scrolling towards the receptors give a visual indication of the time to press the key. This is because the notes being rendered on the screen begin at the top of the screen and scroll towards the receptors in a certain amount of time. This means the time taken for the note to begin at the top of the screen and scroll vertically until it visually appears like it overlaps with the receptors, will coincide with the actual timing value that is within the file of the chart; The note acts like a visual cue for the correct time to press the note. In essence, if the user were to input when the note is perfectly in the center of the receptor visually, the user's input timing will the equal to the exact time of "the beat" (based on the current time of "the beat" in the song). This means that the visual aspects of VSRGs are merely just a visual representation of the interval of time that the user must input (via their input device) within.

Complex Calculations

My adaptation shall require multiple complex calculations, many of which are provided by external libraries. Many of these calculations have been abstracted and therefore do not need to be manually written. However, they must be implemented to allow the essential aspects of gameplay, user accessibility and interface. Most of these calculations have been abstracted by the external math library. These calculations include:

1) Calculating the timing offset to draw the arrow based on the users scroll speed
    a) Calculate the vertical magnitude of the distance from the center of the receptor to the top of the column by subtraction. This is to calculate the time by dividing this distance by the user's note velocity.
        i) Transform local space coordinates to view space coordinates
            (1) Multiply local space coordinates by 4x4 matrices and multiply view, model and projection matrices together. Most of these f
                (a) 4x4 Matrix Multiplication:

$$\begin{bmatrix} a & b & c & d \\ e & f & g & h \\ i & j & k & l \\ m & n & o & p \end{bmatrix} \times \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix} =$$

$$\begin{bmatrix} a + 5b + 9c + 13d & 2a + 6b + 10c + 14d & 3a + 7b + 11c + 15d & 4a + 8b + 12c + 16d \\ e + 5f + 9g + 13h & 2e + 6f + 10g + 14h & 3e + 7f + 11g + 15h & 4e + 8f + 12g + 16h \\ i + 5j + 9k + 13l & 2i + 6j + 10k + 14l & 3i + 7j + 11k + 15l & 4i + 8j + 12k + 16l \\ m + 5n + 9o + 13p & 2m + 6n + 10o + 14p & 3m + 7n + 11o + 15p & 4m + 8n + 12o + 16p \end{bmatrix}$$

                (b) Matrix by vector multiplication (for local space coordinates)

$$\begin{bmatrix} a & b & c & d \\ e & f & g & h \\ i & j & k & l \\ m & n & o & p \end{bmatrix} \times \begin{pmatrix} x \\ y \\ z \\ w \end{pmatrix} = \begin{bmatrix} ax + by + cz + dw \\ ex + fy + gz + hw \\ ix + jy + kz + lw \\ mx + ny + oz + pw \end{bmatrix}$$

b) Divide it by the users scroll speed to get the time in milliseconds

c) Display the notes at the top of the screen and let it scroll vertically at the velocity of the users scroll speed. By the time it scrolls to the center of the receptor, the timing of this event would be the same as the time data in the map file data.

2) Adding 3D perspective rotations for GUI elements integrated within perspective projection

a) Matrix rotation

i) Rotation around the X-axis:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & cos\theta & -sin\theta & 0 \\ 0 & sin\theta & cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \times \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = \begin{bmatrix} x \\ cos\theta \cdot y - sin\theta \cdot z \\ sin\theta \cdot y + cos\theta \cdot z \\ 1 \end{bmatrix}$$

ii) Rotation around the Y-axis:

$$\begin{bmatrix} cos\theta & 0 & sin\theta & 0 \\ 0 & 1 & 0 & 0 \\ sin\theta & 0 & cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \times \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = \begin{bmatrix} cos\theta \cdot x + sin\theta \cdot z \\ y \\ -sin\theta \cdot x + cos\theta \cdot z \\ 1 \end{bmatrix}$$

iii) Rotation around the Z-axis:

$$\begin{bmatrix} cos\theta & -sin\theta & 0 & 0 \\ sin\theta & cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \times \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = \begin{bmatrix} cos\theta \cdot x - sin\theta \cdot y \\ sin\theta \cdot x + cos\theta \cdot y \\ z \\ 1 \end{bmatrix}$$

3) Adding translations to GUI elements to give them aspects of animation

a) Translating a vector coordinate into local view space to a new coordinate on the local space

i) Vector by matrix multiplication (see above)

User Requirements

For my adaptation and for most VSRGs in general, the user does not require any foreknowledge or background information requirements on how to play VSRGs. The user can simply engage with the adaptation/VSRG for the first time and be able to experience gameplay. This is due to the VSRGs game mechanics and interfaces being mostly intuitive and indications on the mechanics of gameplay. However, some knowledge and experience on playing VSRGs would be recommended. This can include general knowledge on how to set up key binds for gameplay and general experience with map editing/creation in VSRGs.

The only feasible user requirements that the user needs are a laptop/desktop with a functioning keyboard and mouse and monitor to play the game. However, certain factors such as a keyboard with a high polling rate (1000Hz+) and a mechanical keyboard (preferably with red switches) shall greatly enhance user experience. These factors are not a requirement.

The user will be able to run the adaptation on any form of operating system if the OpenGL drivers are supported within them. Most computer operating systems such as Windows, MacOS and most Linux kernels have their graphics card manufacturers integrate OpenGL drivers into their systems. This means that use will have a wide range of systems to play my adaptation on. Despite my adaptation conforming to multiple operating systems, my primary operating system will be tailored towards Windows due to it being the operating system that I am using to create the game.

Essential Features

There are multiple factors that are essential to my adaptation. The factors all include different features and their justification.

Feature:

1. A working user interface with different menu screens for the different game modes
    a. A main menu
        i. Working 3D interface and micro interactions
        ii. Section button to allow access to map editing/creation
        iii. Section into gameplay settings
    b. A map selection menu
        i. Displaying map metadata
            1. Song name
            2. Map difficulties
            3. Map BPM
            4. Thumbnail
            5. Map length
            6. Map artist
        ii. Ability to scroll and select each song

Justification:

A working GUI menu is an essential part of gameplay as it is the gateway to a user playing a map and experiencing the main gameplay. If there was no map selection, the user would not be able to decide which maps they can play. This means issues such as playing maps

not tailored to their skill level can arise. Therefore, having the choice of maps of different difficulties is an essential feature

Feature:

2. A working map creation and editing system.
    a. Uploading audio files system
    b. Allowing input of metadata for map file
    c. Previewing of song sections during map editing
        i. Ability to inspect the song in real-time
        ii. Automatically set a preview section that plays during the selection of the map
    d. A visualized map editing system consisting of four vertical columns that can be navigated through each section of the map
        i. The deletion and placement of notes
            1. Placement of long notes
                a. Algorithm to determine the release of hold note
                    i. Account to beat timings
                    ii. Ensure the key release is judged
                    iii. Ensure key press is judged
        ii. Beat snap divisors
            1. Divisors that to the 4^{th} beat up to 32^{nd} beat
                a. Zoom in / scale editing preview factor for sections with densely populated notes
    e. Map saving system
        i. Save map data to text file
        ii. Auto save
            1. Update map in short intervals

Justification:

As mentioned earlier, for the difficulty of a map to be calculated, the map must first be created. Therefore, a system to create and play maps is an essential feature of the adaptation. If there was no visual map system, then maps would need to be created manually via inputting data into a text file. This would mean a very cumbersome process as the timing and column data will need to be input for each note. This would potentially cause errors and difficulties as large quantities of text are data are prone to mistakes.

Feature:

3. The standard working VSRG gameplay – Once a map is selected to play with its desired difficulty, the gameplay shall commence. This consists of a stationary receptor at the bottom of the screen and notes that will scroll from the top of the screen to the bottom of the screen. All of this will happen typically to the "beat" of the map's music.
    a. Notes to be drawn outside the maximum viewport y-value to ensure notes do not just "appear" and seem like they are scrolling in from out of the screen.
    b. Notes to scroll from outside screen to bottom of receptors
        i. Position to be updated every frame
            1. Position updated by distance between the notes starting position and receptor, all divided the user's scroll speed
                a. Scroll speed system
                    i. Accessed through main menu game settings
                    ii. Numerical value between

Justification:
Without the fully adept, core gameplay mechanics of a VSRG, my adaptation will not have succeeded in amending the problem of difficulty calculation. This is because if the main gameplay system does not function fully intended then the difficulty calculated for the map will be a misrepresentation of the user's true ability as the user's true ability is not truly accounted for. Therefore, the functionality of the gameplay must be working to the correct standard.

Feature :

4. An improved difficulty calculation system to accurately and gauge map difficulty.
    a. Algorithms to process the map data and determine the note patterns within maps
        i. Pattern detection system
            1. Conditions for certain patterns
    b. A system to open the map files
        i. System must integrate the difficulty calculation in real-time after map saving
            1. Map saving system
    c. Different Factors to attribute to difficulty calculation
        i. Speed
        ii. Strain
        iii. Readability
        iv. Playability

Justification:

This is the main feature of my adaptation. As mentioned in research, a new and improved difficulty calculation system will prevent the map difficulty of maps with note patterns prone to being miscalculated from being overestimated/underestimated. I conclude in research that the difficulty should not be calculated based on density alone and other factors such as strain and the maps BPM should be considered into calculation. This way an accurate estimate to map difficulty shall be achieved.

Hardware and Software Requirements

For my adaptation, I will be using the latest version of OpenGL (4.6) as of now. However, most features of OpenGL 4.6 are backwards compatible with version 3.3. Therefore, The minimum hardware and software requirements must be suited to run at least OpenGL 3.3. The minimum hardware and software requirements run my adaptation include:

1. Windows (OpenGL 3.3+)
    a. CPU: Any dual-core based processor e.g. Intel core 2 duo E6850 SLA9U
    b. RAM: 1 GB or more
    c. OS: Windows XP SP2 or later
    d. Graphics Card: Must support OpenGL 3.0; typically requires a card from NVIDIA GeForce 8 series or AMD Radeon HD 2000 series or later.
2. macOS
    a. CPU: Intel-based Mac processor
    b. RAM: 2 GB
    c. OS: macOS 10.6 or later
    d. Graphics Card: Supports OpenGL 3.0, typically found in NVIDIA GeForce 8600 or newer, or ATI Radeon HD series.
3. Linux
    a. CPU: Dual-core processor e.g. Intel core 2 duo E6850 SLA9U
    b. RAM: 1 GB
    c. OS: Kernel 2.6 or later
    d. Graphics Card: Supports OpenGL 3.0; NVIDIA GeForce 8 series or AMD Radeon HD 2000 series or later.

Regardless of the operating system and its version, the main requirement for the user is the appropriate drivers installed for your graphics card to utilize OpenGL. This means any system with the essential OpenGL drivers can be able to utilize my adaptation. Proprietary drivers (like NVIDIA or AMD) often provide better support than open-source drivers for this reason.

Limitations

The main limitation to my adaptation is not integrating compatibility for dance machine support. As mentioned in my research earlier, it would not benefit my stakeholders to include dance machine support due to the expensive of owning a machine and that most of my audience prefer to engage with VSRGs on their own personal home device instead of commuting to an arcade to access a dance machine. This is also due to the limitation of not having direct access to a local arcade or branch that owns dance machines.

Another fundamental limitation to my adaptation is it only consists of four keys of input during main gameplay. Many VSRGs such as Konami's Beatmania series have more than four keys for input, allowing users to hit notes on 5 or more columns. This is another feature found in osu!mania. osu!mania allows users to create and edit beatmaps that allow different amounts of keys. The key count of the maps ranges from as low as a single key of input to as high as ten keys of input. This means that you can play a one key map using a single finger and a ten key map with ten fingers. The reason for this limitation is that the original DDR and StepMania did not allow more than four keys, so to keep the aspects of my adaptation like StepMania, it is necessary to maintain strictly four keys of input only.

Success Criteria

For my adaptation to be successful, there are a set of essential criteria that by the end of development, my adaptation must have met. These criteria include:

- The user interface must be fully working with access to the different game sections via clickable buttons
- The user interface must also allow keyboard navigation with the use of left/right/up/down and enter input keys
- The user interface must contain a main menu screen with the logo of the game and the ability to use input to navigate to gameplay
- The main menu must contain a background with aspects such as underlap based on different z-coordinates (layering)
- The main menu must contain a text button in which the user can click to indicate progression in map selection i.e. a play button
- The interaction of the start button must successfully transition you into the map selection screen
- The map selection must contain a list of maps ordered and grouped by artist or difficulty
- The map selection system must allow scrolling through each map via input left/right keys

- The map selection system must display a thumbnail of the maps song cover/ thumbnail image
- The map selection system must display a moving background as the underlay/background for the map selection
- The map selection system must preview a preview of the song in the background of the current selected map
- The map selection should display the maps, metadata and song information within a specified GUI section beside the list of maps
- Each map should have the ability to have more than one difficulty
- The map selection system should allow the changing of the maps different difficulties
- The map selection system should allow the pressing of enter to signify the progression into gameplay
- The map should have standard VSRG gameplay - notes that scroll from out the top of the screen to stationary receptors
- The gameplay must accurately start audio in synchronization of beginning gameplay
- The gameplay must allow the player to hit notes once they are in the timing interval (near) the receptors and give an on-time judgement of accuracy without delay
- The gameplay must keep track of the average accuracy of the user and give an end “grade” based on performance
- The gameplay must save the “grade” as a score inside an external file and
- The user must display their average accuracy as a percentage overall along with their “grade”
- The user must allow the exiting of the “grade” screen and can repeat the cycle of selecting a map and entering gameplay
- The user must have the ability to go back to the main menu via back button GUI
- The user must have the ability to enter map editing/creation via a GUI
- The map editing screen must first have a list of all the current maps
- The map editing screen must have a section to upload an audio file and commence map creation
- The map editing must have columns in which the song can be previewed, and notes can be placed
- The map editing system must have a feature to set a thumbnail/song cover for the map
- The map editing system must have a feature to save maps and update their data in real time

- The map editing system must store the map data in an external folder with an external file in a representable and readable data format e.g. a text file.
- The map editing system must allow returning to the menu screen from saving a map
- The adaptation GUI system must have quick interoperability and interchangeability of each section of game i.e. map selection to map creation to gameplay, etc. Without error
- The menu screen must a GUI to allow access to customization of gameplay i.e. a "settings" button
- The settings section must allow changing of scroll speed and changing of key binds
- The settings section must allow the changing of note/receptor size
- The settings section must allow the changing of scroll direction
- The user must successfully be able to exit for the game by pressing the "x" on the window
- The user can also exit the game via the menu through an "exit" GUI.

Design

To initiate the design of my adaptation, I first must begin with the design decomposition of the components of my game engine that must be formed first when setting up my adaptation. These components will be responsible for allowing my adaptation to have the core features of its system. These features include the ability to create a window, render user interface, handle input and manage audio. With these components, the larger parts of my adaptation that will be accessible by the user e.g. Chart Editing System can be formed.

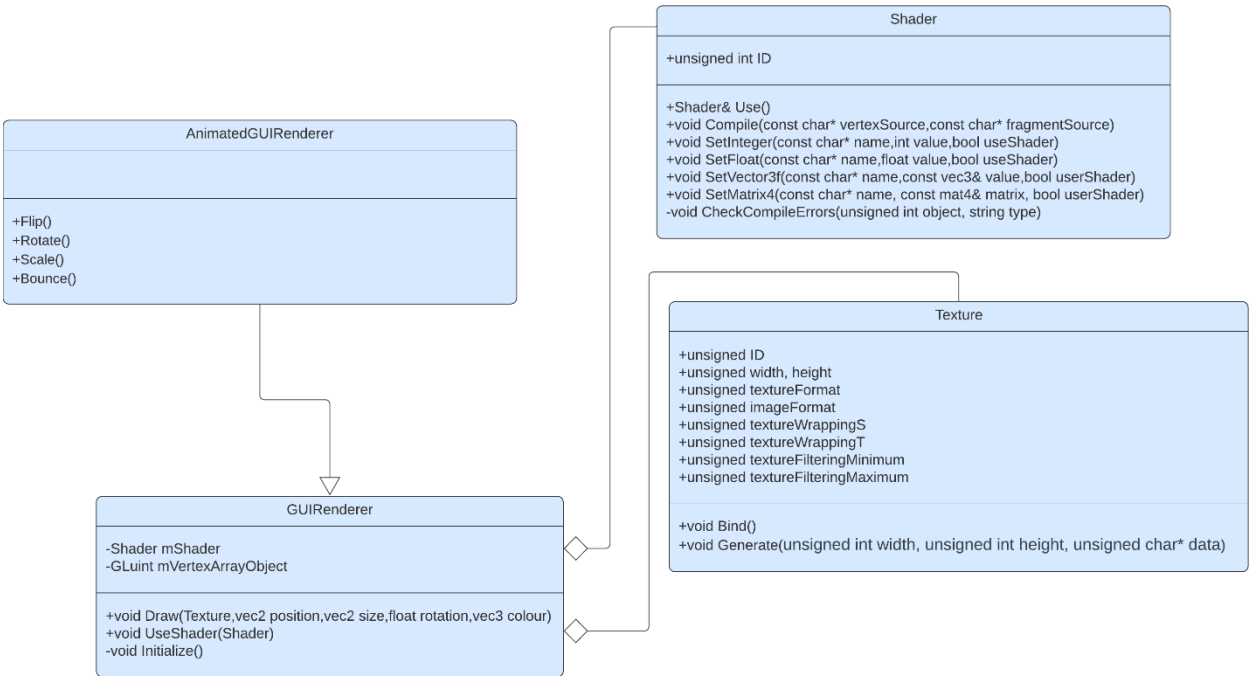
GUI Renderer Class

As mentioned in research, my system will require a GUI system that will simplify the process of rendering UI onto the game window. The GUIRenderer class will act as that "GUI system" and will be the base class in which other classes such as the "AnimatedGUIRenderer" class (more on this later) will be derived from. However, the GUIRenderer class itself is an aggregation of two other sub-components that are required to render any kind of interface when working with OpenGL. These two aggregations are a shader class and texture class and will be discussed in detail later. The GUIRenderer takes these sub-components and combines their features to allow the use of an abstracted and reusable "Draw()" function that will allow quick rendering of user interface. This will allow the real time rendering of interfaces such as buttons and textures for GUIs within my adaptation. For example, the GUIRenderer class will allow both the game logo and the background in adaptation's start menu to be render in just two calls of the "Draw()"

function. This links back to the research in which having a GUIRenderer class allows abstraction of the somewhat cumbersome process of loading textures, generating the texture data, binding the data to a texture bind spot and using the specified shader and prevention of this process being repeated for each instance of user interface in source code.

Class Diagram

The class diagram of the GUIRenderer shows the composition of the class and its main procedures and variables. The class diagram also shows the aggregated Shader and Texture classes and their functions and procedures and the derived AnimatedGUIRenderer class.



Shader Object

The GUIRenderer class will consist of a Shader object and an unsigned integer that is a numerical identifier to a vertex array object. Earlier in the research, discussion about real-time processing in graphics pipeline mentioned that shaders are programmable and are responsible for processing vertex data so that OpenGL can display them as pixels on the viewport. The shader object in the GUIRenderer class specifies which set of vertex and fragment shader files are being used to process the vertex data (that is in a vertex buffer in which an attribute pointer in the VAO points to). This means the GUIRenderer class will have the ability to be instanced with different types of shaders. This means that vertex data

can be processed in different ways at the same time which allows different ways of pixels to be displayed on the screen. For example, an instance of GUIRenderer has its shader variable as a shader that processes the data by moving the texture coordinates on each frame, whilst another instance of GUIRenderer uses a shader that keeps the vertex coordinates static. In this example, the first instance will display an animated “moving” user interface whilst the second instance will display a static use interface. This is crucially important for a system like my adaptation as the rendering of static and animated interfaces will be used frequently in areas such as main gameplay e.g. for the rendering of scrolling notes but the rendering of a background image remaining static.

Vertex Array Object

The vertex array object is responsible for disambiguating the different types of data that are stored within the vertex buffer. In the case of my adaptation, the different types of data within the vertex buffer objects include the texture coordinates (the coordinates of the texturized pixel) and actual vertex coordinates for a quad. These types of data will be stored as an attribute that the vertex array object points to. The reason for having a vertex array object is due to OpenGL 3.3 requiring at least one vertex array object that points to one vertex buffer object for any rendering to be done. Another reason for using a vertex array object is the key feature of OpenGL allowing the binding and unbinding vertex array objects once they are set. What this means is that once the attributes of the vertex array are set, they do not have to be set again for each time the user wants to use the vertex array object.

Initialize() procedure

This key feature ties in with the use of the “Initialize()” procedure for the GUIRenderer class. The Initialize() procedure is responsible for initializing the vertices required to form a quad and the texture coordinates. Once this data has been initialized, it is bound to the vertex attribute object. This means that the same vertices will be used to draw every user interface. This is important as the process of rendering a texture will become standardized which leaves no ambiguity and thus less room for errors .

UseShader() procedure

The “UseShader()” procedure will allow the changing the currently in use shader object to another shader object. This will be useful in circumstances where rendering a different user interface requires a different type of shader. For example, if one interface requires rendering of a GUI with 3D elements and another interface only requires rendering of a 2D GUI, then there will be two different shader files requiring perspective projection and

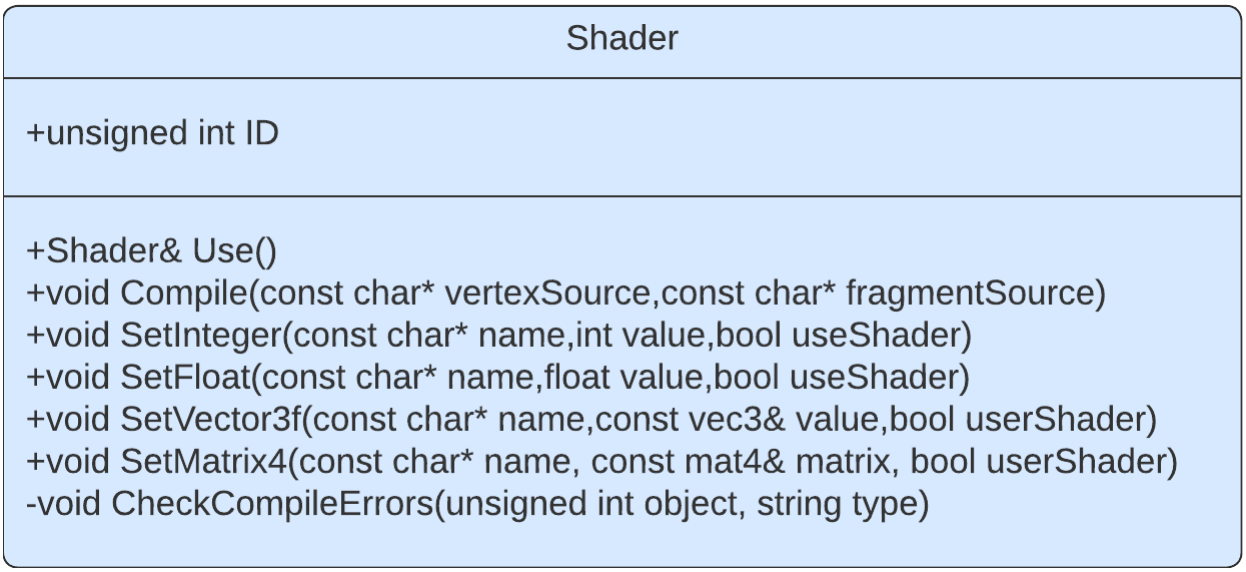
orthographic projection respectively. In this case, the use of the “UseShader()” will allow the dynamic rendering of both 2D and 3D objects in my adaptation.

Shader Class

As shown in the class diagram above, the shader class is used to encapsulate the process of loading and using shader files. The shader class will take the paths of the fragment and vertex shaders as input, read the file’s source code, compile the source code and create the shaders. Once the shaders are created, an OpenGL shader program is made, and the pair of shaders are attached to the program. Afterwards, I can use the shader program to process vertex data program by calling it my code when processing vertex data.

Class Diagram

This is the class diagram of shader in independence.



ID

The shader class contains an unsigned integer variable called “ID” to represent the shaders numerical identifier. Each time OpenGL compiles and creates a shader program, OpenGL stores its reference as an unsigned integer. The ID variable stores a copy of the reference to allow specific use of the shader program for any instance of the shader class.

Use() procedure

The Use() procedure in the shader class will be used for quickly changing the shader program to the program of the current shader object via it’s “ID”. This is useful for allowing multiple scene rendering (2D and 3D) and for rendering in different formats concurrently.

Pseudocode for Use() procedure

The Use() procedure will only have the function of switching the currently in use shader program. The key feature is with this function is that each shader class has an “ID” which is the numerical identifier for the shader. This means that calling the Use() function will switch the program to the instanced shader class program.

```
public procedure Use()
    glUseProgram(this.ID)
endprocedure
```

checkCompileErrors()

A function part of the Shader class that will check for any compilation errors when compiling the source code from a shader file. If any error is found, then the error is logged with a description of the error and the location of the error. The checkCompileErrors() has parameters for the shader ID and the type of compile error that Is being checked.

Compile() procedure

The Compile() procedure is responsible for taking in the sources code of fragment and vertex shaders, reading the source and compiling the source into shaders. Once the shaders are made, they are attached to an OpenGL shader program.

Pseudocode for Compile() procedure

The Compile() pseudocode will consist of creating the shaders using OpenGL’s built in functions, attaching the source code of the shaders then following the procedure to compile and link the shaders.

```
public procedure Compile(vertexShaderSource, fragmentShaderSource)
    // The vertex shader and the fragment shader source code will
    be passed in as a string and will be compiled for each shader
    respectively


    // Create and compile the vertex shader
    vertexShader = glCreateShader(GL_VERTEX_SHADER)
    glShaderSource(vertexShader, vertexShaderSource:ByRef)
```

```
        glCompileShader(vertexShader)
        checkCompileErrors(vertexShader)

        // Create and compile the fragment shader
        fragmentShader = glCreateShader(GL_FRAGMENT_SHADER)
        glShaderSource(fragmentShader, fragmentShaderSource:ByRef)
        glCompileShader(fragmentShader)
        checkCompileErrors(fragmentShader)

        // Create an empty shader program
        this.ID = glCreateProgram()
        // Attach the shaders to the shader program
        glAttachShader(this.ID, vertexShader)
        glAttachShader(this.ID, fragmentShader)

        // Link the shaders once they have been compiled and attached
        glLinkProgram(this.ID)

        // Once the programs have been made, the shaders can be
        discarded to free up memory
        glDeleteShader(vertexShader)
        glDeleteShader(fragmentShader)
endprocedure
```

Uniform Variable Setters

Within the shader class, the SetInteger(), SetFloat(), SetVector3f, SetMatrix4f() procedures will be responsible for updating the uniform variables inside the shader. This means that variables that have been set in the shader to take in some form of data, e.g. an integer variable to update the time in milliseconds, can be dynamically changed to any form from within the source via these functions. These functions will be responsible for changing the

variables required for updating the matrices for the GUI elements. An example use could be dynamically changing the position of an element by performing a matrix transformation on the model matrix and setting the model uniform to the result of the matrix transformation. This will cause the position of the rendered GUI element to be shifted.

Pseudocode for Uniform Variable Setters

The pseudocode for the uniform variable setters will have the option to identify the variable based on its name using the built in OpenGL glGetUniformLocation function. This is to allow easy access to the variable in real time without the need to call the uniform location finder function. The variable setters also give the option to use the shader when setting the variable for ease of access when switching between shaders.

```
// Set Float uniform variable
public procedure SetFloat(name, float value, useShader)
    if useShader then
        this.Use()
    glUniform1f(glGetUniformLocation(this.ID, name), value)
    endif
endprocedure
// Set Integer/Sample2D uniform vairable
public procedure SetInteger(name, value, useShader)
   if useShader then
        this.Use()
    glUniform1i(glGetUniformLocation(this.ID, name), value)
   endif
endprocedure
// Set Vector3 uniform vairable
public procedure SetVector3f(name, value:ByRef, useShader)
    if useShader then
        this->Use()
```

```
    glUniform3f(glGetUniformLocation(this.ID, name), value.x,
value.y, value.z)
    endif
endprocedure
```

Test Data

This table indicates the test data I will use for testing the shader class during the iterative development of my adaptation.

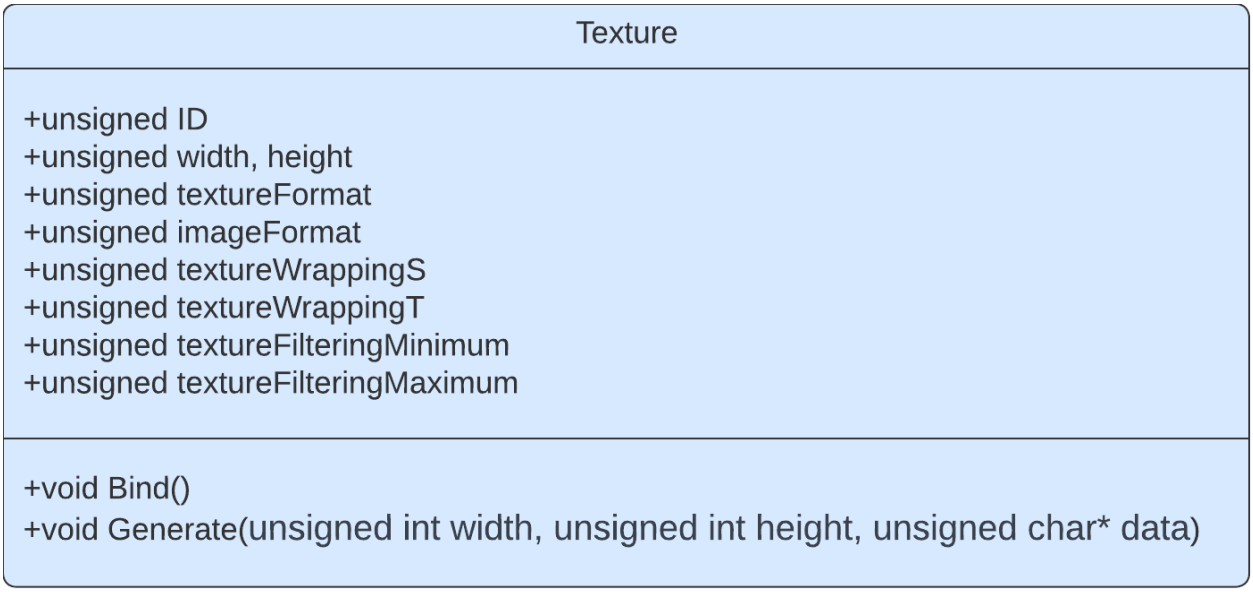
Test	How I will test the data	Justification For Test Data	Expected result
Compile() procedure	Pass in valid fragment and vertex shader source code with no syntax errors or logic errors	This is valid data to show that all OpenGL programs require at least one fragment and vertex shader for any rendering to occur	No shader compilation or syntax errors shown on the console and correctly rendered graphics on viewport
Compile() procedure	Pass in fragment and vertex source code with no syntax errors but with logic errors	This is boundary test data to suggest that shader source code must logically be correct and follow the correctly OpenGL procedures	No compilation or syntax errors shown on the console, however incorrectly rendered graphics or completely black screen on viewport
Compile() procedure	Pass in fragment and vertex source code with syntax errors	This is an invalid test data to test the error logging mechanisms in code	Immediate console errors that indicate the syntax error in source code alongside completely black screen on viewport.

Texture Class

The texture class will be responsible for storing the characteristics that will allow OpenGL to take the path of an image, generate a texture and set a bound OpenGL texture bind slot.

Class Diagram

Below is the class diagram of texture in independence



ID

Much like the shader class, each texture object has an unsigned integer used to represent the texture's numerical identifier. In OpenGL the maximum number of textures that can be bound simultaneously is 16, therefore having numerical identifiers to indicate which texture is bound will allow interchangeability between textures. Furthermore, each texture class stores their "ID" meaning the need to recall the numerical identifier is completely abstracted as the class can call its own "ID" via the "this->ID" pointer dereference.

Texture Format and Image Format

The textureFormat and imageFormat variables within the Texture class will be used to determine whether the texture being generated will require alpha values or not. This will be used for loading transparent images files such as the .PNG file format to be generated as textures.

Texture Wrapping

The textureWrappingS (X-axis) and textureWrappingT (Y-Axis) are the variables that determine the wrapping direction of the texture. This is how texture will be presented if it is a repeating texture or if the texture coordinates get shifted outside the displayed vertices.

Texture Filtering

As discussed in the research, the texture filtering must be determined when setting the texture properties. Texture filtering is the process in which OpenGL must figure out the texture pixel to map the texture coordinate to.

Generate() function

The Generate() function within the texture will carry out the process of taking in image data, setting the OpenGL texture parameters to the image properties and finally binding the texture to an OpenGL bind slot.

Pseudocode Texture Constructor

The constructor for the Texture class will be used to initiate the properties of the texture. The declared values will be the default values; however, they can be set to any value when the texture object is being instanced.

```
// Set default values for when instancing a texture object
class Texure
     public procedure new Texture
         Width = 0
         Height = 0
         iamgeFormat = GL_RGB
         textureFormat = GL_RGB
         textureWrappingS = GL_REPEAT
         textureWrappingT = GL_REPEAT
         textureFilteringMin = GL_LINEAR
         texturingFiltergMax = GL_LINEAR
         glGenTextures(1, &this->ID)
     endprocedure
endclass
```

Pseudocode For Generate() Function

The Generate() function will be responsible for setting the OpenGL texture state variables and calling the bind texture function calls.

```
public procedure Generate(width, height, data)
    this.Width = width
    this.Height = height
```

```
    // Create Texture

    glBindTexture(GL_TEXTURE_2D, this.ID)

    // Set texture image format

    glTexImage2D(this.textureFormat, width, height, 0,
this.imageFormat)

    // Set Texture wrap and filter modes

    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S,
this.textureWrappingS)
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T,
this.textureWrappingT)
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER,
this.textureFilteringMin)
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER,
this.textureFilteringMax)

    // Unbind texture
    glBindTexture(GL_TEXTURE_2D, 0)
endprocedure
```

Test Data

Test	How I will test	Justification For Test Data	Expected result
Generate() function	Pass in correct image data parameters including correct width and height of	This is valid data to suggest that the image data extraction library has successfully	The quad being rendered using the texture should be fully texturized

	image, the correct image format and the actual image data.	extracted image data to be texturized	correctly with no defects
Generate() function	Pass in correct width, height and image data however the image formatting is incorrect. For example, the image has alpha values, but the image formatting is not set to check them	This is boundary test data to show the errors that will occur when trying to load different image types without considering the image format when texturizing. E.g. loading a PNG image as a texture.	The quad being rendered will be texturized but the texture will have visual defects such as tearing, image warping or the image being vertically flipped.
Generate() function	Pass in incorrect image data all together and/or image formatting.	This is invalid data to show that texturization is highly dependent on whether the image data conversion function converts the image correctly.	The quad being rendered will not be texturized at all and will only show the fragment of the quad and potential runtime errors will output on the console.

Draw() procedure

The “Draw()”procedure (for the GUIRenderer class) will consist of the actual texture object that is used in reference when drawing and the transformations that can be applied to change the final output of the interface to be drawn. These transformations include translation, scaling, rotation and an additional parameter to transform the texture coordinate color of the texture. This is another important feature as the process of setting matrix variable and continually updating them in my code for each case of

Pseudocode For Draw() procedure

The Draw() function will take into consideration the parameters that have been passed into the function when drawing and will pass these parameters into the appropriate matrix arithmetic functions. The main key feature of the Draw() function is that the order of transformations is in reverse order. This is because matrix multiplication is non-commutative and is performed right from left.

The mat4 model variable is responsible for the initial identity matrix that does not have any transformations applied to it. For scaling, the model variable is first translated by half of its size. This is because the origin of the texture will be from the top left. To maintain the centered position of the texture must be scaled, translated by half its size, have any other transformations performed and then scaling transformation is reversed.

Another key feature of the Draw() procedure is the use of default parameter variables. This is because the user may not always want to input parameters for all variables.

Once the transformations have been performed and sent to the shader, the texture is bound, and the texturized quad can be drawn onto the viewport.

```
public procedure Draw(Texture, position = (0,0,0), size =
(10,10,10),rotate = 0, colour = (1,0,1.0,1.0))


     // Create identity matrix


         model = mat4(1.0f)


     // Apply any transformations to identity matrix


         model = translate(model, vec3(position, 0.0f))



         model = translate(model, vec3(0.5f * size.x, 0.5f *
size.y, 0.0f))
         model = rotate(model, radians(rotate), vec3(0.0f, 0.0f,
1.0f))
         model = translate(model, vec3(-0.5f * size.x, -0.5f *
size.y, 0.0f));
         model = scale(model, vec3(size, 1.0f))


         this.shader.SetMatrix4("model", model)
```

```
        this.shader.SetVector3f("GUIColor", colour)

        // Bind texture to texture bind slot
        texture.Bind()

        // Set currently in use vertex array object to the
class's vertex array object

        glBindVertexArray(this.vertexArrayObject)

        // Draw the triangles to form the texturized quads

        glDrawArrays(GL_TRIANGLES, 0, 6)
        glBindVertexArray(0)
endprocedure
```

Test Data

Test data	How I will test the data	Justification For Test Data	Expected result
Draw() procedure	Pass texture object and correct position, size, rotation and color value.	This is valid test data to show the process of drawing successfully rendering an image on screen	The texturized quad showing the GUI image being rendered at the correct position, at the correct size, and color value
Draw() procedure	Pass in texture object but not all parameters are inputted. For example, only passing in the size and position of the texture without	This is also valid test data due to the use of default variable declarations within the parameters. This means that if the user does not input anything for the function	The texturized quad being rendered at the correct position and size with no other resultant features. For example, the default value for position is (0,0,0)

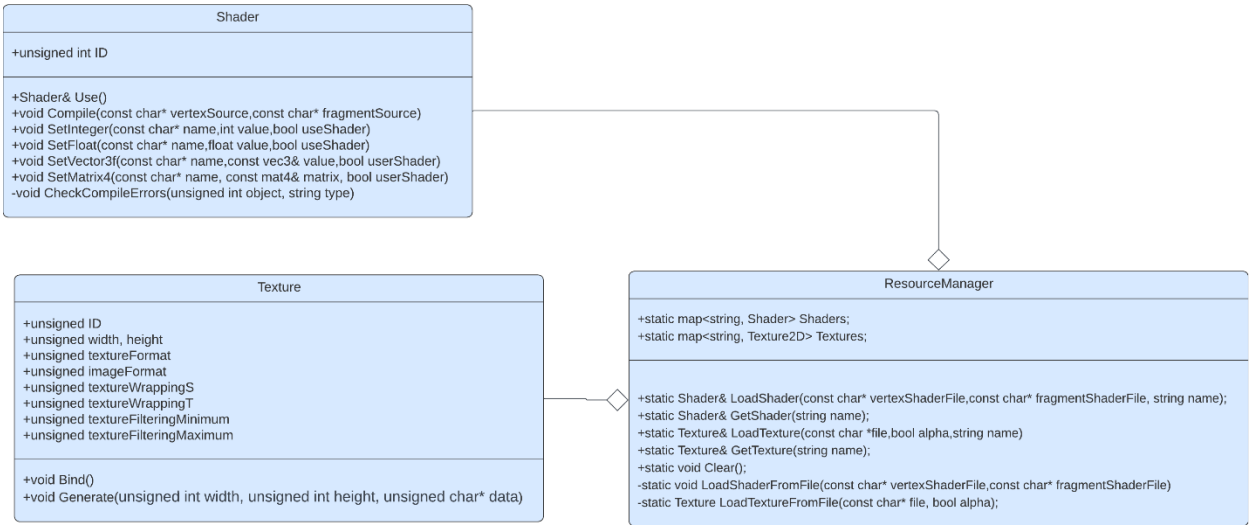
	inputting the color or rotation.	parameters, then it will resort to the default value stored in the function declaration. This is justified because during development, most textures will remain static with size and position. These default values will typically not affect the image in any manner.	meaning if there is no position value entered then the position of the quad will be in the top left corner of the screen.
Draw() procedure	Pass in the texture object with the size at (0,0,0) and the position a as the maximum size of the viewport	This is boundary test data as GUI elements being rendered must be within the limits of viewport otherwise the element's pixels will be clipped (not rendered on screen)	The texturized quad's dimensions will be the size of the viewport's boundaries.
Draw() procedure	Pass in the texture with no input parameters	This is boundary test data as the texture will immediately resort to its default values	The texture will be rendered according to the value of the default variables. In this case it would render in the top left of the screen and with a size of 10x10 pixels.
Draw() Procedure	Pass in the texture with a position and size that is greater than the viewport	This is boundary test data as the data is still valid, however it may not be intentional	The texture will be rendered correctly; However, it will be oversized and parts of the texture that are outside the viewport will be clipped.

Resource Manager Class

The Resource Manager class will be responsible for all the associated asset management. This includes loading shaders from .GLSL files and loading textures from image paths. The resource manager will greatly abstract the process of converting external resources to be useable by OpenGL by aggregating the Shader and Texture classes. The resource manager will not be responsible for the actual conversion of the assets, only the loading of paths of assets to be used by the Shader and Texture Class. However, the resource manager will make use of the Shader and Texture Class by calling the functions that convert the shader paths into shaders and image files into textures within the resource manager function calls. This greatly simplifies and abstracts the process of loading assets as the use of the class will allow the paths to be input into the function petameters and ensure direct conversion into the assets with no further need inducing any other function calls.

Class Diagram

The class diagram for the Resource Manager shows the use of the Shader and Texture class as aggregated data types that will be used in asset management and the associated data structures used for these objects.



Data Structures used in Resource Manager

Within the Resource Manager class, I will use data structures that will be useful for allowing access to the instanced shader and texture objects.

Hash Tables

The Resource Manager class includes the use of the C++ map<> data structure which is a hash table. These hash tables alongside the other variables will be static variables (In C++ and other OOP languages, static refers variables that last the whole length of runtime). This

is because shaders and textures will need to be accessed constantly throughout the program as it runs. I have chosen the use of a hash table to store the shaders and textures due to hash tables having an access time of O(1) and elements being accessible with a key identifier. For example, using the LoadShader() function for loading a shader that uses perspective projection for 3D rendering, the user can store the shader's name parameter in the function as "3D renderer". This means there will not be a need to remember the numerical identifiers for the actual shaders themselves but instead a much more readable and intuitive description of them. As well as this, the use of hash tables is justified due to the need to access the shaders in real-time. As discussed in research, shaders are used in the graphics pipeline. If there are delays loading shaders, then the process of rendering may be delayed and cause dropped frames and unintentional visual delay.

LoadShaderFromFile() function

The LoadShaderFromFile() function will be responsible for retrieving the paths of the shader files and instancing shader objects within the function. As mentioned, the instanced shader objects themselves will be responsible for compiling and creating the OpenGL shader programs via their compile functions.

Pseudocode for LoadShaderFromFile()

The pseudocode for the LoadShaderFromFile() function first begins by loading the contents of shader file from the shader path as a string. Afterwards the function instances the shader objects. This is important as this allows the shader Objects' compile functions to be called. Finally, the function uses the shader objects to compile the string contents. This shows how the resource manager abstracts the process of loading shaders down to only requiring the paths the shader. Once the shader has been created, the object instanced is stored in the shader's hash table to be accessed.

```
public function loadShaderFromFile(vertexShaderPath,
fragmentShaderPath)




    // Open files


    vertexShaderFile = openRead(vertexShaderPath)

    fragmentShaderFile = openRead(fragmentShaderPath)
```

```
    // Read file's buffer contents into streams

    vertexShaderString = myFile.readLine()
    fragmentShaderString = myFile.readLine()

    // Close file handlers

    vertexShaderFile.close()
    fragmentShaderFile.close()



    // Instance the shader object(s)
   shader = new Shader

    // Compile the shader source code

    shader.Compile(vertexShaderString, fragmentShaderString)

    // Return the shader object

    return shader;
endfunction
```

LoadTextureFromFile() function

The LoadTextureFromFile() will follow the same procedure as in the LoadShaderFromFile() function except it will include the additional steps required to convert and image into texture data and pass that data to be generated.

Pseudocode for LoadTextureFromFile()

The function will include a Boolean parameter to determine if the image path's file format contains alpha values e.g. A .PNG file with a transparent background. If there are alpha values, then the texture's image format attributes are updated correctly.

```
public function loadTextureFromFile(file, alpha)
    // Create texture object
    Texture texture

    // Consider the issue of alpha values
    if alpha then
        texture.imageFormat = GL_RGBA
        texture.textureFormat = GL_RGBA
    endif

    // Load image data using library function
   data = stbi_load(file, width:ByRef, height:ByRef, nrChannels,
0)

    // Issue a console error in case of problems loading texture
data
    if data != true then
      print("Failed to load texture from", file)
    endif

    // Generate texture using texture object function

    texture.Generate(data)
    return texture
```

```
endprocedure
```

LoadShader() Function

The LoadShader() function will be responsible for appending shader objects in the hash table. This shows the aggregated link to the GUIRenderer class. This is because to allow different forms of rendering, there will be a need to pass different forms of shaders into the GUIRenderer instance. Thus, the need for loading the different shader files is justified.

Pseudocode for LoadShader()

The LoadShader() pseudocode consists of using the loadShaderFromFile() function to load the shader files from the path and then storing them within the hash table

```
public function LoadShader(vertexShaderPath, fragmentShaderPath, name)
    Shaders[name] = loadShaderFromFile(vertexShaderPath, fragmentShaderPath)
    return Shaders[name]
endfunction
```

LoadTexture() Function

The LoadTexture() functon will be responsible for returning the loaded textures that have been loaded from their image paths.

Pseudocode for LoadTexture()

The LoadTexure() pseudocode consists of using the loadTextureFromFile() function to load the generated textures from image the path and then storing them within the hash table.

```
public function LoadTexture(file, alpha, name)
    Textures[name] = loadTextureFromFile(file, alpha)
    return Textures[name]
endfunction
```

Real Time Processing

I will require real time processing for the main rendering process in the graphics pipeline. Programming the graphics pipeline is the process of turning the raw coordinate and texture data that is currently held in the GPU buffers to the on-screen pixels that are seen on the display monitor. The graphics pipeline must process data and render it onto the window's

framebuffer in real-time. Problems with this process happening in real-time will cause graphical artifacts such as tearing and freezing and affect the user's visual perception of gameplay.

The graphics pipeline is essential as it is fundamental for rendering anything on display. The process of the graphics pipeline must be implemented in my source code programmatically (by writing code). In my development, this process is as follows:

1) Vertex Specification/Generation – Specifying the vertex positions and data in local space coordinates on the GPU.
2) Vertex shading – The programmable part of the graphics pipeline in which we can program what to do with the vertex data. What we program will then be executed on each vertex data. Processes like converting to local to view space coordinates occur here.
3) Primitive assembly – Assembling the vertices to form triangles which in turn make up the basis of the shapes seen on screen
4) Rasterization – The process of determining which pixels to be represented based on factors such as depth testing
5) Fragment shading – Another programmable part of the graphics pipeline that determines the final fragment (color) of each pixel to be displayed on screen.

This process must happen all in real-time, typically before each frame is rendered for my adaptation to have its core functionality.

Test Data

Test Data	How I will test the data	Justification of the data	Expected result
loadShaderFromFile()	Passing in a correct path to both a vertex and fragment shader	This is valid test data to show the successful loading of a shader source file	The shader object will be instanced and will proceed to be compiled via the Shader class Compile() function
loadShaderFromFile()	Passing the same path for the vertex and fragment shader	This is boundary data to show that for a shader object to be correctly instanced, there must be both a separate fragment and vertex shader.	The shader objected will be instanced and compiled however there will possible runtime linking errors due to the ambiguation of uniform variables

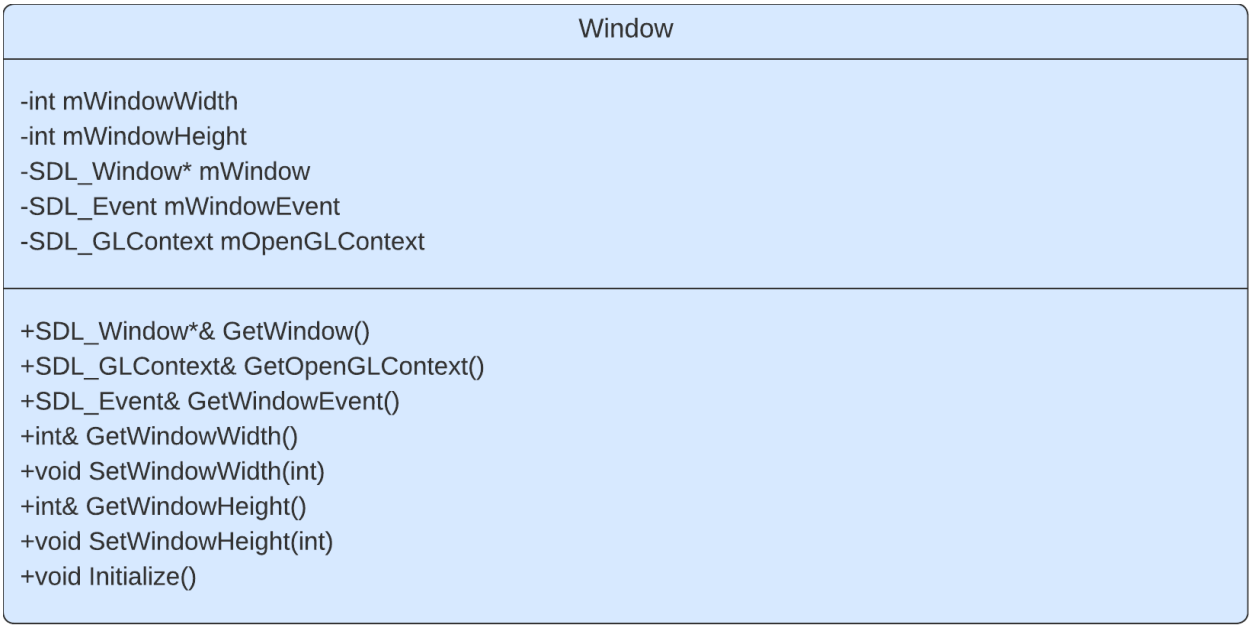
loadShaderFromFile()	Passing in a path to a shader that does not exist or in incorrect	This is invalid test data to show that for shaders to be correctly loaded then the path must be led to a correct shader.	There will be immediate console runtime errors
loadTextureFromFile()	Passing in a correct path an image file that doesn't contain alpha values e.g. JPEG	This is valid test data due to show the consideration of alpha values. This is justified to test allowance interoperability between different image formats	The image will be texturized however any aspects of transparency will be replaced with white pixels
loadTextureFromFile()	Passing in a path that does contain alpha values e.g. PNG	This is valid test data to show the acceptance of images with alpha values	The image will be texturized, and any aspects of transparency will be fully transparent.

Window Class

As discussed in the research, the window class will make use of the SDL2 external window library and its functions to set the properties of the game window and initialize a working game window. The class will hold properties for the window such as its dimensions, the references to the OpenGL context, the pointer to the window itself and a reference to the window events.

Class Diagram

The class diagram of the Window class shows that window class itself is the most independent class. This is because it does not require any other aggregations to set up a window. It only takes SDL2's library functions and organises them into the system required for my adaptation.



Getters and Setters

The GetWindow(), GetOpenGLContext(), GetWindowWidth(), SetWindowWidth(), GetWindowHeight() and SetWindowHeight() functions and procedures will all be responsible for returning and setting the references of the desired variables in the window class. The purpose of this is to maintain the concept of encapsulation and provide clean and authorized access to the member variables within the class.

Initialize() procedure

The initialize procedure will be called when initiating a game window. It is where the window creation happens and it will be responsible for setting OpenGL states vairables. It will also be responsible for checking if all libraries required for my adaptation are functioning.

Pseudocode For Initialize() procedure

The pseudocode for the Initialize) procedure will contain the necessary code to create a window and create an OpenGL context, using SDL2. Furthermore, the pseudocode shows the error checking procedures that will take place in case of any errors during initialization.

```
public procedure Initialize()
      // Initialize SDL

    if SDL_Init(SDL_INIT_VIDEO) < 0 then
```

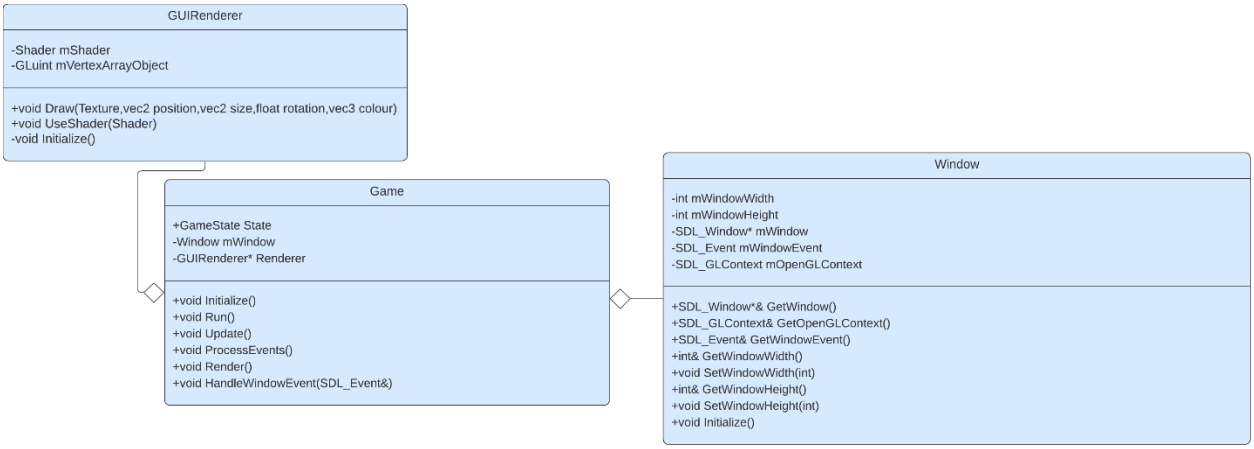
```
        SDL_LogCritical(SDL_GetError())
    endif


      // Create Window


    mWindow = SDL_CreateWindow("Game", SDL_WINDOWPOS_CENTERED,
SDL_WINDOWPOS_CENTERED, mWindowWidth, mWindowHeight,
SDL_WINDOW_OPENGL)
    if mWindow == nullptr then
        SDL_LogCritical(SDL_GetError());
        SDL_Quit()
    endif



     // Create OpenGL context
     // Check for errors


    mOpenGLContext = SDL_GL_CreateContext(mWindow)
    if mOpenGLContext == nullptr then
        SDL_LogCritical(SDL_GetError())
    endif
endprocedure
```

Game Class

The game class will act as an uber class that aggregates the GUIRenderer and Window class. This class will intend to organize the game code whilst also decoupling all the window code from the game code. The game class will also hold functions for processing input and handling window events. It will also contain the “main loop” that makes use of the GUI Renderer and is where all rendering activities happen. Furthermore, the class will also store an enumeration of the current game state and update all necessary functions.

The game class will also heavily utilize the resource manager class for loading the assets required to be used for the game

Class Diagram



Initialize() procedure

The Initialize() procedure for the game class will be responsible for setting up projection matrices, initializing shaders, initializing textures and setting uniform variables. Once these variables are initialized, they will not need to be initialized again (excluding textures). This step of initialization links back to the thinking ahead section of my research. This is because once I implement, I do not need to write a separate system to reproduce more features in my adaptation. This modularity saves time, improves code readability and will reduce the chance of logical and syntax errors.

Initialize() pseudocode

The first key feature of the Initialize() procedure is the use of the orthographic projection matrix. Throughout my research, I discussed the need to use perspective projection for the 3D aspects of my adaptation. As well as that, I will have a majority of 2D aspects such as the menu GUIs. Due to orthographic projection keeping the distance properties of every model from the camera constant, there is no perspective. Thus, everything in orthographic projection remains 2D which is especially useful for rendering models that are strictly meant to be kept in 2D.

The next key feature of the Initialize() is the utilization of the Resource Manager functions to greatly simplify and abstract the process of making new shaders from shader files and loading textures form image files. This is shown as the process of how loading assets works is hidden away. This creates a style of “black box” coding which allows me focus on what the function does and the intention of the use of the function.

Once the shaders have been loaded, the GUI Renderer is instanced, and the game engine is ready to be used correctly.

```
public procedure Initialize()


    // Set up orthographic projection matrix


   orthographicProjection = ortho(0.0f,
mWindow.GetWindowWidth(),mWindow.GetWindowHeight(), 0.0f, -1.0f,
1.0f)


    // load shaders


    ResourceManager::LoadShader("shaders/[YOUR_VERTEX_SHADER_PATH_
HERE" "shaders/[YOUR_FRAGMENT_SHADER_PATH_HERE],"[SHADER NAME]");


    // Set uniform variables within shaders


    ResourceManager::GetShader(“[SHADER
NAME]).Use().SetMatrix4("projection", orthographicProjection);
    ResourceManager::GetShader(“[SHADER
NAME]).Use().SetInteger("image", 0)


    // Instance GUIRenderer


    Renderer = new GUIRenderer(ResourceManager::GetShader("[SHADER
NAME"));
    // load textures
    ResourceManager::LoadTexture(“assets/[PATH_TO_YOUR_IMAGE.PNG]”
, true, “[NAME TO IDENTIFY YOUR TEXTURE]”)
```

```
endprocedure
```

Start Menu

For my adaptation's start menu's design, my approach is a dynamic between the old and retrospective arcade dance design theme that is across older VSRGs whilst keeping the modern functionality and useability. The general aesthetic is meant to conform to the iconic designs of older arcade dance games by using features such as arrows, musical terms and silhouettes of people in dance positions,

Game Cover (Logo)

To begin with, one of the most fundamental aspects of a VSRG is its game cover/logo design. It is often the first impression that users behold and often what they judge the expected outcome of the gameplay to consist of; It forms the initial expectation of what the game is about. For this very reason my game design must include the features of that will suggest that it is aimed at both the older and younger generation alike.

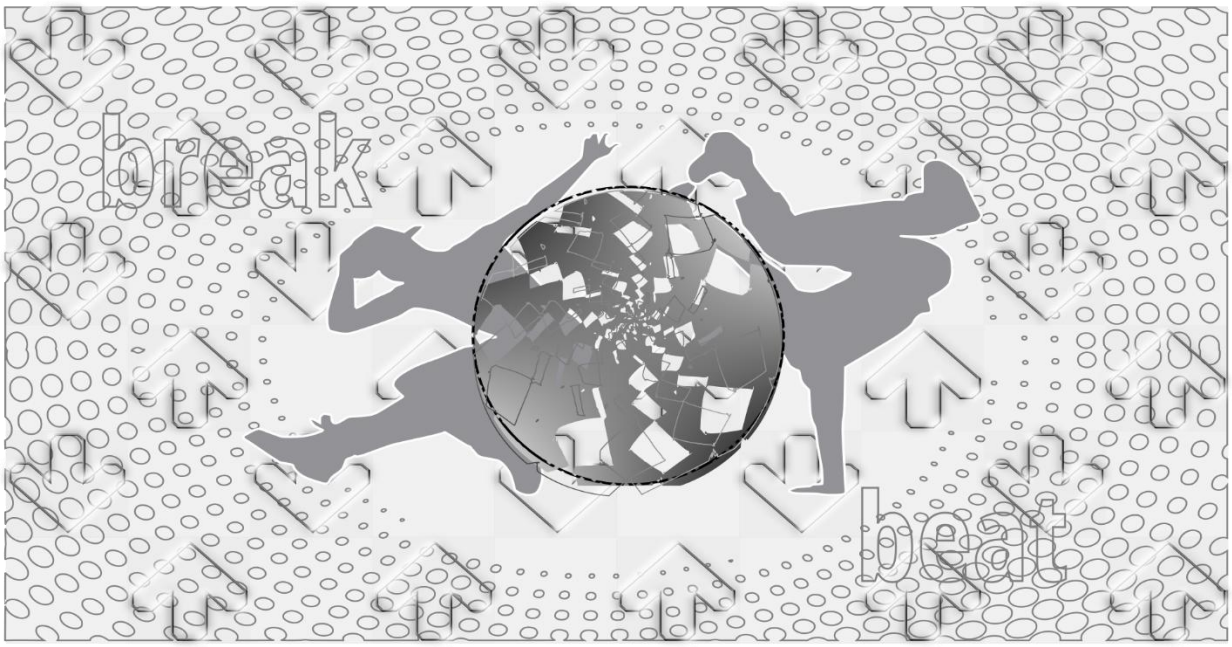
Many older VSRGs designs have a focus on bold, smooth and perspective text. This is prevalent in examples mentioned in research such as ITG and DDR. To stay inclined to the theme of older VSRGs and the general arcade aesthetic found in them, my adaptation's logo design will consist of smooth text with a horizontally tilted perspective. This design choice is intended to mimic the 3D perspective shifts commonly found in older VSRGs designs like DDR.

During the era of many old arcade games and VSRGs the graphical processing power was not advanced as the modern era. This meant the polygon count used to render 3D models and designs was much lower. Therefore, my design consists of a lower sample of curve segments in text rendering. This is to give it the same nostalgic essence found in many older VSRGs and arcade games alike (An example being SuperMario64) that will appeal to the older generation.

Smooth, bold text with perspective is a feature of VSRGs design logos that the older generation tend towards. Examples in which this is found are DDR and Beatmania. These games have their logo design text stylized with a smooth but heavy outline. This is also prevalent in many modern VSRGs such as Friday Night Funkin', too therefore adding outline to my design will appeal both to audiences. Because of this, I implemented my design with an initial white outline of the text then an additional black outline and additional perspective shift. This is to maintain the maintain the general 3D text aspects of older VSRGs.

Background

One theme that is prevalent in many rhythm games and VSRGs alike is distinctive features that make it clear to the user what the game is about. Earlier in research, the evidence I gained suggested that features like arrows, musical terms, people dancing etc. are what make it clear to a user that game is in the genre of a rhythm game. Therefore, for my design, I implemented a background design with the use of an array of alternating hollow arrows to enhance the arcade dance game style theme. I also implemented silhouettes of people dancing, a clear dotted overlay and the use of musical terms to give the 90s music game vibrant feel that is commonly found in old VSRGs. This will benefit the older stakeholder age range and further allude to the design of older VSRGs.



Furthermore, as I commence development of my adaptation, this design will be implemented as an animated moving background to add to the classic dance game aesthetic and generally give the design a fuller and vibrant appeal. My justification for this is the aim provokes the feelings of reminiscence and nostalgia within the older generation as discussed in research earlier.

User Navigation Assist

As mentioned, my design will follow the typical design attributes to older VSRGs however to benefit all my stakeholder audience age ranges, implementing modern functionality and useability is a must. Part of the functionality and useability that will benefit the audience of my younger stakeholder is clear and instructed guidance on how to navigate the user interface. Modern VSRGs such Friday Night Funkin' tell the user how to This is through keystroke indication or user input guidance, typically either by directly addressing the keys to be pressed or by showing an image or sort. To add to the modern functionality and useability of my design, I implemented a user navigation assist bar that will be situated at the bottom of the start menu. The bar will give a clear aid on which keyboard inputs are required to navigate the user interface through buttons to represent keyboard icons. My justification for this design is to generally enhance the intuitiveness of my adaptation and add to the modern functionality and useability.

Start and Exit Buttons

As well as features relating to dance games i.e. arrows, musical terms and the use of bold perspective text, the use of stylish and centered text that a user must interact with via user input to allow progression into main gameplay. This general theme of text signifies to the user that they must press a key, e.g. enter, to start the game is a common occurrence in many old arcade games and VSRGs alike. Furthermore, many older arcade games tend to leave the text without any form of encasement around the text i.e. the text has no aspects of being a “button” but purely just the text alone. Therefore, for my start menu’s user interface, I implemented this style of start and exit buttons to mimic the retrospective menu design of older VSRGs and keep on the pattern of an older design that is appealing to the older audience.

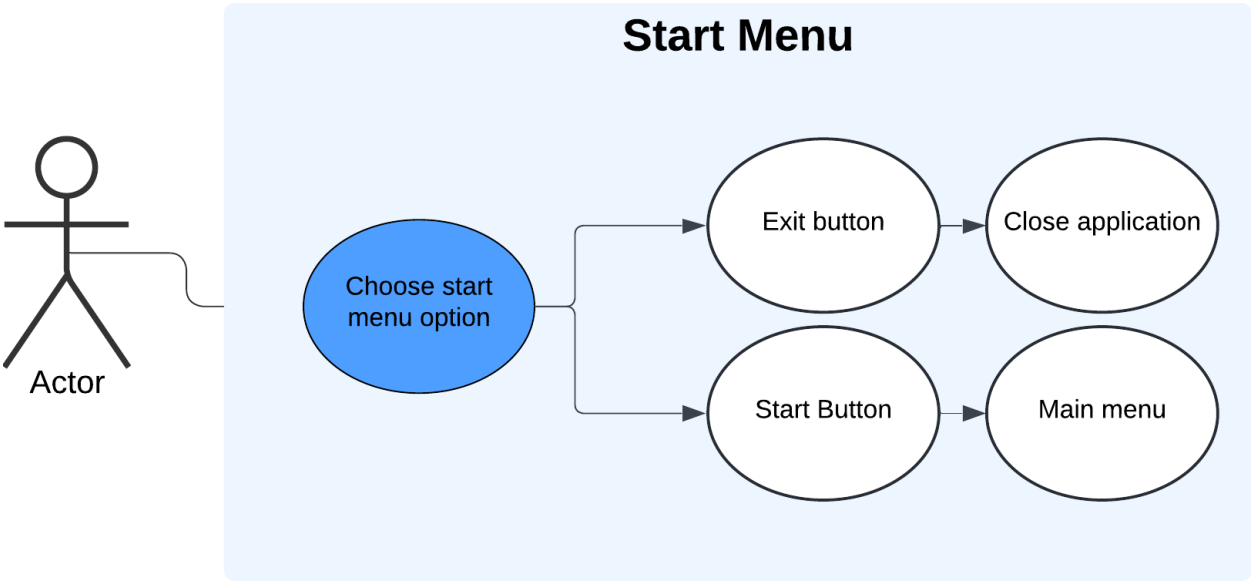
UML Use Case Diagram

The UML use case diagram below visually describes how the user will navigate into the different sections of my adaptation when first loading the game. As mentioned in the design, the start menu will consist of the buttons that will indicate to the user the expected sections of gameplay. The “start” button of the menu will lead into the adaptation’s main menu and the “exit button” will give ability to terminate the process by closing the application

Adding an “exit button” was a feature that was discussed during research and a criterion of the success criteria. Throughout my application, the user interface will emphasize the use of clearly indicated “exit” and “back” buttons to direct termination of the program and return to previous game states. My justification for this can be seen in the example of the user suddenly closing the application via the “x” button (that is built into the window management API) during chart editing process which contains a file currently in use. If the file in use was not last saved properly, the file could be at risk of corruption which may lead to potential memory and storage on the user’s computer. However, if the user were to exit via “exit” button that is built into the programs source code and clearly intended to close

the application, it will allow proper means of cleaning up the resources used for running the application. This can be through calling functions to free memory, calling destructors and automatically saving files e.g. Settings,

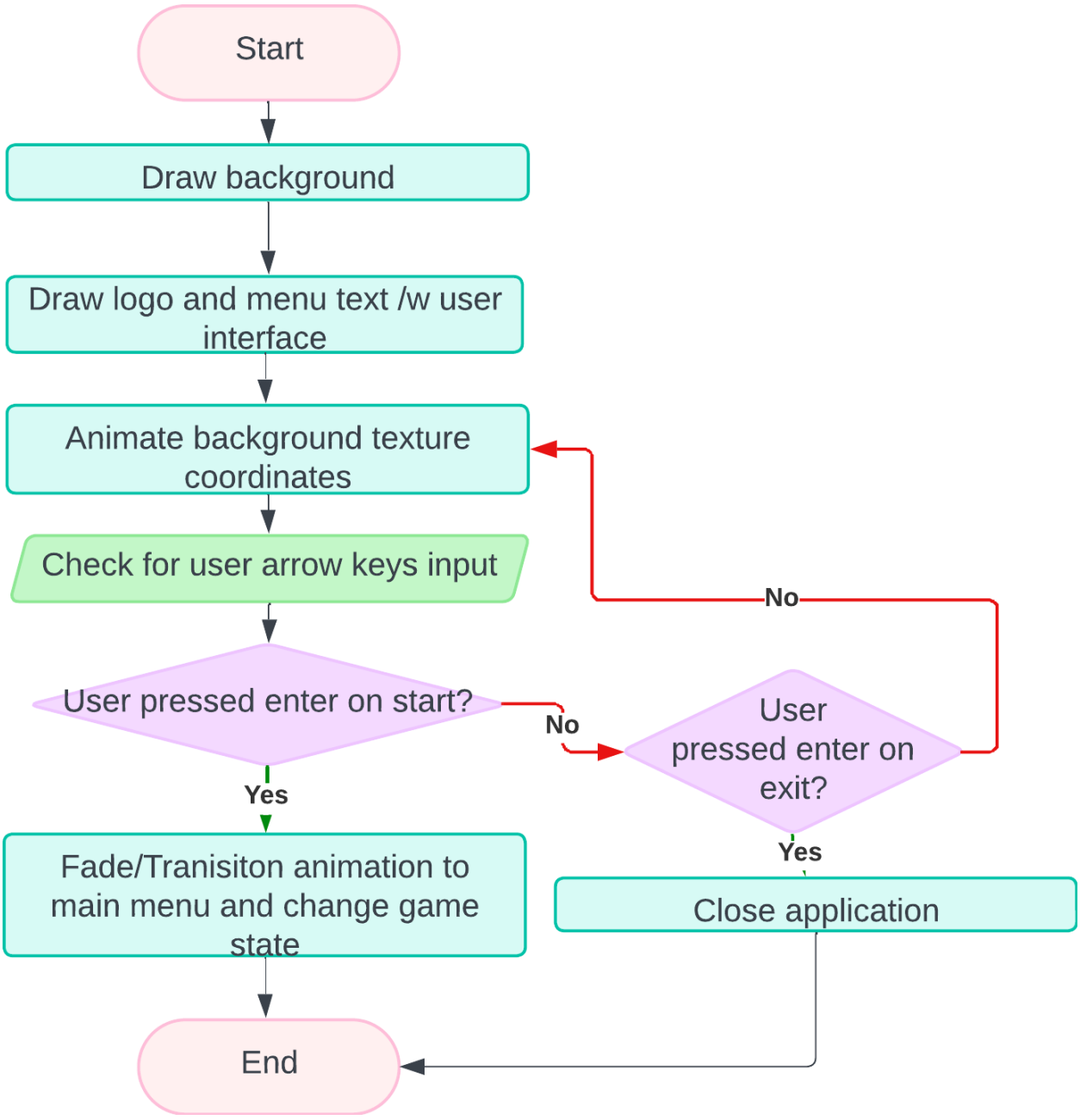
Another justification of adding an “exit button” is to further allude to modern functionality and useability that will appeal to the modern generation and increase the overall intuitiveness of my design. Increasing the intuitiveness of my design further justifies the idea of the user not requiring any foreknowledge of background information to interact with the game. This was stated in the user requirements section of my research and will be a theme that is kept throughout my adaptation.



Flowchart

The flowchart for the start menu begins with drawing the background. This is because for the positioning and layering of the user interface to be correctly organized, the bottom most object must be rendered first. Afterwards, the user interface for the main menu can be drawn on top of the background. Once this happens, the game will begin animating the background by updating the pixel coordinates of the background whilst waiting/processing user input. This process typically happens through using a .GLSL (shader) file and is implemented on the GPU (hardware accelerated). Simultaneously, the game will check for user input via the arrows keys and commence to change the game state based on the menu option selected. When this happens, the transition animation will occur. The animation consists of the entire game window quickly “fading” darker and then brightening on the new interface. In this case the interface will be the main menu screen. My

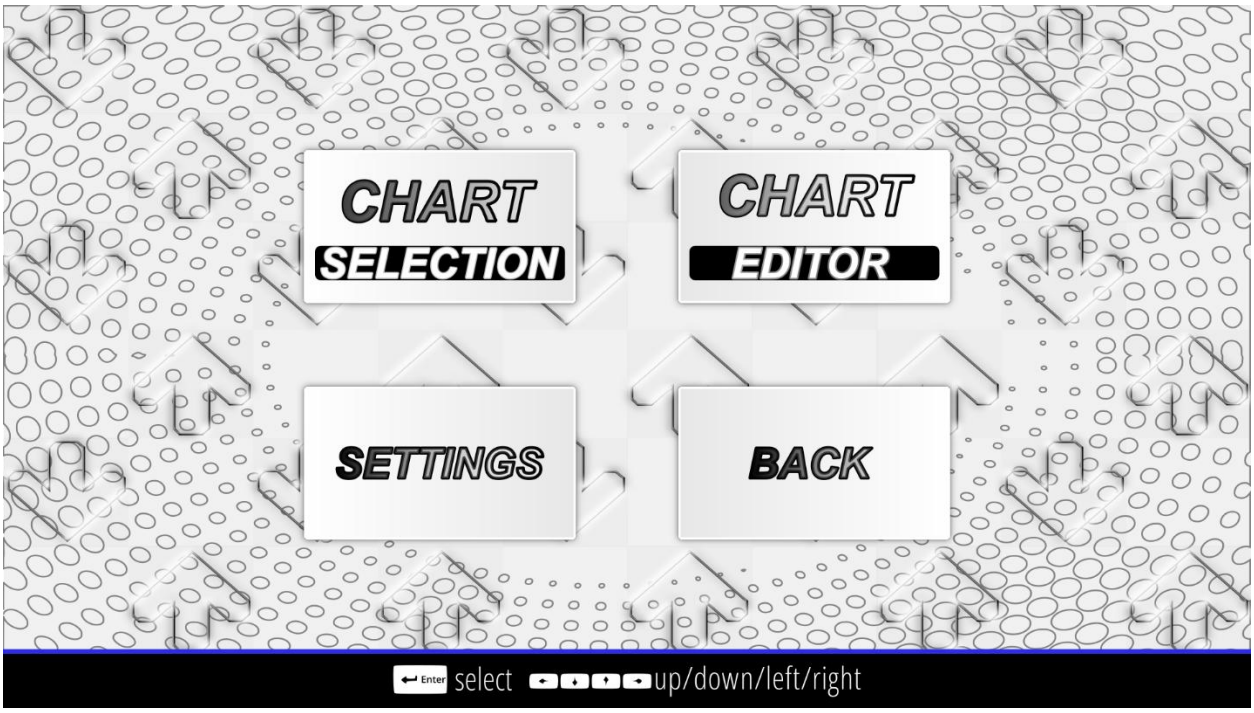
justification for this animation is to keep the design of my adaptation accessible by making it visually clear to the user that the game is transitioning into a different part of the game.



Main Menu

The start menu of my adaptation focused on implementing the desired design aspects. The main menu of my adaptation will focus on navigation and be the core system in which all

aspects of the adaptation are accessed. The menu system will allow progression into features such as gameplay, map creation and customization of gameplay settings.



User Interface

The user interface of the menu system is coordinated using left/right/up/down keys. My justification for this is to keep with pattern of the retrospective style of older arcade dance games that commonly used button input as means of navigation. Furthermore, my intention is to add micro-interaction animations such as 3D rotations around the y axis and scale hover animations to the interfaces whenever a user is selected on it. The intention behind this is to mimic the arcade menu feel that is appealing to the older generation.

As well as the user interface being accessible, as part of the older design features, I implemented a slight drop shadow around the edges of the interfaces and a linear gradient as the background color. This is justified as older VSRGs like DDR do not tend to use a singular still color but often use gradients alongside their perspective backdrops. Furthermore, the aspect of older design is added through a bold, smooth and outlined text.


CHART
SELECTION


CHART
EDITOR

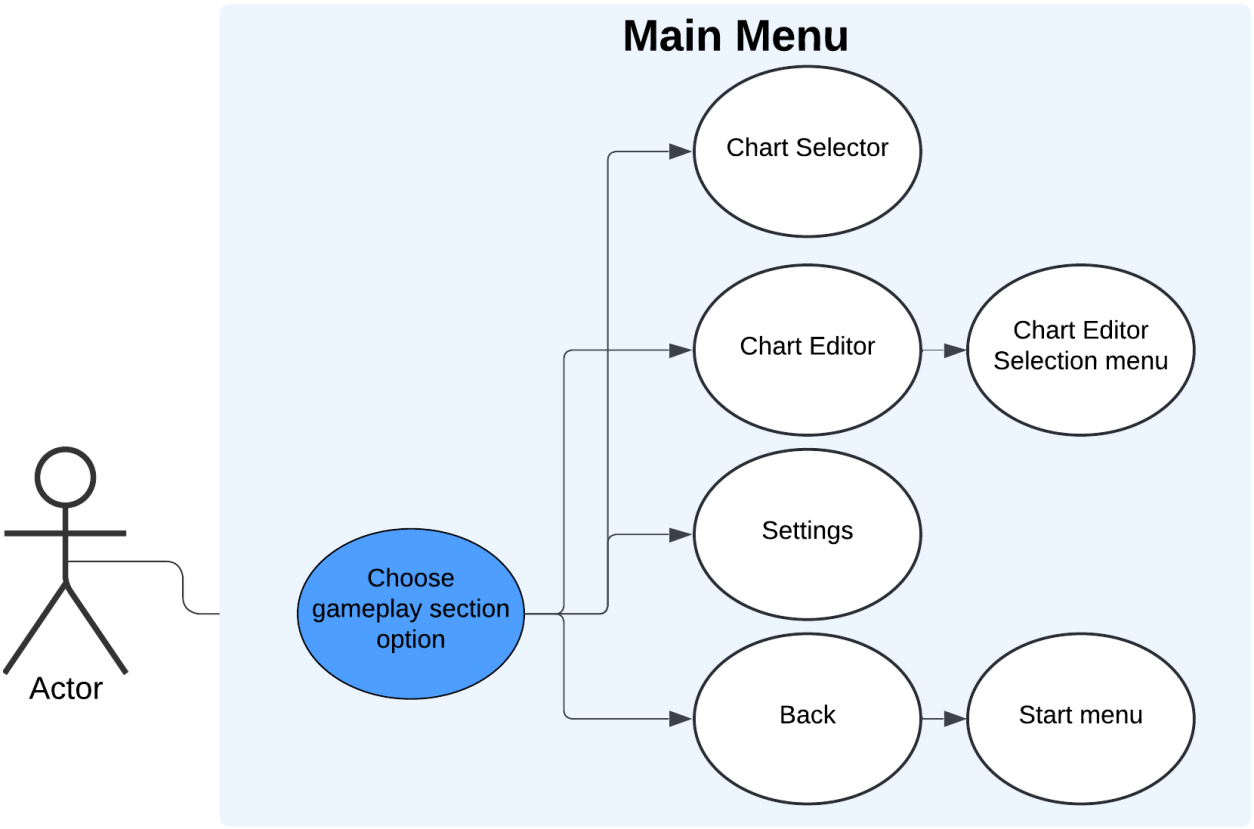

SETTINGS

Another feature that adds to functionality is the method of going back to the start menu either via a “back” button via mouse input or key input navigation. This adds to the ease of access and further enhances the intuitiveness of my adaptation.

Another aspect of useability and functionality is the user navigation assist that is tailored to the navigation of the main menu. This user navigation bar continues to indicate to the user how to maneuver around the menu, via icons indicating the keyboard keys to be pressed. This further keeps the aspects of useability and functionality that are tailored to the modern audience.

UML Use Case Diagram

The use case diagram for the main menu shows the accessibility the user has when changing between game states. This is particularly shown through the user having the ability to return to the start menu via the “back button.” As mentioned earlier in the start menu section, such “back” buttons will not only give the user more control and allow faster access to different game sections, but also allow the program to implement appropriate procedures and functions during the transition of the game state. An example of a procedure that can be implemented when a “back button” is pressed is a confirmation screen that asks the user if they would like to “go back” to the previous screen. This will give the user thinking time and generally add to the modern useability of the interface.



Flowchart

As well as the basic features of a menu user interface like input processing to allow UI navigation, the flowchart for the start menu takes into consideration the steps taken in visually animating the interface. The justification of adding these steps into the flowchart is to ensure the animations are synchronized to the user's keypress input and allow reverting to the original UI state when deselected. Furthermore, the flowchart includes the specific type of transformations such as scale and rotation, that will occur to the user interface when being animated. The justification for this inclusion is due to the animation process being the product of the mathematical process of matrix multiplication to the object's coordinates in the shader, with time as a variable. Within this process, the correct function for matrix multiplication must be used to ensure the objects coordinate are translated correctly.

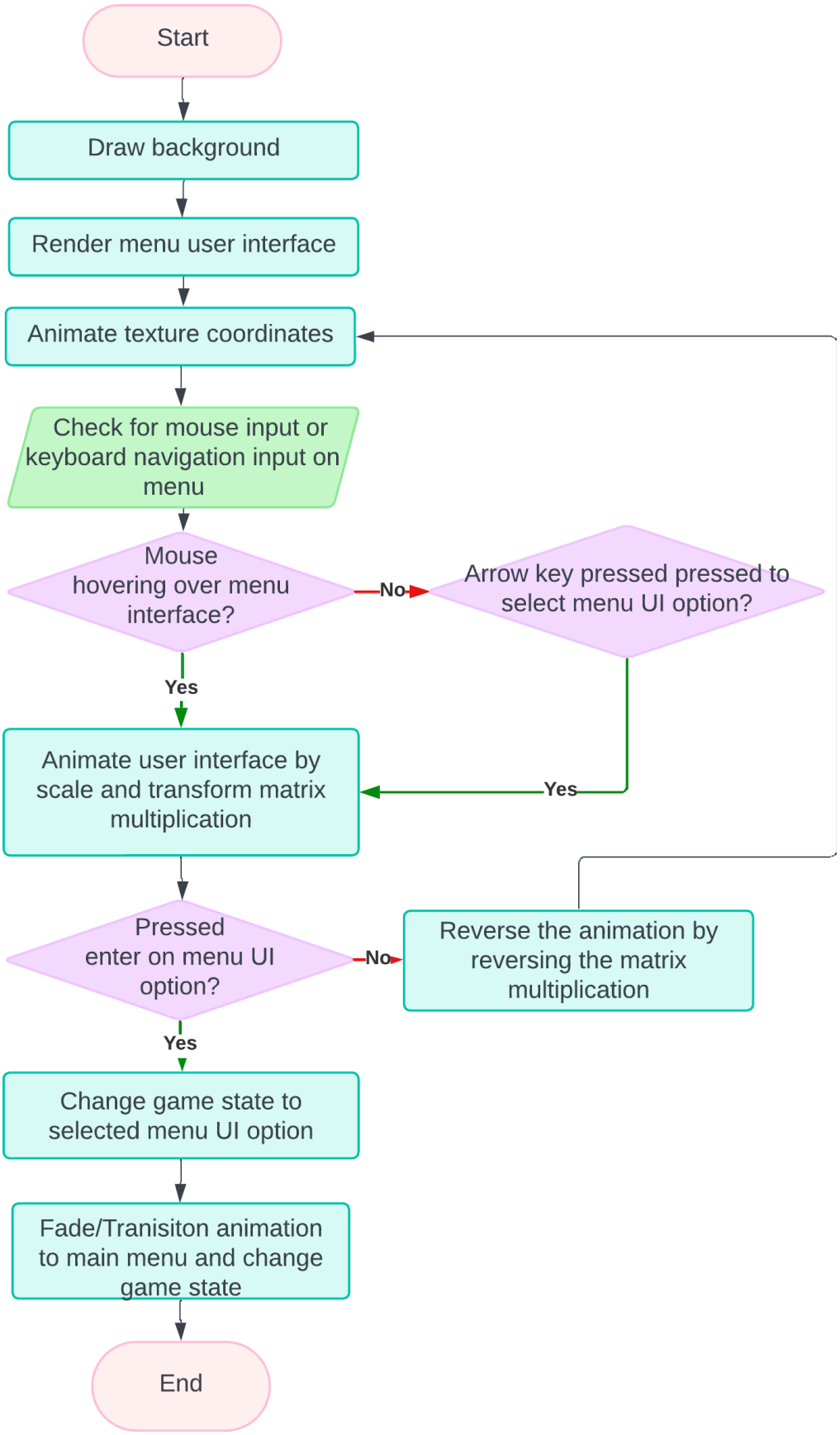

Start
Draw background
Render menu user interface
Animate texture coordinates
Check for mouse input or keyboard navigation input on menu
Mouse hovering over menu interface?
No
Arrow key pressed pressed to select menu UI option?
Yes
Animate user interface by scale and transform matrix multiplication
Yes
Pressed enter on menu UI option?
No
Reverse the animation by reversing the matrix multiplication
Yes
Change game state to selected menu UI option
Fade/Tranisiton animation to main menu and change game state
End

Chart (Map) Selection

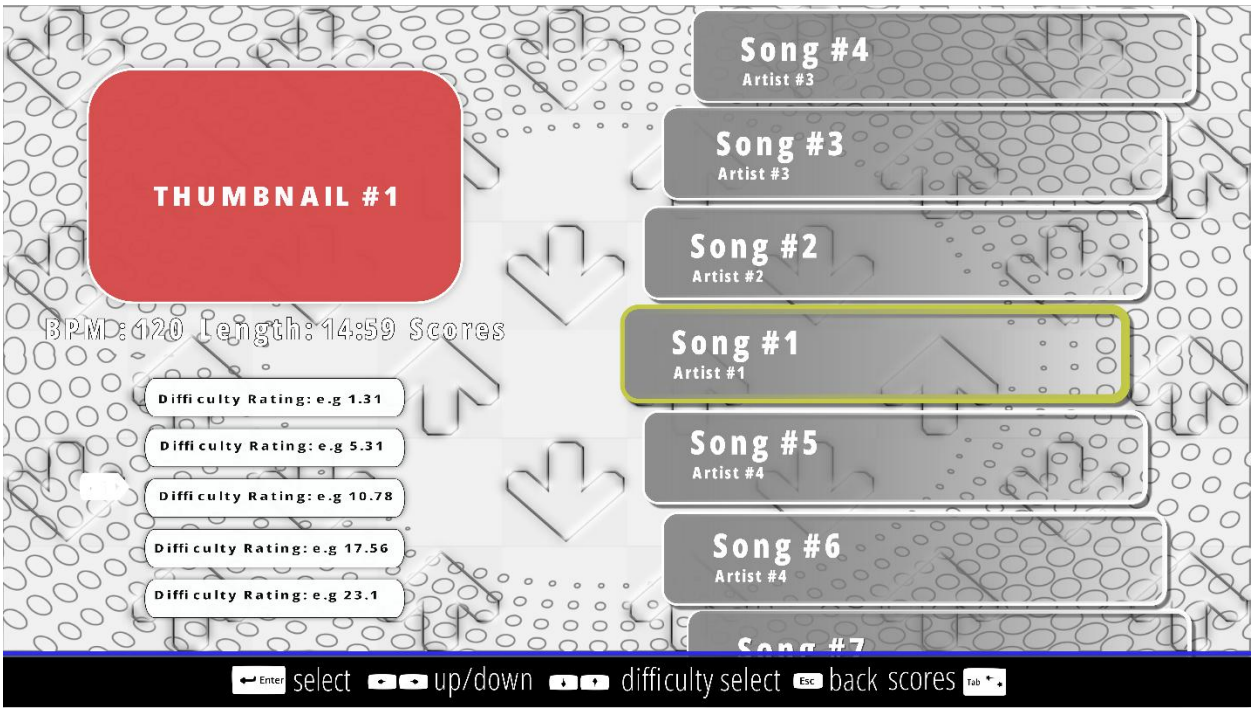
The chart selection menu of my adaptation will allow the user to select a song and commence into the main gameplay. This menu is accessed after the user selects the "chart selection" user interface. The chart selection menu will consist of a list of all the users' maps they are able to access on their device. The design remains like most VSRGs and StepMania itself. This design simplifies and heavily abstracts the actual process of loading a chart file into the game. The user does not need to see the internal process happening within the game system. The user only needs to be able to navigate the user interface and access the maps based on the information the map gives. Therefore, simplifying the process to just a few button presses. This design allows the user to focus on aspects of gameplay such as an evaluation of their skill level, whether they can complete the map, which difficulty they should select for the map and if they have the time for the length of duration of the map. All of this will benefit the user's functionality and allow them to have immersive experience.

As well as the process of loading maps being abstracted, the chart selection menu design also provides a means of ordering the charts in an organized manner. In reality, the files for maps are stored in different locations in secondary storage, but the chart selection design visualizes the charts as one continuous list. This will improve the accessibility of charts and speed up the time taken to get into the main gameplay as the user will not need to search for them in memory. This design makes the game smoother and allows fast repetition of entering gameplay, finishing gameplay and then selecting another chart to enter gameplay. The seamless transition between gameplay and chart selection will benefit the stakeholders of my younger generation as they are more tailored to spending long hours on the game and as mentioned in research. This means they can spend more of their total time practicing playing the game and take less time during navigation.

The chart selection design will also show the most crucial feature to my adaptation, The new and improved difficulty calculations. As mentioned in research, the calculation must be of a quantitative measure and give an accurate measure of the difficulty chart. The actual difficulty of the map is calculated in real-time during and after the map creation process and is displayed in both the editor and selection menu. This feature is a major part of my success criteria and therefore must be a fundamental aspect of my design. As well that, the ability to change between different map difficulties and view scores must be a feature of my design as the difficulty of maps and the scores achieved play a large part in gameplay as they are the main indicators of progression in skill level.

The final feature of my chart selection design is the displaying of a thumbnail/background for the chart. This feature will help in differentiation of charts for when charts may have the

same name. This feature along with an audio preview of the song whilst it is playing will further improve the user experience and meet the standards of my success criteria.

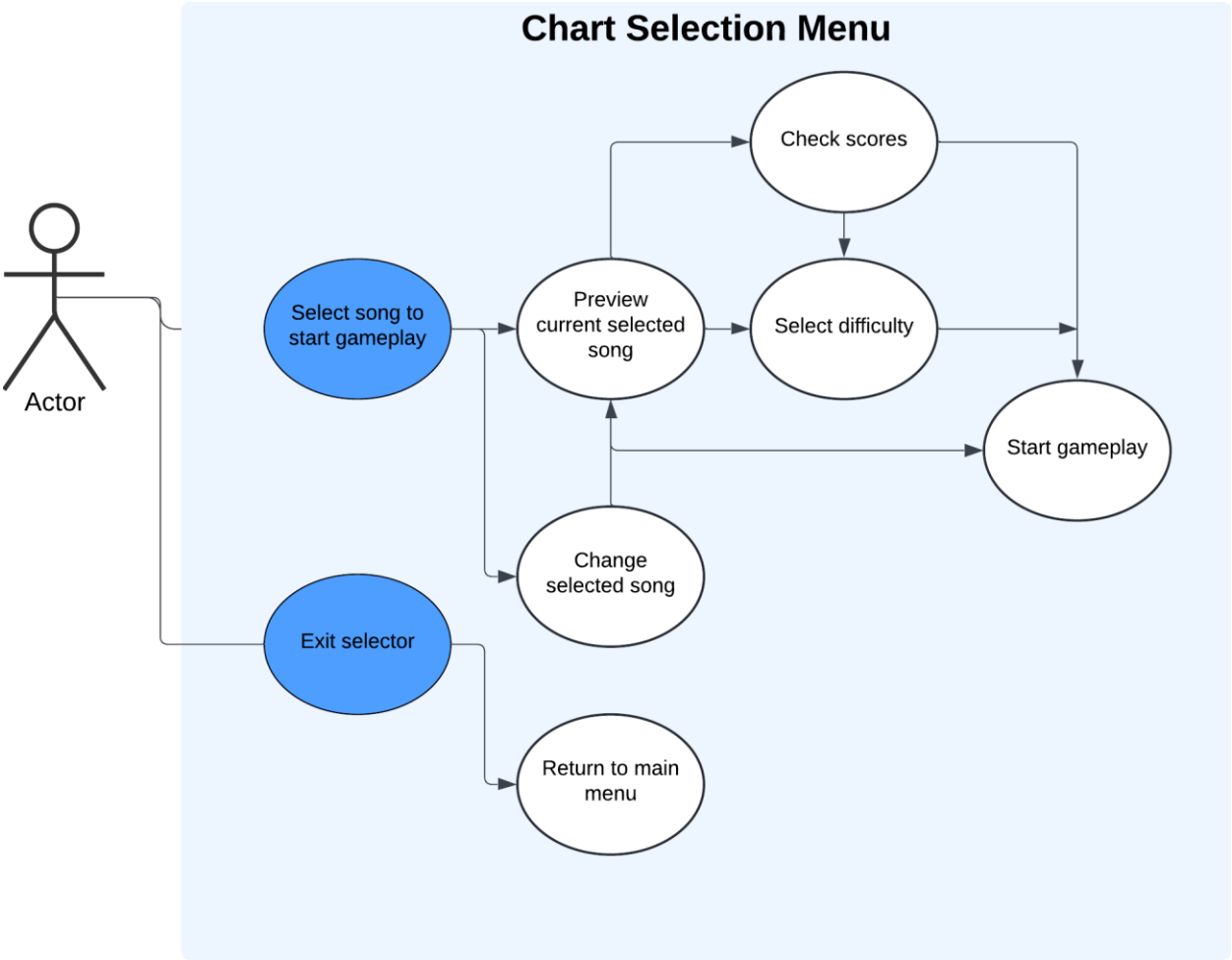


The user navigation assist bar is still present within the design to keep the aspect of usability and functionality. In this section of the game, the bar contains more input icons to tailor to the increased controls to navigate through the chart selection.

UML Use case diagram

The use case diagram for the chart selection menu displays the different and unique mechanisms, such as previewing the most recent play scores for a chart, that can be accessed within the chart selection. The diagram considers the different orders that the user may take when navigating and interacting with the mechanisms . This is shown in the diagram as I have included multiple paths from the “Preview current selected song” that stem in different directions and lead to different use cases. From the user’s perspective this means that when the user is previewing the charts, the user will have the choice of choosing the mechanics in a non-linear fashion; In this case, the user can select the difficulty of the chart via arrows keys then check the scores via the tab key or the user can first check the scores via tab then select the difficulty of the chart. This shows that the user is not restricted to using the interface in a certain order for the game to allow functionality. My justification for this feature is that it gives the user more freedom and ease of access in navigation. For example, if the user was only allowed to check their scores only when they

have selected the difficulty in that specific order then the user would need to learn that specific order that a linear fashion. This would be cumbersome for the user and potentially invoke frustration as the user may only want to check the scores of the chart and not select the difficulty but is forced to select the difficulty first regardless. This will increase the overall fluidity of the navigation system and increase the desired modern functionality of my adaptation.

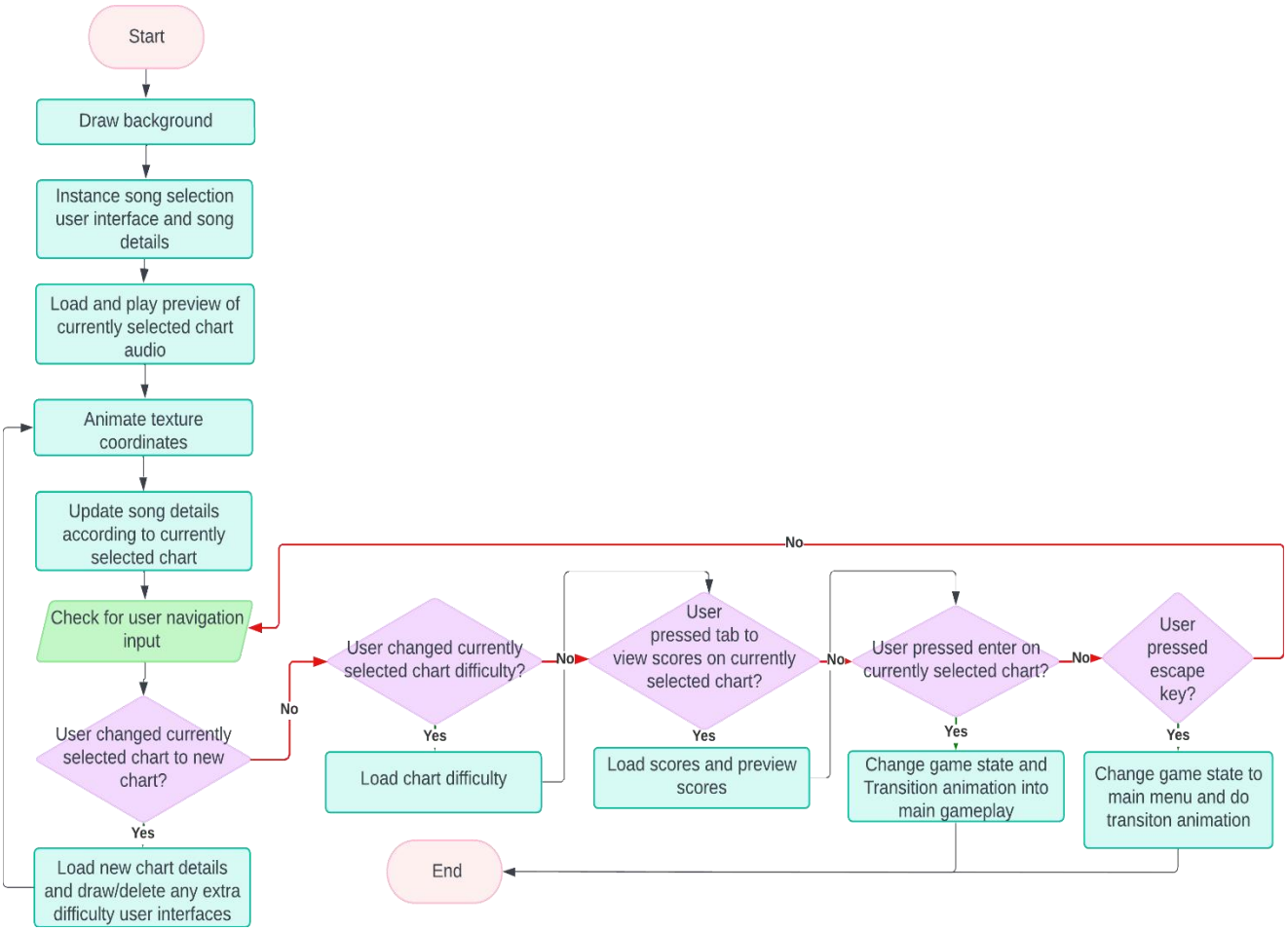


Flowchart

The flowchart for the chart selection menu contains more features than other sections of the game. Most noticeably, the algorithm to implement the allowance of non-linear access to chart selector mechanisms as mentioned in the previous paragraph. This algorithm will consist of a chain of non-nested if-statements. The non-nested nature of these if-statements means that there are no ‘else’ statements at the end of each statement. This means that the game does not need first check for the first condition to move onto the next

condition; In this case, the system does not need to check if the statement of “the user has currently selected a new chart” is false before moving onto the checking if the user has changed currently selected chart. Instead, the system checks if both statements are true or false contemporaneously, thus allowing the execution of menu mechanism based on input in any order (Which ever statement’s Boolean value is true first).

Another notable feature is the use of instancing when drawing song selection user interface. The use of instancing means that each of the GUI elements that encase the song information e.g. song name and song name, can be drawn in a single draw call. This is opposed to having multiple draw calls for each GUI element containing the song information. My justification for this is due to the GUI elements being the same texture thus not requiring a different texture to be bound to the when rendering. Only using one draw call will mean that the CPU/GPU resources will be used more efficiently and lead to an increase in performance factors like frame rate. An increase in performance will lead to smoother gameplay and increase the overall user experience. Factors like frame rate are especially important in VSRGs, as mentioned in the visualization section of research, it will give a more accurate representation of the timing interval for the user to register input in thus leading to a more engaging gameplay experience.

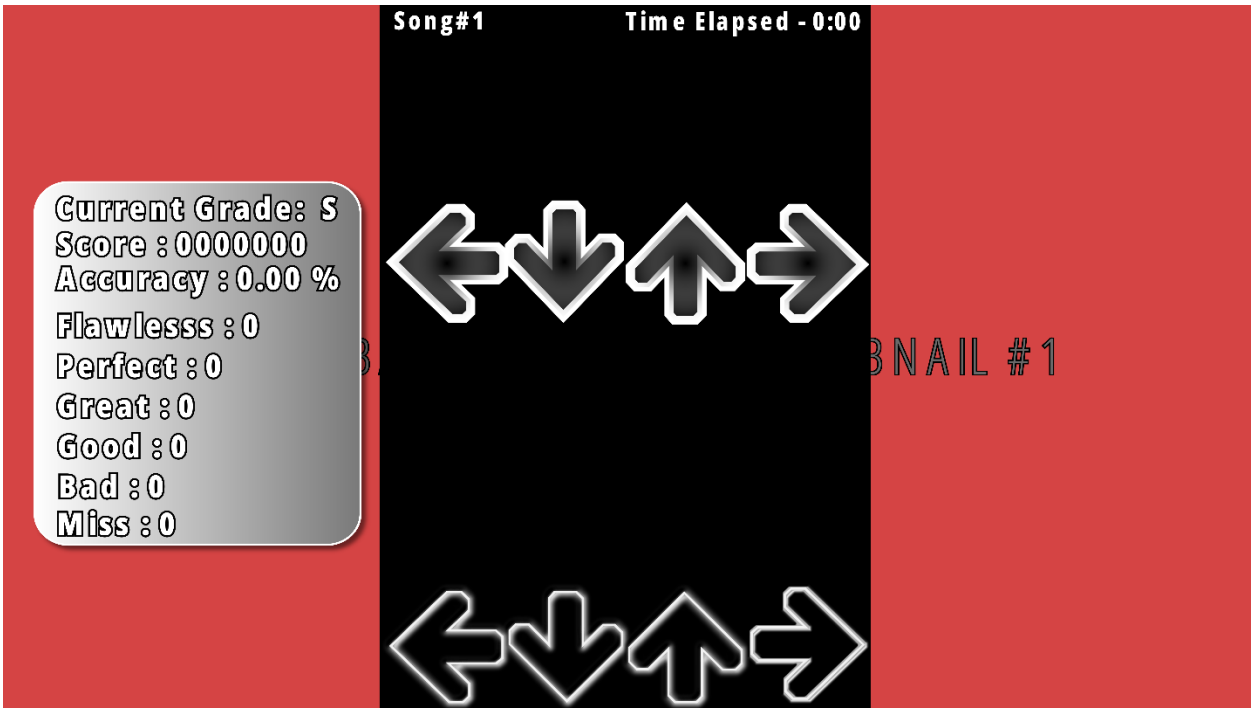


Main Gameplay

The main gameplay will consist of the standard VSRGs gameplay. The user will be able to accordingly respond to notes that are scrolling vertically and yield input based on the visual information. As well as the visual information of how close the notes are to the receptors, the user will also have audio information. There must be audio playing in the background of the main gameplay which will aid in timing the notes and set the basis of the 'rhythm' aspect of the adaptation.

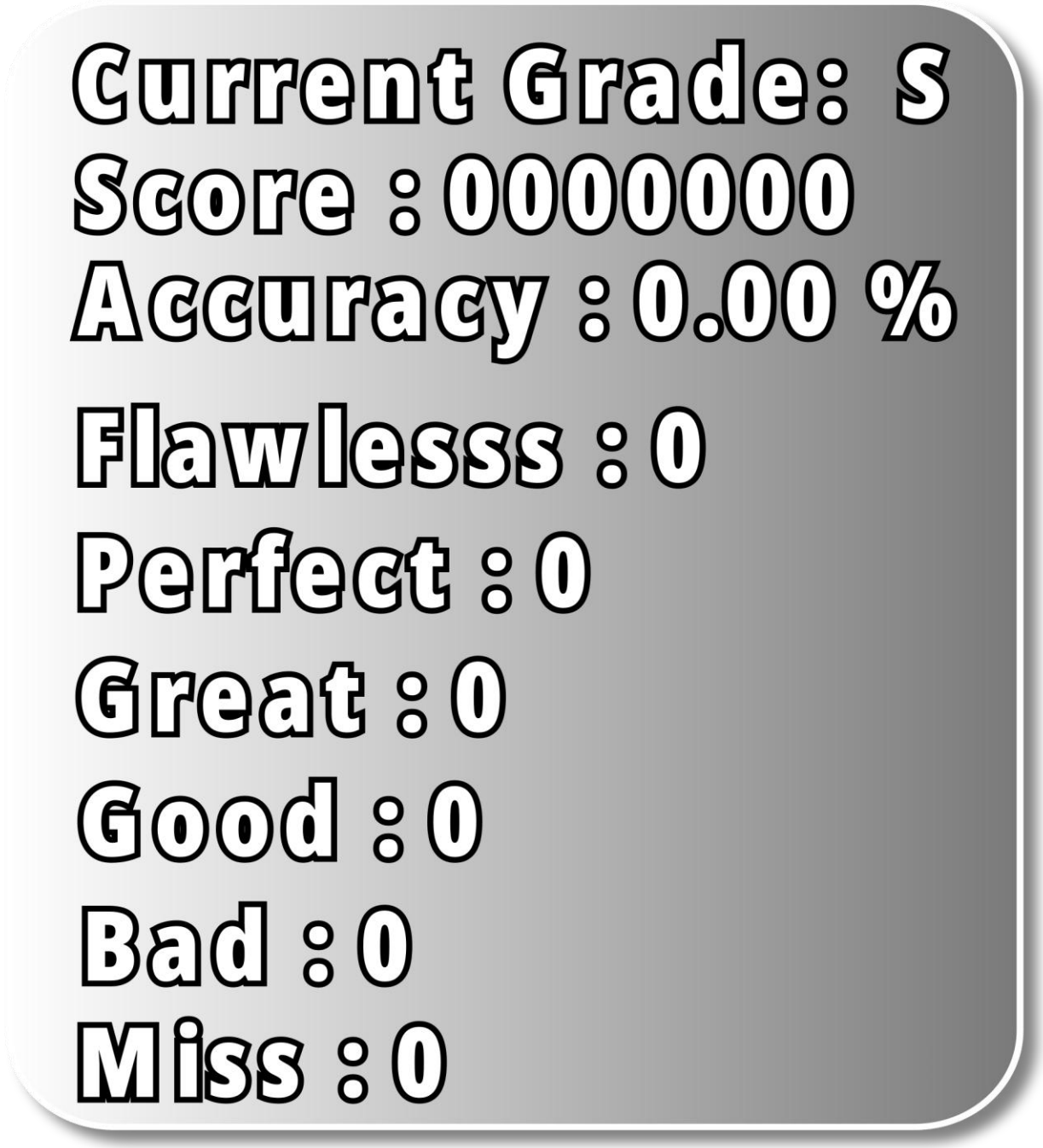
In the background of my adaptation's gameplay design, there shall also be a timing class that will process the timings in real time and give an accurate response on timing judgement. The timing class will form the basis for setting up the user's scoring, thus in turn forming the basis of the grade and accuracy system.

As well as the gameplay and timing, in the actual background of the columns in which the notes scroll, there will be a background image that the user can set in map creation. This



Gameplay Statistics

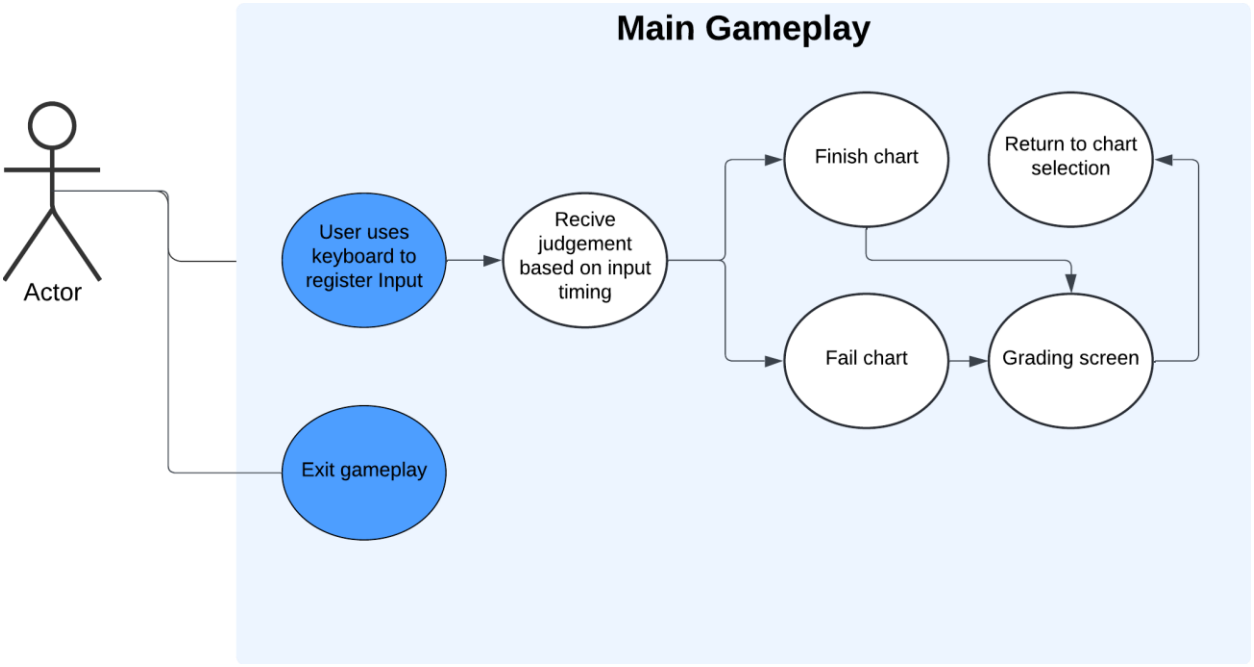
The gameplay statistics will be what is used to indicate the user's performance and the general skill level of the user on the map. The statistics will show the quantity of hit judgements they have received and the average hit accuracy throughout the map. A higher hit accuracy will contribute to a higher accuracy which will increase the score. The displaying of real time gameplay statistics will be beneficial to the user during gameplay. This can be during areas of which there are not many notes in which the user can take a glance. It will also benefit them as it will allow them to monitor their performance progress between their current gameplay and past gameplay score if they are replaying the map. The ability to monitor and track progress statistics in real time adds to the functionality and useability of the adaptation that is appealing to the modern audience of my stakeholders.



UML Use Case Diagram

The use case diagram used for my gameplay clearly considers the different pathways that can form during the user in main gameplay. My justification for this is because it is not known, upon the entering of the main gameplay, what the result of entering the main gameplay will be. For example, the user could enter the main gameplay but fail the chart

being played; Initially the user will have the choice to begin registering input upon the start of audio. From then, if the user decides to yield input, the user will continually receive judgment based on the timing of their input. However, if the user's input is greatly off time during this process, the timing judgement will be registered as a "miss". As this happens, the game health bar will decrease. If the user's health bar diminishes, then main gameplay will terminate and the grade for the user will be a "failure". However, another path the user could take is initially entering main gameplay but then exiting before the end of the chart due reasons such not being satisfied with their current performance. The user could also take the pathway of entering main gameplay and neither exiting nor failing but completing the chart all the way through. In this case the user will be awarded a different grade. Another pathway the user could take is entering main gameplay but immediately exiting the main gameplay. Some reasons for this could be mistakenly selecting the wrong chart or changing their mind on playing the chart. Finally, the user has choice to enter main gameplay, not yield any input, miss the notes and dimmish the life bar and immediately fail the chart. The consideration of different pathways increases the versatility of the gameplay and contributes to the modern useability and functionality that will benefit stakeholders as discussed in research



Flowchart

The two most prominent features of the main gameplay are the rendering of the notes based on the timing in the chart file and calculating the judgement of the input timing.

In this diagram, the process of the how the notes will be rendered has been heavily abstracted. However, it maintains the core aspects of how the note rendering will work; The time the note will be rendered will have an offset of the time by about the time taken for the note to scroll from the top of the screen to the receptors. This is dependent on scroll speed as a faster scroll speed will mean the offset time for notes to be rendered decreases due to the time taken to reach the receptor decreasing.

Note Judgement

The second prominent feature that is subsequent of the note rendering is the note judgment system by comparison of the user's note input timing. This will be done through real time reading of the chart file and the use of switch/case chains to compare the input timing in milliseconds. Below is the table for the range of input timings and their indicated judgement:

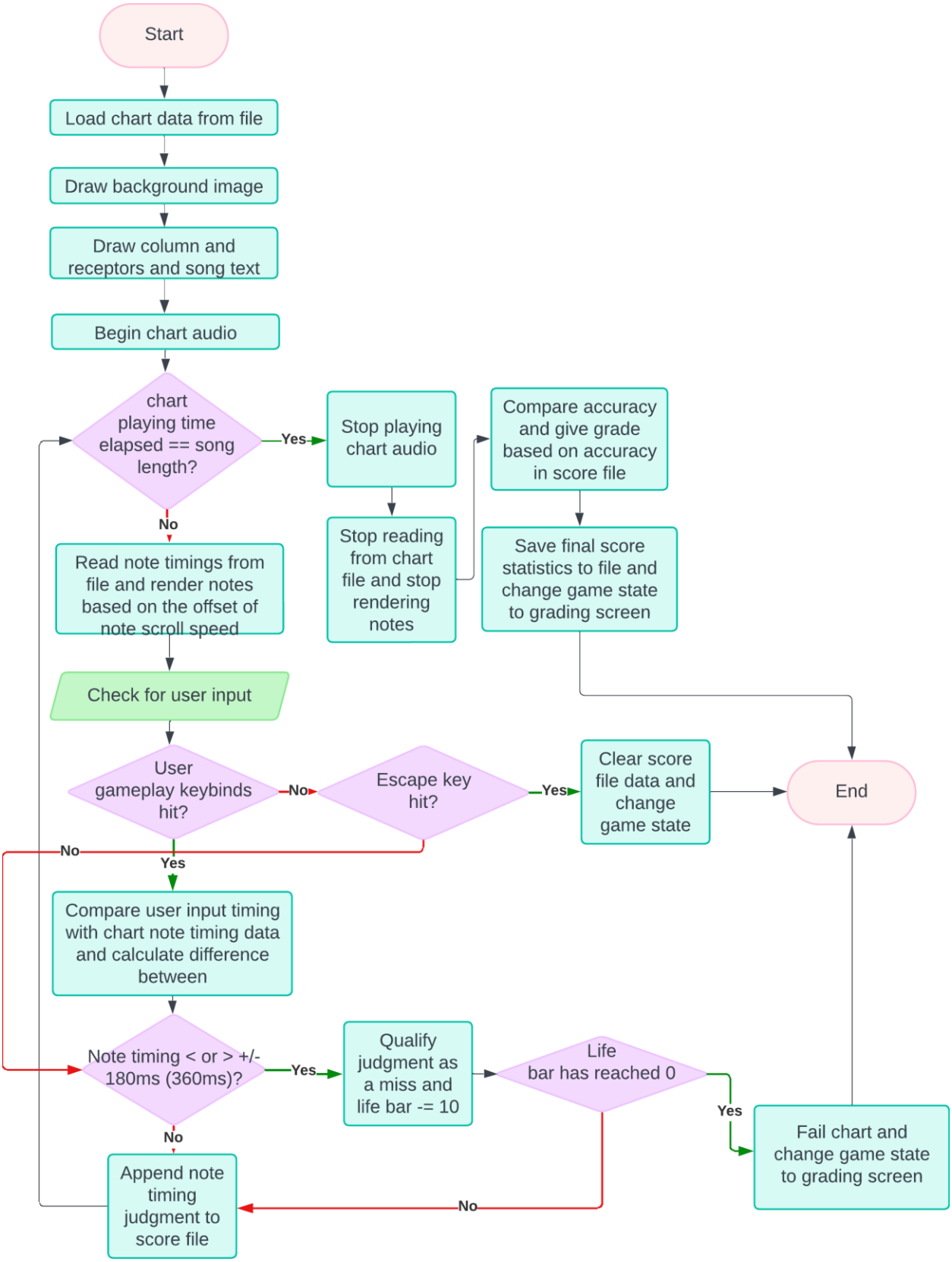
Judgement	**Input timing**
"Flawless"	Within +/- 18.9ms of real timing
"Perfect"	Within +/- 37.8ms of real timing
"Great"	Within +/- 75.6ms of real timing
"Good"	Within +/- 113.4ms of real timing
"Bad"	+/-180ms of real timing
"Miss"	>+180ms or <-180ms of real timing

The judgement system will work by comparing the difference of input time in milliseconds with the target time value that is stored within the file of the chart being played. For example, a note that is roughly at the 4 second mark for the first "beat" of the song is stored in a chart file with a time value of 4000 milliseconds (ms). In my system, a timer will begin from the start of the song audio and will be used to measure the time of the user's input. As mentioned, the note for this note timing will be visually rendered by an offset of based on the user's scroll speed. As the note scrolls, time will pass until the note is on par with its receptor. At this point the user will receive the visual queue to give keyboard input and yield input time. This time is what is compared with the time value that is stored within the chart file. In this case, let's say the user presses the keyboard and the input timing is recorded with a time of 3992ms (from the beginning of the song), the user's input timing will be -8.00ms. This input timing is then compared to the judgement range and the judgement for the input timing is given. In this case, an input time of 8ms is well in range of the "Flawless" timing and the user will be given this judgment in the score file.

Other prominent features in the flowchart include constantly checking for the escape key being inputted alongside the user's key binds and checking if the value of the health bar is

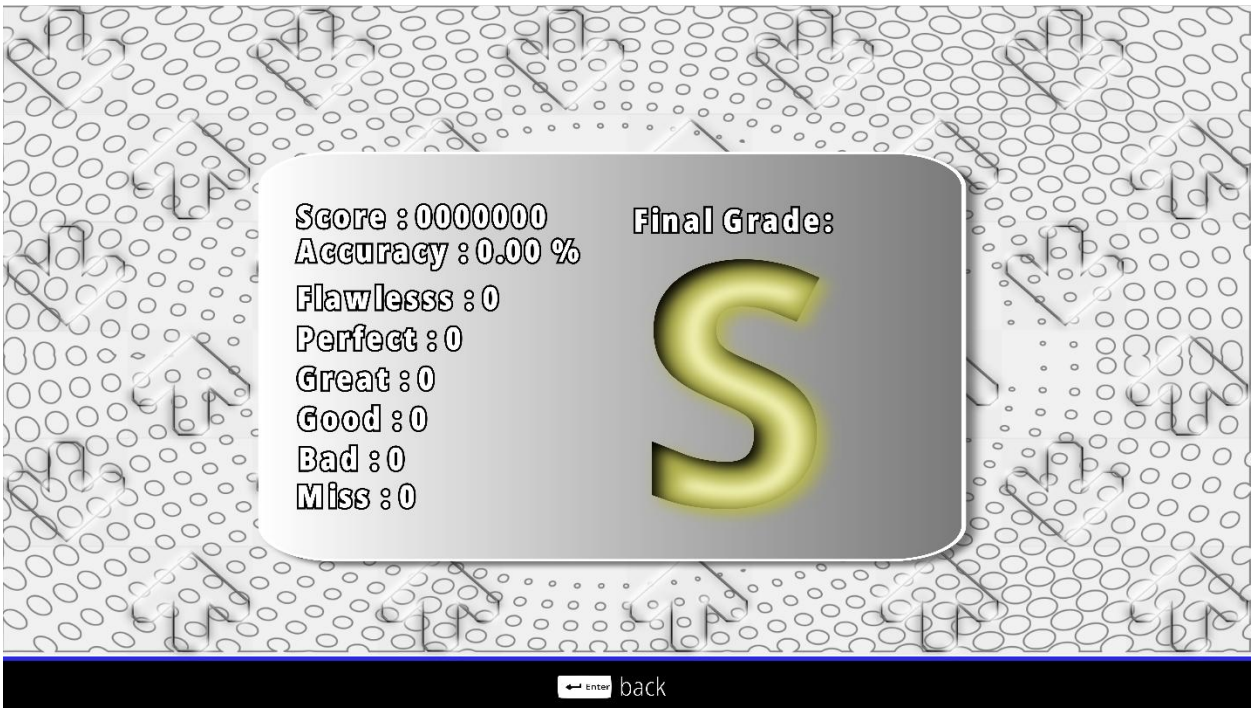
equal to 0. My justification for this is that the health bar and the escape key are the two features that can immediately alter the course of gameplay. This is because the escape key is responsible for immediately terminating the gameplay upon pressing and the health bar has the ability stop gameplay and proceed to the grade screen, once its value reaches 0. This means intended course of gameplay can be altered

The final prominent feature of the flowchart is real time reading and writing to a score file when the user receives a judgement based on their input time. This feature links back to my game statistics design where I stated the user can monitor their improvement over time, based on their accuracy and “grade” being achieved . My justification for this is that if the scores file were not stored in files within user’s secondary storage, the scores would instead be stored in the user’s primary storage (RAM). This is problematic as not only will the usage of main memory increase on every iteration but also due to the volatile nature of primary storage, the user will not be able to monitor their performance over a longer period ,e.g. the next day, without the user losing all their past score data.


Start
Load chart data from file
Draw background image
Draw column and receptors and song text
Begin chart audio
chart playing time elapsed == song length?
Yes
No
Stop playing chart audio
Stop reading from chart file and stop rendering notes
Compare accuracy and give grade based on accuracy in score file
Save final score statistics to file and change game state to grading screen
Read note timings from file and render notes based on the offset of note scroll speed
Check for user input
User gameplay keybinds hit?
No
Escape key hit?
Yes
Clear score file data and change game state
End
No
Yes
Compare user input timing with chart note timing data and calculate difference between
Note timing < or > +/- 180ms (360ms)?
Yes
Qualify judgment as a miss and life bar -= 10
Life bar has reached 0
Yes
Fail chart and change game state to grading screen
No
Append note timing judgment to score file
No

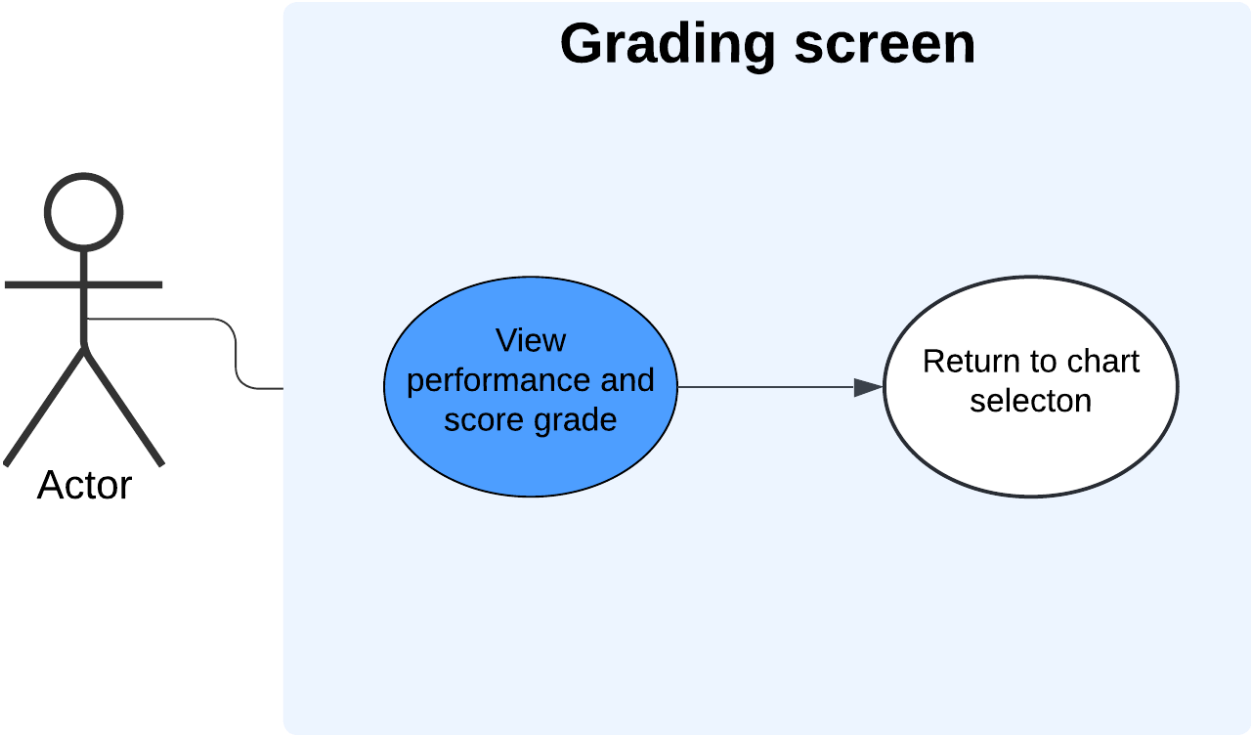
Grade Screen

Once the user has finished the map, the user will be “graded” based on their performance. The user’s grade will be accompanied by the final iteration of the gameplay statistics. This design forms modern functionality and useability of my adaptation. This is because scoring systems can be used to keep track of how the users’ scores change over time and give an indication of improvement in gameplay skill. Being able to monitor and track improvement can give the user a sense of motivation. It can also give the user an indication of what needs improvement. As mentioned earlier, maps can have different sequences of note patterns. If the user’s score is consistently below expectation for maps that are tailored to a specific note pattern, then this design can help indicate which maps the user should focus on in gameplay.



UML Use Case Diagram

The use case diagram for the grading screen shows that the user can only do a limited number of actions when on the grade screen. In this case the user is only able to preview their score and exit the grade screen to chart selection menu.



Flowchart

The flowchart for the grading screen shows the need for real time score file processing when previewing a score file. The process will consist of loading the statistics and rendering the correct text and grade. This process will happen in the background and will be abstracted from the users themselves.

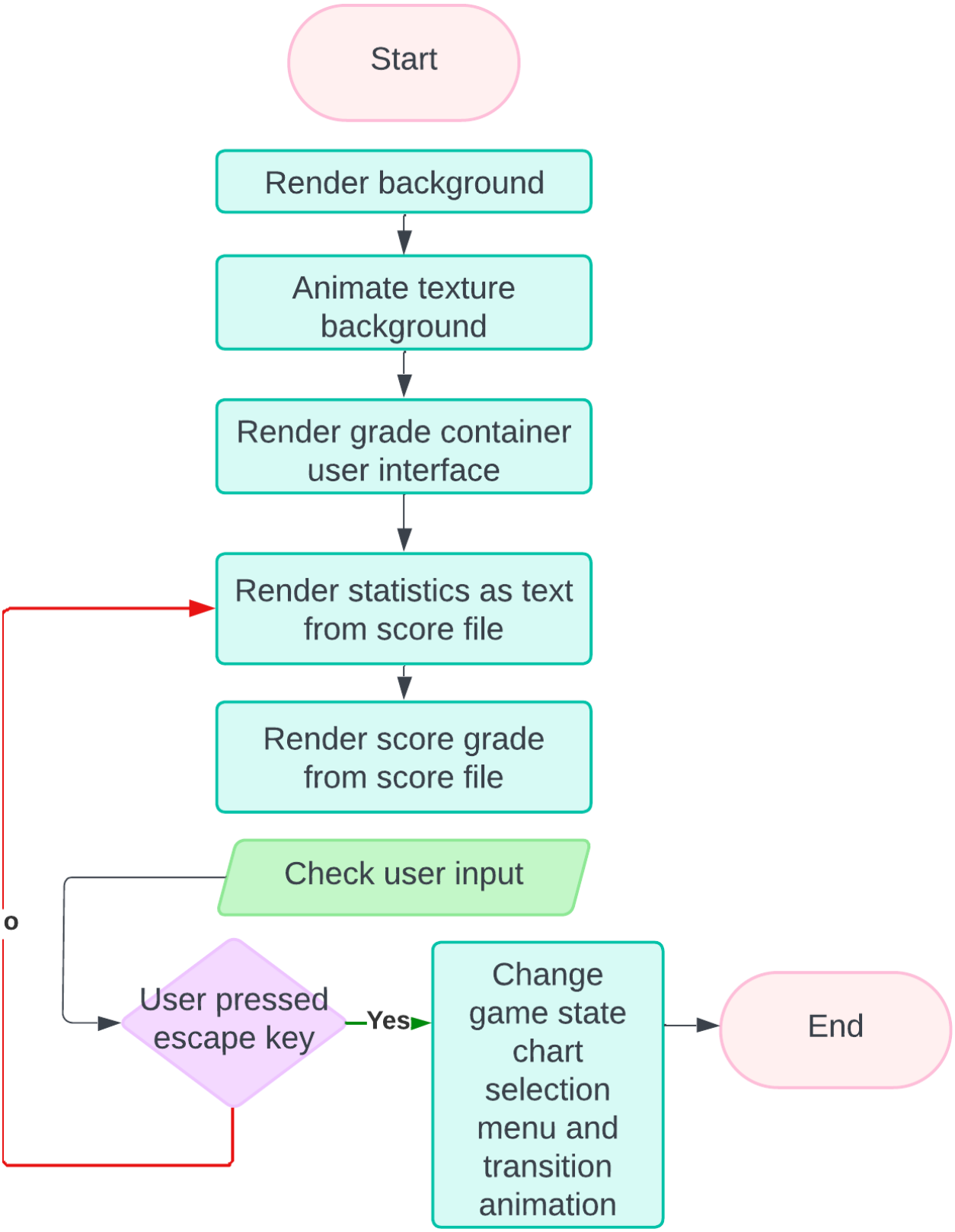
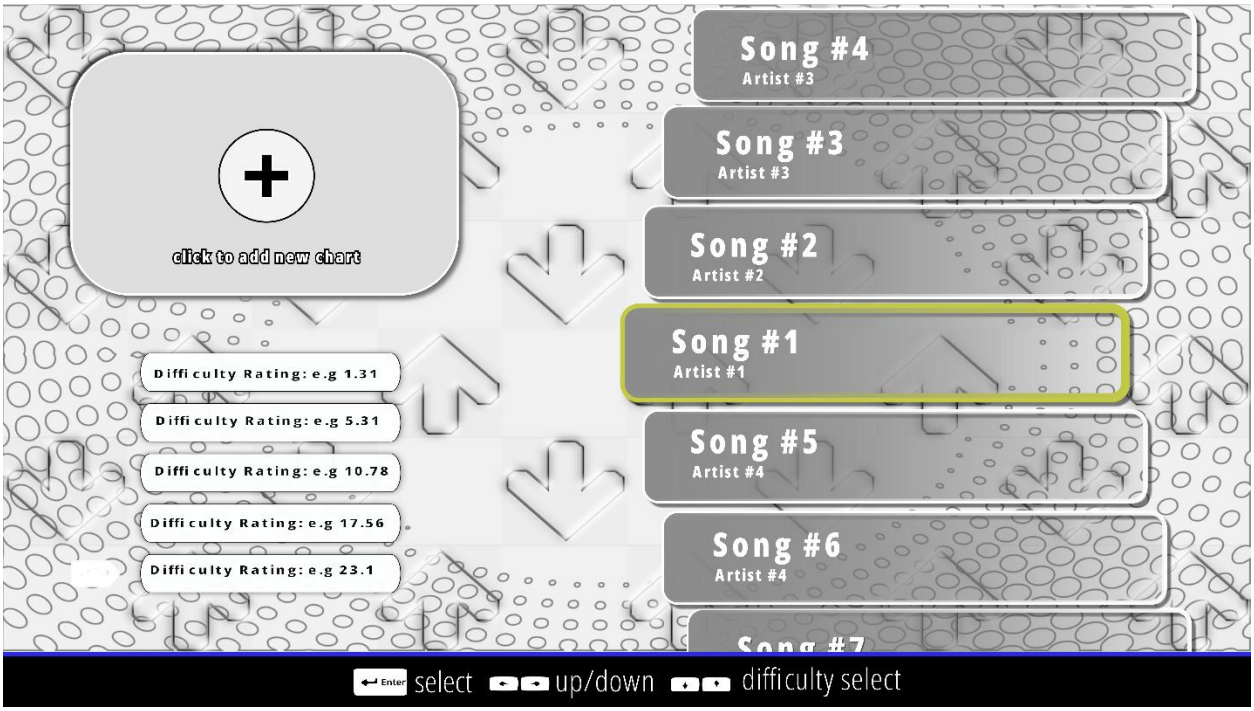
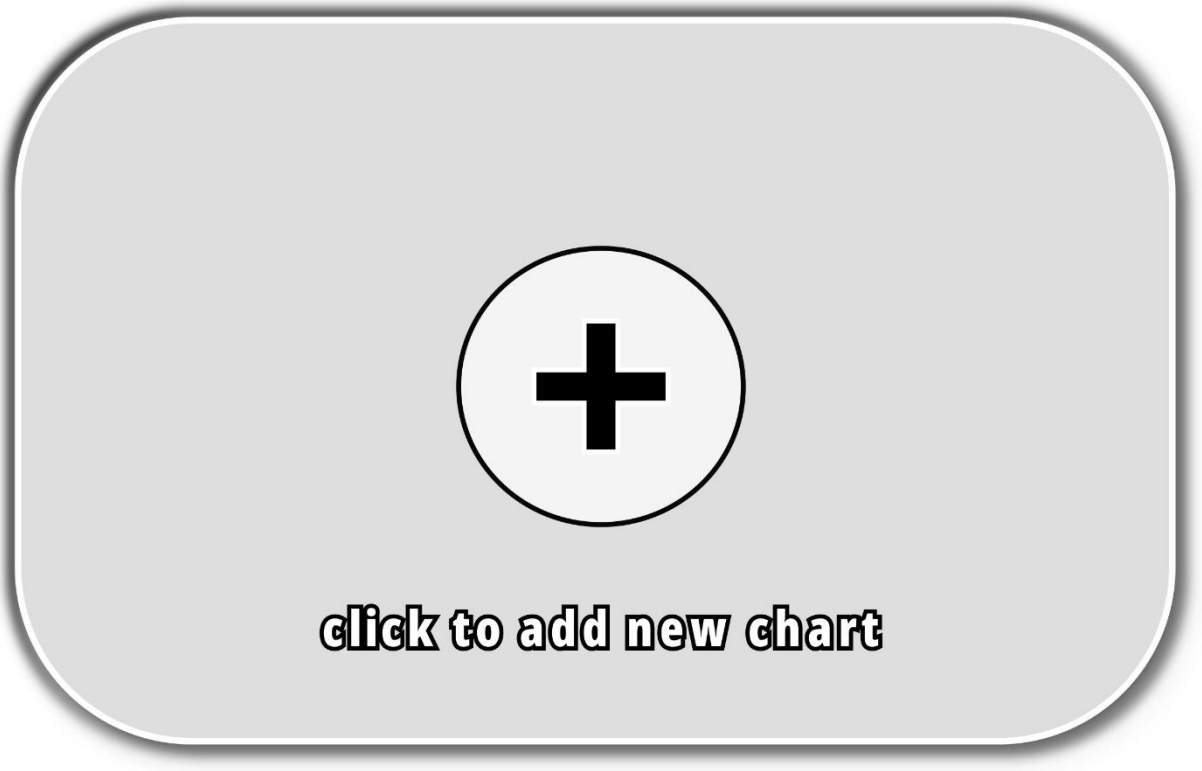

Start
Render background
Animate texture background
Render grade container user interface
Render statistics as text from score file
Render score grade from score file
Check user input
o
User pressed escape key
Yes
Change game state chart selection menu and transition animation
End

Chart Editor Selection

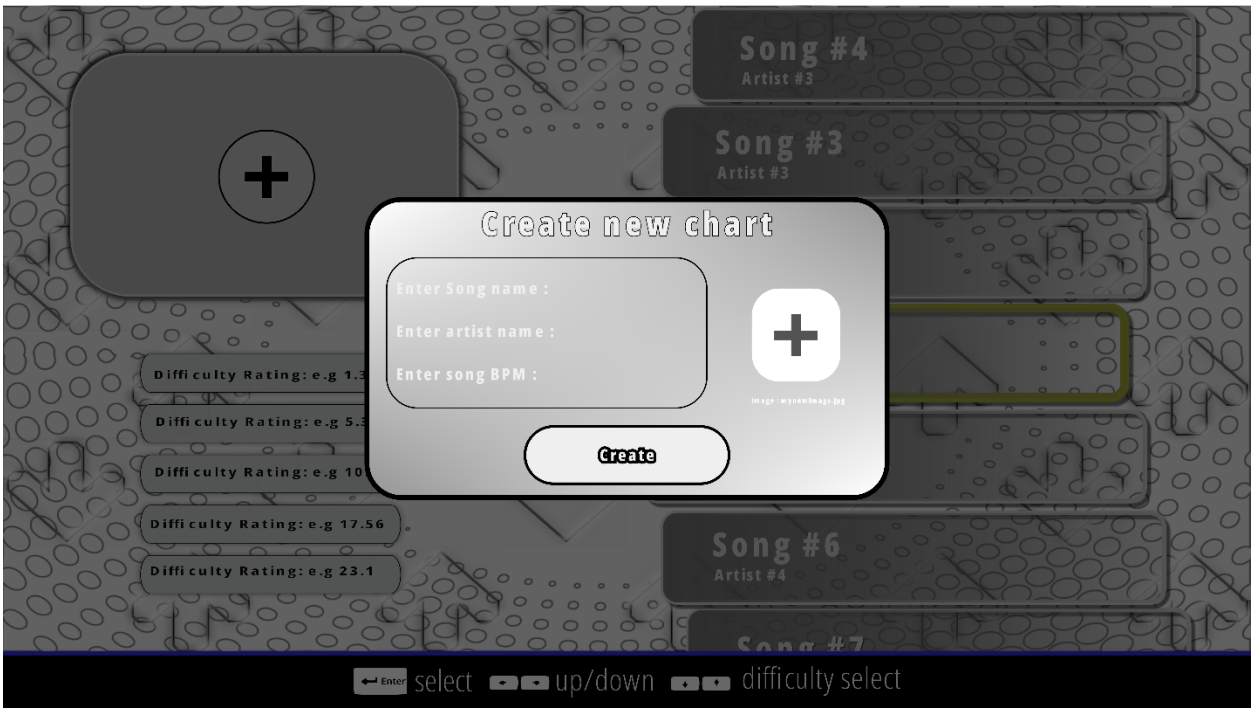
The chart editor is the second choice within the main menu and will mostly resemble aspects of the map selection menu. However, the difference between the chart selection editor and the map selection is the ability to upload audio files to initiate the creation of a new map. As mentioned in research earlier, this is an essential aspect of the adaptation's functionality as it is the gateway into which map the difficulties calculation system is exercised.



The user will be able to initiate the process of creating a new map via the "create new chart" button via mouse input. The reason for limiting it to mouse input only is to limit accidental keyboard mis-input leading to the creation of unwanted new maps.

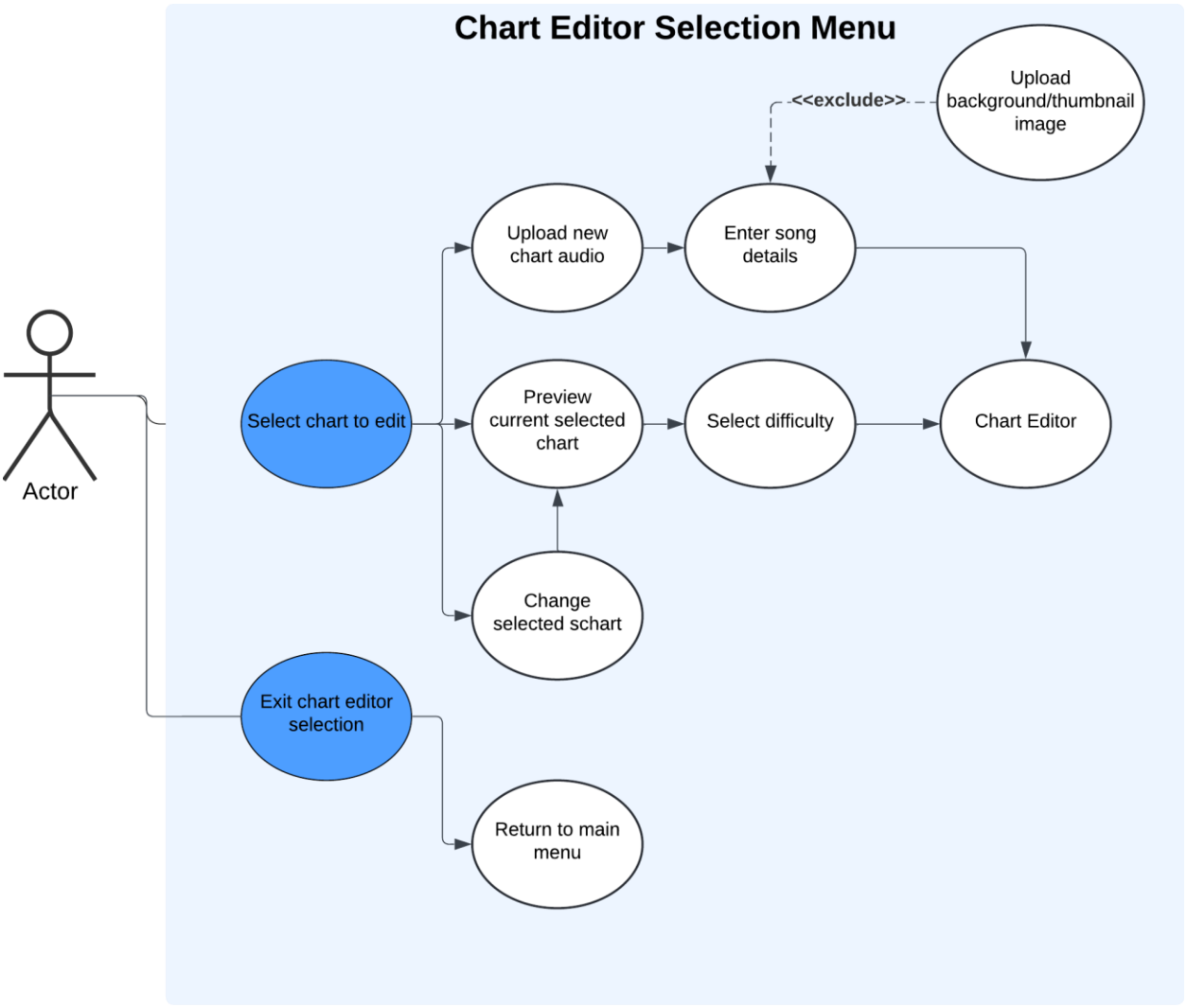


Once the user has clicked on the chart button, the user will be able to enter the song descriptions and choose the accompanying background and thumbnail images. This will form the basis for previewing the song in the map selection editor.



UML Use Case Diagram

The use case diagram for the chart editor selection menu remains most like the chart selection screen however the difference is that the section for previewing charts and their thumbnails, has been completely replaced with the ability to modify or upload new charts for songs. The chart editor system will still retain useability of being able to choose an option from the chart editor in a non-linear fashion.



Flowchart

As described in the chart selection menu, the menu system flowchart consists of more features than other sections adaptation. Despite this, the mechanisms for the chart editor selection remain the same. The difference in the chart editor selection is the consideration of the ability to upload audio files to initiate the creation of new charts or select a chart to edit. This will involve real-time file handling and the processing of new textures. This is

because the user will need to input an audio file with a chart name and upload a new image to be loaded as a texture.

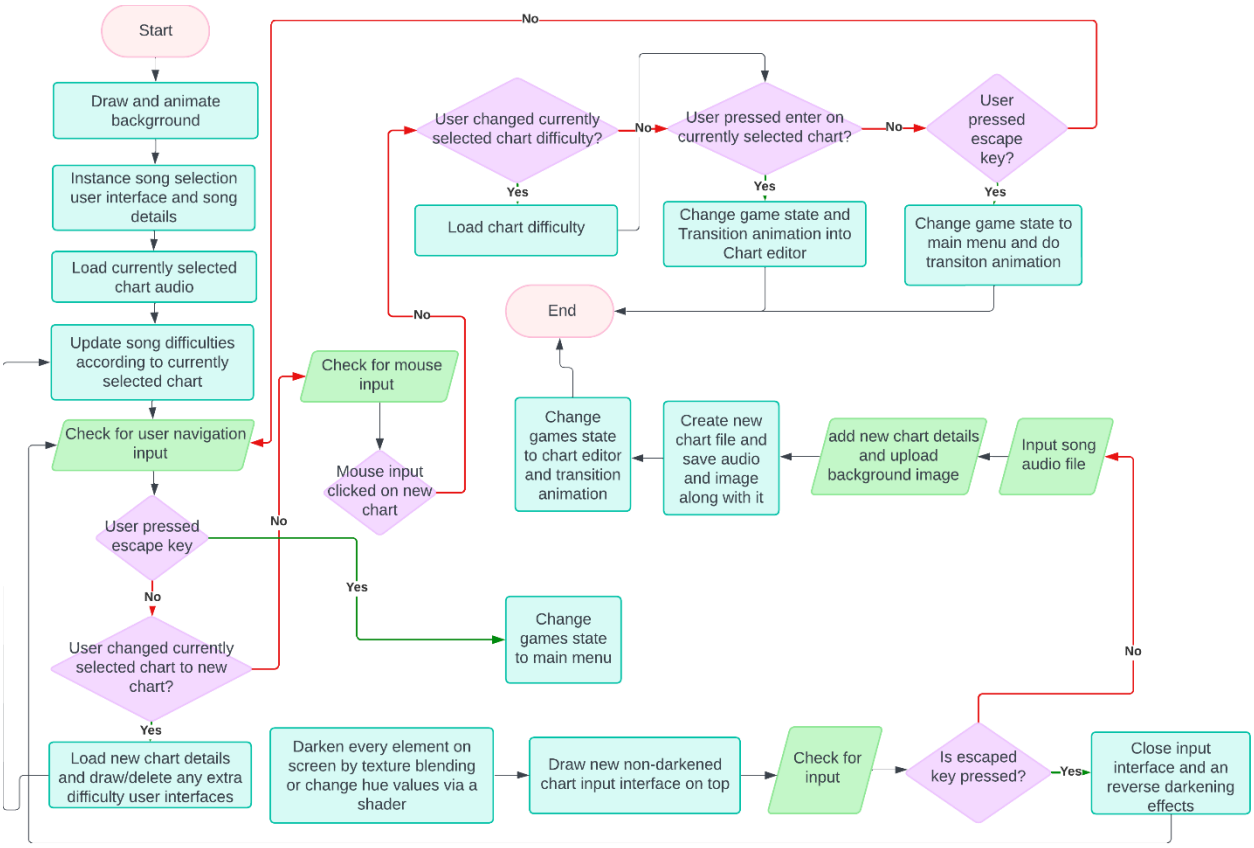


Chart Editor

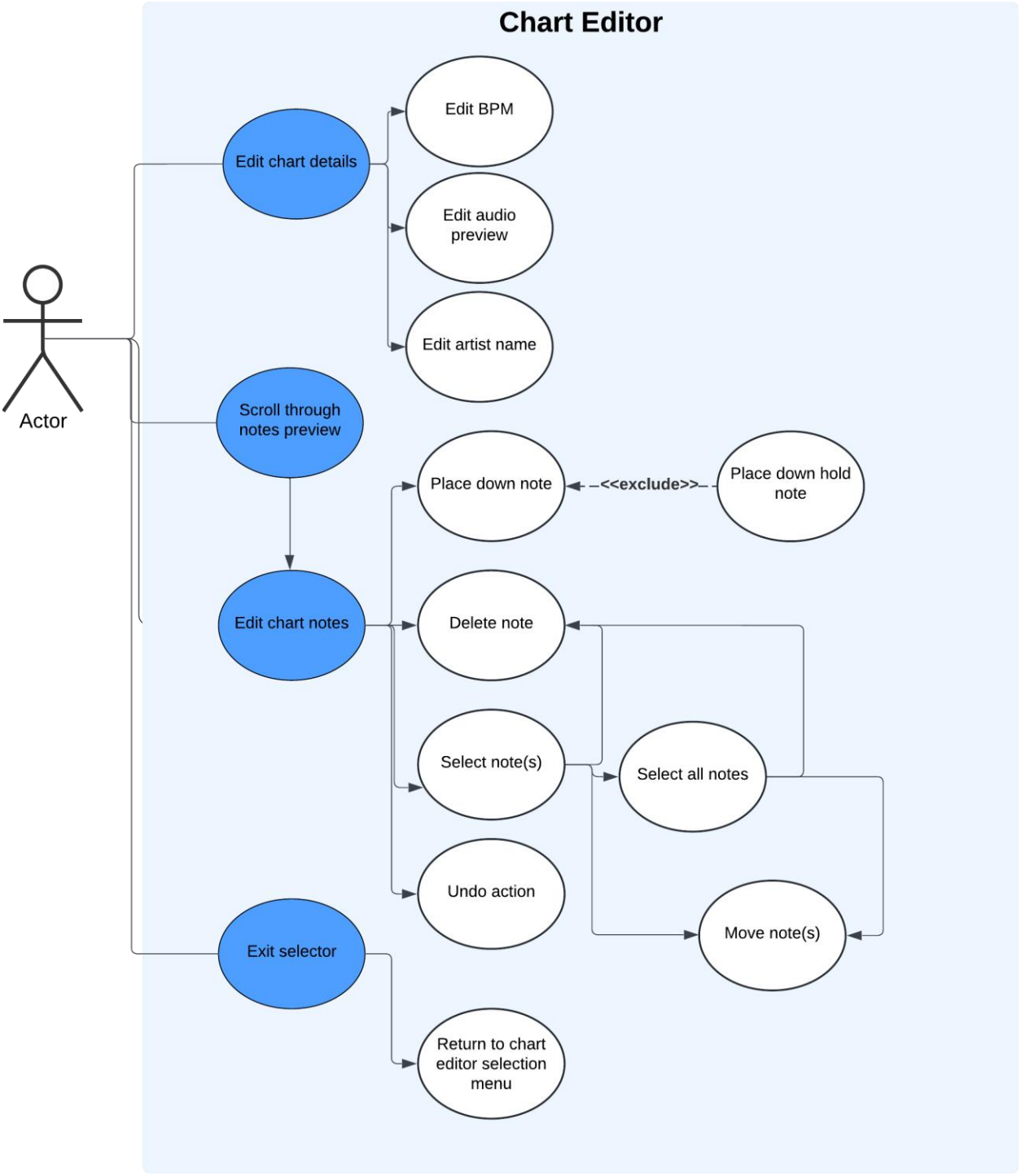
The chart editor is one of the most crucial parts of gameplay as it will be where the main aspect calculation of difficulty will take place. The chart editor will consist of a range of statistics that will be updated in real time as the user is editing a map. The statistics will include the different factors that contribute to difficulty calculation and the terminology for the different note patterns within the map. As well as this, the chart editor will provide the abstracted and visualized process of placing down notes. As mentioned earlier in research, the design of this abstracts all the background process of reading and writing to a chart file down to a few mouse inputs. This further adds to functionality the adaptation.

The process of determining the note statistics such as patterns and counts will be abstracted from the user but will be happening in the background in real time via algorithms to determine them.

As well as displaying the chart statistics, the user will be able to change the beat snapping to allow for more of a precise representation of flow of the music on the notes. The user will also be able to scroll through the length of the map via the scroll bar and preview the notes that are within that section of the chart.

UML Use Case Diagram

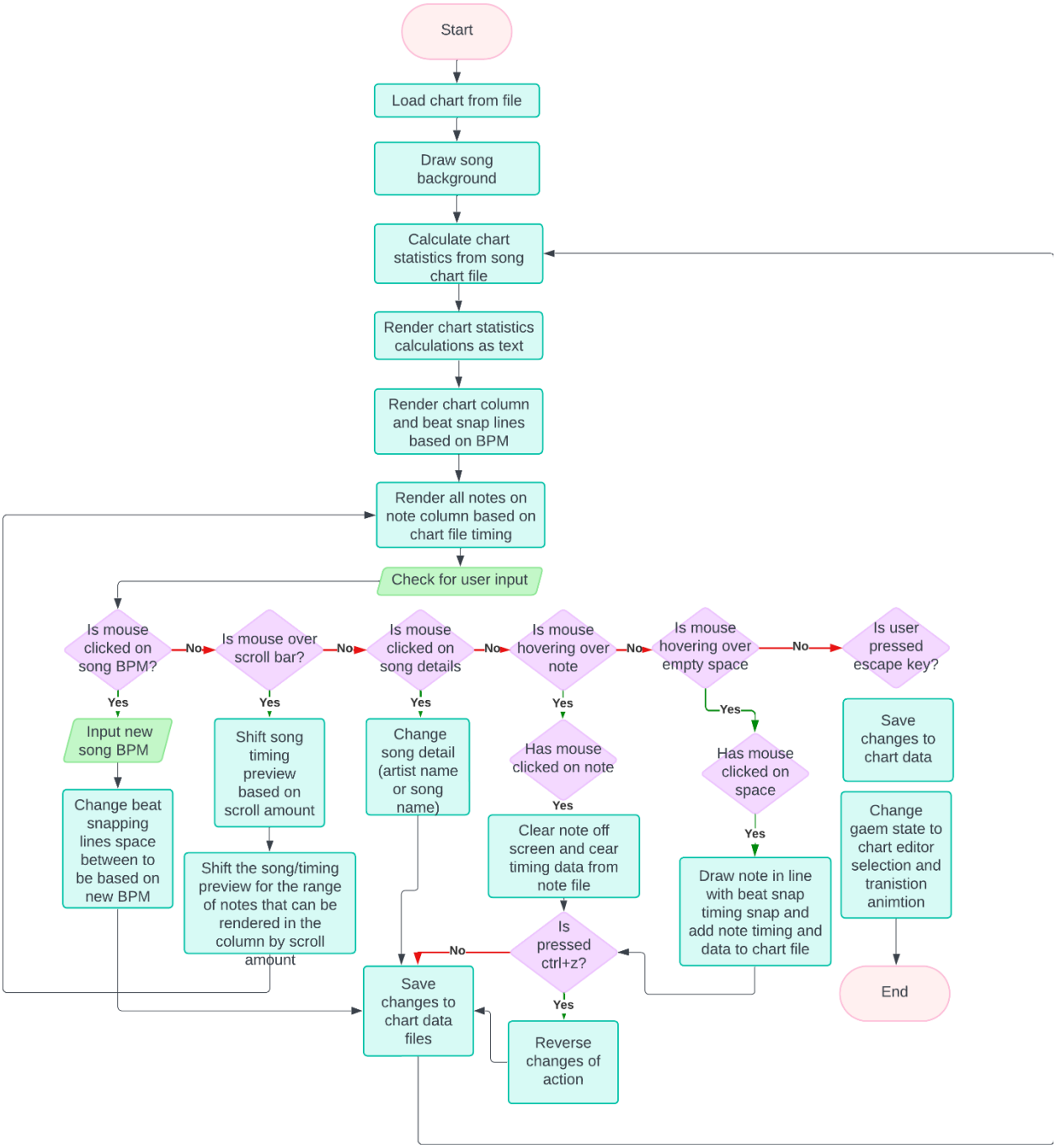
The use case diagram for chart editor is like the chart selection menu in that the user can freely decide what order they wish to operate their actions. In this case the diagram shows the different cases that can happen whilst the user is editing a chart. My justification for this is much like the main gameplay in that it is not known what the user's actions the user will be once is in the chart editor and therefore there is a need to consider the different pathways that can happen. This will be prevalent around real time chart editing where reversibility and flexibility are integrated in the usage. This can be through the user "undoing" an unwanted action or the user making a few edits to a chart without taking up too much time.



Flowchart

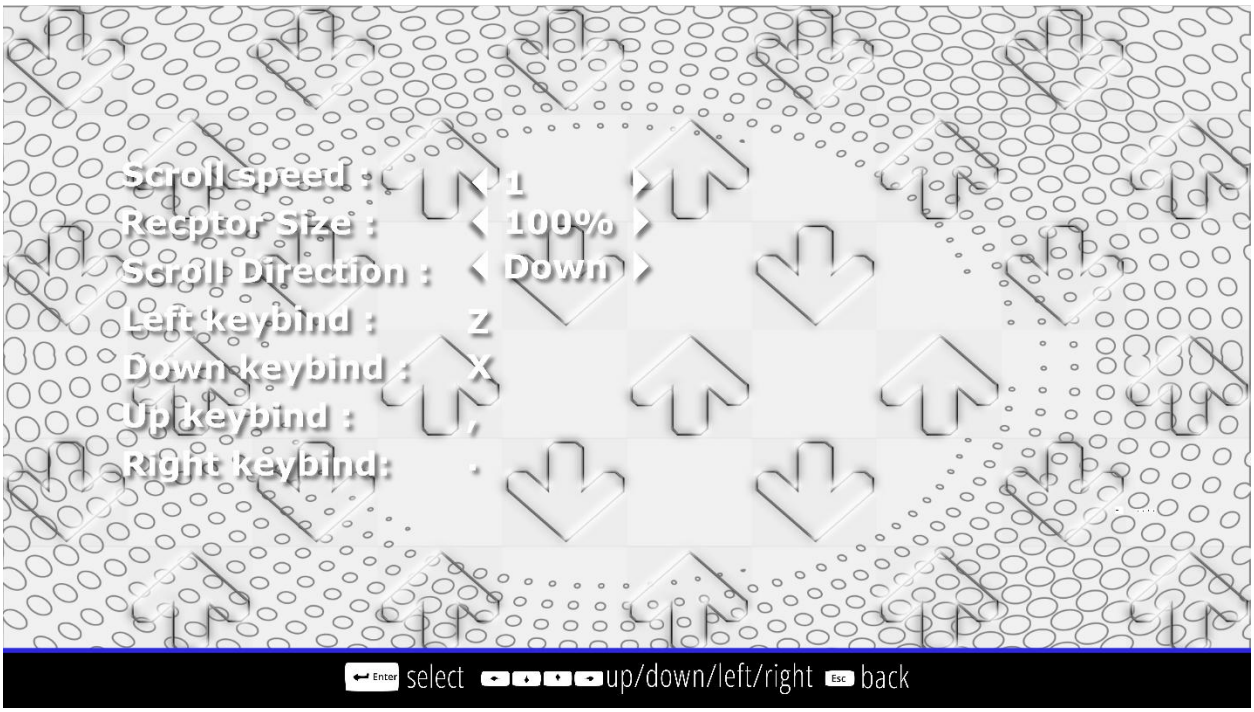
The flowchart for the chart editor follows the principles of main gameplay in that there are many different possibilities that can be taken and thus a need for contemporaneous checking of these possibilities.

Another key feature of the chart editor is the reversibility of using “control + z” key bind. This feature is included to allow efficient change of unwanted actions during chart editing. However, the use of this feature will only apply to placing notes down in the chart column. My justification for adding this feature is to increase the ease of access by not making it a requirement that the user must manually delete each note if they have mistakenly placed it down.



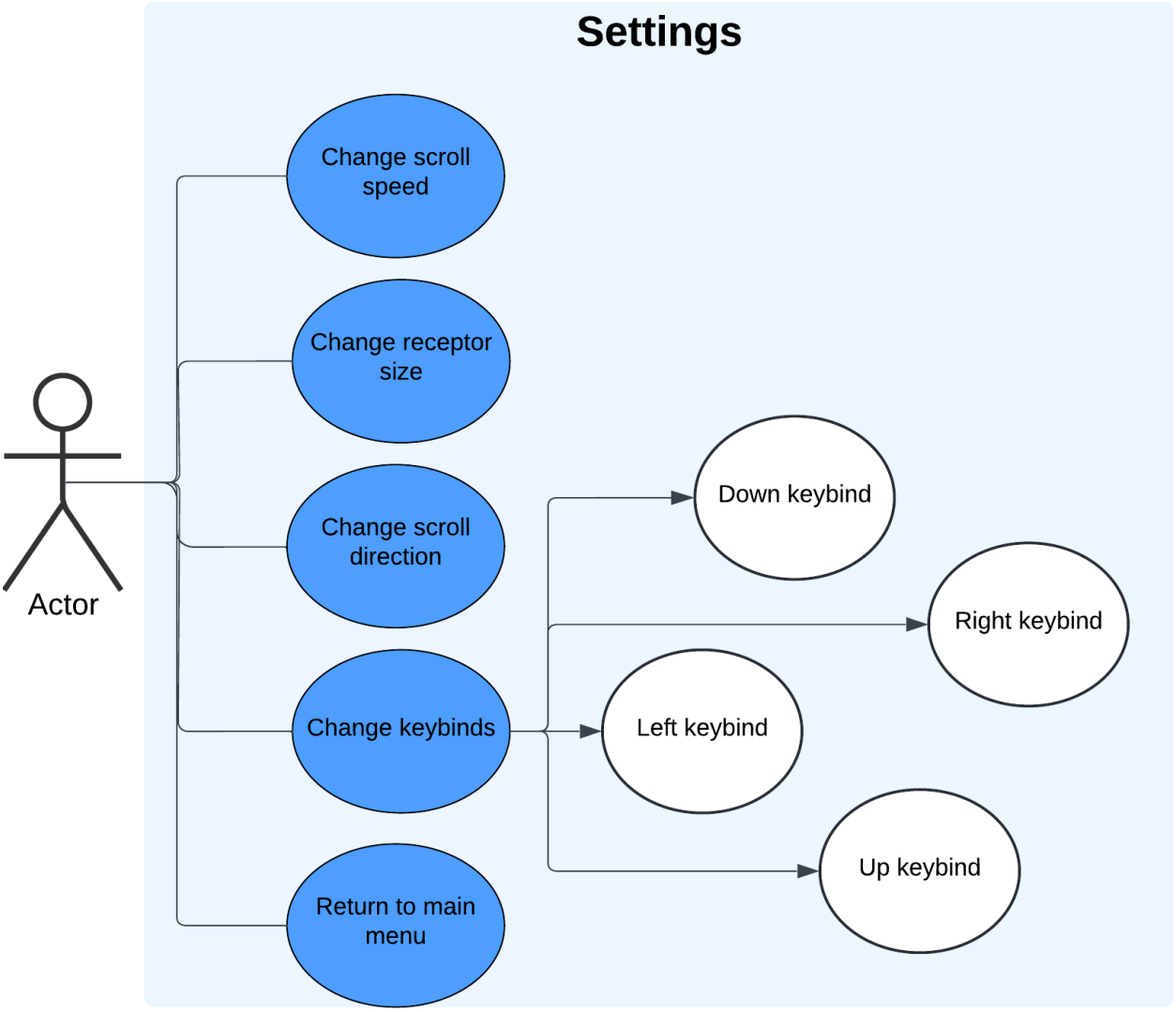
Settings

The final section of the game that use will be able to access is the settings menu. Including settings will add to the useability and functionality of the adaptation as it will allow customizability of gameplay. This will be useful for people who have a preference in the way they engage in gameplay. This is because there are a variety of different preferences that a user may use to achieve the best possible gameplay, with each preference being relative to the user. This will appeal to the modern audience who are more prone to longer hours of gameplay as it will add extensibility for changing styles as they progress. Furthermore, older VSRGs lacked this feature of customizability, therefore adding a settings section will better suit my adaptation for all age ranges of my stakeholders.



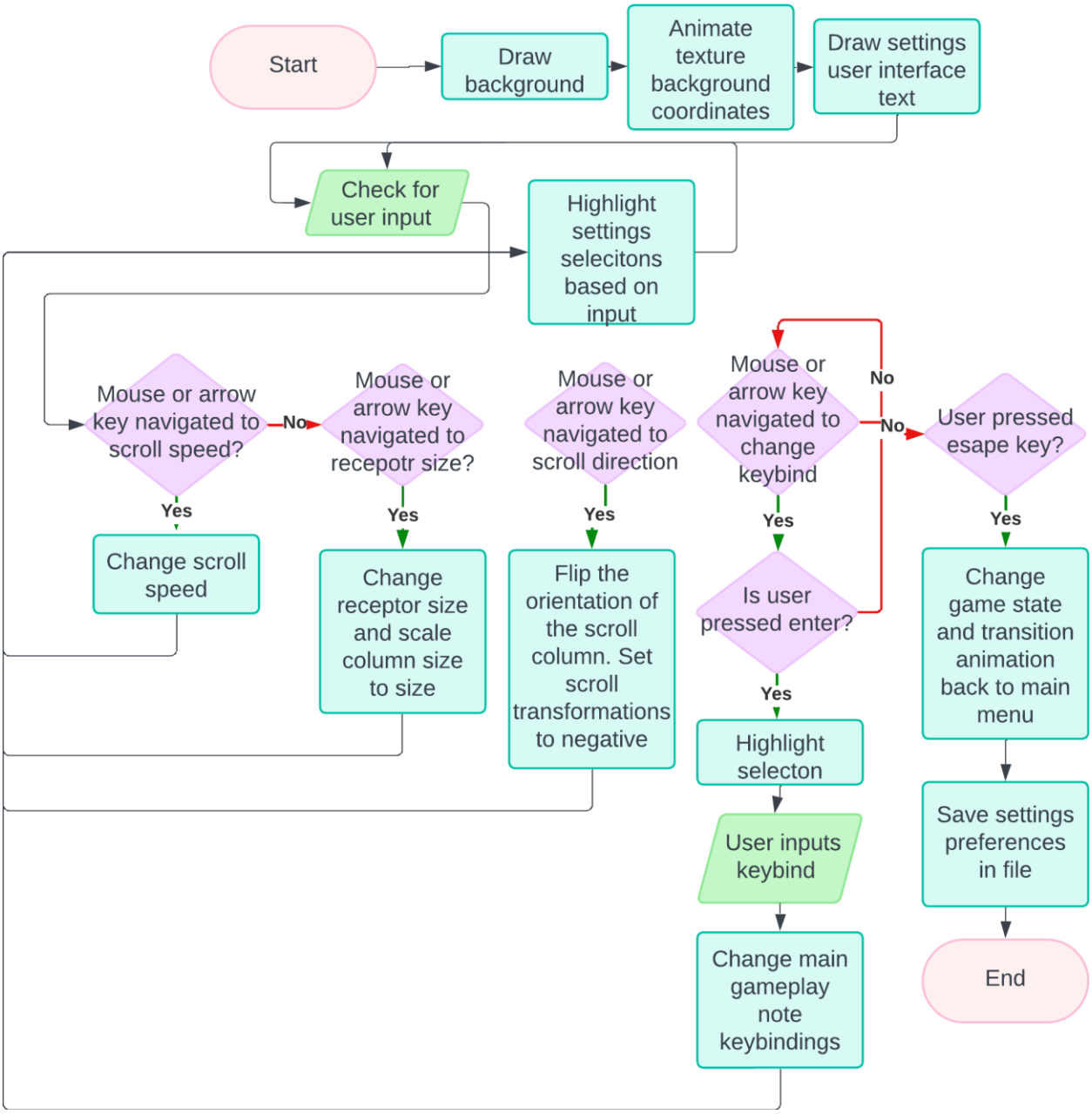
UML Use Case Diagram

The use case diagram shows different cases that the user can make when changing their settings. This continues the use of non-linear option choosing.



Flowchart

The most prominent feature of the flowchart for settings is the feature of setting key binds. This is because I will need a letter representation of each key being pressed on the keyboard. Therefore, I will need to consider the limitations of how many characters are in the font of the text being used.



Further Design Details

From the analysis up to the design, I have been taking the traditional approach to the software development life cycle. This is because I first understood the requirements by gathering information from stakeholders, then I used that data to form the scope and objectives of my adaptation. Afterwards I began to design the most logical architecture model that will conform to the objectives of the adaptation.

As I begin development and testing, I will switch to the agile (iterative) software development life cycle. This means I will begin developing my adaptation in incremental

approaches and perform intermediate testing throughout. My justification for this is because, throughout my development, I will engage in user testing for each section. As well as that, I will need to continue to design areas of my adaptation that may be required to develop my adaptation. However, I did not notice that I needed these areas until the stages of development. This means I may encounter the need to begin designing new pseudocode and class diagrams for a system that I have not considered in design specifically. This will allow me to make changes to my ideas if needed and get user feedback that could potentially improve the overall outcome of the adaptation

With the need for iterative development mentioned, it evident that I may need to change aspects of my design whilst development. Thus, for areas of my user interface design, it is also evident that these areas may be subject to change based on developmental purposes and may not completely reflect the design in the outcome. However, this does not mean that I will greatly deviate from overall architecture.

Development and Testing

Throughout development, I will use object-oriented programming and develop each section of my adaptation in a modular fashion. As discussed, in the design, the use of iterative development will permit sequential testing after each section. This will include any failed tests and the process of amended testing.

Setting Up A Window

To begin development, I will need to develop a functioning game window with an OpenGL context in which all aspects of the adaptation will happen. This will involve developing the Window class and its initiation functions which were discussed in design. It will also involve the creation of the uber game class that will be responsible for decoupling the window code from the game code.

Testing Plan

Test	Expected outcome
Executing the program produced by the compiler	A named window named “breakbeat” that has a resolution of 800x600 and has a black viewport, appears in the center of the users display. Window should also contain a “x” on the top right alongside the minimize button
I can click the “x” on the top right of the window for the program	The program window should disappear immediately, and the program executable process will terminate

Development

To begin with, I first imported the necessary SDL2 library header files that will be responsible for providing the functions for creating the window with the OpenGL context. I also included the header files for the OpenGL toolkit/API that will provide all the subroutines required to use the OpenGL specification for rendering. I also defined the header guards for the .cpp file using the #ifndef and #define macros. This is to prevent the source code in the header file being defined multiple times at each time it included in a separate .cpp file that uses the header. This will allow the usage of the functions and subroutines in other .cpp files without needing to include the actual .cpp file itself.

src > Window.hpp > Window

```
1   /*
2
3   Window.hpp
4
5   The header file for Window.cpp which is used for initalising a window
6   and creating an OpenGL Context within the window using SDL2 and glad
7
8   */
9
10  #ifndef WINDOW_HPP
11  #define WINDOW_HPP
12
13  // Include external libaries for window initialisation
14
15  #include "glad/glad.h"
16  #include "SDL.h"
```

I then defined constants within an anonymous namespace that will be used later to determine the dimensions of the game window. I have set them as constexpr and inline. This means that all uses of the variables are evaluated at compile time and replaced with the rvalue (the actual value, not the identifier) of the variables. My justification for this is that evaluating at compile time is more efficient than evaluating at runtime and will improve performance.

```
18  namespace
19  {
20      inline constexpr int WindowWidth { 800 } ;
21      inline constexpr int WindowHeight { 600 };
22  }
```

I defined the procedures and member variables for the window class that were included in the window class in the design phase and included their scope within the .hpp file. This included setting the member variables to private and the procedure definitions to public. This means that the member variables of the actual window are only accessible by the class and its procedures; There is no access to these variables by any other subroutine outside the class itself. This will preserve the encapsulation of the object's properties and keep the object-oriented style of programming.

The definitions of the procedures are only the function definitions. This means that actual source code of the function will be written and implemented in the Window.cpp file.

```
23
24  class Window
25  {
26  public:
27
28      //  Getters and setters
29      SDL_Window*& GetWindow();
30      SDL_GLContext& GetOpenGLContext();
31      SDL_Event& GetWindowEvent();
32      int& GetWindowWidth();
33      bool& GetWindowClosedBoolean();
34      void SetWindowClosedBoolean(bool);
35      void SetWindowWidth(int);
36      int& GetWindowHeight();
37      void SetWindowHeight(int);
38      // Window constructor containing the width and the height of the window as parameters
39
40      Window();
41      ~Window();
42
43  private:
44
45      // Window class mememberes for window properties
46      int mWindowWidth;
47      int mWindowHeight;
48      SDL_Window* mWindow;
49      SDL_Event mWindowEvent;
50      bool mWindowClosedBoolean;
51
52      // OpenGL context
53      SDL_GLContext mOpenGLContext;
54  };
55
56  #endif
```

I declared the window class in the window .cpp files and began writing the source code for the definition of the Initialize function(). I first began setting the window width and window height variables within the constructor by calling the SDL_Init() function for the SDL2 library and passing in the SDL_INIT_EVERYTHING enumerator. This is to initialize the library and allow usage of its functions within my source code. With this, I added an error checking step during initialization to make sure the function has worked as intended. This is through checking if the result of the SDL_Init() function is greater than 0. If it is not, then an error message will be logged to the console when running the adaptation and the contents of the actual error will be appended to the string on the console error using the SDL_GetError() function.

The actual window is created using the SDL_CreateWindow() function. During this, the window position on the screen when it first loads is determined using the

SDL_WINDOWPOS_CENTERED enumerators. This means that the window will be in the center of the users display when the game loads. Afterwards, the additional error check step is also implemented for the window creation except the difference is that it will check if the mWindow variable pointers to nothing. This will indicate that the result of the SDL_CreateWindow() function has not returned a window pointer due to an error.

src > Window.cpp > ...

```cpp
/*

Window.cpp

The source file for Window.cpp for initalising a window,
OpenGL states and the OpenGL context for rendering

*/

// Include libararies
#include "Window.hpp"

// Window constructor function to initialize window and its properties
Window::Window() :
 mWindowWidth { ::WindowWidth },
 mWindowHeight { ::WindowHeight }
{
    // Initialize SDL
    // Check if the initialization function was sucessful
    if (SDL_Init(SDL_INIT_EVERYTHING) < 0)
    {
        // Log a console error to show that initialization was unsucessfull
        // Append the actual error that is provided by SDL2 onto the string of the console error
        SDL_LogCritical(SDL_LOG_CATEGORY_APPLICATION, "Failed to initialize SDL! %s\n", SDL_GetError());
    }

    // Create the window and define its position, width and height properties and the type of window it is
    mWindow = SDL_CreateWindow("breakbeat", SDL_WINDOWPOS_CENTERED, SDL_WINDOWPOS_CENTERED,
        mWindowWidth, mWindowHeight, SDL_WINDOW_OPENGL);

    // If the value return to the mWindow vairable is null then the window failed to be initiazlaied
    if (mWindow == nullptr)
    {
        SDL_LogCritical(SDL_LOG_CATEGORY_APPLICATION, "Failed to create window! %s\n", SDL_GetError());
        SDL_Quit();
    }

```

I then wrote the code to determine the OpenGL version that my adaptation is going to use. This was discussed during the Hardware Specifications chapter of my research. As discussed, the latest version of OpenGL 4.6 will be backwards compatible with the previous version and thus allow the different operating systems with older graphics drivers to run my adaptation.

I also set the OpenGL profile version to the core profile through the SDL_CONTEXT_PROFILE_CORE enumerator. The core profile is used for all versions of OpenGL above 3.3 due to OpenGL switching from the immediate mode profile (immediate mode refers to the older and easier method of drawing graphics) to the core profile mode (the core profile is the much more modern but harder to use method of drawing graphics).

This means that the deprecated functions that use the immediate mode of OpenGL in OpenGL versions 3.2 and under will not function when using OpenGL 3.3

```
38      // Set OpenGL version to 4.6
39      SDL_GL_SetAttribute(SDL_GL_CONTEXT_MAJOR_VERSION, 4);
40      SDL_GL_SetAttribute(SDL_GL_CONTEXT_MINOR_VERSION, 6);
41      SDL_GL_SetAttribute(SDL_GL_CONTEXT_PROFILE_MASK, SDL_GL_CONTEXT_PROFILE_CORE);
42
```

After setting the OpenGL version, I then proceeded to instantiate the OpenGL context on the window using the SDL_GL_CreateContext function and add the additional error checking code that will be useful during the debugging process.

Finally, I loaded the glad OpenG toolkit library using the gladLoadGLLoader() function. This is required to allow access to OpenGL functions. I also implemented the additional error checking procedure, however this time I used the glGetError() function instead of the SDL_GetError() function. My justification for this is because the glad OpenGL toolkit library and SDL2 are separate from each other and therefore require different function calls to retrieve errors.

```
43      // Create the OpenGL context
44      mOpenGLContext = SDL_GL_CreateContext(mWindow);
45      if (mOpenGLContext == nullptr)
46      {
47          SDL_LogCritical(SDL_LOG_CATEGORY_APPLICATION, "Failed to create OpenGL context! %s\n", SDL_GetError());
48      }
49
50      // Load OpenGL functions
51      if (!gladLoadGLLoader(SDL_GL_GetProcAddress))
52      {
53          SDL_LogCritical(SDL_LOG_CATEGORY_APPLICATION, "Failed to initialize glad! %s\n", glGetError());
54      }
55  }
```

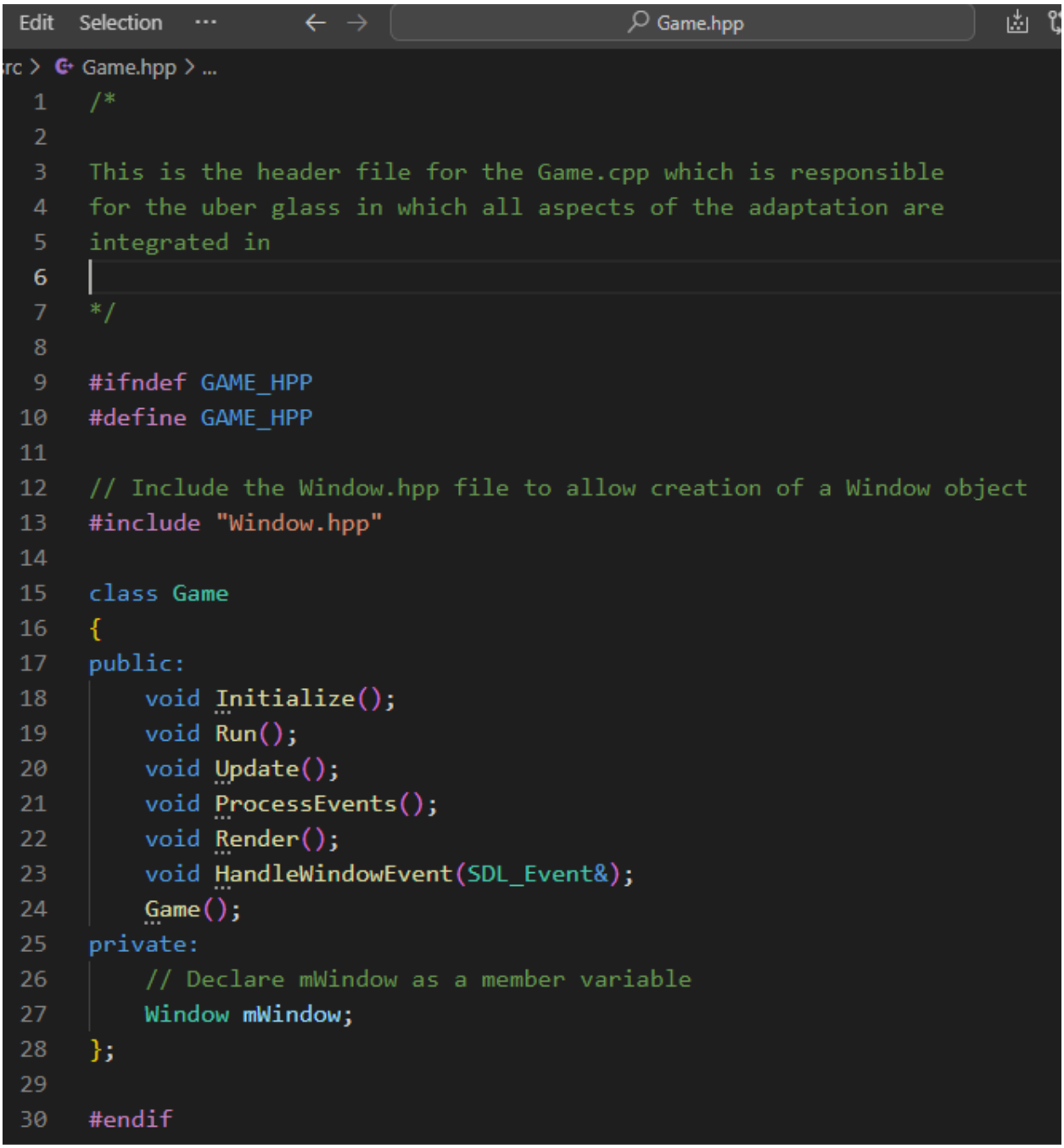
After the source code for the initialize() function was written, I proceeded to write the source code for the getters and setters in the window class according to the type of function required. This involved using the this-> keyword and the & keyword to return the variable relative to the class and to return the variable by reference respectively. My justification for returning by reference is to prevent inefficient copy initialization of the variables during the use of the function. This will save memory and optimize performance. The this-> keyword was also used to set the member variable relative to the class itself. This is because there can be many instances of a class in object-oriented programming therefore there must be a form of disambiguation for the different member variables.

```
56
57  SDL_Window*& Window::GetWindow()
58  {
59      return mWindow;
60  }
61  SDL_GLContext& Window::GetOpenGLContext()
62  {
63      return this->mOpenGLContext;
64  }
65
66  int& Window::GetWindowWidth()
67  {
68      return this->mWindowWidth;
69  }
70  int& Window::GetWindowHeight()
71  {
72      return this->mWindowHeight;
73  }
74
75  SDL_Event& Window::GetWindowEvent()
76  {
77      return this->mWindowEvent;
78  }
79
80  void Window::SetWindowWidth(int width)
81  {
82      this->mWindowWidth = width;
83  }
84  void Window::SetWindowHeight(int height)
85  {
86      this->mWindowHeight = height;
87  }
88
```

I wrote the source code for the window destructor which deletes the OpenGL context and destroys the window once the use of the window class goes out of scope during the program.

```
88
89  Window::~Window()
90  {
91      SDL_GL_DeleteContext(mOpenGLContext);
92      SDL_DestroyWindow(mWindow);
93      SDL_Quit();
94  }
```

After creating the window class, I proceeded to write the function definitions for the game class in the game.hpp file. This involved defining the subroutines that were defined in the game class diagram, defining the class header files and declaring a private window object member within the class.



I began writing the actual source for the game class's procedures. I first began by instantiating the window object variable with its default constructor. This means that window class construction is called as the window object is being instantiated.

```
src > Game.cpp > Game()
1   #include "Game.hpp"
2
3   Game::Game() :
4       mWindow(Window())
5   {
6   }
```

As I was about to begin writing the source code for the main game loop, I realized that for my game loop to loop continuously, it must have some form of condition to keep it looping. This condition must be decided when the user wants. Therefore, the condition for my game loop to keep running must be when the user decides to close the game window. To develop this feature, introduced a window closed Boolean in the private member variables and a getter and setter for the Boolean function. This is a feature that allows meeting of the user being able to exit the game via the “x” button success criterion.

```
27      bool& GetWindowClosedBoolean();
28      void SetWindowClosedBoolean(bool);
29      void SetWindowWidth(int);
30      int& GetWindowHeight();
31      void SetWindowHeight(int);
32      // Window constructor containing the width and the
33
34      Window();
35      ~Window();
36
37  private:
38
39      // Window class mememberes for window properties
40      int mWindowWidth;
41      int mWindowHeight;
42      SDL_Window* mWindow;
43      SDL_Event mWindowEvent;
44      bool mWindowClosedBoolean;
```

I also wrote the source code functionality of the additional window closed getter and setter

```
88
89  void Window::SetWindowClosedBoolean(bool boolean)
90  {
91      this->mWindowClosedBoolean = boolean;
92  }
93
94  bool& Window::GetWindowClosedBoolean()
95  {
96      return this->mWindowClosedBoolean;
97  }
```

For me to develop the use of an “x” button to exit the window, I must start writing the source for the ProcessEvents() source code. Within this procedure I began checking for window events using the written GetWindowEvent() function. This function returns the current SDL event that is being referenced in the mWindowEvent() variable. Once the window event has been returned, the contents of event variable are continually checked using the SDL_PollEvent and a while loop. Within this while loop, the event variable is checked using a switch statement. In this, the case of whether the event type is an SDL_QUIT is checked. This is possible as all event types in SDL2 are enumerated types. The SDL_QUIT enumerator is only called when the current window event is when the user is clicked on the “x” button on the window. Therefore, it is evident to set the mWindowClosedBoolean to true when this happens. Once this happens the main loop’s condition will be broken and the game will stop running.

```
8   void Game::ProcessEvents()
9   {
10      SDL_Event& event = mWindow.GetWindowEvent();
11      while (SDL_PollEvent(&event))
12      {
13          switch (event.type)
14          {
15              case SDL_QUIT:
16                  mWindow.SetWindowClosedBoolean(true);
17                  break;
18          }
19      }
20  }
```

After I wrote this code, I began writing the source code for the Run() procedure. I initated the main loop while loop and called the ProcessEvents() procedure within the main loop. This means the game will continually check for window events on each iteration of the game loop. My justification for this is that if the game loop did not check for window events on each iteration, then when the user clicks on the “x” button whilst the game is running, the game will would not close immediately and potentially risk of running indefinitely on the users system.

```
21
22  void Game::Run()
23  {
24      // Main loop for the game
25      while (!mWindow.GetWindowClosedBoolean())
26      {
27          // Handle input and window events
28          ProcessEvents();
29
30          // Update the window with OpenGL
31          SDL_GL_SwapWindow(mWindow.GetWindow());
32      }
33  }
34
```

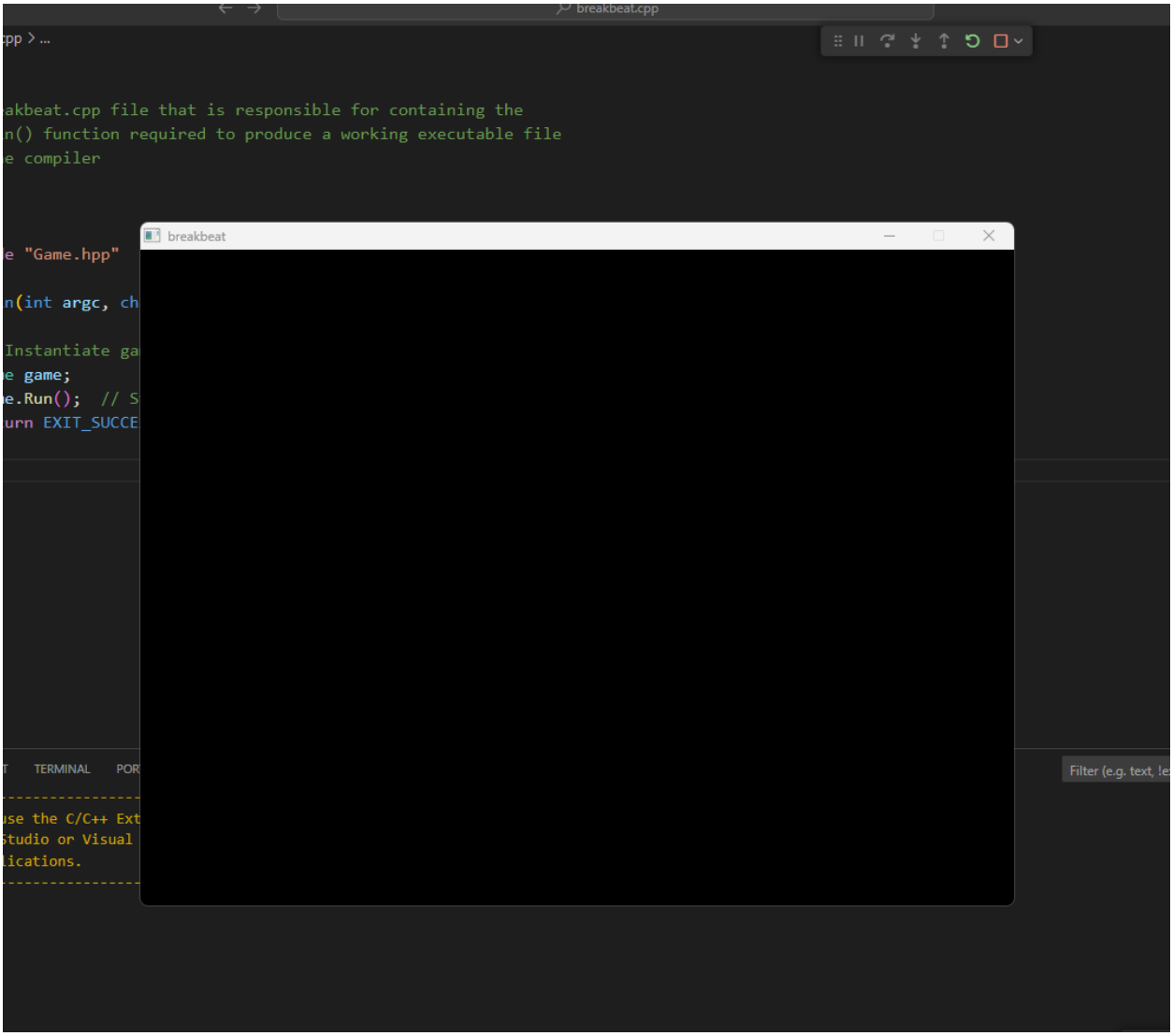
I then wrote the boilerplate int main() function source code that is must for all C++ programs that are being compiled. However, the difference in in the main() code is the usage of the int arc and char* argv[] as parameters in the function. My justification for this is because the SDL2 library uses a macro to define the usage of the main function. SDL2 uses #define main SDL_main within the headers for their source code. This means every instance of “main” is replaced with SDL_main. Because of this, the preprocessor will replace the int main() function with int SDL_main(). The actual int main() function is located within the SDL code. This is because the actual int main() is needed to perform initialization of SDL2 first. After the initialization has occurred, SDL2 then calls our main int main() function as SDL_main() and execution of our program that uses SDL2 begins. However, since SDL2 is a C based API, there is no function overloading meaning there can be only one prototype for the SDL_main() function. The SDL developers decided that it should be int SDL_main(int, char **) thus meaning we must use this protype for all SDL2 based programs.

src > breakbeat.cpp > main(int, char * [])

```cpp
/*

The breakbeat.cpp file that is responsible for containi
int main() function required to produce a working execu
from the compiler

*/

#include "Game.hpp"

int main(int argc, char* argv[])
{
    // Instantiate game object
    Game game;
    game.Run();  // Start the main game loop
    return EXIT_SUCCESS;
}

```

Testing

To commence testing, I compiled all the source files and ran the executable produced by the compiler. Upon running the program, a named window with the correct height and width properties appeared. This window was also freely movable on my display.



I was also able to click the “x” in the top left corner to test if the window event handling successfully worked and the window successfully closed itself. Thus, the success criteria of the user being able to exit the game via the “x” has been met.

Resizable Window

In this section of development, I will adjust the already formed window’s dimensions to be adjustable and dynamically adapt to the user’s display resolution upon initial execution. The window being resizable allows users to play with the adaptation on different resolutions and adjust the resolution according to their gameplay and performances needs. This will increase the overall accessibility of my adaptation. Therefore, the justification for making the window to be resizable is to continue to follow the basis of functionality and useability that is appealing to the modern audience as discussed in my analysis.

Testing Plan

Test	Expected Result
Executing the program produced by the compiler	A named window that has the same resolution as the user's maximum display resolution and is on full screen. The viewport should be completely black and thus the user's entire screen should be black.
Pressing the f11 key for the first time after executing the game.	The window should immediately resize the window to the minimum 640x480 resolution and allow the user to change the window's resolution by dragging the borders with user' s mouse.
Pressing the f11 for the second time after executing the game	The window should immediately return to full screen mode and the user's screen should be completely black
Pressing the f11 key for all subsequent times after the second press	The window should return to the last resized dimensions and should return to full screen mode seamlessly on each press.

Development

To begin the development of the resizable window, I first began by editing the Initialize() procedure within the Window class. I began by appending the SDL_WINDOW_RESIZABLE and the SDL_WINDOW_FULLSCREEN the flags parameter passed into the SDL_CreateWindow() function. With this code added, the window should now be resizable and start maximized upon initial startup. This was done because it is the actual flags themselves that determine the type of interface the window will be. This is because, as discussed in analysis, the external SDL2 library abstracts most of the window creation process. As well as the window creation process, SDL2 adds extra abstraction by abstracting the features that determine how the window will behave down to just a few enumerated type flags.

```
27      // Create the window and define its position, width and height properties and the type of window it is
28      mWindow = SDL_CreateWindow("breakbeat", SDL_WINDOWPOS_CENTERED, SDL_WINDOWPOS_CENTERED,
29          mWindowWidth, mWindowHeight, SDL_WINDOW_OPENGL | SDL_WINDOW_FULLSCREEN | SDL_WINDOW_RESIZABLE);
```

I began writing the subroutines that will allow the window to change between full screen mode and windowed mode and to be resizable even after coming out of full screen mode. This involved defining functions and procedures to update the OpenGL viewport and to save the resolution of the window when it was last resized.

```
27  class Window
28  {
29  public:
30
31      //  Getters and setters
32      SDL_Window*& GetWindow();
33      SDL_GLContext& GetOpenGLContext();
34      SDL_Event& GetWindowEvent();
35      int& GetWindowWidth();
36      void SetLastWindowedSize(int, int);
37      pair<int, int> GetLastWindowedSize() const;
38      bool& GetWindowClosedBoolean();
39      void SetWindowClosedBoolean(bool);
40      void SetWindowWidth(int);
41      int& GetWindowHeight();
42      void SetWindowHeight(int);
43      void HandleWindowResize(SDL_Event&);
44      void UpdateViewport(int width, int height);
45      void ToggleFullscreen();
```

I then defined the variables that will be involved in saving the last resolution of the window, checking if the window is in resize mode and if the window is in full screen. I also added in a Boolean variable that determines whether the game has just been executed and the resolution of the window has not changed yet. My justification for adding a resize mode variable is to factor in that the window must only save its last resize resolution only when it is not in full screen. This was the last saved resolution will not get set to the full screen resolution when the video changes to full screen mode.

```
52  private:
53
54      // Window class mememberes for window properties
55      int mWindowWidth;
56      int mWindowHeight;
57      int mLastWindowedWidth;
58      int mLastWindowedHeight;
59      SDL_Window* mWindow;
60      SDL_Event mWindowEvent;
61      bool mWindowClosedBoolean;
62
63      // Boolean value to check if window is closed
64      bool mWindowClosed;
65      bool mIsFullscreen;
66      bool mResizeMode;
67      bool mFirstTimeWindowed;
68
69      // OpenGL context
70      SDL_GLContext mOpenGLContext;
71  };
```

I wrote the code to dynamically determine the user's display resolution using the SDL_DisplayMode variable. This variable will return the user's display information. used the variable to retrieve the user's width and height properties. I then used the user's width and height in the window creation process. My justification for this action is because the user's display resolution needs to be known at runtime as the user's resolution depending on the user's device.

```
27      // Query the display's usable display bounds
28      SDL_DisplayMode displayMode;
29
30      if (SDL_GetDesktopDisplayMode(0, &displayMode) != 0)
31      {
32          SDL_LogError(SDL_LOG_CATEGORY_APPLICATION, "Could not get display mode: %s", SDL_GetError());
33      }
34      else
35      {
36          mWindowWidth = displayMode.w;
37          mWindowHeight = displayMode.h;
38      }
39
40      // Create the window and define its position, width and height properties and the type of window it is
41      mWindow = SDL_CreateWindow("breakbeat", SDL_WINDOWPOS_CENTERED, SDL_WINDOWPOS_CENTERED,
42          mWindowWidth, mWindowHeight, SDL_WINDOW_OPENGL | SDL_WINDOW_FULLSCREEN | SDL_WINDOW_RESIZABLE);
```

I wrote the code to update the viewport resolution of the window. This is because as well as the window resolution being updated, the content that OpenGL is rendering needs to be updated as well. This is to prevent the contents of the screen not being scaled up with the window

```
112  // Function to handle viewport adjustment on window resize
113  void Window::UpdateViewport(int width, int height)
114  {
115      mWindowWidth = width;
116      mWindowHeight = height;
117      glViewport(0, 0, width, height);
118  }
```

I then wrote the source code for the getters and setters to return and set the variables for the last windowed resolution.

```
120  void Window::SetLastWindowedSize(int width, int height)
121  {
122      this->mLastWindowedWidth = width;
123      this->mLastWindowedHeight = height;
124  }
125
126  pair<int, int> Window::GetLastWindowedSize() const
127  {
128      return { mLastWindowedWidth, mLastWindowedHeight };
129  }
```

I wrote the main code for allowing the window to change between full screen mode and resizable window mode. This involved first checking if the window is in resize or fullscreen mode and negating the Boolean variable of the code. I did this by first checking If the window is fullscreen mode. If it is then I used the SDL_GetWindowSize(), SDL_GetCurrentDisplayMode , SDL_SetWindowSize() and SDL_SetWindowFullscreen procedures, to get the current window size, get the user's maximum display resolution and set the window size to the user's maximum display resolution and finally set the window to full screen. My justification for setting the window to the maximum display resolution before setting it to full screen mode is to prevent unwanted resolution upscaling suddenly changing the window size. This is because unwanted resolution upscaling can cause the windows aspect ratio to be irregular and create a widened "stretched" visual distortion effect on the content on the user's window.

```
131  void Window::ToggleFullscreen()
132  {
133      mIsFullscreen = !mIsFullscreen;
134      mResizeMode = !mResizeMode;
135
136      if (mIsFullscreen)
137      {
138          // Save current window size before switching to fullscreen
139          SDL_GetWindowSize(mWindow, &mWindowWidth, &mWindowHeight);
140
141          // Set fullscreen with native display resolution
142          SDL_DisplayMode displayMode;
143          if (SDL_GetCurrentDisplayMode(0, &displayMode) == 0)
144          {
145              SDL_SetWindowSize(mWindow, displayMode.w, displayMode.h);
146              SDL_SetWindowFullscreen(mWindow, SDL_WINDOW_FULLSCREEN);
147              UpdateViewport(displayMode.w, displayMode.h);
148          }
149      }
150      else
151      {
152          // Exit fullscreen mode, restore windowed mode size
153          SDL_SetWindowFullscreen(mWindow, 0);
154
155          // Retrieve last windowed size
156          auto [width, height] = GetLastWindowedSize();
157          SDL_SetWindowSize(mWindow, width, height);
158          UpdateViewport(width, height);
159
160          // Center the window only the first time it exits fullscreen mode
161          if (mFirstTimeWindowed)
162          {
163              SDL_SetWindowPosition(mWindow, SDL_WINDOWPOS_CENTERED, SDL_WINDOWPOS_CENTERED);
164              mFirstTimeWindowed = false; // Set to false after the first-time centering
165          }
166      }
167  }
```

I then changed the window width and height constants to the be the variables for the minimum possible dimensions that user can resize the window. My justification for this is to prevent the user resizing the window to such a point that the user will not be able to see the user interface. This will also increase the robustness of adaptation and prevent misuse.

```
// Constants to define the smallest possible window resolution
namespace
{
    inline constexpr int MinWindowWidth  { 640 };
    inline constexpr int MinWindowHeight { 480 };
}
```

I then updated the window constructors to consider the variables for the last saved resolution, if the window is in full screen mode or resize mode and if it is the first time the user is changing the window size

```
// Window constructor function to initialize window and its properties
Window::Window() :
    mLastWindowedHeight { ::MinWindowHeight },
    mLastWindowedWidth { ::MinWindowWidth },
    mIsFullscreen { true },
    mResizeMode { false },
    mFirstTimeWindowed { true }
{
```

I then implemented the code that will be responsible for setting the actual limit on the window size. This involved checking if the window dimensions on each window resize event are on bounds of the minimum limits. If the window size reaches these limits, then the window will automatically “clamp” it dimensions to the minimum limits using the SDL_SetWindowSize() procedure.

```
172  void Window::HandleWindowResize(SDL_Event& event)
173  {
174      int newWidth = event.window.data1;
175      int newHeight = event.window.data2;
176
177      // Clamp the width and height to the minimum values
178      if (newWidth < ::MinWindowWidth || newHeight < ::MinWindowHeight)
179      {
180          newWidth = max(newWidth, ::MinWindowWidth);
181          newHeight = max(newHeight, ::MinWindowHeight);
182
183          // Resize the window to the maximum allowed size if too small
184          SDL_SetWindowSize(mWindow, newWidth, newHeight);
185      }
186
187      // If not in fullscreen, update the last windowed size and viewport
188      if (!mIsFullscreen)
189      {
190          SetLastWindowedSize(newWidth, newHeight);
191          UpdateViewport(newWidth, newHeight);
192      }
193  }
```

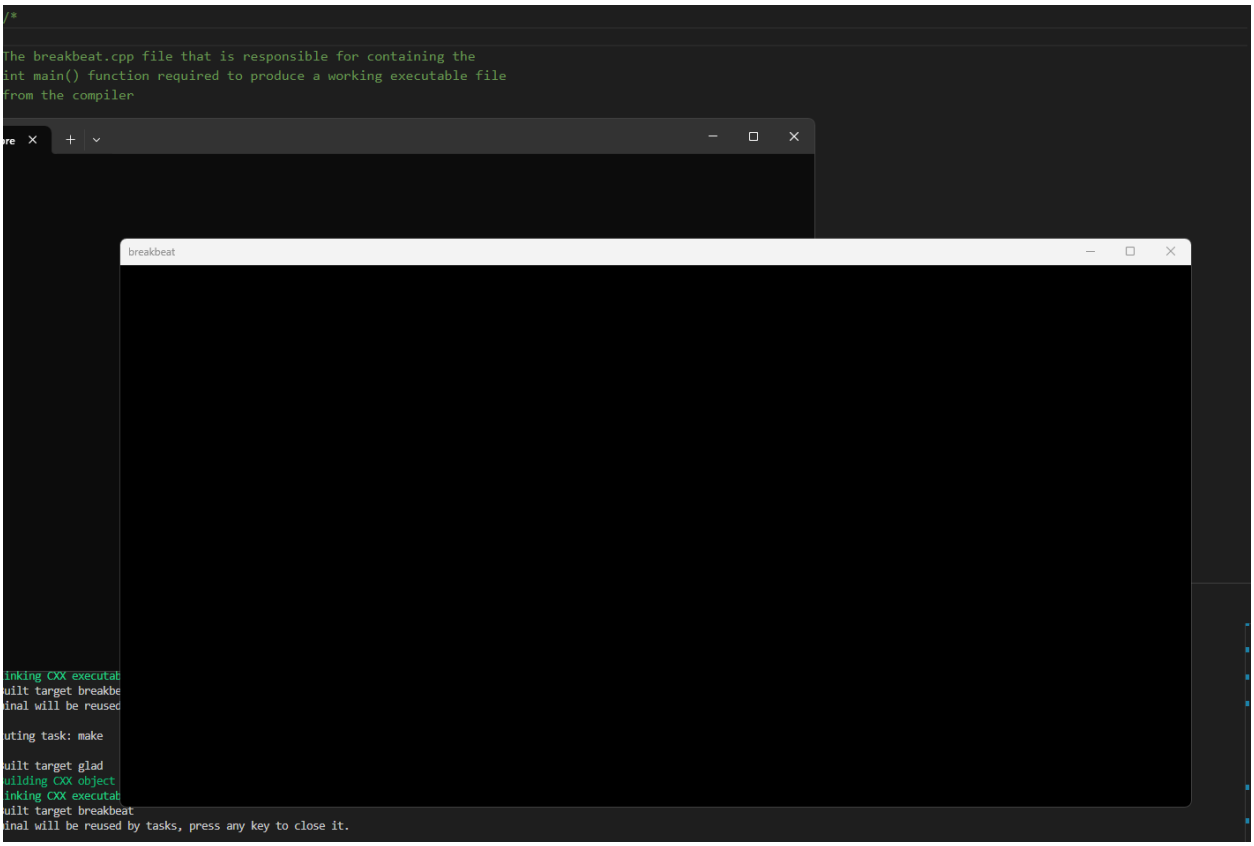
Finally, I implemented the ability to toggle full screen mode and change into resize mode by pressing F11 in the Game class's ProcessEvents() procedure. This was done by further building upon the SDL_Event checking code as discussed in the Setting Up A Window section. This time I included checking for window events using the SDL_WINDOWEVENT enumerated window type and for keypresses using the SDL_KEYDOWN enumerated type.

```
 8  void Game::ProcessEvents()
 9  {
10      SDL_Event& event = mWindow.GetWindowEvent();
11      while (SDL_PollEvent(&event))
12      {
13          switch (event.type)
14          {
15              case SDL_QUIT:
16                  mWindow.SetWindowClosedBoolean(true);
17                  break;
18
19              case SDL_WINDOWEVENT:
20                  HandleWindowEvent(event);
21                  break;
22
23              case SDL_KEYDOWN:
24                  if (event.key.keysym.sym == SDLK_F11)
25                      mWindow.ToggleFullscreen();
26                  break;
27
28              default:
29                  break;
30          }
31      }
32  }
```

Testing

To commence testing, I tested the first test case and ran the program for the first time. As soon as this happened, my screen was entirely black as intended and upon pressing the tab, I could see that the black screen was the window with its black viewport taking up the screen. This shows that my window is successfully on a full screen. This can be seen in the “Resizing A Window Test 1”

I then tested the test case of pressing F11 for the first time and the window resized to its minimum set dimensions immediately. I then pressed F11 for the second time and tried resizing the window and the windows dimensions successfully updated whilst using the mouse to drag.



I then tested the test case of pressing F11 for the third time and the window returned to full screen. I tested pressing F11 for the fourth time and the window returned to window mode and to the last saved dimensions of the window when it was last being resized.

For the final test I tested whether the window would let itself be resized to beyond the minimum limits. I tested this by dragging the window beyond the resize limit and as I did this window "snapped" back to the limit when I let of dragging the mouse. This can all be seen in the "Resizable Window Test 1" video.

Start Menu

This section will cover the creation of the adaptation's start menu user interface design. This will involve implementing the classes that were discussed as per the design stage.

Testing Plan

For my testing plan, I will also include the test data cases that were discussed in design as well as the main testing plan as shown in the table below.

Test	Expected Outcome
Execution of the program produced by the compiler	A window with the user interface of the game menu which was shown in the design section with the background animated

Pressing the up and down arrow keys whilst on the game menu	The user interface for the “start” and exit buttons should be highlighted based on the users’ input
Pressing the enter key on the currently highlighted start menu selection	The game window should fade into black and transition into a different state to show a change in game state.

Development

The development of the main menu will cover the development of shader, texture, GUIRenderer and resource manager class that was previously discussed in design

Shader class

I started by writing the shader class and subroutine definitions for the Shader class that was discussed in design. This involved creating the header and .cpp files and. This also involved defining the default value for the Boolean variable useShader to false. The reason for doing this is to allow flexibility when interacting with shader files. This flexibility comes in the form of allowing changes to the shader file without needing to use the shader to change it. This is justified as it will make my game engine more robust.

src > Shader.hpp > Shader

```
#ifndef SHADER_H
#define SHADER_H

#include <string>

#include <glad/glad.h>
#include <glm/glm.hpp>
#include <glm/gtc/type_ptr.hpp>

using glm::vec2;
using glm::vec3;
using glm::vec4;
using glm::mat4;

using std::string;

// Shader class for handling both vertex and fragment shaders
class Shader
{
public:
    // state
    unsigned int ID;
    // constructor
    Shader() { }
    // sets the current shader as active
    Shader& Use();
    // compiles the shader from given source code
    void Compile(const char *vertexSource, const char *fragmentSource);
    // utility functions for uniform varibles
    void SetFloat(const char *name, float value, bool useShader = false);
    void SetInteger (const char *name, int value, bool useShader = false);
    void SetVector3f (const char *name, float x, float y, float z, bool useShader = false);
    void SetMatrix4 (const char *name, const mat4 &matrix, bool useShader = false);
private:
    // checks if compilation or linking failed and if so, print the error logs
    void checkCompileErrors(unsigned int object, std::string type);
};

#endif
```

I began defining and specifying the utility setters as discussed in the design pseudocode. The key difference between the pseudocode and the real source code is the use of glGetUniformLocation() function. This is the function that is responsible for retrieving the name of the uniform variable in the identified shader. Another key feature is the checking of the useShader Boolean. By default, it is set to false, however if the parameter passed is true, then the shader will simultaneously update the uniform variable and be used at the same time. This feature is to allow shaders to be updated in real time without the need to call the Use() function on each instance. My justification for this is that it will make the code more modular and reproducible.

src > Shader.cpp > string

```cpp
#include "Shader.hpp"

#include <iostream>

using glm::mat4;
using glm::value_ptr;
using std::string;

Shader &Shader::Use()
{
    glUseProgram(this->ID);
    return *this;
}
void Shader::SetFloat(const char *name, float value, bool useShader)
{
    if (useShader)
        this->Use();
    glUniform1f(glGetUniformLocation(this->ID, name), value);
}
void Shader::SetInteger(const char *name, int value, bool useShader)
{
    if (useShader)
        this->Use();
    glUniform1i(glGetUniformLocation(this->ID, name), value);
}
void Shader::SetVector3f(const char *name, float x, float y, float z, bool useShader)
{
    if (useShader)
        this->Use();
    glUniform3f(glGetUniformLocation(this->ID, name), x, y, z);
}
void Shader::SetMatrix4(const char *name, const mat4 &matrix, bool useShader)
{
    if (useShader)
        this->Use();
    glUniformMatrix4fv(glGetUniformLocation(this->ID, name), 1, false, value_ptr(matrix));
}
```

I then wrote the checkCompileErrors() function for vertex and fragment shader source code. This function will be used within the Compile() function intermediately. It will be used to check for errors during the stages of the shaders being created, compiled, attached to a program and finally linked. This is to allow error checking for any causes of rendering problems that could be related to the shader files. This feature of error checking will make it easier to locate logic and syntax errors. This is because the IDE and C++ compiler itself does not provide error checking for the .GLSL shader files, it us up the developer to implement the error checking. Therefore, what this function will do is internally call the glGetShaderInfoLog() function to check for status of the shaders. If there are any issues with the status of the shader then the issue will be printed on the command line. My justification for adding error checking is to increase the robustness of my code,

make the code more maintainable and decrease the overall time spot errors and debug them.

```
39  void Shader::checkCompileErrors(unsigned int object, string type)
40  {
41      int success;
42      char infoLog[1024];
43      if (type != "PROGRAM")
44      {
45          glGetShaderiv(object, GL_COMPILE_STATUS, &success);
46          if (!success)
47          {
48              glGetShaderInfoLog(object, 1024, NULL, infoLog);
49              std::cout << "| ERROR::SHADER: Compile-time error: Type: " << type << "\n"
50                  << infoLog << "\n -- --------------------------------------------------- -- "
51                  << std::endl;
52          }
53      }
54      else
55      {
56          glGetProgramiv(object, GL_LINK_STATUS, &success);
57          if (!success)
58          {
59              glGetProgramInfoLog(object, 1024, NULL, infoLog);
60              std::cout << "| ERROR::Shader: Link-time error: Type: " << type << "\n"
61                  << infoLog << "\n -- --------------------------------------------------- -- "
62                  << std::endl;
63          }
64      }
65  }
```

I developed the Compile() procedure that was discussed in design pseudocode into its real written source code. This source code mainly follows the basis of the pseudocode except the key difference being the use of type definition. A key feature of the Compile() code is the deletion of the shaders using the glDeleteShader() procedure once they have been created. This is because the actual shader files themselves are not needed once the OpenGL shader program has been compiled and created. My justification for doing this is to save overall GPU memory usage and allow the freed memory to be used for other aspects of the program that will be noticeable.

```
void Shader::Compile(const char* vertexSource, const char* fragmentSource)
{
    // The vertex shader and the fragment shader source code will be passed in as a string and will be compiled for each shader respectively
    unsigned int vertexShader, fragmentShader;
    // create and compile the vertex shader
    vertexShader = glCreateShader(GL_VERTEX_SHADER);
    glShaderSource(vertexShader, 1, &vertexSource, NULL);
    glCompileShader(vertexShader);
    checkCompileErrors(vertexShader, "VERTEX");
    // create and compile the fragment shader
    fragmentShader = glCreateShader(GL_FRAGMENT_SHADER);
    glShaderSource(fragmentShader, 1, &fragmentSource, NULL);
    glCompileShader(fragmentShader);
    checkCompileErrors(fragmentShader, "FRAGMENT");

    // create shader program
    this->ID = glCreateProgram();
    glAttachShader(this->ID, vertexShader);
    glAttachShader(this->ID, fragmentShader);

    // Link the shaders once they have been compiled and attached
    glLinkProgram(this->ID);
    checkCompileErrors(this->ID, "PROGRAM");
    // delete the shaders as they're linked into our program now and no longer necessary
    glDeleteShader(vertexShader);
    glDeleteShader(fragmentShader);
}
```

Texture Class

I began the process of writing the subroutine declarations in a .hpp header file for the Texture class as per design.

```
src > Texture.hpp > Texture
#ifndef TEXTURE_HPP
#define TEXTURE_HPP

#include <glad/glad.h>

// Texture2D is able to store and configure a texture in OpenGL.
// It also hosts utility functions for easy management.
class Texture
{
public:
    // holds the ID of the texture object, used for all texture operations to reference to this particular texture
    unsigned int ID;
    // texture image dimensions
    unsigned int width, height; // width and height of loaded image in pixels
    // texture Format
    unsigned int textureFormat; // format of texture object
    unsigned int imageFormat; // format of loaded image
    // texture configuration
    unsigned int textureWrappingS; // wrapping mode on S axis
    unsigned int textureWrappingT; // wrapping mode on T axis
    unsigned int textureFilteringMinimum; // filtering mode if texture pixels < screen pixels
    unsigned int textureFIlteringMaximum; // filtering mode if texture pixels > screen pixels
    // constructor (sets default texture modes)
    Texture();
    // generates texture from image data
    void Generate(unsigned int width, unsigned int height, unsigned char* data);
    // binds the texture as the current active GL_TEXTURE_2D texture object
    void Bind() const;
};

#endif
```

I then wrote the constructor for the texture class that was discussed in the pseudocode section for the design of the texture class. This involved determining the default values for a texture object.

src > Texture.cpp > Texture()

```cpp
#include <iostream>

#include "texture.hpp"


Texture::Texture()
    : width(0),
    height(0),
    textureFormat(GL_RGB),
    imageFormat(GL_RGB),
    textureWrappingS(GL_REPEAT),
    textureWrappingT(GL_REPEAT),
    textureFilteringMinimum(GL_LINEAR),
    textureFIlteringMaximum(GL_LINEAR)
{
    glGenTextures(1, &this->ID);
}
```

I wrote the Generate() function and bind() procedure for the texture class that was also discussed in the pseudocode for design. This involved taking the width and height parameters and setting the texture object member variables to their value. Afterwards the a numerical identifier for the texture is set and the texture is bound using glBindTexture(). Once the texture is bound, the image and texture formats are set and image data is passed is used to create the texture. This is done through glTexImage2D. Finally image parameters for the texture wrapping and texture filtering for the bound texture are set according to the passed in parameters. My justification for texture filtering due to OpenGL texture coordinates can be any floating-point number and are not precisely on the pixels themselves. Thus, OpenGL must find a way to map the texture pixel correctly via a filtering method. There are different kinds of texture filtering and each one can be applied when scaling the image resolution up or down.

```
19  void Texture::Generate(unsigned int width, unsigned int height, unsigned char* data)
20  {
21      this->width = width;
22      this->height = height;
23      // create Texture
24      glBindTexture(GL_TEXTURE_2D, this->ID);
25      glTexImage2D(GL_TEXTURE_2D, 0, this->textureFormat, width, height, 0, this->imageFormat, GL_UNSIGNED_BYTE, data);
26      // set Texture wrap and filter modes
27      glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, this->textureWrappingS);
28      glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, this->textureWrappingT);
29      glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, this->textureFilteringMinimum);
30      glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, this->textureFIlteringMaximum);
31      // unbind texture
32      glBindTexture(GL_TEXTURE_2D, 0);
33  }
34
35  void Texture::Bind() const
36  {
37      glBindTexture(GL_TEXTURE_2D, this->ID);
38  }
```

GUI Renderer Class

I then began writing the GUI Renderer class and its procedure definitions in its header file, as discussed in design.

```
src > GUIRenderer.hpp > ...
1   #ifndef GUI_RENDERER_HPP
2   #define GUI_RENDERER_HPP
3
4   #include "Shader.hpp"
5   #include "Texture.hpp"
6
7   #include <glm/glm.hpp>
8
9   #include <iostream>
10
11  using glm::vec2;
12  using glm::vec3;
13
14  class GUIRenderer
15   {
16   public:
17      void UseShader(Shader&);
18      void Draw(Texture& texture, vec2 position,vec2 size = vec2(10.0f,10.0f), float rotate=0.0f, vec3 color = vec3(1.0f));
19      GUIRenderer(Shader&);
20      ~GUIRenderer();
21  private:
22      Shader mShader;
23      unsigned int mVertexArrayObject;
24      void Initialize();
25  };
26
27  #endif
```

Afterwards, I wrote the source code for the Initialize() procedure. This involved first declaring a vertex buffer object and the vertices to form a rectangle. These rectangle vertices will be the vertex coordinates that all interfaces will be rendered onto. Once

src > GUIRenderer.cpp > ...

```cpp
#include "GUIRenderer.hpp"

void GUIRenderer::Initialize()
{
    unsigned int vertexBufferObject;
    float vertices[] = {
        // Position       // Tex Coords
        0.0f, 1.0f,      0.0f, 1.0f,
        1.0f, 0.0f,      1.0f, 0.0f,
        0.0f, 0.0f,      0.0f, 0.0f,

        0.0f, 1.0f,      0.0f, 1.0f,
        1.0f, 1.0f,      1.0f, 1.0f,
        1.0f, 0.0f,      1.0f, 0.0f
    };

    glGenVertexArrays(1, &this->mVertexArrayObject);
    glGenBuffers(1, &vertexBufferObject);

    glBindBuffer(GL_ARRAY_BUFFER, vertexBufferObject);
    glBufferData(GL_ARRAY_BUFFER, sizeof(vertices), vertices, GL_STATIC_DRAW);

    glBindVertexArray(this->mVertexArrayObject);
    // Position attribute
    glEnableVertexAttribArray(0);
    glVertexAttribPointer(0, 2, GL_FLOAT, GL_FALSE, 4 * sizeof(float), (void*)0);
    // Texture coordinate attribute
    glEnableVertexAttribArray(1);
    glVertexAttribPointer(1, 2, GL_FLOAT, GL_FALSE, 4 * sizeof(float), (void*)(2 * sizeof(float)));

    glBindBuffer(GL_ARRAY_BUFFER, 0);
    glBindVertexArray(0);
}
```

I then began writing the constructor and destructor for the GUIRenderer class as per design. The key feature of this class is the destructor which is responsible for the deletion of the vertex array object once it goes out of scope. My justification for this is to save memory and properly manage memory on the GPU in a safe manner.

```
src > GUIRenderer.cpp > ~GUIRenderer()
1   #include "GUIRenderer.hpp"
2
3   GUIRenderer::GUIRenderer(Shader &shader)
4   {
5       this->mShader = shader;
6       this->Initialize();
7   }
8
9   GUIRenderer::~GUIRenderer()
10  {
11      glDeleteVertexArrays(1, &this->mVertexArrayObject);
12  }
```

I began writing the Draw() function.

```
48  void GUIRenderer::Draw(Texture &texture, vec2 position, vec2 size, float rotate, vec3 color)
49  {
50      this->mShader.Use();
51
52      mat4 model = mat4(1.0f);
53      model = translate(model, vec3(position, 0.0f));
54      model = translate(model, vec3(0.5f * size.x, 0.5f * size.y, 0.0f));
55      model = glm::rotate(model, radians(rotate), vec3(0.0f, 0.0f, 1.0f));
56      model = translate(model, vec3(-0.5f * size.x, -0.5f * size.y, 0.0f));
57      model = scale(model, vec3(size, 1.0f));
58
```

I encountered an error. The error was because there was no function to take in the vec3 data type as a parameter.

```
54  model = translate(model, vec3(0.5f * size.x, 0.5f * size.y, 0.0f));
55  model = glm::rotate(model, radians(rotate
56  model = translate(model, vec3(-0.5f * siz
57  model = scale(model, vec3(size, 1.0f));
58
59  this->mShader.SetMatrix4("model", model);
60  this->mShader.SetVector3f("spriteColor", color);
```

no suitable conversion function from "glm::vec3" (aka "glm::vec<3, float, glm::packed_highp>") to "float" exists C/C++ (413)

glm::vec3 color

View Problem (Alt+F8) Quick Fix... (Ctrl+.)

The solution to the error was to write an overloaded function that allows passing in of the vec3 data type. I defined the overloaded function in the header file of the Shader class

```
33      void SetVector3f (const char *name, const vec3 &value, bool useShader = false);
```

I then wrote the overloaded function's source code. It remains similar to the previous uniform variable setters.

```
60  void Shader::SetVector3f(const char *name, const vec3 &value, bool useShader)
61  {
62      if (useShader)
63          this->Use();
64      glUniform3f(glGetUniformLocation(this->ID, name), value.x, value.y, value.z);
65  }
```

The error was no longer encountered, and I continued to write the rest of the Draw() function

```
48  void GUIRenderer::Draw(Texture &texture, vec2 position, vec2 size, float rotate, vec3 color)
49  {
50      this->mShader.Use();
51
52      mat4 model = mat4(1.0f);
53      model = translate(model, vec3(position, 0.0f));
54      model = translate(model, vec3(0.5f * size.x, 0.5f * size.y, 0.0f));
55      model = glm::rotate(model, radians(rotate), vec3(0.0f, 0.0f, 1.0f));
56      model = translate(model, vec3(-0.5f * size.x, -0.5f * size.y, 0.0f));
57      model = scale(model, vec3(size, 1.0f));
58
59      this->mShader.SetMatrix4("model", model);
60      this->mShader.SetVector3f("spriteColor", color);
61
62      glActiveTexture(GL_TEXTURE0);
63      texture.Bind();
64
65      glBindVertexArray(this-> mVertexArrayObject);
66      glDrawArrays(GL_TRIANGLE_STRIP, 0, 6);
67      glBindVertexArray(0);
68  }
```

Resource Manager

I then wrote the resource manager's class definitions in the header file as discussed in design. I added in the definition of the Clear() function for later use. This function is for properly deallocating the textures in memory and for unbinding

src > ResourceManager.hpp > ...

```cpp
#ifndef RESOURCE_MANAGER_H
#define RESOURCE_MANAGER_H

#include <map>
#include <string>

#include <glad/glad.h>

#include "Texture.hpp"
#include "Shader.hpp"

using std::map;

// A static singleton ResourceManager class that hosts several
// functions to load Textures and Shaders. Each loaded texture
// and/or shader is also stored for future reference by string
// handles. All functions and resources are static and no
// public constructor is defined (singleton class)

class ResourceManager
{
public:
    // Shaders and textures stored within a hash table
    static map<string, Shader>    Shaders;
    static map<string, Texture> Textures;
    // loads (and generates) a shader program from file loading vertex, fragment
    static Shader& LoadShader(const char *vShaderFile, const char *fShaderFile, const char *gShaderFile, string name);
    // retrieves a stored sader
    static Shader& GetShader(string name);
    // loads (and generates) a texture from file
    static Texture& LoadTexture(const char *file, bool alpha, string name);
    // retrieves a stored texture
    static Texture& GetTexture(string name);
    // properly de-allocates all loaded resources
    static void Clear();
private:
    // private constructor, that is we do not want any actual resource manager objects.
    ResourceManager() { }
    // loads and generates a shader from file
    static Shader loadShaderFromFile(const char *vShaderFile, const char *fShaderFile, const char *gShaderFile = nullptr);
    // loads a single texture from file
    static Texture loadTextureFromFile(const char *file, bool alpha);
};
```

I then began writing the source for the resource manger .cpp file. I first started by trying to instantiate the hash tables within the constructors, however I was met with an error. The error is due to the shader and texture hash tables being static members and c++ not permitting the setting of static members with the constructor. The error is also because the resource manager is a singleton class and therefore its default constructor is already defined within the header file. The purpose of this is to prevent resource manger objects being formed so that the resource manager keeps its singleton status.

```
src > ResourceManager.cpp > Shaders
1   #include "ResourceManager.hpp"
2
3   #include <iostream>
4   #include <sstream>
5   #include <fstream>
6
7   #define STB_IMAGE_IMPLEMENTATION
8   #in  "Textures" is not a nonstatic data member or base class of class "ResourceManager" C/C++(292)
9
10  //   static std::map<std::string, Texture> ResourceManager::Textures
11  Res  View Problem (Alt+F8)  Quick Fix... (Ctrl+.)
12  :  Textures{},
13     Shaders{}
14  { }
```

Therefore, to resolve this problem, I removed the constructor and instantiated the variables without a constructor. This resolved the issue, and the hash table variables were instantiated as normal.

```
10  // Instantiate static variables
11
12  std::map<std::string, Texture>    ResourceManager::Textures;
13  std::map<std::string, Shader>       ResourceManager::Shaders;
```

I then began writing the source for the loadShaderFromFile() as discussed in design. I started writing the source code for opening the files

```
15  Shader ResourceManager::loadShaderFromFile(const char* vertexShaderPath, const char *fragmentShaderPath)
16  {
17      // 1. retrieve the vertex/fragment source code from filePath
18      std::string vertexCode;
19      std::string fragmentCode;
20      try
21      {
22          // open files
23          std::ifstream vertexShaderFile(vertexShaderPath);
24          std::ifstream fragmentShaderFile(fragmentShaderPath);
25          std::stringstream vertexShaderStringStream, fragmentShaderStringStream;
```

I then wrote the code for reading the files and converting their contents into strings

```
26          // read file's buffer contents into streams
27          vertexShaderStringStream << vertexShaderFile.rdbuf();
28          fragmentShaderStringStream << fragmentShaderFile.rdbuf();
29          // close file handlers
30          vertexShaderFile.close();
31          fragmentShaderFile.close();
32          // convert stream into string
33          string vertexShaderString = vertexShaderStringStream.str();
34          string fragmentShaderString = fragmentShaderStringStream.str();
35      }
```

I wrote the code within a try catch exception. If the process of reading the shader encounters an error, then the error will be caught in the exception and a message stating the issue with reading the shader file will be sent to the console.

```
36      catch (std::exception exception)
37      {
38          std::cout << "ERROR::SHADER: Failed to read shader files" << std::endl;
39      }
```

I then converted the strings into c string so they can be inputted as const char* and passed into the compile() function. I then instantiated a shader object and used its Compile() function to compile the shade files into a shader object. Afterwards the shader object that has been successfully made is returned via the return statement.

```
40      const char *vertexShaderCode = vertexCode.c_str();
41      const char *fragmentShaderCode = fragmentCode.c_str();
42      // 2. now create shader object from source code
43      Shader shader;
44      shader.Compile(vertexShaderCode, fragmentShaderCode);
45      return shader;
46  }
```

Afterwards, I then wrote the loadShader() function that loads the shader path, stores the shader in the hash table and then returns the loaded shader simultaneously. This is all implemented with the use of the loadShaderFromFile() function. I also wrote the GetShader() function which simply returns the value of the identified shader in the hash table. This assumes that the shader has already been loaded.

```
15  Shader& ResourceManager::LoadShader(const char *vertexShaderPath, const char *fragmentShaderPath, string name)
16  {
17      Shaders[name] = loadShaderFromFile(vertexShaderPath, fragmentShaderPath);
18      return Shaders[name];
19  }
20
21  Shader& ResourceManager::GetShader(std::string name)
22  {
23      return Shaders[name];
24  }
```

I then wrote the load texture function from design as discussed in pseudocode.

```
59  Texture ResourceManager::loadTextureFromFile(const char *file, bool alpha)
60  {
61      // create texture object
62      Texture texture;
63      if (alpha)
64      {
65          texture.textureFormat = GL_RGBA;
66          texture.imageFormat = GL_RGBA;
67      }
68      // load image
69      int width, height, nrChannels;
70      unsigned char* data = stbi_load(file, &width, &height, &nrChannels, 0);
71      // now generate texture
72      if (!data) {
73      std::cerr << "Failed to load texture from " << file << std::endl;
74      }
75      texture.Generate(width, height, data);
76      // and finally free image data
77      stbi_image_free(data);
78      return texture;
79  }
```

I then wrote the LoadTexture() and GetTexture() functions.

```
25
26  Texture& ResourceManager::LoadTexture(const char *file, bool alpha, std::string name)
27  {
28      Textures[name] = loadTextureFromFile(file, alpha);
29      return Textures[name];
30  }
31
32  Texture& ResourceManager::GetTexture(std::string name)
33  {
34      return Textures[name];
35  }
36
```

Game Class Updates

I then started updating the Game class Initialize function. I started off by first initializing the orthographic projection matrix. This will be used for the bulk of the 2D designs of the game.

```
34  void Game::Initialize()
35  {
36      // Set up orthgraphic projection matrix
37      mat4 proj = glm::ortho(0.0f, static_cast<float>(mWindow.GetWindowWidth()),
38          static_cast<float>(mWindow.GetWindowHeight()), 0.0f, -1.0f, 1.0f);
```

Before I could start using the shaders themselves, I needed to define a vertex and fragment shader. I first began by defining the standard source code for the vertex shader. This code works by taking in the vertex coordinates as uniform variables and setting the OpenGL position attribute for vertex coordinates to the value of the uniform variable multiplied by the model and projection variables. Finally, the code sets the textureCoordinate variables that will used in the fragment shader to the value of the texture coordinate attribute in the vertex array object.

This shader code is responsible for determining the actual positioning of each pixel for every element rendered on the screen.

shaders > VertexShader.glsl

```
1   #version 330 core
2   layout (location = 0) in vec2 vertexCoordinateAttribute;
3   layout (location = 1) in vec2 textureCoordinateAttribute;
4
5   out vec2 textureCoordinate;
6
7   uniform mat4 model;
8   uniform mat4 projection;
9
10  void main()
11  {
12      gl_Position = projection * model * vec4(vertexCoordinate, 0.0, 1.0);
13      textureCoordinate = textureCoordinateAttribute;
14  }
```

I then wrote the standard fragment shader code. This code takes in the output of the textureCoordinate unform and the output of the texture object sampler and uses the texture() function to finally determine the color of the pixel being rendered.

shaders > FragmentShader.glsl

```
#version 330 core
out vec4 fragmentColor;
in vec2 textureCoordinate;

uniform sampler2D image;
uniform vec3 color;

void main()
{
    fragmentColor = texture(image, textureCoordinate) * vec4(color, 1.0);
}
```

I then wrote the shader code for animating the background that was discussed in design. This code consists of using a time variable to move the texture coordinates. This works because the time variable is the time in milliseconds from since the program started. This means that upon starting the game, the background's texture coordinates will immediately begin to be incremented by 0.001. This is because I will be using the SDL_GetTicks() procedure which starts counting from 0 as soon as the program starts which will happen at the same time the background is being rendered. My justification for using this approach is that is because if I were to use another approach such incrementing the texture coordinates using a while loop or similar in the shader, it would be more computationally expensive and take up more GPU resources than needed.

shaders > BackgroundVertexShader.glsl

```
#version 330 core
layout(location = 0) in vec3 vertexCoordinate;
layout(location = 1) in vec2 textureCoordinateAttribute;

out vec2 textureCoordinate;

uniform mat4 model;
uniform mat4 projection;
uniform float time;

void main() {
    gl_Position = projection * model * vec4(vertexCoordinateAttribute, 1.0);
    textureCoordinate = 2 * textureCoordinateAttribute + vec2(0, time * 0.1); // Adjust speed by changing the multiplier
}
```

Afterwards I began loading the shaders using the resource manager to be used. This involved passing the relative paths to the shader files themselves and setting a name for the shader object.

```
    // load shaders
    ResourceManager::LoadShader("../shaders/VertexShader.glsl", "../shaders/FragmentShader.glsl", "default");
```

I then set the projection matrix uniform variable in the shader to the orthographic projection matrix. I also set the texture image sampler to currently in use texture unit. In

this case the index of the default texture unit is 0. I am doing this only for initialization stage. During the loaded textures stage, this will change to the indexes of the current in use textures.

```
43    ResourceManager::LoadShader("../shaders/VertexShader.glsl", "../shaders/FragmentShader.glsl", "default");
44    ResourceManager::GetShader("default").Use().SetMatrix4("projection", proj);
45    ResourceManager::GetShader("default").Use().SetInteger("image", 0);
```

I then repeated this process for the background shader.

```
46    ResourceManager::LoadShader("../shaders/BackgroundVertexShader.glsl","../shaders/FragmentShader.glsls","background");
47    // configure shaders
48    ResourceManager::GetShader("background").Use().SetMatrix4("projection", proj);
49    ResourceManager::GetShader("background").Use().SetInteger("image", 0);
50    ResourceManager::GetShader("background").SetMatrix4("projection", proj);
```

I then defined the GUI renderer pointers.

```
34    GUIRenderer* Renderer;
35    GUIRenderer* AnimRenderer;
```

I then defined the GUIRenderer object

```
47        Renderer = new GUIRenderer(ResourceManager::GetShader("defualt"));
```

I then defined the GUIRenderer oobject for the background.

```
51        AnimRenderer = new GUIRenderer(ResourceManager::GetShader("background"));
```

I then loaded the texture assets for the start menu using the resouce manager. I used the “..” within the function path to indicate the relatative path to the working directory. This way, when the user runs the adaptation, the assets for textures and shaders will be based on their directory; If I kept the paths relative to my computer only, then the textures and shaders will never be loaded.

```
ResourceManager::LoadTexture("../assets/png/breakbeat-background-dots-dots.png",true,"background");
ResourceManager::LoadTexture("../assets/png/breakbeat-background-dots-dots.png",true,"background-dots");
ResourceManager::LoadTexture("../assets/png/start menu/breakbeat-start-menu-logo-2x-new.png", true, "game-logo");
ResourceManager::LoadTexture("../assets/png/start menu/breakbeat-start-menu-start-button.png", true, "start-button");
ResourceManager::LoadTexture("../assets/png/start menu/breakbeat-start-menu-exit-button.png", true, "exit-button");
ResourceManager::LoadTexture("../assets/png/start menu/breakbeat-start-menu-user-navigation-assist.png", true, "start-menu-ua"
```

I then set up the render code by using the Draw() function, this involved opening the original design files and copying the positions coordinates of textures on the interface. This is because I designed my user interface with the same dimensions as my screen so all the object’s positions used to form interface will correlate with the positions needed to render the same textures on screen. I also set the clear color to for a

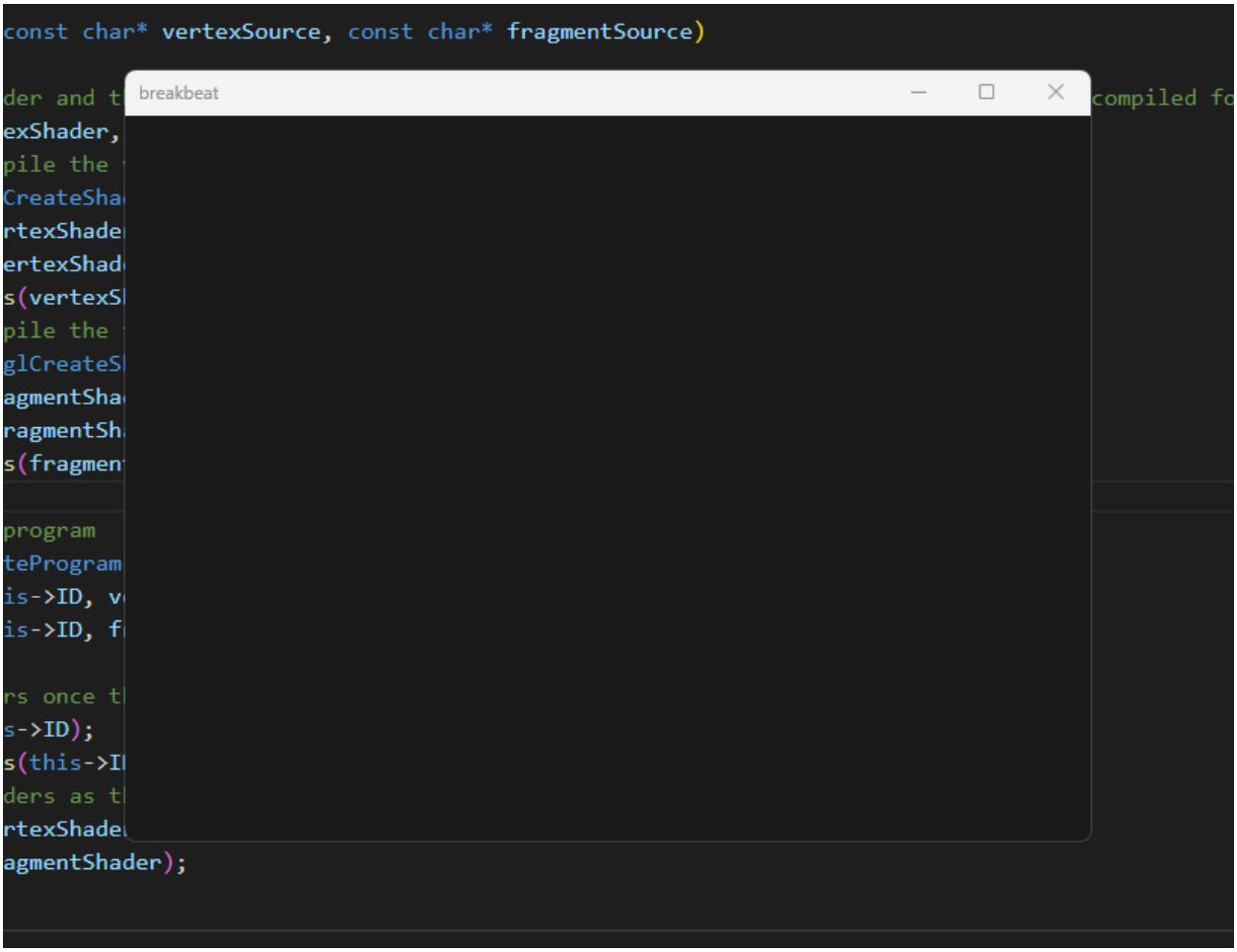
```
61  void Game::Render()
62  {
63      glClearColor(0.1f, 0.1f, 0.1f, 1.0f); // Ensure alpha is set to 1.0f for s
64      glClear(GL_COLOR_BUFFER_BIT);
65
66      float timeValue = (SDL_GetTicks() / 1000.0f);
67
68      // Draw Background
69      AnimRenderer->Draw(
70          ResourceManager::GetTexture("background"),
71          glm::vec2(0, 0),
72          glm::vec2(1920, 989)
73      );
74
75      // Draw Logo
76      Renderer->Draw(
77          ResourceManager::GetTexture("game-logo"),
78          glm::vec2(372.256, 193.333),
79          glm::vec2(1162.667, 573.998)
80      );
81
82      // Draw Start Button
83      Renderer->Draw(
84          ResourceManager::GetTexture("start-button"),
85          glm::vec2(862.230f, 720.000f),
86          glm::vec2(195.541f, 73.988f)
87      );
88
89      // Draw Exit Button
90      Renderer->Draw(
91          ResourceManager::GetTexture("exit-button"),
92          glm::vec2(889.921f, 824.483f),
93          glm::vec2(143.693f, 78.957f)
94      );
95
96      // Draw User Navigation Assist
97      Renderer->Draw(
98          ResourceManager::GetTexture("start-menu-ua"),
99          glm::vec2(0, 988.918), // Adjust Y position if needed
100         glm::vec2(1920, 91.082)
101     );
102
103     ResourceManager::GetShader("background").SetFloat("time",-timeValue,true);
104 }
```

With this system in place, I then decided to perform an intermediate testing phase to ensure that the user interface was rendering properly. My justification for doing this is to ensure that I have user interface to work with before commencing to programming the functionality for the user interface. If I did not test whether the user interface was being rendered and I started to write the functionality for user interface assuming that it was working, then if the user interface was not working, I could potentially write functionality code that is incorrect and does not match with the working interface code.

Testing Phase One

Error Encountered – Dark Grey Screen

Upon compiling and running the executable file, a completely dark grey screen appeared and there were errors on the command line. This was not expected behavior for the program.

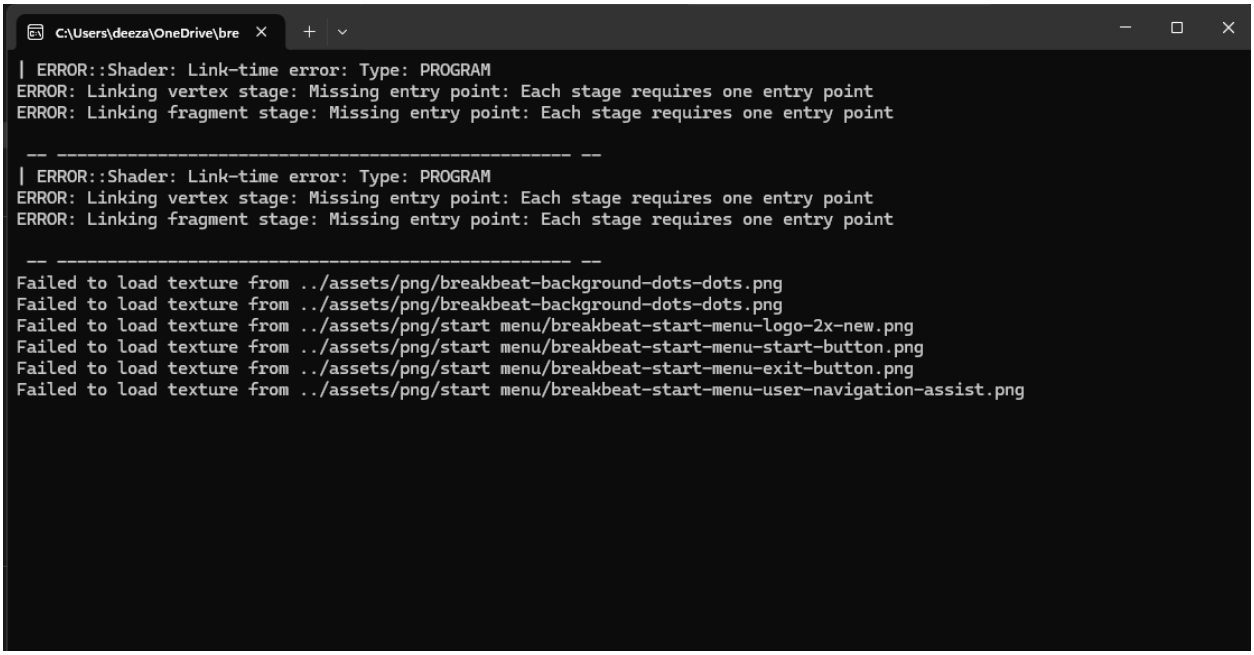


The errors on the command line clearly showed that the error checking code that is in the checkCompileErrors() procedure in the loadTextureFromFile() function is working

correctly. This result successfully links back to the test data as shown in the design section for the Shader class. The test data in design showed the invlaid test data for passing an invalid shader source code with syntax errors into the Compile() function would yield a test result of immediate console errors. This test data was successfully met.

Reason For the Error

The errors themselves suggest that there is an issue with loading the path to shaders. This is especially shown through the “Link-time error” messages which suggests that the two default shader files required for OpenGL to do rendering are not being compiled properly.



Solution To the Error

To resolve this problem, I removed all uses of “..” and replaced them with “\\”, to determine the root directory when loading file paths and then used the <filesystem> library to retrieve the relative path for the working directory. I did this by concatenating the working directory for the project directory onto the file path string in the loadShader() and loadTexture() function. This resolved the issue because using “..” was not determining the root directory of the source folder and instead was simply being loaded as two dots at the beginning of the string.

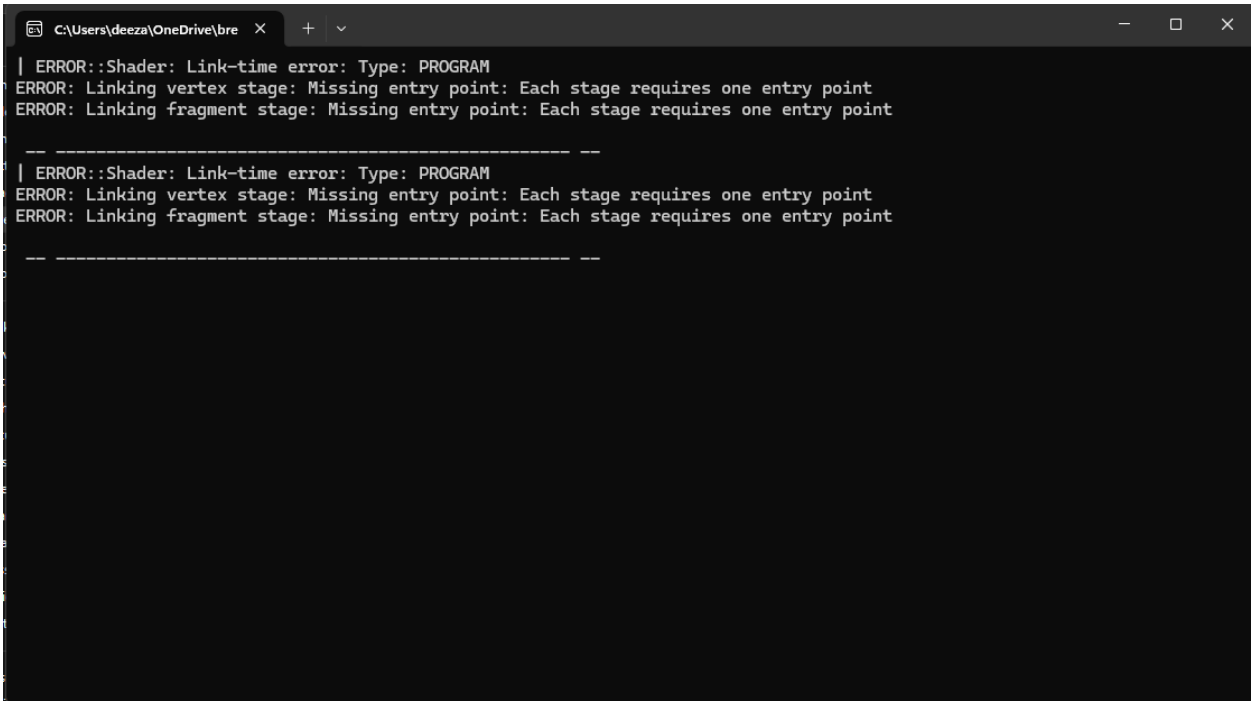
```
30  Texture& ResourceManager::LoadTexture(const char *file, bool alpha, string name)
31  {
32      Textures[name] = loadTextureFromFile((fs::current_path().string() + file).c_str(), alpha);
33      return Textures[name];
34  }
```

```
18  Shader& ResourceManager::LoadShader(const char *vertexShaderPath, const char *fragmentShaderPath, string name)
19  {
20      Shaders[name] = loadShaderFromFile((fs::current_path().string() + vertexShaderPath).c_str(),
21          (fs::current_path().string() + fragmentShaderPath).c_str());
22      return Shaders[name];
23  }
```

```
43      ResourceManager::LoadShader("\\shaders\\DefaultVertexShader.glsl", "\\shaders\\DefaultFragmentShader.glsl", "default");
44      ResourceManager::GetShader("default").Use().SetMatrix4("projection", projection);
45      ResourceManager::GetShader("default").Use().SetInteger("image", 0);
46      Renderer = new GUIRenderer(ResourceManager::GetShader("defualt"));
47      // configure shaders
48      ResourceManager::LoadShader("\\shaders\\BackgroundVertexShader.glsl","\\shaders\\DefaultFragmentShader.glsl", "background");
49      ResourceManager::GetShader("background").Use().SetMatrix4("projection", projection);
50      ResourceManager::GetShader("background").Use().SetInteger("image", 0);
51      AnimRenderer = new GUIRenderer(ResourceManager::GetShader("background"));
52      // load textures
53      ResourceManager::LoadTexture("\\assets\\png\\breakbeat-background-dots-dots.png",true,"background");
54      ResourceManager::LoadTexture("\\assets\\png\\breakbeat-background-dots-dots.png",true,"background-dots");
55      ResourceManager::LoadTexture("\\assets\\png\\start menu\\breakbeat-start-menu-logo-2x-new.png", true, "game-logo");
56      ResourceManager::LoadTexture("\\assets\\png\\start menu\\breakbeat-start-menu-start-button.png", true, "start-button");
57      ResourceManager::LoadTexture("\\assets\\png\\start menu\\breakbeat-start-menu-exit-button.png", true, "exit-button");
58      ResourceManager::LoadTexture("\\assets\\png\\start menu\\breakbeat-start-menu-user-navigation-assist.png", true, "start-menu-ua");
```

Error Encountered – Shader Link Errors

Upon testing again, the texture loading errors disappeared, however the shader link time errors persisted.



Upon reading my code I discovered that the issue laid in misnamed variables causing the shader source code to not be converted into a string and there being a typo in the name used in my shaders.

Reason For the Error

Upon realization I corrected the misnamed variables from “vertexShaderString” and “fragmentShaderString” to just “vertexCode” and “fragmentCode.” This is because I had already declared “vertexCode” and “fragmentCode” as string variables before the file reading process and intended to use them to hold the contents of the file being read. My intention was for them to be converted into c strings, but I was not using the variables themselves to hold the results of the file being read. Instead, I declared another pair of local variables to hold the contents of the file. This meant that once the local variables contained the file contents, they immediately went out of scope and the file contents were lost. Thus, “vertexCode” and “fragmentCode” were left as empty strings.

Solution To the Error

```
70          vertexCode = vertexShaderStringStream.str();
71          fragmentCode = fragmentShaderStringStream.str();
72      }
73      catch (std::exception exception)
74      {
75          std::cout << "ERROR::SHADER: Failed to read shader files" << std::endl;
76      }
77      const char *vertexShaderCode = vertexCode.c_str();
78      std::cout<<vertexShaderCode;
79      const char *fragmentShaderCode = fragmentCode.c_str();
```

I then fixed the typo from “defualt” to “default” when naming the shader.

```
Renderer = new GUIRenderer(ResourceManager::GetShader("default"));
```

Error Encountered – Shader Compilation Errors

Upon testing one more time, the shader entry point linking errors disappeared however I still received errors during the shader compilation stage.

```
C:\Users\deeza\OneDrive\bre
| ERROR::Shader: Link-time error: Type: PROGRAM
Program Link Failed for unknown reason.

 -- --------------------------------------------------- --
C:\Users\deeza\OneDrive\breakbeat\shaders\BackgroundVertexShader.glsl#version 460 core
layout(location = 0) in vec3 vertexCoordinate;
layout(location = 1) in vec2 textureCoordinateAttribute;

out vec2 textureCoordinate;

uniform mat4 model;
uniform mat4 projection;
uniform float time;

void main() {
    gl_Position = projection * model * vec4(vertexCoordinateAttribute, 1.0);
    textureCoordinate = 2 * textureCoordinateAttribute + vec2(0, time * 0.1); // Adjust speed by changing the multiplier
}
| ERROR::SHADER: Compile-time error: Type: VERTEX
ERROR: 0:12: 'vertexCoordinateAttribute' : undeclared identifier
ERROR: 0:12: '' : compilation terminated
ERROR: 2 compilation errors.  No code generated.


 -- --------------------------------------------------- --
| ERROR::Shader: Link-time error: Type: PROGRAM
Program Link Failed for unknown reason.

 -- --------------------------------------------------- --
```

Reason For the Error

The issue laid within another misnamed variable error. It turned out that I had forgotten to write “vertexCoordinateAttribute” and wrote “vertexCoordinate” in the vertex shader for the background. I had also done the same in the default vertex shader.

Solution To the Error

Upon realization, I fixed the shader variable names.

shaders > DefaultVertexShader.glsl

```
#version 460 core
layout (location = 0) in vec2 vertexCoordinateAttribute;
layout (location = 1) in vec2 textureCoordinateAttribute;

out vec2 textureCoordinate;

uniform mat4 model;
uniform mat4 projection;

void main()
{
    gl_Position = projection * model * vec4(vertexCoordinateAttribute, 0.0, 1.0);
    textureCoordinate = textureCoordinateAttribute;
}
```

shaders > BackgroundVertexShader.glsl

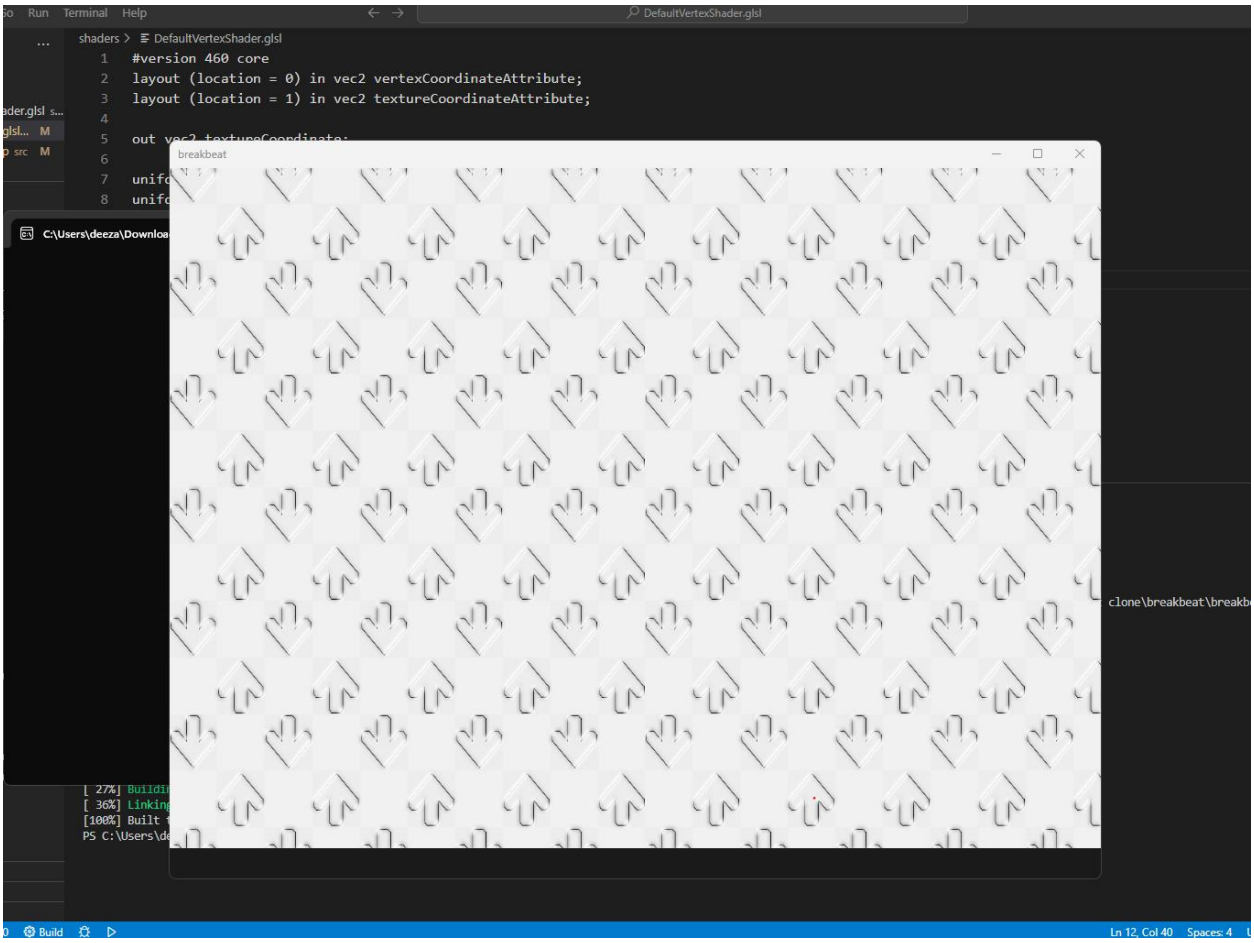
```
#version 460 core
layout(location = 0) in vec3 vertexCoordinateAttribute;
layout(location = 1) in vec2 textureCoordinateAttribute;

out vec2 textureCoordinate;

uniform mat4 model;
uniform mat4 projection;
uniform float time;

void main() {
    gl_Position = projection * model * vec4(vertexCoordinateAttribute, 1.0);
    textureCoordinate = 2 * textureCoordinateAttribute + vec2(0, time * 0.1); // Adjust speed by changing the multiplier
}

```

Upon testing, the errors disappeared, and the user interface was successfully displayed. The results of this test can be seen in the “Testing Phase One Video”.

To further continue showcasing the test data for the Compile() function that was discussed in design. I altered my vertex shader source to include a logic error within the code. The logic error was swapping the matrix multiplication order from “projection * model” to “model * projection”, for the model and projection matrices. This should yield a logic error due to matrix multiplication being non-commutative. This test data will act as boundary test data as mentioned in the shader design section.

```
10  void main()
11  {
12      gl_Position = model * projection * vec4(vertexCoordinateAttribute, 0.0, 1.0)
13      textureCoordinate = textureCoordinateAttribute;
14  }
```

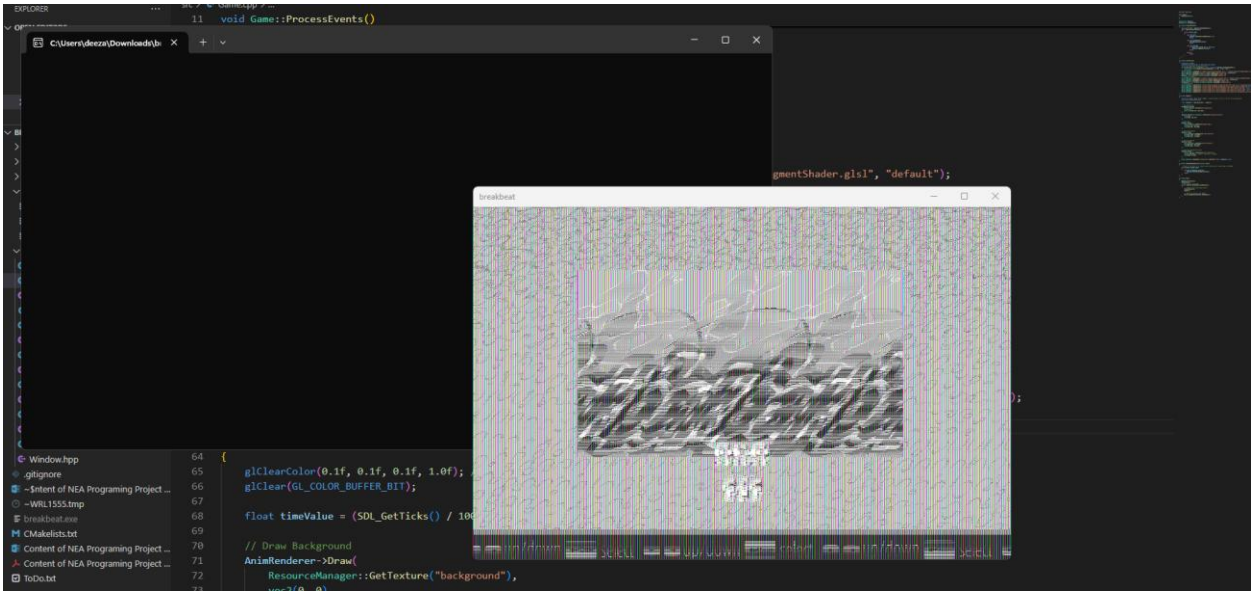
Upon testing the program, the window was correctly rendered and there were no console errors. However, the start menu user interface had completely disappeared and the only interface remaining was the background interface. This is the test result that was mentioned in the test data of my design. This shows that the boundary test data of passing in shader source code with logic errors for the compile() procedure was successfully met.



To continue testing the test data in design I tested the Generate() function's boundary test data by passing in correct image data but leaving the alpha values for the images to be incorrectly set. I did this my setting the alpha Boolean variable to false when loading the textures using the Resource Manager class during the Initialize() procedure. This should yield rendered textures but with visual defects. This is because all of my images being loaded in are in the .PNG texture format and have transparent backgrounds.

```
52    ResourceManager::LoadTexture("\\assets\\png\\breakbeat-background-dots-dots-cropped.png",false,"background-dots");
53    ResourceManager::LoadTexture("\\assets\\png\\breakbeat-background-arrows-squares-edit-cropped.png",false,"background");
54    ResourceManager::LoadTexture("\\assets\\png\\start menu\\breakbeat-start-menu-logo-2x-new.png", false, "game-logo");
55    ResourceManager::LoadTexture("\\assets\\png\\start menu\\breakbeat-start-menu-start-button.png", false, "start-button");
56    ResourceManager::LoadTexture("\\assets\\png\\start menu\\breakbeat-start-menu-exit-button.png", false, "exit-button");
57    ResourceManager::LoadTexture("\\assets\\png\\start menu\\breakbeat-start-menu-user-navigation-assist.png", false, "start-menu-ua");
```

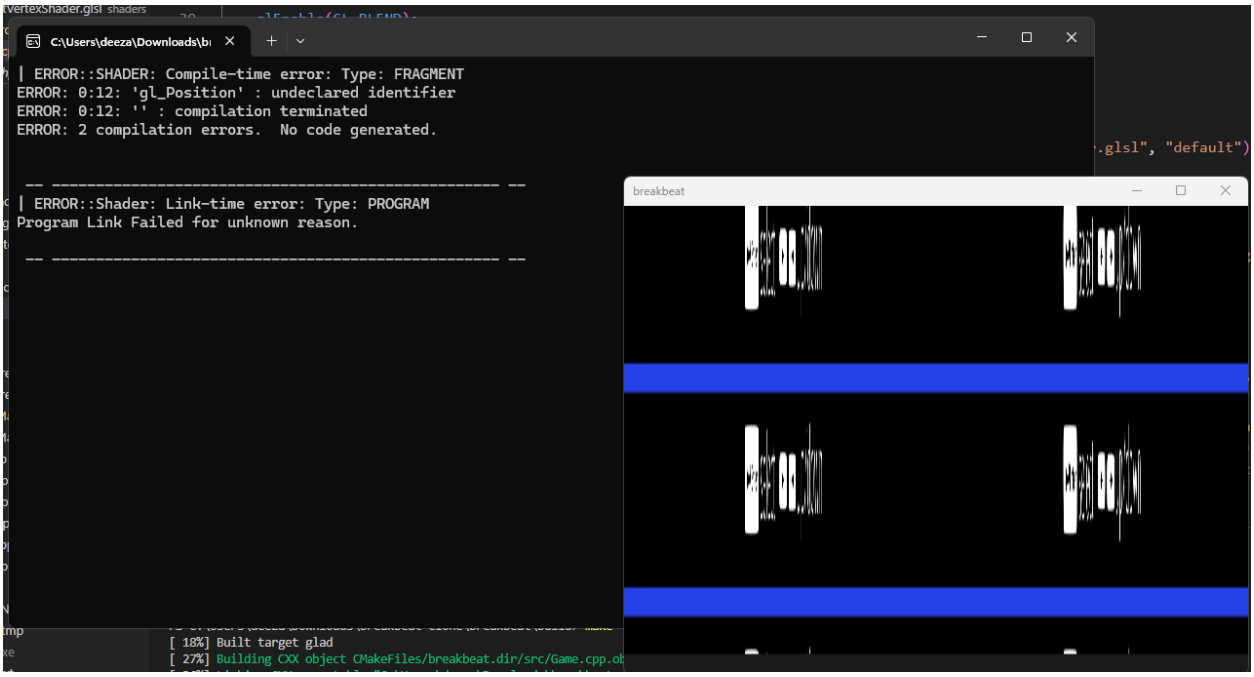
Upon testing this test data, the test result shows that there are no immediate console errors and texturized quads are being rendered. However, there are clear visual defects in the textures. This was the test result as described in the test data shown in design. All of this evidence suggests that the Generate() function is clearly working as intended.



I then continued to test the test data for the resource manager class as discussed in design. I first tested the test data of using the same vertex shader path for the fragment shader path when passing in the shader paths. I tested this by setting the fragment shader path for the LoadShader() procedure for the resource manager to the same path as the vertex shader. This will inwardly test the loadShaderFromFile() function within the LoadShader() procedure.

```
45    ResourceManager::LoadShader("\\shaders\\DefaultVertexShader.glsl", "\\shaders\\DefaultVertexShader.glsl", "default");
46    ResourceManager::GetShader("default").Use().SetMatrix4("projection", projection);
47    ResourceManager::GetShader("default").Use().SetInteger("image", 0);
48    Renderer = new GUIRenderer(ResourceManager::GetShader("default"));
```

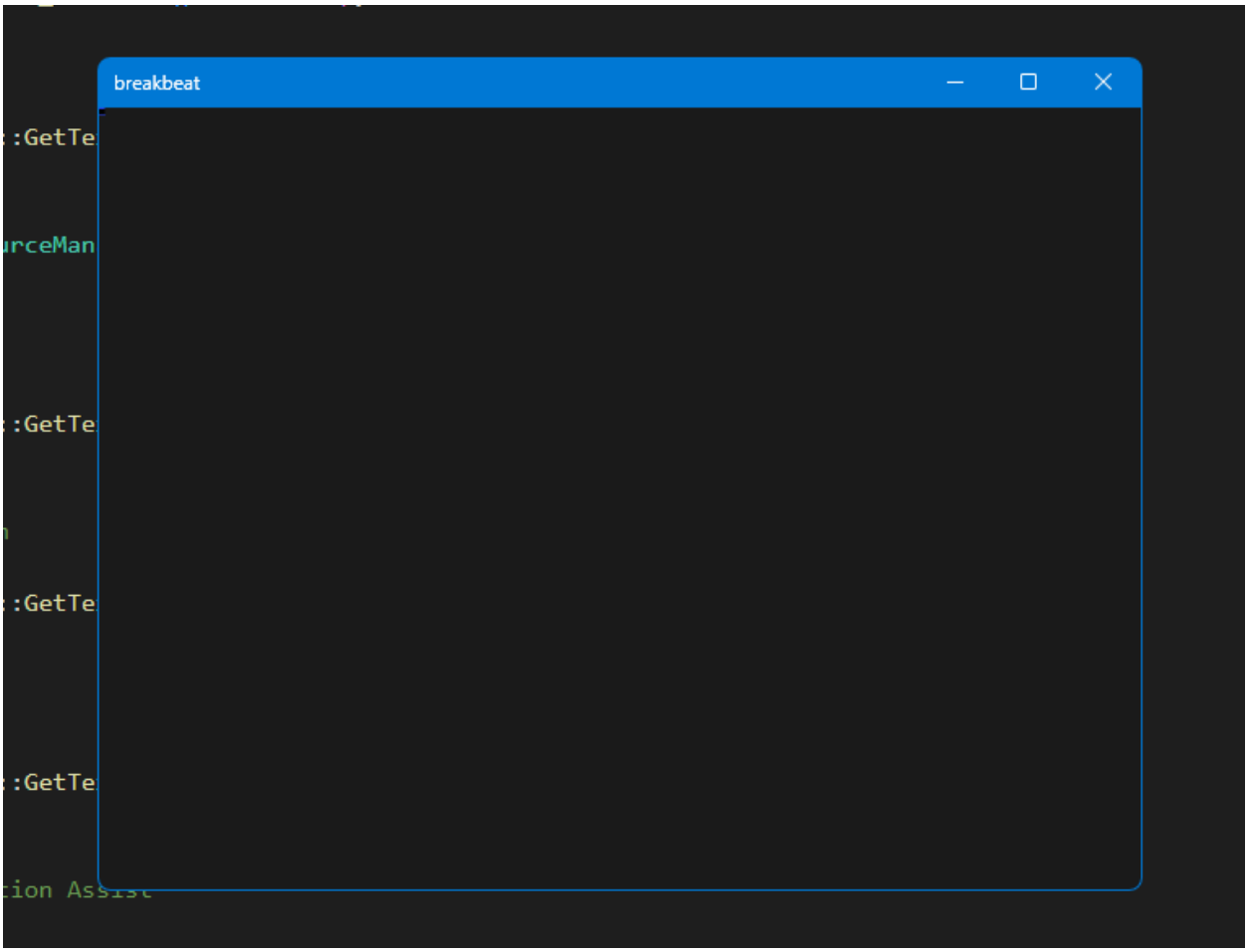
Upon testing the program, the result showed that there were actually immediate compile-time errors when running the program and there were incorrect textures being renderered for the animated background. This was slightly different to the predicted test result of the as discussed in design but still correlates to the test results by there being immediate runtime errors. This evidence suggests that loadShaderFromFile() is working as intended.



Finally, I began testing the Draw() function as discussed in design. Most of the tests that were discussed in design were successfully passed due to user interface correctly rendering. However, I tested the remaining test data. I first tested passing the Draw() procedure function without any parameters except the actual texture being rendered. The test result should resort to the Draw() function drawing the object based on the default parameters as discussed in design. These default parameters should include the texture being 10x10 pixels and the objects all in the top left corner.

```
63  void Game::Render()
64  {
65      glClearColor(0.1f, 0.1f, 0.1f, 1.0f); // Ensure alpha is set to 1.0f for solid backgroun
66      glClear(GL_COLOR_BUFFER_BIT);
67
68      float timeValue = (SDL_GetTicks() / 1000.0f);
69
70      // Draw Background
71      AnimRenderer->Draw(
72          ResourceManager::GetTexture("background")
73      );
74
75      Renderer->Draw(ResourceManager::GetTexture("background-dots")
76      );
77
78      // Draw Logo
79      Renderer->Draw(
80          ResourceManager::GetTexture("game-logo")
81      );
82
83      // Draw Start Button
84      Renderer->Draw(
85          ResourceManager::GetTexture("start-button")
86      );
87
88      // Draw Exit Button
89      Renderer->Draw(
90          ResourceManager::GetTexture("exit-button")
91      );
92
93      // Draw User Navigation Assist
94      Renderer->Draw(
95          ResourceManager::GetTexture("start-menu-ua")
96      );
97
98      ResourceManager::GetShader("background").SetFloat("time",-timeValue,true);
99  }
```

Upon testing, the test case was exactly as described in the test result; All the textures were rendered as 10x10 rectangles and were all situated in the top left corner of viewport. This evidence suggests that the Draw() function was working as intended.


breakbeat

This concluded the first intermediate phase of testing the user interface. For the rest of the development, I proceeded to work on implementing the functionality of the start menu's user interface.

Development Continued

To begin the development of the start menu's functionality I first began with making the user interface for the start and exit button correctly highlighted. This is to ensure that the user will be able to see what menu option they are navigating during the first opening of the adaptation. My justification for doing is to increase the intuitiveness of my adaptation by ensuring that any first-time user will clearly be able to learn the input controls to navigate my adaptation. Increasing the overall intuitiveness of my design will also contribute to adding to the modern useability that was described in during analysis.

To do this I needed a method to animate the interface's color/fragment over time and continually. Therefore, I decided to take the same approach I used for animating the background of using the timeValue variable for incrementing the properties of the rendered object over time. However, this time I used the time timeValue variable will to make use of the color parameter that was written into the Draw() function. I did this by passing the

timeValue variable into the color parameter in the Draw() function for the start and exit menu button. This involved passing in a vec3 data type to represent the RGB values of the texturized quad's fragment. I passed the timeValue on the third values of the vec3. This represents the B (Blue) values of the RGB values. I left the values of R (Red) and G (Green) as 1. Passing 1 into R and G will keep the values of R and G value white whilst the B (Blue) value is changing. This will give the text object a yellow color. My justification for this is to ensure that highlight color is clearly contrasting from the other user interface colors.

Afterwards, to achieve the continual highlighting of the start menu text I implemented the use of the sin() function. My justification for using the sin() function is because the mathematical properties of the sine function will always cause values of timeValue to oscillate between -1 and 1 indefinitely, thus giving the desired "looping" effect required to animate the color of the object.

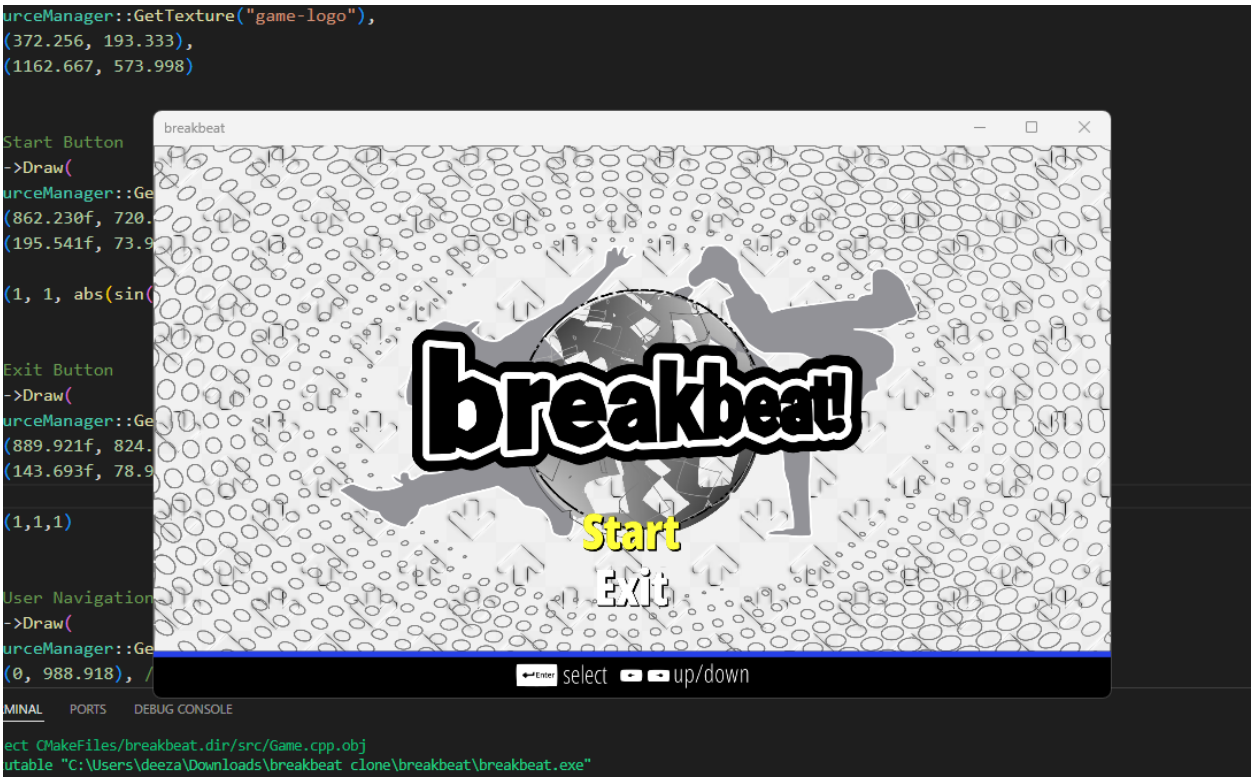
Finally, to make sure the time frame between each oscillation is even, I used the abs() (absolute value) function to ensure that the values of the RGB values will never be negative. This is because the sine function naturally oscillates between -1 and 1. However, all RGB values below 0 have no effect on the color. Therefore, during the time the output of the sin(timeValue) function is below -1, the color will remain completely yellow until the value of sin(timeValue) becomes positive again. Thus, it will cause the color of the start-button to remain yellow for a longer period before returning to its white color. Therefore, using the abs() function will make sure that the value of B will immediately begin to increase, as soon as it reaches 0.

```
89      // Draw Start Button
90      Renderer->Draw(
91          ResourceManager::GetTexture("start-button"),
92          vec2(862.230f, 720.000f),
93          vec2(195.541f, 73.988f),
94          0,
95          vec3(1, 1, abs(sin(timeValue)))
96      );
97
98      // Draw Exit Button
99      Renderer->Draw(
100         ResourceManager::GetTexture("exit-button"),
101         vec2(889.921f, 824.483f),
102         vec2(143.693f, 78.957f),
103         0,
104         vec3(1,1,1)
105     );
```

I then began another short intermediate test phase to test the implementation of this feature.

Testing Phase Two

Upon compiling my program, the user interface successfully rendered the start menu with the “flashing” start-menu. The results of this test can also be seen in the “Test Phase Two Video”.



GUIRenderer Redesign

Upon reviewing my code and thinking ahead for the remainder of my system, I concluded my initial design of my GUIRenderer class system was inefficiently designed. This is due to the aspect of needing to manually update the parameters of the objects being rendered by passing in variables into the Draw() function. It would be more efficient to have each object being rendered as a separate object with built-in functions to update their properties such as position, size, color and scale. As well as that, if each object being rendered is independent, it will make management of their attributes easier and more direct. This is especially useful for the later stages of my game development in which I will each object being rendered to behave differently. An example use case in later stages of game development can include menu selection screens where interfaces such as the song selection interface will need to be highlighted and move to show indication of the song being selection

Sprite Renderer Class

Therefore, I decided to remake the GUIRenderer class into a sprite renderer system where each object being rendered will be represented as a “sprite” with its own position, size, scale, color and texture position attributes that can all be accessed directly. These attributes can also be updated via their utility functions such as Move(), SetColor(), SetSize(), etc... My overall justification for implementing this system is to allow the feature

of updating object's properties in other sections of code outside of the Render() loop. This will be essential for having input handling and implementing the rest of the functionality for the user interfaces.

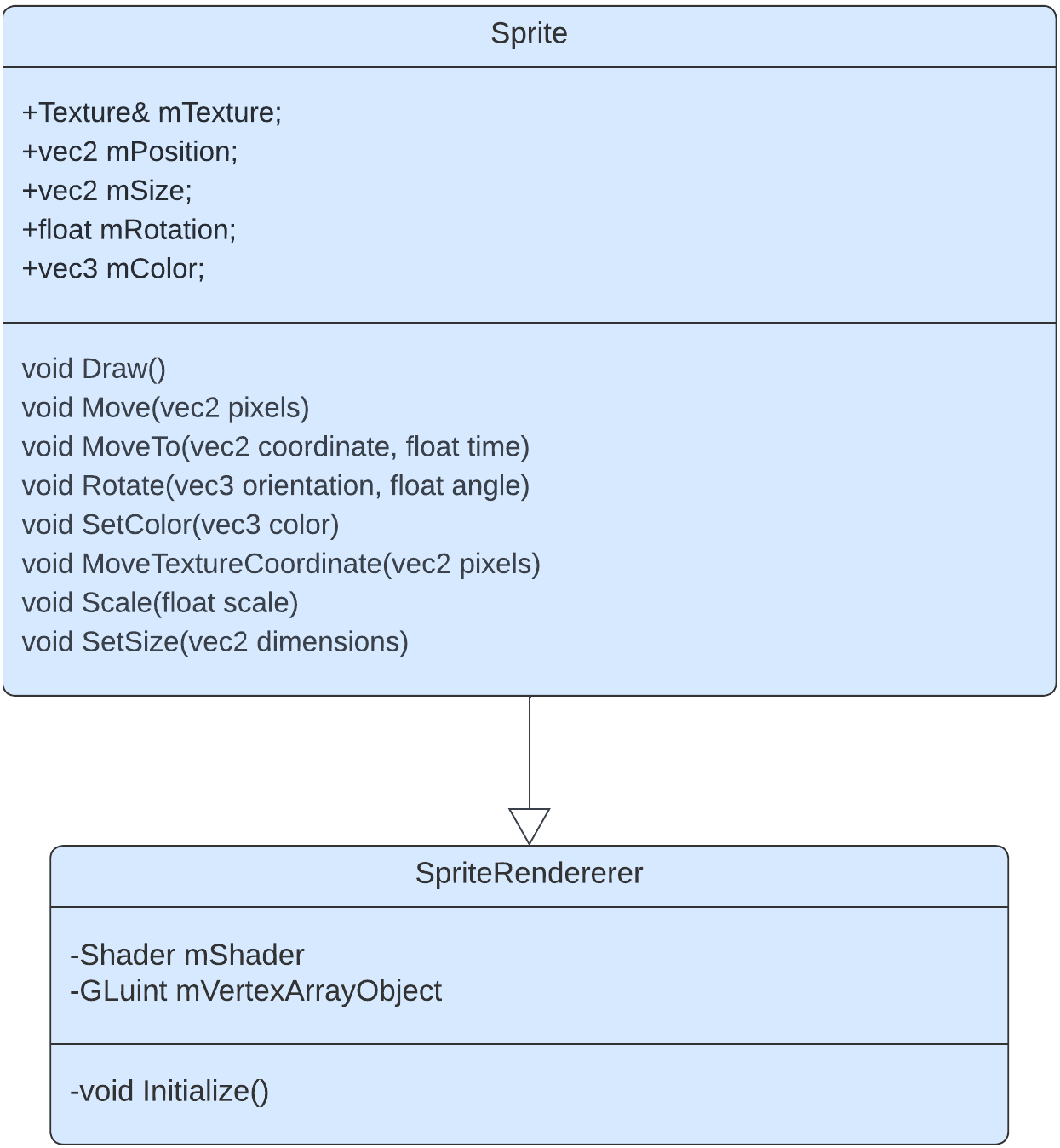
The Sprite Renderer system will still leverage the already made shader, texture, resource manager, game and window system. It will remain like the GUI Renderer class except the main difference is that it will be the derived class in which all "Sprite" objects are inherited, and it has no Draw() function. The class is still responsible for initial initialization of the vertex buffers and vertex array objects that will be used to draw the quads required for rendering. However, each derived sprite object will have its own use of the Draw() function.

Sprite Class

As mentioned above, the Sprite objects themselves will not require parameters to be passed into the Draw() function due to each object having their properties that can be updated independently. This means that the only purpose of the Draw() function will to be draw the quads that make up the sprites and substitute the position, size, color, and rotation into the quad attributes themselves. This is the basis of my new GUI system.

Class Diagram

Below is the diagram of the sprite diagram and the derived sprite class. The sprite class diagram shows its member attributes that will be substituted into the object's Draw() function.



Pseudocode for Draw() function

As mentioned above, the Draw() function will take in no parameters and use the values of the sprite's positions, size, rotation, texture and color to determine the attributes of the the quad being rendered.

```
public procedure Draw()
    model = mat4(1.0f)
```

```
    model = translate(model, vec3(this.mPosition, 0.0f));
    model = translate(model, vec3(0.5f * sprite.mSize.x, 0.5f *
mSize.y, 0.0f))
    model = rotate(model, radians(this.mRotation), vec3(0.0f,
0.0f, 1.0f))
    model = translate(model, vec3(-0.5f * this.sprite.mSize.x, -
0.5f * this.mSize.y, 0.0f));
    model = scale(model, vec3(this.mSize, 1.0f))


    this.mShader.Use()
    this.mShader.SetMatrix4("model", model)
    this.mShader.SetVector3f("color", this.mColor)


    glActiveTexture(0)


    glBindVertexArray(this.mVertexArrayObject)
    glDrawArrays(GL_TRIANGLE_STRIP, 0, 6)
endprocedure
```

Pseudocode for Move() function

Below is the pseudocode of the many utility functions. The basis of the utility functions will remain the same; It involves updating the sprites attributes based on the parameters given.

```
public procedure Move(pixels)
    this.mPosition += pixels
endprocedure
```

Sprite Renderer Prototype 1

To begin the development of the new Sprite renderer system I first began by rewriting the header file of the old GUIRenderer class and replacing the function definitions with the function definitions as shown in the sprite renderer class. This involved renaming the "GUIRenderer" .cpp file to "SpriteRenderer". This also involved defining the member variables for the class to be protected. This is because in C++, for any derived classes to have access to its base class's member variables, the member variables must be defined as protected.

src > SpriteRenderer.hpp > ...

```
1   #ifndef SPRITERENDERER_HPP
2   #define SPRITERENDERER_HPP
3
4   #include <glm/glm.hpp>
5   #include <glm/gtc/matrix_transform.hpp>
6   #include "Texture.hpp"
7   #include "Shader.hpp"
8
9   using namespace glm;
10
11  class SpriteRenderer {
12  public:
13      // Constructor, Destructor, and other necessary member functions
14      SpriteRenderer(Shader& shader);
15      virtual ~SpriteRenderer();
16
17      // Virtual Draw method for flexibility
18      void Initialize();
19  protected:
20      Shader mShader;
21      GLuint mVertexArrayObject;
22  };
23
24  #endif // SPRITERENDERER_HPP
```

Afterwards, I created the Sprite class header that is derived from the SpriteRenderer class. I then wrote the function definitions for the class as shown in the class diagram.

src > Sprite.hpp > Sprite > Rotate(vec3, float)

```
#ifndef SPRITE_HPP
#define SPRITE_HPP

#include "SpriteRenderer.hpp"
#include "ResourceManager.hpp"

using glm::radians;
using glm::perspective;

class Sprite : public SpriteRenderer {
public:
    // Constructor
    // Remove the default arguments for previous parameters
    Sprite(Texture& texture, vec2 position, vec2 size, float rotate, vec3 color, Shader& shader, bool perspective = false);

    // Override Draw method
    virtual void Draw();
    void Move(vec2 pixels);
    void MoveTo(vec2 coordinate, float time);
    void Rotate(vec3 orientation, float angle);
    void SetColor(vec3 color);
    void MoveTextureCoordinate(vec2 pixels);
    void Scale(float scale);
    void SetSize(vec2 dimensions);

    // Additional member functions specific to Sprite...
private:
    Texture& mTexture;
    vec2 mPosition;
    vec2 mSize;
    float mRotation;
    vec3 mColor;
    bool mPerspective;
};

#endif // SPRITE_HPP
```

Afterwards I removed the Draw() function from the GUIRenderer class and renamed the class handles to and the class its self to “SpriteRenderer.” The Initialize() procedure remains the same.

src > SpriteRenderer.cpp > Initialize()

```cpp
#include "SpriteRenderer.hpp"

SpriteRenderer::SpriteRenderer(Shader& shader)
{
    this->mShader = shader;
    this->Initialize();
}

SpriteRenderer::~SpriteRenderer()
{
    glDeleteVertexArrays(1, &this->mVertexArrayObject);
}

void SpriteRenderer::Initialize()
{
    unsigned int vertexBufferObject;
    float vertices[] = {
        // Position       // Tex Coords
        0.0f, 1.0f,      0.0f, 1.0f,
        1.0f, 0.0f,      1.0f, 0.0f,
        0.0f, 0.0f,      0.0f, 0.0f,

        0.0f, 1.0f,      0.0f, 1.0f,
        1.0f, 1.0f,      1.0f, 1.0f,
        1.0f, 0.0f,      1.0f, 0.0f
    };

    glGenVertexArrays(1, &this->mVertexArrayObject);
    glGenBuffers(1, &vertexBufferObject);

    glBindBuffer(GL_ARRAY_BUFFER, vertexBufferObject);
    glBufferData(GL_ARRAY_BUFFER, sizeof(vertices), vertices, GL_STATIC_DRAW);

    glBindVertexArray(this->mVertexArrayObject);
    // Position attribute
    glEnableVertexAttribArray(0);
    glVertexAttribPointer(0, 2, GL_FLOAT, GL_FALSE, 4 * sizeof(float), (void*)0);
    // Texture coordinate attribute
    glEnableVertexAttribArray(1);
    glVertexAttribPointer(1, 2, GL_FLOAT, GL_FALSE, 4 * sizeof(float), (void*)(2 * sizeof(float)));

    glBindBuffer(GL_ARRAY_BUFFER, 0);
    glBindVertexArray(0);
}
```

Aftwards, I then wrote the class constructor for the Sprite class and wrote the source code for the Draw() function as shown in the pseudocode.

```
src > Sprite.cpp > MoveTo(vec2, float)
#include "Sprite.hpp"

// Constructor that initializes sprite properties
Sprite::Sprite(Texture& texture, vec2 position, vec2 size, float rotate, vec3 color, Shader& shader, bool perspective)
    : SpriteRenderer(shader),  // Call the base class constructor with the shader
      mTexture(texture),
      mPosition(position),
      mSize(size),
      mRotation(rotate),
      mColor(color),
      mPerspective(perspective)
{
}

// Overridden Draw method for Sprite
void Sprite::Draw()
{
    mat4 model = mat4(1.0f);
    model = translate(model, vec3(mPosition, 0.0f));
    model = translate(model, vec3(0.5f * mSize.x, 0.5f * mSize.y, 0.0f));
    model = glm::rotate(model, glm::radians(mRotation), vec3(0.0f, 0.0f, 1.0f));
    model = translate(model, vec3(-0.5f * mSize.x, -0.5f * mSize.y, 0.0f));
    model = scale(model, vec3(mSize, 1.0f));

    this->mShader.Use();
    this->mShader.SetMatrix4("model", model);
    this->mShader.SetVector3f("color", mColor);

    glActiveTexture(GL_TEXTURE0);
    mTexture.Bind();

    glBindVertexArray(this->mVertexArrayObject);
    glDrawArrays(GL_TRIANGLE_STRIP, 0, 6);
    glBindVertexArray(0);
}
```

I then started to implement the utility functions that will be used to update each sprite's attributes. I first began to write the utility functions for updating the position of the sprite. I implemented an extra MoveTo() function to allow time-based movement. My justification for doing this is to think ahead for the future main gameplay system of my game in which notes will be rendered over time.

```
// utility Functions
void Sprite::Move(vec2 pixels) {
    mPosition += pixels;
}

void Sprite::MoveTo(vec2 coordinate, float time) {
    // time-based movement (interpolation)
    vec2 increment = (coordinate - mPosition) / time;
    mPosition += increment;  // Simplified for one update
}
```

I then implemented a rotation function which will consider both perspectives of rotation. This means that function will allow both 2D and 3D rotation based on whether the sprite's mPerspective variable is true. If the variable is true, then the shader used will switch to

using perspective projection and the object will be rendered as a 3D object and rotated in the 3D plane. This links backs to analysis section in where I stated 3D aspects will be included in the user interface to appeal to the older audience.

```
// More complex rotation function that considers perspective rotation
void Sprite::Rotate(vec3 orientation, float angle) {
    mat4 model = mat4(1.0f);

    if (mPerspective)
    {
        // Switch to perspective shader
        // This part assumes you have a mechanism to change shaders in the SpriteRenderer
        mat4 view = lookAt(vec3(0.0f, 0.0f, 3.0f), vec3(0.0f), vec3(0.0f, 1.0f, 0.0f));
        model = rotate(model, glm::radians(angle), orientation);
        this->mShader.SetMatrix4("view", view);
    } else
    {
        // Use orthographic shader for normal 2D rotation
        model = rotate(model, radians(angle), orientation);
    }

    this->mShader.SetMatrix4("model", model);
}
```

I then finished implementing the rest of the utility functions for Sprite class. This mostly involved passing in parameters and incrementing the Sprite's attribute by the parameter.

```
void Sprite::SetColor(vec3 color)
{
    this->mColor=color;
}

void Sprite::MoveTextureCoordinate(vec2 pixels)
{
    this->mShader.SetVector2f("textureCoordiante", pixels);
}

void Sprite::Scale(float scale)
{
    this->mSize *= scale;
}

void Sprite::SetSize(vec2 dimensions)
{
    this->mSize = dimensions;
}
```

Afterwards, I began updating the Game class header file to include the new SpriteRenderer System. This involved replacing the GUIRenderer object with the SpriteRenderer system.

Furthermore, I created a neseted <unordered_map> data structure (hash table) that will hold the groups of sprites to be rendered based on their game state. This hash table data structure will store hash tables filled sprites within itself. This means that each sprite will be able to be easily identified via the key given to it when it is stored within the hash table. This also means that each sprite will be able to be accessed in any section of the Game .cpp source code. My justification for this is to allow sprites to updated outside the Render() loop and to be accessed via their name. This will also make handling sprites themselves more efficient.

Lasty, I created an enumerable listing he game states of the game. These enumerated types will hold the list of the different game states my game can be in and will be used for comparison during input handling.

src > Game.hpp > GAME_HPP

```cpp
/*

This is the header file for the Game.cpp which is responsible
for the uber glass in which all aspects of the adaptation are
integrated in

*/

#ifndef GAME_HPP
#define GAME_HPP

enum class GameState {
    START_MENU,
    GAMEPLAY,
    PAUSE_MENU,
    OPTIONS_MENU,
    // Add more states as needed
};

// Include the Window.hpp file to allow creation of a Window object
#include "Window.hpp"
#include <unordered_map>
#include <vector>
#include <memory>
#include "ResourceManager.hpp"
#include "Sprite.hpp"

#include <vector>
#include <memory> // For smart pointers

class Game
{
public:
    void Initialize();
    void Run();
    void Update();
    void ProcessEvents();
    void Render();
    void HandleWindowEvent(SDL_Event&);
    Game();
private:
    // Declare mWindow as a member variable
    GameState mCurrentGameState;
    Window mWindow;
    std::unordered_map<GameState, std::map<string, std::unique_ptr<Sprite>>> spriteGroups;
};

#endif
```

I then began the process of redefining the process of rendering objects by instantiating the sprites themselves. This involved creating the sprite objects by passing the texture, position, size, color and shader variables and appending them to the hash table. I first

began by redefining the sprites for all of the components of the user interface used for the start menu and then appending them to the nested hash table under the hash table key of the enumerated type START_MENU. This also involved taking all the previous parameters that used in the previous Draw() function system and copying them into the sprite attributes

```
    // Set initial game state to start menu
    mCurrentGameState = GameState::START_MENU;

    // Load Sprites
    // Clear previous sprites
    spriteGroups.clear();

    // Initialize sprites for the START_MENU state

    spriteGroups[GameState::START_MENU]["background"] = (std::make_unique<Sprite>(
        ResourceManager::GetTexture("background"),
        vec2(0, 0),
        glm::vec2(1960.178, 1033.901),
        0.0f,
        vec3(1.0f),
        ResourceManager::GetShader("background")
    ));


    spriteGroups[GameState::START_MENU]["background-dots"] = (std::make_unique<Sprite>(
        ResourceManager::GetTexture("background-dots"),
        vec2(0, 0),
        vec2(1920, 994.167),
        0.0f,
        vec3(1.0f),
        ResourceManager::GetShader("default")
    ));


    spriteGroups[GameState::START_MENU]["game-logo"] = (std::make_unique<Sprite>(
        ResourceManager::GetTexture("game-logo"),
        vec2(372.256, 193.333),
        vec2(1162.667, 573.998),
        0.0f,
        vec3(1.0f),
        ResourceManager::GetShader("default")
    ));

    spriteGroups[GameState::START_MENU]["start-button"] = (std::make_unique<Sprite>(
        ResourceManager::GetTexture("start-button"),
        vec2(862.230f, 720.000f),
        vec2(195.541f, 73.988f),
        0.0f,
        vec3(1.0f),
        ResourceManager::GetShader("default")
    ));
```

```
    spriteGroups[GameState::START_MENU]["exit-button"] = (std::make_unique<Sprite>(
        ResourceManager::GetTexture("exit-button"),
        vec2(889.921f, 824.483f),
        vec2(143.693f, 78.957f),
        0.0f,
        vec3(1.0f),
        ResourceManager::GetShader("default")
    ));

    spriteGroups[GameState::START_MENU]["start-menu-ua"] = (std::make_unique<Sprite>(
        ResourceManager::GetTexture("start-menu-ua"),
        vec2(0, 988.918),
        vec2(1920, 91.082),
        0.0f,
        vec3(1.0f),
        ResourceManager::GetShader("default")
    ));
}
```

I then began to rewrite the Render() function's source code to draw each new sprite. This involved using an if statement to search for the hash table within the nested hash table that has the key to the current game state and using a for each loop to iteratively loop through each sprite and call its Draw() function. This means that each sprite will be rendered independently. This is the main purpose of making the new sprite renderer system.

```
void Game::Render()
{
    glClearColor(0.1f, 0.1f, 0.1f, 1.0f); // Ensure alpha is set to 1.0f for solid background
    glClear(GL_COLOR_BUFFER_BIT);

    if (mSpriteGroups.find(mCurrentGameState) != mSpriteGroups.end())
    {
        for (auto& [key, sprite]: mSpriteGroups[mCurrentGameState])
        {
            sprite->Draw();
        }
    }
```

This section of development marks the core aspect of the new SpriteRenderer system. The rest of the development of the new system will involve creating new utility functions for edifying sprites. These utility functions will include functions for specific animations and for editing the internal attributes of the sprites.

Sprite Renderer Prototype 2

Upon thinking about the rest of the implications of my new system. I decided to add a few improvements to my new system to increase the efficiency and modularity of the source code and the overall system.

The first improvement I implemented was making the hash tables to store the sprites to be inside the sprite renderer class instead of the game class. This is because I am shifting the design of the sprite renderer system to be based off composition rather than inheritance. My justification for this change is for the purpose of encapsulation of all things to do with the sprites to be within their associated classes. I also changed the name of the sprite hash table to “mCurrentlyRenderedSprites” to store the sprites that are currently being used in rendering and added in another hash table to store the default positions of the sprites. This will be for loading the default states of sprites upon game state transitions.

```
21  class SpriteRenderer {
22  public:
23      // Constructor, Destructor, and other necessary member functions
24      SpriteRenderer();
25      ~SpriteRenderer();
26
27      void Initialize();
28
29      // Hash table to store the sprites currently in use in the render loop
30      unordered_map<GameState, map<string, Sprite*>> mCurrentlyRenderedSprites;
31
32  private:
33      // Vertex array object that will be used across all sprites
34      GLuint mVertexArrayObject;
35      // Hash table to store the defualt positions of the sprites
36      // This will be used to restore sprites to their defualt states
37      // This will be when loading different games states
38      unordered_map<GameState, map<string, Sprite*>> mDefaultSprites;
39  };
```

The next change I made to my system was creating functions for the sprite renderer class that will involve simplifying the creation of sprites down to a few function calls. This is what was discussed during analysis and was the main intention of the GUI design. This involved defining the CreateSprite(), LoadDefaultSprite() and adding an extra hash table for loading the default values of sprites into the game.

```
29  void Initialize();
30  Sprite* CreateSprite(GameState gameState, string name, Texture& texture, vec2 position, vec2 size, float rotate, vec3 color,Shader& shader, bool perspective, vec2 texturePositon = vec2(0,0), float textureScale = 1);
31
32  void DrawSprites(GameState gameState);
33  void LoadDefaultSprites(GameState gameState);
34
35
36  // Hash table to store the sprites currently in use in the render loop
37  unordered_map<GameState, map<string, Sprite*>> mCurrentlyRenderedSprites;
38  unordered_map<GameState, map<string, Sprite*>> mDefaultSprites;
39
```

Then I continued development by making the Sprite class part of the composition of the Sprite Renderer. This involved removing the inheritance of the SpriteRenderer class from the Sprite class and using the parameters that were in the CreateSprite() procedure to determine the Sprite object.

```
class Sprite
{
public:
    // Constructor
    // Remove the default arguments for previous parameters
    Sprite(Texture& texture, vec2 position, vec2 size, float rotate, vec3 color,Shader& shader, bool perspective = false,  vec2 texturePositon = vec2(0,0), float textureScale = 1);

```

I then began to develop the CreateSprite() procedure. The procedure to create new sprites involved taking in the sprite's texture, position, size and other variables such as rotation. It then involved setting the Sprite's vertex array object that will render the quads for the sprite to the vertex array object that has been initialized in SpriteRenderer. Once the sprite has been created, it is stored in the sprites hash table. This will allow the sprite easy access to it via its key.

```
Sprite* SpriteRenderer::CreateSprite(GameState gameState, string name, Texture& texture, vec2 position, vec2 size, float rotate, vec3 color, Shader& shader, bool perspective, vec2 texturePositon, float textureS
{
    // Create a unique_ptr to a new Sprite instance
    auto sprite = new Sprite(texture, position, size, rotate, color, shader, perspective, texturePositon, textureScale);

    // Store the unique_ptr in the default sprite map
    GLuint& VAO = this->mVertexArrayObject;
    sprite->mVertexArrayObject = VAO;
    mDefaultSprites[gameState][name] = sprite;

    // Return the raw pointer for convenience if needed
    return mDefaultSprites[gameState][name];
}
```

Afterwards, I wrote the functions that will be used to draw the sprites and load the default sprites. The DrawSprites() procedure is vitally important as it the key difference between the old GUIRenderer system and the new system. This is because as explained earlier, the sprites are updated without passing in any variables into the draw() procedure. The actual variables that will be used to render the sprites can be independently updated via utility functions.

The LoadDefaultSprites() will be used to load the default states of sprites between transisitong to game states. The reason for is because the currently rendered Sprites states will be dynamically updated within the hash table and their attributes will differ from their original states. However, the sprites need to revert to their normal states when transitioning from one game state to another e.g. start menu to main menu. Thererfore a justifiable approach to do this is to store the default values of the sprites and then load then on each transition of game state.

```
96  void SpriteRenderer::DrawSprites(GameState gameState)
97  {
98      // Iterate through each sprite in sprite hash table and call its draw function
99      for (auto& [key, sprite] : mCurrentlyRenderedSprites[gameState])
100     {
101         sprite->Draw(); // Use sprite directly
102     }
103 }
104
105 void SpriteRenderer::LoadDefaultSprites(GameState gameState)
106 {
107     // Iterate through each sprite and copy it to the currently rendered sprites hash table
108     for (auto& [key, sprite] : mDefaultSprites[gameState])
109     {
110         // Use the copy constructor to create a copy of the sprite
111         mCurrentlyRenderedSprites[gameState][key] = sprite;
112     }
113 }
```

Afterwards, I wrote the function definitions for simplifying the sprite creation process and began to write out the function procedures to allow games state transitions. This involved defining the subroutines in the header file. I also added an UpdateSprites() procedure. This is because on top of adding a Draw() function for rendering sprites. I decided to add an Update() procedure into the sprites for future use to allow one-time animations and continual animations.

```
41      Sprite* GetSprite(GameState, string);
42      void UpdateSprites(GameState);
43      void LoadDefaultSprites(GameState gameState);
44
45      void CheckForTransitionState();
46      void TransitionToGameState(GameState newGameState);
47
48      void Transition(GameState newGameState);
```

I then continued to define more utility functions and began to define animation functions for the sprite class that will be used in dynamically updating the sprite as it being rendered. This involved creating functions and procedures that will be used to get the sprite's attributes and set the value of the sprite's attributes. These functions will mostly involve returning the value of the attribute and setting it to the value of parameter passed in.

```
19      //  Draw method
20      void Draw();
21
22      // Utility functions
23
24      void Move(vec2 pixels);
25      void Rotate(vec3 orientation, float angle);
26      void MoveTextureCoordinate(vec2 pixels);
27      void Scale(float scale);
28
29      // Setters for the sprite attributes
30      void SetPosition(const vec2& position);
31      void SetSize(const vec2& size);
32      void SetRotation(float rotate);
33      void SetColor(const vec3& color);
34      void SetShader(Shader& shader);
35      void SetTextureScale(float scale);
36
37      /*  Animation functions */
38      // Will be used to perform transformations and changes to sprite over time
39      void MoveTo(vec2 coordinate, float time);
40      void SetMoveTo(bool state, vec2 coordinate, float time);
41
42      // Procedure to darken the sprites color over time
43      void SetDarken(bool enable, float darkenTime = 1.0f);  // New method to start/stop darkening
44      void Darken();  // Existing method to handle darkening process
45
46      // Procedure to brighten the sprutes color from the darkened sate over time
47      void SetBrighten(bool enable, float invertTime = 1.0f);  // New method to start/stop inverting
48      void Brighten();  // Existing method to handle inversion (lightening
49
50      // Getters for the sprite attributes
51      vec2 GetPosition() const;
52      vec2 GetSize() const;
53      float GetRotation() const;
54      vec3 GetColor() const;
55      Shader& GetShader() const;
56      bool GetBrightenState();
57      bool GetDarkenState();
58
```

Aswell as defining the subroutines for dynamically updating the sprites. I also created the member Boolean variables that will be needed to perform the sprite updates once within the update loop. This is because, the sprites will be updated on every frame due to its Draw() and Update() functions be called each time. Thus, to prevent performance issues and to render animations only once, a multitude of Boolean variables will be required to track the state the of the animation and the states of the attributes within the sprite. The most notable Boolean variables is the mHasUpdated variable that will only evaluate to true once all other state variables such as mIsMovingTo and mDarkening are all false. This is because these variables will only be true during the sprites being animated. My justification for adding such Booleans is because setting them to false during the sprites update loops will be allow the part of code that is updating the sprite to be checked again

only if the Boolean is true. This will allow seamless one time updates of sprites despite every sprite being updated on frame.

```
64      // Additional member functions specific to Sprite...
65  private:
66      Texture& mTexture;
67      vec2 mTexturePosition;
68      float mTextureScale;
69      vec2 mPosition;
70      vec2 mSize;
71      float mRotation;
72      vec3 mColor;
73      Shader& mShader;
74      // Used to determine whether the sprite is being rendererd as a 3D object
75      bool mPerspective;
76
77      // Universal update vairable to handle sprite state updates
78      bool mHasUpdated;
79
80      float mCurrentTime;
81      float mTotalTime;
82
83      /* Animation state varibles */
84
85      // Start and end coordinates for the MoveTo() function
86      vec2 mStartCoordinate;
87      vec2 mEndCoordinate;
88      int mDistanceBetween;
89      vec2 mIncrement;
90
91      // Animation state for MoveTo()
92      bool mIsMovingTo;
93
94      // Time varibles for the brighten and darken utility functions();
95      // Used to determine start times to time the time elapsed
96      float mDarkenStartTime = 0.0f;
97      float mBrightenStartTime = 0.0f;
98      float mDarkenTime = 1.0f;
99      float mBrightenTime = 1.0f;
100
101     // State varible to indicate when the sprite is becoming darker
102     bool mDarkening = false;
103     bool mBrightening = false;
104
105     // Store the original colors of varibles when they are being darkened/brightened
106     // Store the darkest color to lighten from
107     // Store the original color to darken from
108     vec3 mOriginalColor;
109     vec3 mDarkenedColor;
110
111     // State varible to determine when the varible is dark or bright
112     bool mDarkened = false;
113     bool mBrightened = false;
```

Afterwards, I then began to write out more of the getters and setters that will return the sprite's variables. I then implemented the utility functions for updating the sprites attributes in the update() function.

```
100  void Sprite::MoveTextureCoordinate(vec2 offset)
101  {
102      // Get the current texture position if stored, or initialize to zero
103     this->mTexturePosition += offset;
104  }
105
106  void Sprite::SetTextureScale(float scale)
107  {
108      // Get the current texture position if stored, or initialize to zero
109     this->mTextureScale = scale;
110  }
111
112  void Sprite::Scale(float scale)
113  {
114      this->mSize *= scale;
115  }
116
117   // Setters
118  void Sprite::SetPosition(const vec2& position) {
119      mPosition = position;
120  }
121
122  void Sprite::SetSize(const vec2& size) {
123      mSize = size;
124  }
```

For the consideration of the texture coordinates utility fucntion

```
126  void Sprite::SetRotation(float rotate) {
127      this->mRotation = rotate;
128  }
129
130  void Sprite::SetColor(const vec3& color) {
131      this->mColor = color;
132  }
133
134  void Sprite::SetShader(Shader& shader) {
135      this->mShader = shader;
136  }
137
138  // Getters
139  vec2 Sprite::GetPosition() const {
140      return mPosition;
141  }
142
143  vec2 Sprite::GetSize() const {
144      return mSize;
145  }
146
147  float Sprite::GetRotation() const
148  {
149      return mRotation;
150  }
151
152  vec3 Sprite::GetColor() const
153  {
154      return mColor;
155  }
156
157  Shader& Sprite::GetShader() const
158  {
159      return this->mShader;
160  }
```

I then wrote the logic for the utility function that will be used to darken the sprites over time. This involved using the SDL_GetTicks() function to instantaneously get the current time passed since the initialization of SDL. This essentially starts a timer from the moment the variable SetDarken() function is enabled. Upon enabling, another time variable that gets the instantaneous time that has passed will continually be updated. This time variable will subtract the initial value of the time passed from it. This is because the value of this variable will be greater than the initial variable. Therefore, subtracting them will yield the difference between them and give the total time that has elapsed. The time elapsed is then used as measure of the progression of the sprite to be darkened. This means that the sprite will be darkened based on how much time has passed; When exactly a second has passed the sprite will be fully darkened (The value is multiplied by a 1000 as it is all in milliseconds).

Once a second has passed the sprite will set its state variables for being darkened to false and the Update() function will stop darkening the sprite.

```
174  void Sprite::SetDarken(bool enable, float darkenTime)
175  {
176      if(!mDarkened || mBrightened && !mDarkening)
177      {
178          mDarkenTime = darkenTime;
179          mOriginalColor = mColor;
180          mDarkening = true;
181          mDarkenStartTime = SDL_GetTicks();
182      }
183  }
184
185  void Sprite::Darken()
186  {
187      float timeElapsed = SDL_GetTicks() - mDarkenStartTime;
188      float progress =  timeElapsed / (mDarkenTime * 1000);
189      mColor = mOriginalColor - progress;
190      if(mColor.x <= 0 && mColor.y <= 0 && mColor.z <= 0 || timeElapsed >= 1000)
191      {
192          mColor = vec3(0);
193          mDarkened = true;
194          mDarkening = false;
195          mBrightened = false;
196      }
197  }
```

```
234      // If the sprite is in the darkening state it will call the darken function
235      if (mDarkening)
236          Darken();
237
238      // If the sprite is the in the brightening state then brighten the function
239      if (mBrightening)
240          Brighten();
241
242      // If the sprite is fully black it has been darkened
243      if(mColor == vec3(0))
244          mDarkened = true;
245
246      // If the sprite is fully bright it has been brightened
247      if(mColor == vec3(1))
248          mBrightened = true;
249
250      // If the sprite is not in any update state it has been fully updated
251      if( (mDarkened || mBrightened) &&
252          mIsMovingTo == false)
253      {
254          mHasUpdated == true;
255      }
256  }
```

I then applied the same logic for the Brighten() function except I used a set of differently named variables that will disambiguate between brighten and darkening and incremented the value of mColor using the progress varible instead of decrementing it.

```
199  // Start or stop the Brightening process
200  void Sprite::SetBrighten(bool enable, float brightenTime)
201  {
202      if(!mBrightened || mDarkened && !mBrightening)
203      {
204          mBrightenTime = brightenTime;
205          mDarkenedColor = mColor;
206          mBrightening = true;
207          mBrightenStartTime = SDL_GetTicks();
208      }
209  }
210
211  void Sprite::Brighten()
212  {
213      float timeElapsed = SDL_GetTicks() - mBrightenStartTime;
214      float progress =  timeElapsed / (mBrightenTime * 1000);
215      mColor = mDarkenedColor + progress;
216      if(
217       timeElapsed >= 1000
218       )
219      {
220          mColor = mOriginalColor;
221          mBrightened = true;
222          mBrightening = false;
223          mDarkened = false;
224      }
225  }
```

I then began to modify the Game class to include utility functions for incorporating the newly designed sprite renderer class. This involved creating functions that will simplify the use of writing “mSpriteRender.mCurrentlyRenderedSprites.CreateSprite()” down to just CreateSprite(). Furthermore, I also defined a procedure to make use of the sprite hash tables, specifically to load the default sprites for the current game state. I also wrote the function definition to call each spite’s Update() procedure.

Another function I defined is the CalculateDeltaTime() function which will be used to calculate the time between update in the game loop.

```
34      double mDeltaTime;
35      double mLastFrame;
36
37      void CalculateDeltaTime();
38
39      void InitMenu();
40
41      Sprite* GetSprite(GameState, string);
42      void UpdateSprites(GameState);
43      void LoadDefaultSprites(GameState gameState);
```

I then wrote the procedure to calculate delta time. This involves using two variables to get the current time elapsed in milliseconds. The second variable is declared after the first and the difference between the second and first is the value of time that has passed between each update. Afterwards the second variable is set to the value of the first variable and the cycle repeats. This means that the time between every update will remain constant past a certain point. My justification for using delta time will be used to make sure that objects are being animated and moved at a constant rate regardless of if the user is at a very high frame rate.

```
275  void Game::CalculateDeltaTime()
276  {
277      Uint64 currentTime = SDL_GetTicks();
278      mDeltaTime = (currentTime - mLastFrame
279      mLastFrame = currentTime;
280  }
```

Finally, I then wrote the procedure to call each sprite's Update() function that was discussed earlier, using a for loop. Then I wrote out the utility functions to simplify creating sprites and load the default sprites states based on the game state. This marks the end of the redesign of the GUIRenderer class into the new SpriteRenderer class.

```
341  void Game::UpdateSprites(GameState gameState)
342  {
343      for (auto& [key, sprite] : mSpriteRenderer.mCurrentlyRenderedSprites[gameState])
344      {
345          sprite->Update(mDeltaTime);
346      }
347  }
348
349  Sprite* Game::GetSprite(GameState gameState, string name)
350  {
351      // Simplify Getting sprite without the need to write out the entire line
352      return mSpriteRenderer.mCurrentlyRenderedSprites[gameState][name];
353  }
354
355  void Game::LoadDefaultSprites(GameState gameState)
356  {
357      // Clear currently rendered sprites for the new game state
358      mSpriteRenderer.mCurrentlyRenderedSprites[gameState].clear();
359
360      // Load default sprites for the given game state
361      for (const auto& [key, sprite] : mSpriteRenderer.mDefaultSprites[gameState])
362      {
363          // Clone or reference the default sprite into the currently rendered sprites
364          mSpriteRenderer.mCurrentlyRenderedSprites[gameState][key] = sprite;
365      }
366  }
```

Development Continued

To begin development again, I wrote the new sprite definitions using the new CreateSprite() procedures. This involved copying the old parameters into the parameters of the CreateSprite() procedure.

```
133     mSpriteRenderer.CreateSprite(
134         GameState::START_MENU,
135         "background",
136         ResourceManager::GetTexture("background"),
137         vec2(0, 0),
138         glm::vec2(1960.178, 1033.901),
139         0.0f,
140         vec3(1.0f),
141         ResourceManager::GetShader("default"),
142         false,
143         vec2(0,0),
144         2
145     );
146 
147     mSpriteRenderer.CreateSprite(
148         GameState::START_MENU,
149         "background-dots",
150         ResourceManager::GetTexture("background-dots"),
151         vec2(0, 0),
152         vec2(1920, 994.167),
153         0.0f,
154         vec3(1.0f),
155         ResourceManager::GetShader("default"),
156         false
157     );
158 
159     mSpriteRenderer.CreateSprite(
160         GameState::START_MENU,
161         "game-logo",
162         ResourceManager::GetTexture("game-logo"),
163         vec2(372.256, 193.333),
164         vec2(1162.667, 573.998),
165         0.0f,
166         vec3(1.0f),
167         ResourceManager::GetShader("default"),
168         false
169     );
170 
171     mSpriteRenderer.CreateSprite(
172         GameState::START_MENU,
173         "start-button",
174         ResourceManager::GetTexture("start-button"),
175         vec2(862.230f, 720.000f),
176         vec2(195.541f, 73.988f),
177         0.0f,
178         vec3(1.0f),
179         ResourceManager::GetShader("default"),
180         false
181     );
182 
183     mSpriteRenderer.CreateSprite(
184         GameState::START_MENU,
185         "exit-button",
186         ResourceManager::GetTexture("exit-button"),
187         vec2(889.921f, 824.483f),
188         vec2(143.693f, 78.957f),
189         0.0f,
190         vec3(1.0f),
191         ResourceManager::GetShader("default"),
192         false
```

```
195     mSpriteRenderer.CreateSprite(
196         GameState::START_MENU,
197         "start-menu-ua",
198         ResourceManager::GetTexture("start-menu-ua"),
199         vec2(0, 988.918),
200         vec2(1920, 91.082),
201         0.0f,
202         vec3(1.0f),
203         ResourceManager::GetShader("default"),
204         false
205     );
```

I then finalized this process by calling the DrawSprites() method from the SpriteRenderer class. This is the code responsible for rendering every sprite on screen.

```
318 void Game::Render()
319 {
320     // Clear the screen with a solid background color
321     glClearColor(0.1f, 0.1f, 0.1f, 1.0f);
322     glClear(GL_COLOR_BUFFER_BIT);
323 
324     // Render all sprites for the current game state
325     mSpriteRenderer.DrawSprites(mCurrentGameState);
326 }
```

With the sprites now being rendering I needed to create a menu system for the start menu. I decided to create a menu system class that will be responsible for all types of menu systems across my adaptation. Having a generalized menu class system will make the aspects of menus in my game more modular and reusable.

Menu Class

The menu class will make use of the <vector> data structure which a list to store sprites objects that will be used in the menu system. It will consist of an index to the current sprite selected in the menu list and will make use of function pointers to allow customization of the menu animations. The menu class will also contain functionality for changing the current menu choice index. This will allow menu system to “wrap around”

Class Diagram

The menu class diagram shows below the definitions of the functions that will make up the menu system. Alongside the functions, specified member functions are described within the diagram. Additionally, upon considering the aspect of a time delay whilst selection. I added member variables that will allow time delay logic to take place. Furthermore, I will be using the “Callback” type definition for the void(*) (function pointer) definition.

Menu
vector<Sprite*> mSprites int mMenuChoice Sprite* mCurrentlySelectedMenuChoice bool mWrapAround Callback mHighlightAnimationCallback Callback mSelectionAnimationCallback float mSelectionDelay Uint32 mLastSelectionTime Uint32 currentTime
Menu(vector<Sprite*>,bool) void AppendSprite(Sprite*) void IncrementMenuChoice() void DecrementMenuChoice() void SetHighlightAnimation(Callback animationCallback) void SetSelectionAnimation(Callback animationcCallback) void PlayHighlightAnimation() void UpdateCurrentTime()

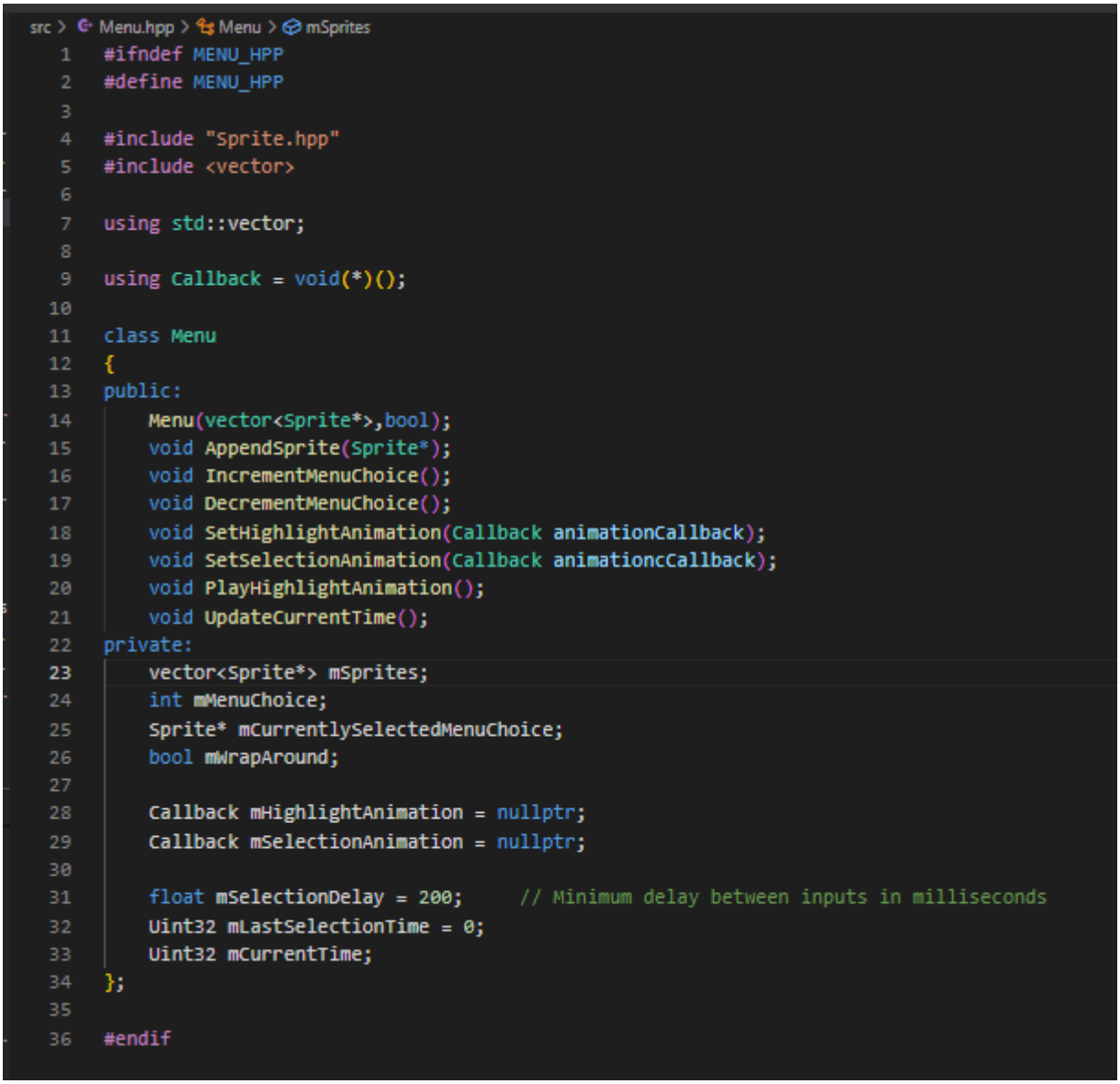
Pseudocode for IncrementMeuChoice()

Below is the pseudocode for the menu logic to change the current menu choice index. It involves incrementing the menu choice index based on wether the wrap around varible is true. If the wrap around varible is true it will modulo the index by the list size, this way it will always “wrap around” back to 0 when the index is greater than the menu size

```
public procedure IncrementMenuChoice()
    if mWrapAround==true  then
        mMenuChoice = (mMenuChoice + 1) MOD mSprites.size()
    elseif mMenuChoice < mSprites.size() – 1 then
        ++mMenuChoice
    mCurrentlySelectedMenuChoice = mSprites[mMenuChoice]
endprocedure
```

Development Continued

To implement the menu class, I first began by writing the function definitions for the class diagram in the header files.



I then wrote out the class constructor and implemented the increment menu function as described in pseudocode.

src > Menu.cpp > DecrementMenuChoice()

```cpp
#include "Menu.hpp"


Menu::Menu(vector<Sprite*> sprites, bool wrapAround)
    : mSprites(sprites),
      mWrapAround(wrapAround),
      mMenuChoice(0),
      mCurrentlySelectedMenuChoice(mSprites[mMenuChoice])
{
}

void Menu::AppendSprite(Sprite* sprite)
{
    mSprites.push_back(sprite);
}

void Menu::IncrementMenuChoice()
{
    if (mWrapAround)
    {
        mMenuChoice = (mMenuChoice + 1) % mSprites.size();
    }
    else
    {
        if (mMenuChoice < mSprites.size() - 1)
        {
            ++mMenuChoice;
        }
    }
    mCurrentlySelectedMenuChoice = mSprites[mMenuChoice];
}
```

I then wrote out the DecrementMenuChoice() function that will be responsible for decreasing the index of the current menu choice. The only difference between the increment and the decrement function is that the negative function decreases the index and will wrap around when it is less than 0. This is shown through the menu choice having the menu size added onto it before it is modulo by the menu size(). This is to ensure that no negative indexes are reached. My justification for this is because trying to access a negative list index may lead to undefined behavior.

As well as that I wrote out the setter functions which were relatively straightforward; It only involved taking in the parameters and setting the member variable to it. The unique attribute about the SetHighlightAnimation() procedure is that it will be take in a callback that will be customized via a lambda function. This means that the animation code can be dynamically determined relative to each menu instance.

```
33  void Menu::DecrementMenuChoice()
34  {
35      if (mWrapAround)
36      {
37          mMenuChoice = (mMenuChoice - 1 + mSprites.size()) % mSprites.size(); // Handle negative mod
38      }
39      else
40      {
41          if (mMenuChoice > 0)
42          {
43              --mMenuChoice;
44          }
45      }
46      mCurrentlySelectedMenuChoice = mSprites[mMenuChoice];
47  }
48
49  void Menu::SetHighlightAnimation(Callback callback)
50  {
51      mSelectionAnimation = callback;
52  }
53
54  void Menu::SetSelectionAnimation(Callback callback)
55  {
56      mHighlightAnimation = callback;
57  }
58
59  void Menu::PlayHighlightAnimation()
60  {
61      if(mSelectionAnimation)
62          mSelectionAnimation();
63  }
64
65  void Menu::UpdateCurrentTime()
66  {
67      mCurrentTime = SDL_GetTicks();
68  }
```

I then continued development by implementing the menu class in the game class as a hash table in the game class. This is because I intend to have multiple menu objects based on the different menus per games state. I then defined utility functions that will be used to create menu objects and get menu objects relative to their game states.

```
62      // Store menu class hash table
63      unordered_map<GameState, unordered_map<string, Menu*>> mCurrentMenus;
```

```
49      void CreateMenu(GameState, vector<Sprite*>, bool, string);
50      Menu* GetMenu(GameState, string);
```

```
510 void Game::CreateMenu(GameState gamestate, vector<Sprite*> sprites,bool wrapAround, string name)
511 {
512     Menu* menu = new Menu(sprites,wrapAround);
513     mCurrentMenus[gamestate][name] = menu;
514 }
515
516 Menu* Game::GetMenu(GameState gamestate,string name)
517 {
518     return mCurrentMenus[gamestate][name];
519 }
```

I then began to create the menu sprites for the separate menu system based on their game states. This involved getting the default game states of the sprites and creating a menu object using CreateMenu(). I also set the highlight animation for the menu that is selected. This involved setting the color of the sprite to yellow.

src > Menus.cpp > InitializeMenus()

```cpp
#include "Game.hpp"

void Game::InitializeMenus()
{

    // Create menu system for start menu
    CreateMenu(GameState::START_MENU,
        vector<Sprite*>
        {
          GetDefaultSprite(GameState::START_MENU, "start-button"),
          GetDefaultSprite(GameState::START_MENU, "exit-button"),
        },
        true,
        "start-menu"
    );

    Menu* startMenu = GetMenu(mCurrentGameState, "start-menu");

    startMenu->SetHighlightAnimation(
        [this,startMenu](){
        startMenu->GetCurrentMenuOption()->SetColor(vec3(1.0f, 1.0f, 0.0f));
    });
}
```

I then had the idea to implement the same approach of initializing the sprite within the separate source file and inside a function. My justification for doing this is to decouple the repetitive sprite creation code from the actual game logic. This also involved the creation of the InitializeSprites() function.

src > Sprites.cpp > InitializeSprites()

```cpp
#include "Game.hpp"

void Game::InitializeSprites()
{
        mSpriteRenderer.CreateSprite(
        GameState::START_MENU,
        "background",
        ResourceManager::GetTexture("background"),
        vec2(0, 0),
        glm::vec2(1960.178, 1033.901),
        0.0f,
        vec3(1.0f),
        ResourceManager::GetShader("default"),
        false,
        vec2(0,0),
        2
    );

    mSpriteRenderer.CreateSprite(
        GameState::START_MENU,
        "background-dots",
        ResourceManager::GetTexture("background-dots"),
        vec2(0, 0),
        vec2(1920, 994.167),
        0.0f,
        vec3(1.0f),
        ResourceManager::GetShader("default"),
        false
    );

    mSpriteRenderer.CreateSprite(
        GameState::START_MENU,
        "game-logo",
        ResourceManager::GetTexture("game-logo"),
        vec2(372.256, 193.333),
        vec2(1162.667, 573.998),
        0.0f,
        vec3(1.0f),
        ResourceManager::GetShader("default"),
        false
    );

    mSpriteRenderer.CreateSprite(
        GameState::START_MENU,
        "start-button",
        ResourceManager::GetTexture("start-button"),
        vec2(862.230f, 720.000f),
        vec2(195.541f, 73.988f),
        0.0f,
        vec3(1.0f),
        ResourceManager::GetShader("default"),
        false
    );

    mSpriteRenderer.CreateSprite(
        GameState::START_MENU,
        "exit-button",
        ResourceManager::GetTexture("exit-button"),
        vec2(889.921f, 824.483f),
        vec2(143.693f, 78.957f),
        0.0f,
        vec3(1.0f),
        ResourceManager::GetShader("default"),
```

```
121     // Initialize sprites
122     InitializeSprites();
123
124     mSpriteRenderer.LoadDefaultSprites(GameState::START_MENU);
125
126     // Initialize menus;
127     InitializeMenus();
128
```

Pseudocode for Transition()

After creating the menus, I then began to write the transition function pseudocode that will be used in changing the game state and achieving the success criterion of having a transition state to give visual indication that the game is changing.

The general basis of the function involved defining Boolean variables that will be triggered based on wether the previous condition has been meet. The conditions themselves will make use of Sprite's utility functions that will enable the animation for sprites to occur and constantly checking the state of the animation. In this case, the procedure involves checking if all the sprites in the sprite hash table are darkened, loading the new sprites as darkened sprites first, and then brightening them to give the "fade in" effect of the transition.

```
public procedure Transition(newGameState)

// If it is the first frame and the sprites are not being darkened
enable the darken animation for all sprites in current game state


    if(NOT mTransitioningDark AND mFirstTransitionFrame) then

            for each sprite in
CurrentlyRenderedSprites[CurrentGameState]

                sprite.SetDarken(true)

        mTransitioningDark = true

        mFirstTransitionFrame = false


// If the sprites are in their darkening state, check if all the
sprites are fully dark. If they are not break out


    if(mTransitioningDark AND NOT mAllDark) then
```

```
        mAllDark = true


        for each sprite in
CurrentlyRenderedSprites[CurrentGameState])
            if sprite.GetDarkenState() != true then
                mAllDark = false


// Once all the sprites are dark then load the new sprites then
set the darkening Boolean to false and set the brightening Boolean
to true


    If mAllDark then
        mTransitioningDark = false
        mTransitioningLight = true
        LoadDefaultSprites(newGameState)
        mCurrentGameState = newGameState


// Set each new load sprite to initially be dark


        for each sprite in
CurrentlyRenderedSprites[CurrentGameState])
            Sprite.SetColor(0)
        mAllLight = false


// Immediately enable the brighten animation in each sprite
        for each sprite in
CurrentlyRenderedSprites[CurrentGameState]
            sprite.SetBrighten(true)
```

```
        mAllDark = false


     // If sprites are being brightened check if they're fully
bright
    if(mTransitioningLight AND NOT mAllLight)


        mAllLight = true;


        for each sprite in
CurrentlyRenderedSprites[CurrentGameState])
            if(sprite.GetBrightenState() != true)
                mAllLight = false
    // Once each sprite is fully brightened end the transition and
reset the transition varibles
    if(mAllLight) then
        mAllDark = false
        mAllLight = true
        mTransitioningDark = false
        mTransitioningLight = false
        mTransitioningGameState = NOT_TRANSITIONING
        mHasTransitioned = true
        mFirstTransitionFrame = true
```

Development Continued

I then implemented the Transition function within the game class. This involved writing the pseudocode above in proper syntax and defining the member variables used in the function within the game class. Upon writing the code I realised that game needs to constantly check for transition calls during the update loop to ensure that no transition calls are missed. Therefore, I need to implement the procedure as a recursive procedure and update it within the main Update() game loop. This involved me creating the

CheckForTransitionState() and TransitionToGameState() procedures that will be called in the main game loop

```
    bool mTransitioningDark = false;
    bool mTransitioningLight = false;
    bool mAllDark = false;
    bool mAllLight = true;
    bool mHasTransitioned = true;
    GameState mTransitioningGameState = GameState::NOT_TRANSITIONING;
    bool mFirstFrame = true;
    bool mFirstTransitionFrame = true;
};
```

```
235  void Game::Transition(GameState newGameState)
236  {
237      if(!mTransitioningDark && mFirstTransitionFrame)
238      {
239          std::cout << "Darkening sprites!\n";
240          for (auto& [key, sprite] : mSpriteRenderer.mCurrentlyRenderedSprites[mCurrentGameState])
241          {
242              sprite->SetDarken(true);
243          }
244          mTransitioningDark = true;
245          mFirstTransitionFrame = false;
246      }
247
248      if(mTransitioningDark && !mAllDark)
249      {
250          std::cout << "Checking if sprites are darkened!\n";
251          mAllDark = true;
252
253          for (auto& [key, sprite] : mSpriteRenderer.mCurrentlyRenderedSprites[mCurrentGameState])
254          {
255              if(sprite->GetDarkenState() != true)
256                  mAllDark = false;
257              return;
258          }
259      }
260
261      if(mAllDark)
262      {
263          std::cout << "Loading new sprites!\n";
264          mTransitioningDark = false;
265          mTransitioningLight = true;
266          LoadDefaultSprites(newGameState);
267          mCurrentGameState = newGameState;
268          std::cout << "Loading new sprites and setting them to color 0!\n";
269          for (auto& [key, sprite] : mSpriteRenderer.mCurrentlyRenderedSprites[mCurrentGameState])
270          {
271              sprite->SetColor(vec3(0));
272          }
273          mAllLight = false;
274          std::cout << "Brightening new sprites!\n";
275          for (auto& [key, sprite] : mSpriteRenderer.mCurrentlyRenderedSprites[mCurrentGameState])
276          {
277              sprite->SetBrighten(true);
278          }
279          mAllDark = false;
280      }
281
282      if(mTransitioningLight && !mAllLight)
283      {
284          std::cout << "Checking if new sprites are brightened!\n";
285          mAllLight = true;
286
287          for (auto& [key, sprite] : mSpriteRenderer.mCurrentlyRenderedSprites[mCurrentGameState])
288          {
289              if(sprite->GetBrightenState() != true)
290                  mAllLight = false;
291              return;
292          }
293      }
294
```

```
308  void Game::CheckForTransitionState()
309  {
310      if(!mHasTransitioned && (mTransitioningGameState != GameState::NOT_TRANSITIONING))
311      {
312          Transition(mTransitioningGameState);
313      }
314  }
315
316  void Game::TransitionToGameState(GameState newGameState)
317  {
318      mTransitioningGameState = newGameState;
319      mHasTransitioned = false;
320  }
321
```

```
142  void Game::Update()
143  {
144      GetSprite(mCurrentGameState, "background")->MoveTextureCoordinate(vec2(0, -0.1 / (1000 / mDeltaTime)));
145
146      CheckForTransitionState();
147
148      UpdateSprites(mCurrentGameState);
149  }
```

```
295      if(mAllLight)
296      {
297          std::cout << "Finished transition!\n";
298          mAllDark = false;
299          mAllLight = true;
300          mTransitioningDark = false;
301          mTransitioningLight = false;
302          mTransitioningGameState = GameState::NOT_TRANSITIONING;
303          mHasTransitioned = true;
304          mFirstTransitionFrame = true;
305      }
306  }
```

After the transition function was implemented, I began loading the user interface for the main menu game state using the CreateSprite() function. Beforehand, I loaded the sprites' textures using the LoadTextures function that was discussed earlier. The key difference here is that I rendered the main user interface using the perspective shader. This means the sprites are now 3D objects. My justification for this is to meet the requirements of having 3D aspects to mimic the 3D aspects of older VSRGs and benefit the older stakeholders. This was fully discussed in analysis.

```
110      /* MAIN_MENU */
111      ResourceManager::LoadTexture("\\assets\\png\\main menu\\breakbeat-main-menu-user-navigation-assist-new.png",true,"main-menu-ua");
112      ResourceManager::LoadTexture("\\assets\\png\\main menu\\breakbeat-main-menu-settings-user-interface.png",true,"main-menu-settings-button");
113      ResourceManager::LoadTexture("\\assets\\png\\main menu\\breakbeat-main-menu-chart-editor-user-interface.png",true,"main-menu-chart-editor-button");
114      ResourceManager::LoadTexture("\\assets\\png\\main menu\\breakbeat-main-menu-chart-selection-user-interface.png",true,"main-menu-chart-selection-button");
115      ResourceManager::LoadTexture("\\assets\\png\\main menu\\breakbeat-main-menu-back-button-user-interface.png",true,"main-menu-back-button");
```

```
79        /* Initialize Sprites for the MAIN_MENU game state*/
80
81      mSpriteRenderer.CreateSprite(
82          GameState::MAIN_MENU,
83          "background",
84          ResourceManager::GetTexture("background"),
85          vec2(0, 0),
86          glm::vec2(1960.178, 1033.901),
87          0.0f,
88          vec3(1.0f),
89          ResourceManager::GetShader("default"),
90          false,
91          vec2(0,0),
92          2
93      );
94
95      mSpriteRenderer.CreateSprite(
96          GameState::MAIN_MENU,
97          "start-menu-ua",
98          ResourceManager::GetTexture("main-menu-ua"),
99          vec2(-0.866, 988.918),
100         vec2(1920.866, 91.082),
101         0.0f,
102         vec3(1.0f),
103         ResourceManager::GetShader("default"),
104         false
105     );
106
107     mSpriteRenderer.CreateSprite(
108         GameState::MAIN_MENU,
109         "main-menu-settings-button",
110         ResourceManager::GetTexture("main-menu-settings-button"),
111         vec2(448.689, 571.618),
112         vec2(447.212, 264.223), // <---- THIS RIGHT HERE
113         180.0f,
114         vec3(1.0f),
115         ResourceManager::GetShader("default-3D"),
116         true
117     );
118
119     mSpriteRenderer.CreateSprite(
120         GameState::MAIN_MENU,
121         "main-menu-chart-editor-button",
122         ResourceManager::GetTexture("main-menu-chart-editor-button"),
123         vec2(1024.394, 208.425),
124         vec2(447.212, 264.223), // <---- THIS RIGHT HERE
125         180.0f,
126         vec3(1.0f),
127         ResourceManager::GetShader("default-3D"),
128         true
129     );
130
131     mSpriteRenderer.CreateSprite(
132         GameState::MAIN_MENU,
133         "main-menu-chart-selection-button",
134         ResourceManager::GetTexture("main-menu-chart-selection-button"),
135         vec2(448.394, 208.425),
136         vec2(447.212, 264.223), // <---- THIS RIGHT HERE
137         180.0f,
138         vec3(1.0f),
139         ResourceManager::GetShader("default-3D"),
```

Before I could begin the write the menu logic, I realized that I need to create a separate set of vertex array objects for the 3D objects. Therefore, I decided to copy a new set of vertex array objects and vertex buffers objects in the game class's Initialize() procedure code. Another major reason for doing this is because the vertices for the 3D objects need to be in the 3D Coordinate space and therefore require a different set of vertex data.

I then modified the sprite draw() code to consider which vertex array object to bind to. This is based on whether the object is in 3D which is determined on the perspective variable

```
15      GLuint vertexBufferObject, vertexBufferObject3D;
```

```
30      // 3D sprite vertex data (Position + Tex Coords, no normals)
31      float vertices3D[] =
32      {
33          // Position         // Tex Coords
34          0.0f, 1.0f, 0.0f,   0.0f, 1.0f,  // Top-left
35          1.0f, 0.0f, 0.0f,   1.0f, 0.0f,  // Bottom-right
36          0.0f, 0.0f, 0.0f,   0.0f, 0.0f,  // Bottom-left
37
38          0.0f, 1.0f, 0.0f,   0.0f, 1.0f,  // Top-left
39          1.0f, 1.0f, 0.0f,   1.0f, 1.0f,  // Top-right
40          1.0f, 0.0f, 0.0f,   1.0f, 0.0f   // Bottom-right
41      };
```

```
62      // Set up 3D VAO/VBO
63      glGenVertexArrays(1, &this->mVertexArrayObject3D);
64      glGenBuffers(1, &vertexBufferObject3D);
65
66      glBindVertexArray(this->mVertexArrayObject3D);
67      glBindBuffer(GL_ARRAY_BUFFER, vertexBufferObject3D);
68      glBufferData(GL_ARRAY_BUFFER, sizeof(vertices3D), vertices3D, GL_STATIC_DRAW);
69
70      // Position attribute (vec3)
71      glEnableVertexAttribArray(0);
72      glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 5 * sizeof(float), (void*)0);
73
74      // Texture coordinate attribute (vec2)
75      glEnableVertexAttribArray(1);
76      glVertexAttribPointer(1, 2, GL_FLOAT, GL_FALSE, 5 * sizeof(float), (void*)(3 * sizeof(float)));
77
78      // Unbind VAO and VBO
79      glBindVertexArray(0);
80      glBindBuffer(GL_ARRAY_BUFFER, 0);
81  }
```

```
89      // Determine whether the VAO is going to used 3D or 2D based on the mPersepctive parameter
90      sprite->mVertexArrayObject = perspective ? this->mVertexArrayObject3D : this->mVertexArrayObject;
```

I then began to write the function to convert world space normalized device coordinates (NDC) as discussed in analysis. This involved taking the size of the coordinate multiplying it by two and then dividing it by the screen size. This is to a get fraction of how much of the

coordinate is into the screen dimension. This is because OpenGL uses a right-handed object space and its normalized device coordinates are between -1 and 1. Therefore, to ensure that coordinate lies in that range that range I must subtract 1. Afterwards, to convert from world space to NDC, I must take the inverse of the projection and view matrix. This is to reverse the effects of perspective and ensure that the object being rendered remains in the same plane as every other 2D object being rendered. Essentially, the 3D object's vertices are transformed in such a way that they are directly in front of the camera to give the effect of the object still being 2D.

To make use of the function and to make the 3D objects rendered on the same plane as the 2D objects, I used an if statement to consider the perspective Boolean for the object and converted its mPosition coordinates using ScreenToWorldSpace();

```
269  vec2 Sprite::ScreenToWorldSpace(vec2 screenCoord, mat4 view)
270  {
271      // NORMALIZED DEVICE COORDINATES (NDC)
272      double x = (2.0 * screenCoord.x / 1920) - 1.0;
273      double y = 1.0 - (2.0 * screenCoord.y / 1080); // Flip Y axis for NDC
274
275      // Homogeneous clip space
276      glm::vec4 screenPos = glm::vec4(x, y, 1.0f, 1.0f);
277
278      // Perspective projection matrix
279      float aspectRatio = static_cast<float>(1920) / 1080;
280      mat4 perspectiveProjection = glm::perspective(glm::radians(45.0f), aspectRatio, 0.1f, 100.0f);
281
282      // Transform from NDC to world space
283      glm::mat4 viewProjectionInverse = glm::inverse(perspectiveProjection * view);
284      glm::vec4 worldPos = viewProjectionInverse * screenPos;
285
286      return glm::vec2(worldPos.x, worldPos.y);
287  }
288
289
```

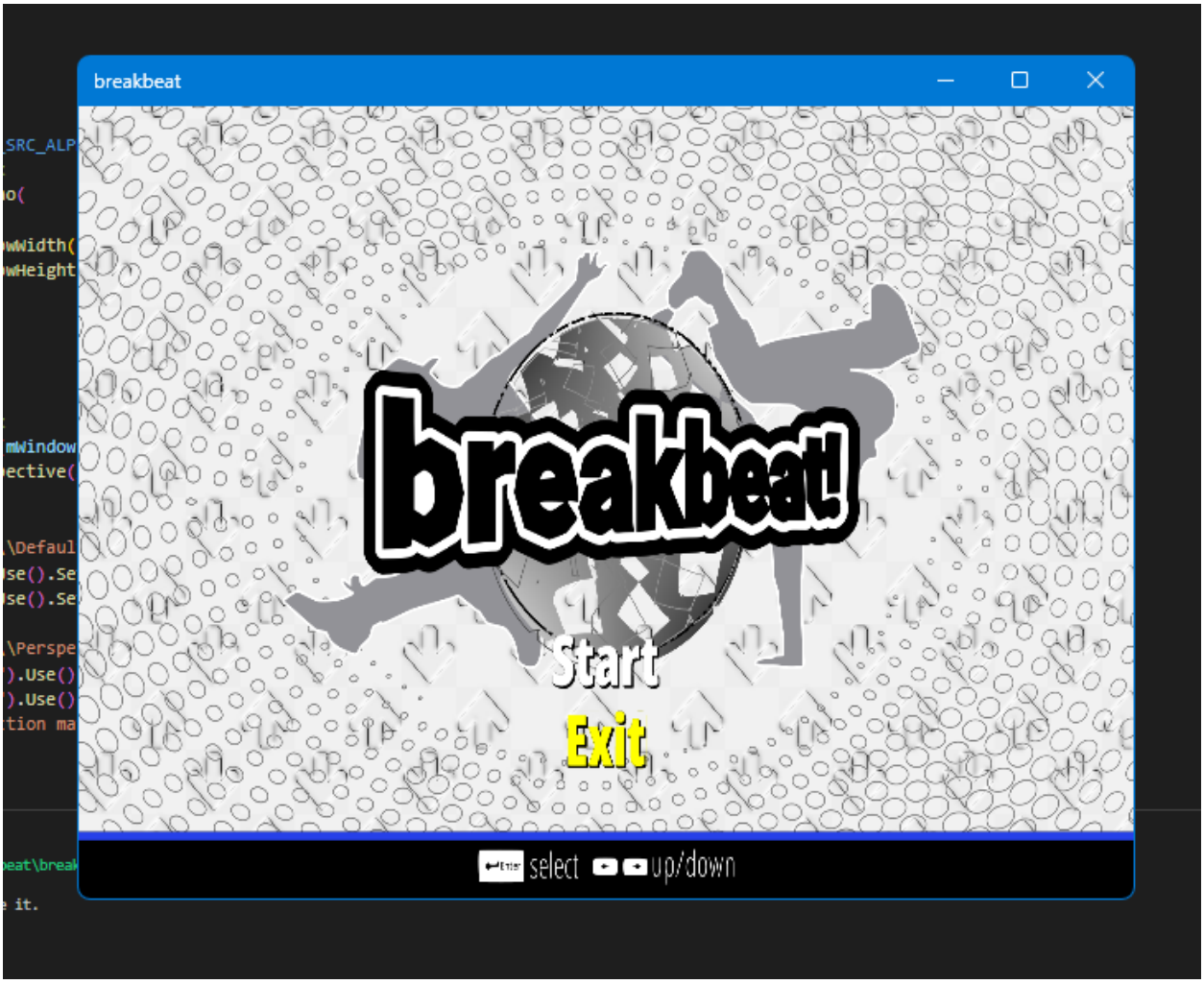
```
34      if (mPerspective)
35      {
36          // Fixed camera position for 3D
37          view = glm::translate(mat4(1.0f), glm::vec3(0.0f, 0.0f, -2.0f));
38          this->mShader.SetMatrix4("view", view);
39
40          // Compute world position and size
41          vec2 worldPos = ScreenToWorldSpace(vec2(mPosition.x, mPosition.y), view);
42          vec2 worldSize = ScreenToWorldSpace(mSize, view);
43
44          model = glm::translate(model, vec3(worldPos.x, worldPos.y, 0.0f)); // Move to world position
45          model = glm::translate(model, vec3(worldSize / 2.0f, 0.0f)); // Center sprite
46          model = glm::rotate(model, glm::radians(mRotation), vec3(0.0f, 0.0f, 1.0f)); // Rotate around center
47          model = glm::translate(model, vec3(-worldSize / 2.0f, 0.0f)); // Translate back
48          model = glm::scale(model, vec3(worldSize, 1.0f)); // Scale to match size
49
50          this->mShader.SetMatrix4("model", model);
51      }
```

To finalize development, I implemented the menu logic for the user interface. This involved updating the ProcessEvents() procedure. The logic involved checking for key inputs pressed and getting the sprite reference of the sprite in the game state's menu object and calling the transition function based on the game state being transitioned to.

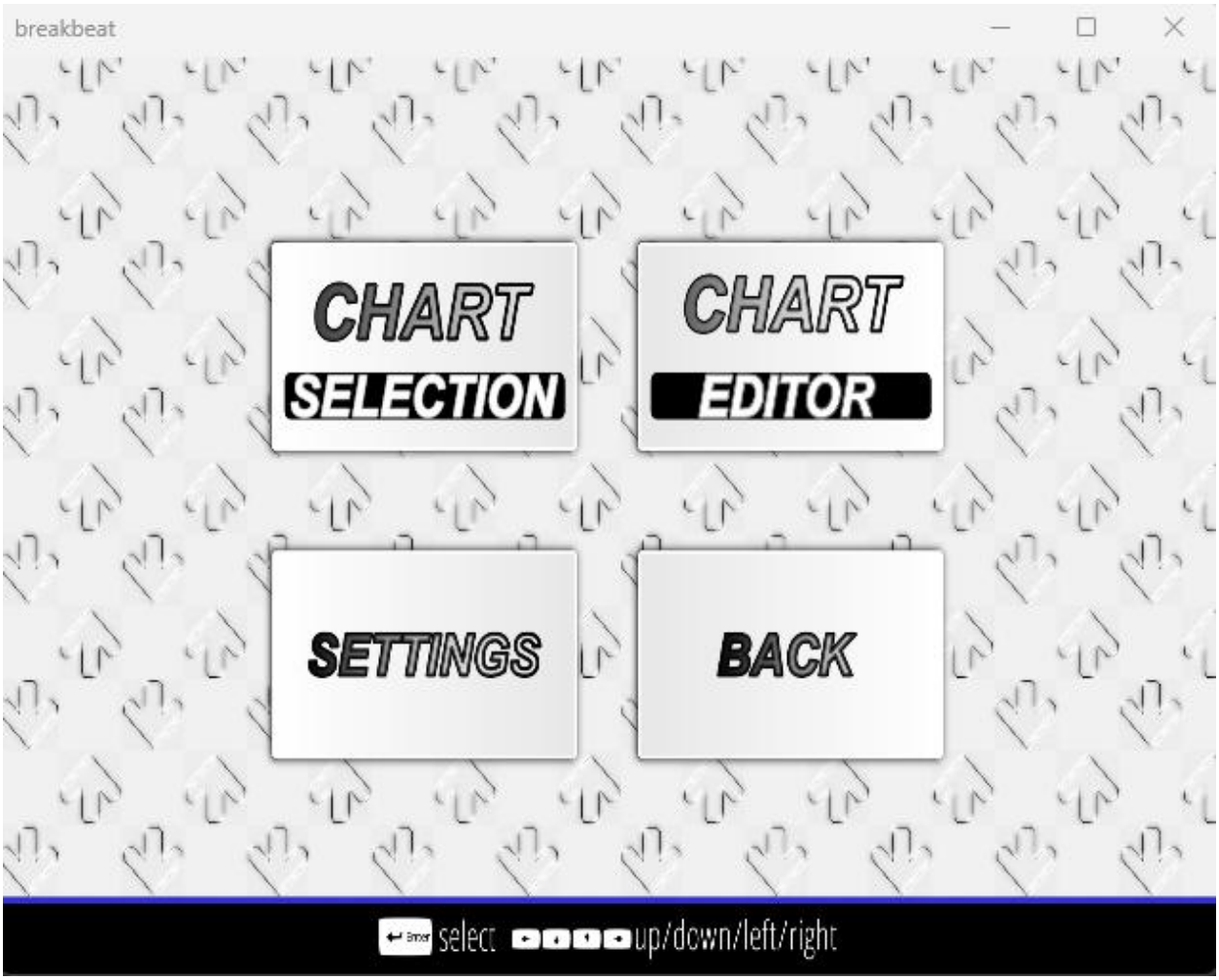
```
        // Check for key inputs
        else if (event.type == SDL_KEYDOWN)
        {
            if (event.key.keysym.sym == SDLK_F11)
            {
                mWindow.ToggleFullscreen();
            }

            // If the current game state is start menu then check for input
            if (mCurrentGameState == GameState::START_MENU)
            {
                // Store a pointer to the start menu object
                Menu* startMenu = GetMenu(mCurrentGameState, "start-menu");

                // Update the value of the current time at this moment
                startMenu->UpdateCurrentTime();

                // If the time elapsed between the last update is greater than 200ms
                if (startMenu->CheckSelectionTime())
                {
                    // IF the key pressed is the left key
                    if (event.key.keysym.sym == SDLK_LEFT)
                    {
                        startMenu->GetCurrentMenuOption()->SetColor(vec3(1.0f));
                        // Update menu choice index with wrap-around
                        startMenu->IncrementMenuChoice();
                        startMenu->PlayHighlightAnimation();
                        startMenu->UpdateCurrentTime();
                    }
                    // If the key pressed is the right key
                    if (event.key.keysym.sym == SDLK_RIGHT)
                    {
                        startMenu->GetCurrentMenuOption()->SetColor(vec3(1.0f));
                        // Update menu choice index with wrap-around
                        startMenu->DecrementMenuChoice();
                        startMenu->PlayHighlightAnimation();
                        startMenu->UpdateCurrentTime();
                    }
                    // If they pressed in the return key
                    if (event.key.keysym.sym == SDLK_RETURN)
                    {
                        // If the currrent menu option is the start button sprite then call the transition animation
                        if (startMenu->GetCurrentMenuOption() == GetSprite(mCurrentGameState, "start-button"))
                        {
                            TransitionToGameState(GameState::MAIN_MENU);
                        }
                        // If the current menu option is the exit button sprite then close the application
                        else if (startMenu->GetCurrentMenuOption() == GetSprite(mCurrentGameState, "exit-button"))
                        {
                            mWindow.SetWindowClosedBoolean(true);  // Exit game
                        }
                    }
                }
            }
        }
    }
}
```

Testing

To commence testing I ran the program compiled and the start menu appeared again successfully. Furthermore, the menu screen's user interface was fully useable when pressing the arrow keys. This meets the success criteria that was mentioned in the test plan. This can be seen in the "Start Menu Test 1 Video".



Upon pressing enter on the return button, the program was successfully able to transition into the main menu and the user interface was correctly rendered. This meets the criteria that was mentioned in the test plan and analysis. The footage of the transition can be seen in the "Start Menu Test 1" Video



Main Menu

The main menu system will consist of adding animations for the main menu option selected and then allow transition into different game states.

Test Plan

Below are the test plans for the main menu system.

Test	Expected Outcome
Pressing the arrow keys to navigate the menu choices	Each of the menu options should have a one-time animation of a rotation the z axis to indicate which game state selected as well as being highlighted
Pressing enters on the current menu option	The currently selected menu option should rotate around the y axis and the game screen should fade into the separate game screens with their user interface however with no functionality.

Pressing enters on the back button menu option	The back button user interface should perform a momentary full 360-degree rotation around the y axis before the game screen darkening and transitioning back to the start menu.

Development

To begin development, I first began by creating the menu logic for the menu sprites. This involved creating rotation procedures and scaling procedures within the sprite class and creating a new set of variables for the logic behind the procedures. The logic behind these procedures was mainly the same as other previous procedures discussed. The reason for making the scale and rotation procedures for the sprites is to give an indication for what menu option the user is selecting via a micro-interaction animation. Furthermore, the rotation function considers whether the sprite is in perspective mode and will perform a 3D rotation on the sprite. My justification for the sprites having 3D animations was discussed in analysis; The intention is to mimic the 3D animations that are found in older arcade games. This is to contribute to the older aesthetic of the game that benefits older stakeholders of my game.

```
266  void Sprite::SetRotation(bool enable, vec3 orientation, float rotationTime, float angle, bool looping)
267  {
268      if(!mIsRotating && enable)
269      {
270          mRotationOrientation = orientation;
271          mStartRotationAngle = mRotation;
272          mRotationTime = rotationTime;
273          mRotationAngle = angle;
274          mRotationStartTime = SDL_GetTicks();
275          mIsRotating = true;
276          mIsLoopRotation = looping;
277      }
278
279  }
280
281  void Sprite::SetScale(bool enable, float targetScale, float scaleTime, bool looping)
282  {
283      if (!mIsScaling && enable)
284      {
285          mStartScale = mCurrentScale; // Start from the current scale
286          mTargetScale = targetScale;
287          mScaleTime = scaleTime;
288          mScaleStartTime = SDL_GetTicks();
289          mIsScaling = true;
290          mIsLoopScaling = looping;
291      }
292  }
```

```
        if (mIsRotating)
        {
            float timeElapsed = (SDL_GetTicks() - mRotationStartTime) / 1000.0f; // Convert to seconds
            float progress = timeElapsed / mRotationTime; // Progress as a ratio (0 to 1)

            if (timeElapsed >= mRotationTime)
            {
                mIsRotating = false; // Stop rotating if done
                progress = 1.0f; // Clamp to final state
            }

            // Interpolate between the current angle and target angle
            mRotation = glm::mix(mStartRotationAngle, mRotationAngle, progress);
        }

        model = rotate(model, glm::radians(mRotation), mRotationOrientation);

        model = glm::translate(model, vec3(-worldSize / 2.0f, 0.0f)); // Translate back

```

```
    model = glm::translate(model, vec3(worldPos.x, worldPos.y, 0.0f)); // Move to world position
    model = glm::translate(model, vec3(worldSize / 2.0f, 0.0f)); // Center sprite

    if (mIsRotating)
    {
        float timeElapsed = (SDL_GetTicks() - mRotationStartTime) / 1000.0f; // Convert to seconds
        float progress = timeElapsed / mRotationTime; // Progress as a ratio (0 to 1)

        if (timeElapsed >= mRotationTime && !mIsLoopRotation)
        {
            mIsRotating = false; // Stop rotating if done
            progress = 1.0f; // Clamp to final state
        }

        // Interpolate between the current angle and target angle
        mRotation = glm::mix(mStartRotationAngle, mRotationAngle, progress);
    }

    model = rotate(model, glm::radians(mRotation), mRotationOrientation);

    model = glm::translate(model, vec3(-worldSize / 2.0f, 0.0f)); // Translate back


    if (mIsScaling)
    {
        float timeElapsed = (SDL_GetTicks() - mScaleStartTime) / 1000.0f; // Convert to seconds
        float progress = timeElapsed / mScaleTime; // Progress as a ratio (0 to 1)

        if (timeElapsed >= mScaleTime && !mIsLoopScaling)
        {
            mIsScaling = false; // Stop scaling if done
            progress = 1.0f;    // Clamp to the final value
        }

        mCurrentScale = glm::mix(mStartScale, mTargetScale, progress); // Interpolate between scales
    }

    // Correctly position and scale the sprite

    model = glm::translate(model, vec3((-worldSize * mCurrentScale / 2.0f, 0.0f))); // Center sprite
    model = glm::scale(model, vec3(worldSize * mCurrentScale, 1.0f)); // Scale to match size
```

The menu logic involved the same logic as the start menu but instead I had to add additional functionality for vertical menu movements. This involved making procedures that will update the index of the current menu choice based on the number of the menu's

columns. The reasoning behind these procedures is part of my success criteria in which the user will have the ability to navigate not only using the left and right arrow keys but the up and down keys also. Thus, to have vertical movement using the up and down keys I needed to create additional functions that handle the indexes differently to moving the menu horizontally.

```
49  void Menu::MoveMenuChoiceUp()
50  {
51      int totalItems = mSprites.size(); // Get totalItems
52
53      if (mWrapAround) // If menu wrap around is true
54      {
55          // Minus the length of the menu and the length of the menu columns
56          mMenuChoice = (mMenuChoice - mHorizontalLength + totalItems) % totalItems;
57      }
58      else
59      {
60          if (mMenuChoice >= mHorizontalLength)
61          {
62              mMenuChoice -= mHorizontalLength;
63          }
64      }
65
66      mCurrentlySelectedMenuChoice = mSprites[mMenuChoice];
67  }
68
69  void Menu::MoveMenuChoiceDown()
70  {
71      int totalItems = mSprites.size();
72
73      if (mWrapAround)
74      {
75          mMenuChoice = (mMenuChoice + mHorizontalLength) % totalItems;
76      }
77      else
78      {
79          if (mMenuChoice + mHorizontalLength < totalItems)
80          {
81              mMenuChoice += mHorizontalLength;
82          }
83      }
84
85      mCurrentlySelectedMenuChoice = mSprites[mMenuChoice];
86  }
```

I then programmed the menu animation logic via passing in the appropriate function pointers into the menu class callbacks. This involved using the sprite class utility functions that were discussed and programmed earlier. Furthermore, I also moved all menu related into one source file and encapsulated the code to make the code more readable and maintainable. I then finished the menu logic by adding in the sections to transition to the game settings. Furthermore, I included the menu option to be highlighted yellow to give a

clear indication in which menu option is being selected. This is to make the main menu more accessible and friendly to the user.

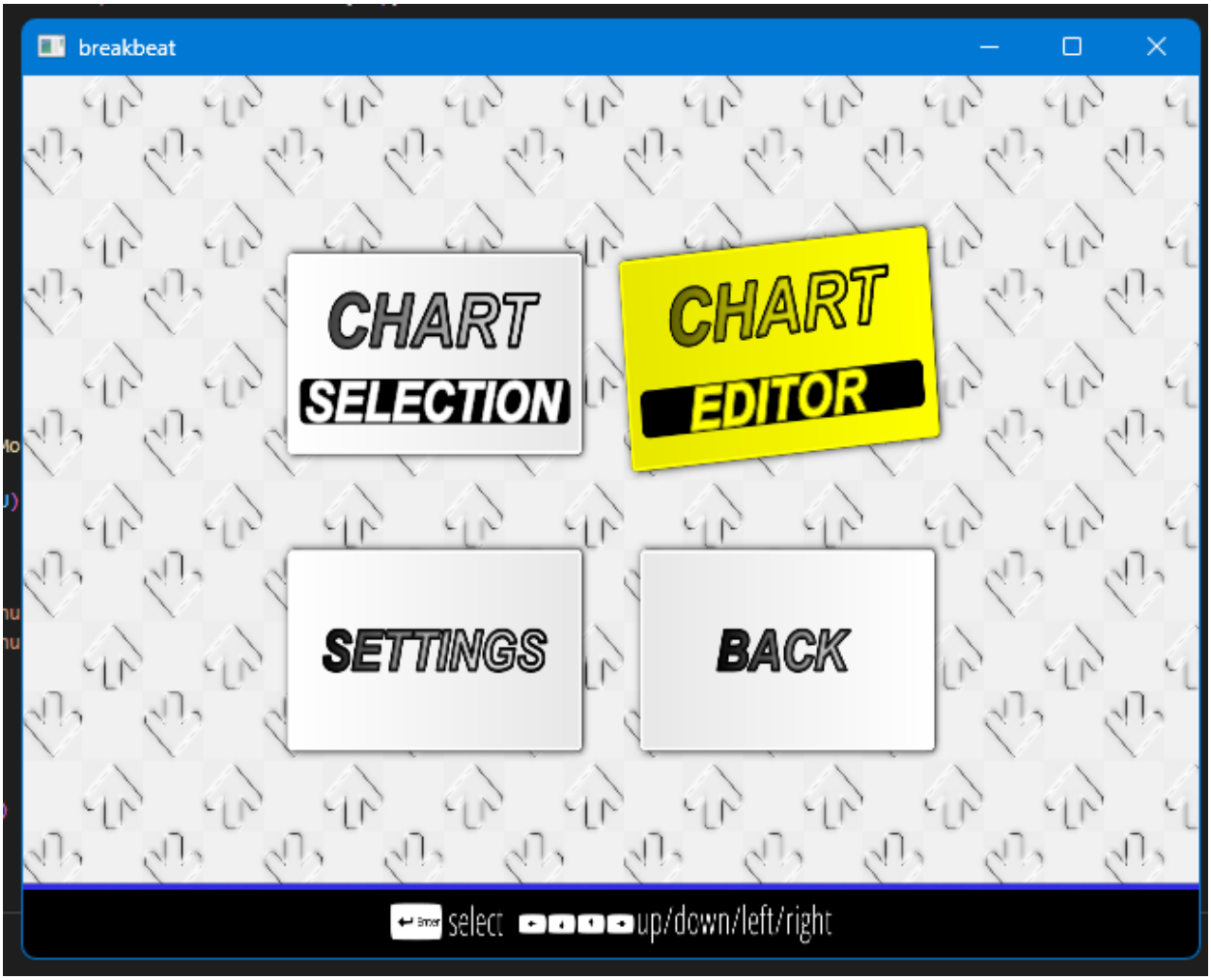
```
42      mainMenu->SetHighlightAnimation(
43          [this,mainMenu](){
44          mainMenu->GetCurrentMenuOption()->SetRotation(true,vec3(0,0,1),0.1,5); // Rotate the sprite when its being highlighted
45          mainMenu->GetCurrentMenuOption()->SetScale(true,1.05,0.2);
46          mainMenu->GetCurrentMenuOption()->SetColor(vec3(1.f,1.0f,0.0f)); // Highlight the sprite
47      });
48
49      mainMenu->SetSelectionAnimation(
50          [this,mainMenu](){
51          mainMenu->GetCurrentMenuOption()->SetRotation(true,vec3(0.0f,1.0f,0),0.5,360); // Play a spinning animation when enter is pressed
52      });
53
54      mainMenu->SetUnhighlightAnimation(
55          [this,mainMenu](){
56          mainMenu->GetCurrentMenuOption()->SetRotation(true,vec3(0,0,1),0.05,0);
57          mainMenu->GetCurrentMenuOption()->SetScale(true,1.0,0.1);
58          mainMenu->GetCurrentMenuOption()->SetColor(vec3(1.0f));
59      });
```

src > Menus.cpp > InitializeMenus()

```
60  void Game::HandleMenuInput(SDL_Event& event)
63      if (mCurrentGameState == GameState::START_MENU)
109     else if(mCurrentGameState == GameState::MAIN_MENU)
110     {
111         Menu* mainMenu = GetMenu(mCurrentGameState, "main-menu");
112
113         mainMenu->UpdateCurrentTime();
114
115         // If the time elapsed between the last update is greater than 200ms
116         if (mainMenu->CheckSelectionTime())
117         {
118             if (event.key.keysym.sym == SDLK_LEFT)
119             {
120                 mainMenu->PlayUnhighlightAnimation();
121                 // Update menu choice index with wrap-around
122                 mainMenu->DecrementMenuChoice();
123                 mainMenu->PlayHighlightAnimation();
124                 mainMenu->UpdateLastSelectedTime();
125             }
126             // If the key pressed is the right key
127             if (event.key.keysym.sym == SDLK_RIGHT)
128             {
129                 mainMenu->PlayUnhighlightAnimation();
130                 // Update menu choice index with wrap-around
131                 mainMenu->IncrementMenuChoice();
132                 mainMenu->PlayHighlightAnimation();
133                 mainMenu->UpdateLastSelectedTime();
134             }
135             if (event.key.keysym.sym == SDLK_DOWN)
136             {
137                 mainMenu->PlayUnhighlightAnimation();
138                 // Update menu choice index with wrap-around
139                 mainMenu->MoveMenuChoiceDown();
140                 mainMenu->PlayHighlightAnimation();
141                 mainMenu->UpdateLastSelectedTime();
142             }
143             if (event.key.keysym.sym == SDLK_UP)
144             {
145                 mainMenu->PlayUnhighlightAnimation();
146                 // Update menu choice index with wrap-around
147                 mainMenu->MoveMenuChoiceUp();
148                 mainMenu->PlayHighlightAnimation();
149                 mainMenu->UpdateLastSelectedTime();
150             }
151             // If they pressed in the return key
152             if (event.key.keysym.sym == SDLK_RETURN)
153             {
154                 // If the currrent menu option is settings button call the transition animation
155                 if (mainMenu->GetCurrentMenuOption() == GetSprite(mCurrentGameState, "main-menu-settings-button"))
156                 {
157                     mainMenu->PlaySelectionAnimation();
158                     if(mainMenu->GetCurrentMenuOption()->IsAnimationComplete())
159                         TransitionToGameState(GameState::SETTINGS);
160                 }
161                 if (mainMenu->GetCurrentMenuOption() == GetSprite(mCurrentGameState, "main-menu-back-button"))
162                 {
163                     mainMenu->PlaySelectionAnimation();
164                     TransitionToGameState(GameState::START_MENU);
165                 }
166             }
167         }
168     }
169 }
```

Testing

To commence testing the main menu I ran the compiled program and navigated to the main menu of the game. I first tested if pressing the left and right arrows keys will yield horizontal movement of the menu option. This test case was successful as the currently selected menu option grew and rotated on its z axis. This can be seen in the “Main Menu Test 1” video. Furthermore, I tested whether I could use the up and down keys to allow vertical navigation of the main menu, and this was successful.



I then tested pressing the enter key on the back menu option and the game successfully played the transition animation back into the start menu. Furthermore, the back button played a 3D rotation animation of spinning around its y axis. This test result successfully meets the case of having 3D micro-interaction animations that was discussed in analysis.

Settings Menu

This section of development will cover the settings section of the game that will be for setting the main game play attributes as discussed in design. This will include storing the user's gameplay preferences within a settings file.

Test Plan

Test	Expected Outcome
Running the compiled program	Textures should load correctly, and all textures should load despite the supposed texture bind limit being surpassed
Running the compiled program and entering the start menu	The mouse should appear and track my mouse movement in real time
Hovering a sprite in different game states	The sprite that is being hovered over should be highlighted yellow and play its hover animation.
Navigating to the settings user interface	The settings user interface as shown in design should successfully render after being faded in
Navigating the settings user interface using up and down arrow keys	The settings options text should play an animation to indicate that they are being highlighted. In this case the animation should be text growing
Pressing the enter key on a settings option	The text for settings option should be highlighted yellow and should still be enlarged
Using the left and right keys on the currently selected text option	The currently selected text option should either increment of decrement and upon inspecting the settings file the file should be updated in real time.
Pressing enter on a key bind settings menu text option and pressing another key afterwards	The key bind text option should turn yellow and then should be the letter of the next key pressed
Pressing escape on a currently selected settings menu option	The selected menu option should return to its normal color. In this case the text should go from yellow to white.
Pressing escape again on the settings menu	The settings menu will fade out and the game will transition back to the main menu as a fade in animation.

Development

Bindless Textures

Before I begin developing the settings menu of the game. First, I need to consider an issue with my system regarding textures. In OpenGL, there is a limit to the amount textures that can be stored in the current OpenGL context. This is because each OpenGL context has a limited number of textures bind slots. This means that with my current system , I can only load a limited number of textures before I cannot load anymore. In my case, I can only load up to 32 textures, however this limit is variable based on the user's graphics card and operating system. This is an issue as the current assets required for my adaptation are greater than my texture limit.

However, there is a solution to this problem; The solution is the use of bindless textures. Bindless textures do not require a texture to bound to a bind slot when creating the texture. Instead, a 64-bit unsigned integer is used to represent the texture instead of binding it to a binding slot.

Therefore, I decided to implement the use of bindless textures within my code adaptation. This involved making changes to the Texture class Generate() function and including the bindless textures extension to my shaders.

The actual changes made involve using the bindless texture OpenGL extension to make the 64-bit unsigned integer reference to the texture and the making the texture reference resident. The texture reference being resident means that the texture reference has direct access to the GPU memory. As well as that, I passed the texture reference into the shader so that it can load the texture from the texture reference.

My overall justification for adding bindless textures is to remove the texture limit and make future development of my adaptation easier. This is because, if I were not to use bindless textures, I would require alternative techniques such as texture atlases and texture arrays that are time consuming and tedious to implement.

```
class Texture
{
public:
    // holds the ID of the texture object, used
    unsigned int ID;
    GLuint64 handle; // Bindless texture handle
```

```
33      glBindTexture(GL_TEXTURE_2D, 0);
34      this->handle = glGetTextureHandleARB(this->ID); // Convert the texture to a unsigned 64 bit interger reference
35      glMakeTextureHandleResidentARB(this->handle); // Make the texture reference resident
36  }
```

```
#extension GL_ARB_bindless_texture : require // Include bindless texture extensions
#extension GL_ARB_gpu_shader5 : require // Allow the textures reference to be accessed directly

in vec2 textureCoordinate;

out vec4 fragmentColor;
layout (bindless_sampler) uniform sampler2D image; // Allow the texture uniform to be stored as two 32 bit unsinged integers
uniform vec3 color;

```

```
    // Get the location of the uniform
    GLint uniformLocation = glGetUniformLocation(this->mShader.ID, "image");

    // Pass the sprite's texture handle to the shader
    glUniformHandleui64ARB(uniformLocation, mTexture.handle);
```

Settings User Interface

After implementing bindless textures, I began the development of the settings menu. I first began by loading the textures for the settings menu using the resource manager system that was implemented earlier and creating the settings menu sprites using the Sprite renderer system. The code shown below is not the full sprite creation or texture loading code and is only the section for the settings menu. The rest of the sprite creation and texture loading code for all sections will be shown in the appendix.

```
/* SETTINGS MENU */
ResourceManager::LoadTexture("\\assets\\png\\settings menu\\breakbeat-settings-user-navigation-assist.png",true,"settings-menu-ua");
ResourceManager::LoadTexture("\\assets\\png\\settings menu\\breakbeat-settings-information-text-2.png",true,"settings-menu-text");
ResourceManager::LoadTexture("\\assets\\png\\settings menu\\breakbeat-settings-increment-right.png",true,"settings-increment-right");
ResourceManager::LoadTexture("\\assets\\png\\settings menu\\breakbeat-settings-increment-left.png",true,"settings-increment-left");
```

```
    GameState::SETTINGS,
    "settings-menu-text",
    ResourceManager::GetTexture("settings-menu-text"),
    vec2(0, 200),
    vec2(1203.980, 542.850),
    0.0f,
    vec3(1.0f),
    ResourceManager::GetShader("default"),
    false
);

mSpriteRenderer.CreateSprite(
    GameState::SETTINGS,
    "settings-increment-left-1",
    ResourceManager::GetTexture("settings-increment-left"),
    vec2(808.060, 240.958),
    vec2(44.331, 68.262),
    0.0f,
    vec3(1.0f),
    ResourceManager::GetShader("default"),
    false
);

mSpriteRenderer.CreateSprite(
    GameState::SETTINGS,
    "settings-increment-left-2",
    ResourceManager::GetTexture("settings-increment-left"),
    vec2(808.060, 301.370),
    vec2(44.331, 68.262),
    0.0f,
    vec3(1.0f),
    ResourceManager::GetShader("default"),
    false
);

mSpriteRenderer.CreateSprite(
    GameState::SETTINGS,
    "settings-increment-left-3",
    ResourceManager::GetTexture("settings-increment-left"),
    vec2(808.060, 369.703),
    vec2(44.331, 68.262),
    0.0f,
```

Text Renderer Class

To continue development of the settings and the overall adaptation, I must introduce a new element to my game engine which is text rendering. This is because I will need text throughout my adaptation to indicate to user what is happening within files and in menu systems and in gameplay. In the early days of text rendering, text was rendered using a bitmap font texture and each character of the font had a specific texture coordinate. Each character on the font was known as a glyph.

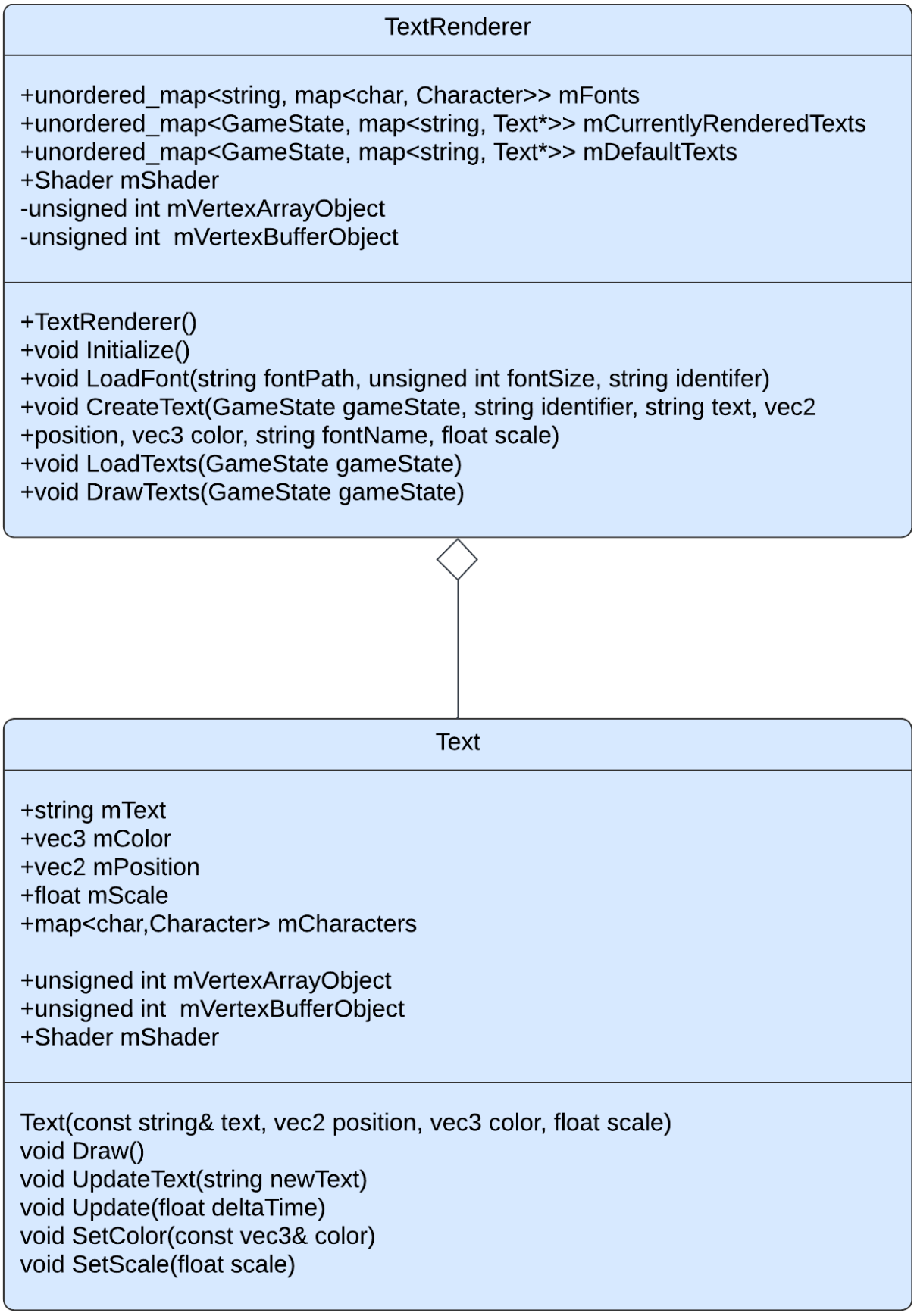
Text Class

The text renderer system will be like the sprite renderer system. However, instead of sprites, it will be composed of text objects and will not contain as many utility functions as there are for sprite objects. As well as that, the drawing mechanism for text objects is slightly different due to the need for the additional FreeType OpenGL library to load the .tff (TrueType) fonts for the text. The .ttf font is not based on being defined by pixels or any other non-scalable solution but the use mathematical equations that can procedurally generate the rasterized images for character glyph, a bit like vector graphics. The reason for using .ttf is that it will allow the generation of character glyphs of various sizes without any loss of quality. My justification for this is that if I were to use an older character technique such as bitmap fonts then any attempt to try and scale font will lead to quality loss and the need to recompile a entirely new bitmap font for the different sized font.

The text renderer will follow the same use of the nested unordered_map<> (hash table) data structure as in the sprite renderer prototype sections however there is an additional nested hash table to store the fonts of the texts that will be loaded by the FreeType OpenGL library. This reason for doing this is for the same reason as the sprite renderer class before; It will allow text objects to be dynamically updated outside the Render() loop and make development more reuseable.

Class Diagram

The class diagram for the text renderer shows the use of functions that are like the sprite renderer class. These include a procedure to create text objects and two similar functions that will be used to load text objects. Furthermore, there are procedures used to initialize the text renderer class as whole. The justification for including a separate Initialize() procedure is due to each text object being separate from a sprite object and so it requires a different set of vertex array objects and buffers to initialize.


TextRenderer
+unordered_map<string, map<char, Character>> mFonts
+unordered_map<GameState, map<string, Text*>> mCurrentlyRenderedTexts
+unordered_map<GameState, map<string, Text*>> mDefaultTexts
+Shader mShader
-unsigned int mVertexArrayObject
-unsigned int mVertexBufferObject
+TextRenderer()
+void Initialize()
+void LoadFont(string fontPath, unsigned int fontSize, string identifer)
+void CreateText(GameState gameState, string identifier, string text, vec2
+position, vec3 color, string fontName, float scale)
+void LoadTexts(GameState gameState)
+void DrawTexts(GameState gameState)
Text
+string mText
+vec3 mColor
+vec2 mPosition
+float mScale
+map<char,Character> mCharacters
+unsigned int mVertexArrayObject
+unsigned int mVertexBufferObject
+Shader mShader
Text(const string& text, vec2 position, vec3 color, float scale)
void Draw()
void UpdateText(string newText)
void Update(float deltaTime)
void SetColor(const vec3& color)
void SetScale(float scale)

Initialize() Pseudocode

As mentioned above, the initialization procedure for the text renderer will be used to render the text object from a different set of vertex data. Therefore, the initialization procedure will be like the sprite renderer initialize procedure and will involve the creation of vertex buffers and vertex arrays and the assignment of a shader to the class.

```
public procedure Initialize()
    this.mShader = GetShader("text")
    this.mShader.SetInteger("text", 0)

    // configure VAO/VBO for texture quads
    glGenVertexArrays(1, &this.mVertexArrayObject)
    glGenBuffers(1, &this.mVertexBufferObject)
    glBindVertexArray(this.mVertexArrayObject)
    glBindBuffer(GL_ARRAY_BUFFER, thismVertexBufferObject)
    glBindVertexArray(0)
    mFonts.clear()
```

Character() Pseudocode

Each of the glyphs that are loaded by the FreeType library are different sizes and each have different metrics. Each of the glyphs can reside on a horizontal baseline or slightly below it. These metrics define the exact offsets of to properly position each glyph on the baseline e.g. How large each glyph should be, how many pixels to advance to render the next glyph. Therefore, in order to store these metrics, I will need to construct a character struct that will be used to store and load it metrics to generate a texture. My justification for storing the metrics in a data structure is because it is more efficient to store the generated data somewhere in the application and query it rather than to load it every frame. Each character data structure will be stored within another hash table.

```
/// Holds all metrics relative to a character glyph as loaded
using FreeType
class Character
    TextureID // ID handle of the glyph texture
    Size     // size of glyph
```

```
    Bearing   // offset from baseline to left/top of glyph
    Advance   // horizontal offset to advance to next glyph
```

LoadFont() Pseudocode

The LoadFont() procedure will consist of the loading the first 128 ASCII characters of the font and storing its relevant data in character data structure. Within the for loop of the pseudocode, the program generates a texture and stores its metrics. However, the most notable part of the code is the disabling of byte alignment. Normally OpenGL requires that textures have the texture size (in bytes) is a multiple of 4. This is because the bitmap generated from the glyph is a grayscale 8-bit image where each colour is represented by a single byte.

```
public procedure LoadFont(fontPath,fontSize,identifer)
        // Load font using FreeType
       ft;
        // Load empty font face data structure
       face;
        // Set font size
        FT_Set_Pixel_Sizes(face, 0, fontSize)

        // Disable byte-alignment restriction
        glPixelStorei(GL_UNPACK_ALIGNMENT, 1);

        for i=0 to 128
            // Load character glyph
            FT_Load_Char(face, c, FT_LOAD_RENDER)

            // Generate texture
           texture
           glGenTextures(1, &texture)
           glBindTexture(GL_TEXTURE_2D, texture)
```

```
            // Set texture image properties
            glTexImage2D
            (
                GL_TEXTURE_2D,
                0,
                GL_RED,
                face.glyph.bitmap.width,
                face.glyph.bitmap.rows,
                0,
                GL_RED,
                GL_UNSIGNED_BYTE,
                face.glyph.bitmap.buffer
            );

            // Set texture options
            glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S,
GL_CLAMP_TO_EDGE)
            glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T,
GL_CLAMP_TO_EDGE)
            glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER,
GL_LINEAR)
            glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER,
GL_LINEAR)

            // Store character
            Character character =
                texture
```

```
                ivec2(face.glyph.bitmap.width,
face.glyph.bitmap.rows),
                ivec2(face.glyph.bitmap_left,
face.glyph.bitmap_top),
                face.glyph.advance.x


            characters.insert(Character)



        mFonts[identifer] = characters


        glBindTexture(GL_TEXTURE_2D, 0)
        // Finish using font face and clear memory
        FT_Done_Face(face)
        // Finish using free type library
        FT_Done_FreeType(ft)
```

Development Continued

To continue the development, I began by creating the character data structure within a separate header file. This is because I will require the character data structure to be shared within the TextRenderer.cpp to store the text objects and the Text.cpp file to allow texturized glyphs to access their character metrics.

src > Character.hpp > ...

```
#ifndef CHARACTER_HPP
#define ChARACTER_HPP

#include <glm/glm.hpp>

/// Holds all state information relevant to a character as loaded using FreeType
struct Character {
    unsigned int TextureID; // ID handle of the glyph texture
    glm::ivec2   Size;      // size of glyph
    glm::ivec2   Bearing;   // offset from baseline to left/top of glyph
    unsigned int Advance;   // horizontal offset to advance to next glyph
};

#endif
```

I then began writing out the code for the text renderer class and writing out the proper code for the Initialize() procedure as discussed and explained in pseudocode. Furthermore, I included the hash table for the fonts and the default and currently rendered texts, much like the sprite renderer system. The justification for doing this remains the same; To allow the text objects to loaded and unloaded based on the current game state.

```
class TextRenderer
{
public:
unordered_map<string, map<char, Character>> mFonts;
unordered_map<GameState, map<string, Text*>> mCurrentlyRenderedTexts;
unordered_map<GameState, map<string, Text*>> mDefaultTexts;

Shader mShader;
// constructor
TextRenderer();
void Initialize();
// pre-compiles a list of characters from the given font
void LoadFont(string fontPath, unsigned int fontSize, string identifer);
void CreateText(GameState gameState, string identifier, string text, vec2 position, vec3 color, string fontName, float scale);
void LoadTexts(GameState gameState);
void DrawTexts(GameState gameState);
private:
    unsigned int mVertexArrayObject, mVertexBufferObject;
};

#endif
```

```
18 void TextRenderer::Initialize()
19 {
20     // load and configure shader
21     this->mShader = ResourceManager::GetShader("text");
22     this->mShader.SetMatrix4("projection", glm::ortho(0.0f, static_cast<float>(1920), static_cast<float>(1080), 0.0f), true);
23     this->mShader.SetInteger("text", 0);
24     this->mShader.SetVector3f("shadowColor", glm::vec3(0.0f, 0.0f, 0.0f)); // Black shadow
25     this->mShader.SetVector2f("shadowOffset", glm::vec2(0.005f, -0.005f)); // Offset slightly down and right
26     this->mShader.SetFloat("shadowBlur", 0.002f); // Adjust for more/less blur
27
28     // configure VAO/VBO for texture quads
29     glGenVertexArrays(1, &this->mVertexArrayObject);
30     glGenBuffers(1, &this->mVertexBufferObject);
31     glBindVertexArray(this->mVertexArrayObject);
32     glBindBuffer(GL_ARRAY_BUFFER, this->mVertexBufferObject);
33     glBufferData(GL_ARRAY_BUFFER, sizeof(float) * 6 * 4, NULL, GL_DYNAMIC_DRAW);
34     glEnableVertexAttribArray(0);
35     glVertexAttribPointer(0, 4, GL_FLOAT, GL_FALSE, 4 * sizeof(float), 0);
36     glBindBuffer(GL_ARRAY_BUFFER, 0);
37     glBindVertexArray(0);
38
39     // Clear any values within the font hash table on first initalization
40     mFonts.clear();
41 }
```

I then began wiring the LoadFont() procedure as described and discussed in pseudocode. Whilst writing the code I included further debug statements that will indicate whether the FreeType library is working correctly. This is to ensure that the code is more maintainable and will help in future cases of debugging.

```
void TextRenderer::LoadFont(string fontPath, unsigned int fontSize, string identifer)
{
        // std::cout<<"Loading font!"<<'\n';
        // Load font using FreeType
        FT_Library ft;
        if (FT_Init_FreeType(&ft)) {
            std::cerr << "ERROR::FREETYPE: Could not init FreeType Library" << std::endl;
            return;
        }

        FT_Face face;
        if (FT_New_Face(ft, (fs::current_path().string() + fontPath.c_str()).c_str(), 0, &face)) {
            std::cerr << "ERROR::FREETYPE: Failed to load font" << std::endl;
            return;
        }
```

I then began writing out the rest of the LoadFont() procedure as described in the pseudocode. The difference here is the instantiation of a character hash table that will be put inside the nested hash table for the texts. Within this hash table the font name will the identifier to the hash table of characters.

```
59      // Set font size of font
60      FT_Set_Pixel_Sizes(face, 0, fontSize);
61
62      glPixelStorei(GL_UNPACK_ALIGNMENT, 1); // Disable byte-alignment restriction
63
64      // Create character hash table to be put in nested hash table
65      std::map<char, Character> characters;
66
67      // Generate first 128 ASCII characters
68      for (GLubyte c = 0; c < 128; c++) {
69          // Load character glyph
70          if (FT_Load_Char(face, c, FT_LOAD_RENDER)) {
71              std::cerr << "ERROR::FREETYPE: Failed to load Glyph" << std::endl;
72              continue;
73          }
74          // Generate texture
75          unsigned int texture;
76          glGenTextures(1, &texture);
77          glBindTexture(GL_TEXTURE_2D, texture);
78          glTexImage2D
79          (
80              GL_TEXTURE_2D,
81              0,
82              GL_RED,
83              face->glyph->bitmap.width,
84              face->glyph->bitmap.rows,
85              0,
86              GL_RED,
87              GL_UNSIGNED_BYTE,
88              face->glyph->bitmap.buffer
89          );
90
91          // Set texture options
92          glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_CLAMP_TO_EDGE);
93          glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_CLAMP_TO_EDGE);
94          glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
95          glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);
96
```

```
97              // Store character
98              Character character =
99              {
100                 texture,
101                 glm::ivec2(face->glyph->bitmap.width, face->glyph->bitmap.rows),
102                 glm::ivec2(face->glyph->bitmap_left, face->glyph->bitmap_top),
103                 static_cast<unsigned int>(face->glyph->advance.x)
104             };
105
106             characters.insert(std::pair<char, Character>(c, character));
107         }
108
109         mFonts[identifer] = characters;
110
111         glBindTexture(GL_TEXTURE_2D, 0);
112         FT_Done_Face(face);
113         FT_Done_FreeType(ft);
114 }
```

Afterwards, I began writing the code for the text class as mentioned in the above class diagram. `

src > Text.hpp > ...

```
#ifndef TEXT_HPP
#define TEXT_HPP

#include <map>
#include <unordered_map>
#include <string>
#include <glm\glm.hpp>
#include "SDL.h"

using namespace glm;
using std::string;
using std::map;

#include "Shader.hpp"
#include "Character.hpp"

class Text {
public:
    string mText;
    vec3 mColor;
    vec2 mPosition;
    float mScale = 1.0f;
    map<char,Character> mCharacters;

    unsigned int mVertexArrayObject,  mVertexBufferObject;
    Shader mShader;

    Text(const string& text, vec2 position, vec3 color, float scale);
    void Draw();
    void UpdateText(const std::string& newText);
    void Update(float deltaTime);
    void SetColor(const glm::vec3& color);
    void SetScale(float scale);
}
```

I then began writing the code for the Draw() procedure for the text class. The logic remained the same as the sprite class however the key difference is that I am using the raw vertex data to perform the positioning and transformation of the text. This is instead of use the model matrix. This is because the character glyphs have metrics that need to be passed into the vertex data as mentioned earlier.

```
void Text::Draw()
{
    mShader.Use();
    mShader.SetVector3f("textColor", mColor);
    glActiveTexture(GL_TEXTURE0);
    glBindVertexArray(this->mVertexArrayObject);

    float x = mPosition.x;
    for (const char& c : mText)
    {
        Character ch = mCharacters[c];
        float xpos = x + ch.Bearing.x;
        float ypos = mPosition.y + (this->mCharacters['H'].Bearing.y - ch.Bearing.y);
        float w = ch.Size.x * mScale;
        float h = ch.Size.y * mScale;

        float vertices[6][4] =
        {
            {xpos,     ypos + h,    0.0f, 1.0f},
            {xpos + w, ypos,        1.0f, 0.0f},
            {xpos,     ypos,        0.0f, 0.0f},

            {xpos,     ypos + h,    0.0f, 1.0f},
            {xpos + w, ypos + h,    1.0f, 1.0f},
            {xpos + w, ypos,        1.0f, 0.0f}
        };

        glBindTexture(GL_TEXTURE_2D, ch.TextureID);
        glBindBuffer(GL_ARRAY_BUFFER, this->mVertexBufferObject);
        glBufferSubData(GL_ARRAY_BUFFER, 0, sizeof(vertices), vertices);
        glBindBuffer(GL_ARRAY_BUFFER, 0);
        glDrawArrays(GL_TRIANGLES, 0, 6);
        x += (ch.Advance >> 6) * mScale;
    }
    glBindVertexArray(0);
    glBindTexture(GL_TEXTURE_2D, 0);
}
```

Upon thinking ahead for the rest of my system, I realised that my text objects will need to change colour and scaled for future menu user interface systems. Therefore, I decided to copy the already existing sprite code for darkening and setting colour into the text class. This involved changing the variables names, but the logic was entirely the same. This is because text objects will need to highlight to indicate which menu option is being selected in future systems. This further proves my previous points about the sprite code allowing the code to reusable and maintainable.

```
33      void SetScale(float scale);
34      void SetDarken(bool enable, float darkenTime = 1.0f);  // New method to start/stop darkening
35
36      void Darken();  // Existing method to handle darkening process
37      // Procedure to brighten the sprutes color from the darkened sate over time
38      void SetBrighten(bool enable, float invertTime = 1.0f);  // New method to start/stop inverting
39      void Brighten();  // Existing method to handle inversion (lightening
40      void SetScale(bool enable, float targetScale, float scaleTime = 1.0f, bool looping = false);
41      const string& GetText();
42  private:
43      bool mIsScaling = false;
44      float mStartScale = 1.0f;
45      float mTargetScale = 1.0f;
46      float mScaleTime;
47      float mScaleStartTime;
48      bool mIsLoopScaling;
49
50      bool mHasUpdated;
51
52      float mDarkenStartTime = 0.0f;
53      float mBrightenStartTime = 0.0f;
54      float mDarkenTime = 1.0f;
55      float mBrightenTime = 1.0f;
56
57      bool mDarkening = false;
58      bool mBrightening = false;
59
60      vec3 mOriginalColor;
61      vec3 mDarkenedColor;
62
63      // State varible to determine when the varible is dark or bright
64      bool mDarkened = false;
65      bool mBrightened = false;
66  };
```

```cpp
const string& Text::GetText()
{
    return mText;
}

void Text::UpdateText(const std::string& newText) {
    mText = newText;
}

void Text::SetColor(const glm::vec3& color) {
    mColor = color;
}

void Text::SetScale(float scale) {
    mScale = scale;
}

void Text::SetScale(bool enable, float targetScale, float scaleTime, bool looping)
{
    if (!mIsScaling && enable)
    {
        mStartScale = mScale; // Start from the current scale
        mTargetScale = targetScale;
        mScaleTime = scaleTime;
        mScaleStartTime = SDL_GetTicks();
        mIsScaling = true;
        mIsLoopScaling = looping;
    }
}

void Text::SetDarken(bool enable, float darkenTime)
{
    if(!mDarkened || mBrightened && !mDarkening)
    {
        mDarkenTime = darkenTime;
        mOriginalColor = mColor;
        mDarkening = true;
        mDarkenStartTime = SDL_GetTicks();
    }
}
```

```
91  void Text::Darken()
92  {
93      float timeElapsed = SDL_GetTicks() - mDarkenStartTime;
94      float progress =  timeElapsed / (mDarkenTime * 1000);
95      mColor = mOriginalColor - progress;
96      if(mColor.x <= 0 && mColor.y <= 0 && mColor.z <= 0 || timeElapsed >= 1000)
97      {
98          mColor = vec3(0);
99          mDarkened = true;
100         mDarkening = false;
101         mBrightened = false;
102     }
103 }
104
105 void Text::SetBrighten(bool enable, float brightenTime)
106 {
107     if(!mBrightened || mDarkened && !mBrightening)
108     {
109         mBrightenTime = brightenTime;
110         mDarkenedColor = mColor;
111         mBrightening = true;
112         mBrightenStartTime = SDL_GetTicks();
113     }
114 }
115
116 void Text::Brighten()
117 {
118     float timeElapsed = SDL_GetTicks() - mBrightenStartTime;
119     float progress =  timeElapsed / (mBrightenTime * 1000);
120     mColor = mDarkenedColor + progress;
121     if(
122      timeElapsed >= 1000
123      )
124     {
125         mColor = mOriginalColor;
126         mBrightened = true;
127         mBrightening = false;
128         mDarkened = false;
129     }
130 }
```

```
132  void Text::Update(float deltaTime)
133  {
134      // If the sprite is in the darkening state it will call the darken function
135      if (mDarkening)
136          Darken();
137
138      // If the sprite is the in the brightening state then brighten the function
139      if (mBrightening)
140          Brighten();
141
142      // If the sprite is fully black it has been darkened
143      if(mColor == vec3(0))
144          mDarkened = true;
145
146      // If the sprite is fully bright it has been brightened
147      if(mColor == vec3(1))
148          mBrightened = true;
149
150      // If the sprite is not in any update state it has been fully updated
151      if(mDarkened || mBrightened)
152      {
153          mHasUpdated == true;
154      }
155
156      if (mIsScaling)
157      {
158          float timeElapsed = (SDL_GetTicks() - mScaleStartTime) / 1000.0f; // Convert to seconds
159          float progress = timeElapsed / mScaleTime; // Progress as a ratio (0 to 1)
160
161          if (timeElapsed >= mScaleTime && !mIsLoopScaling)
162          {
163              mIsScaling = false; // Stop scaling if done
164              progress = 1.0f;    // Clamp to the final value
165          }
166
167          mScale = glm::mix(mStartScale, mTargetScale, progress); // Interpolate between scales
168      }
169  }
```

Afterwards I wrote out the create text procedures that are copy of the CreateSprite() procedure but instead create text objects within the text renderer class and store them within the class's hash tables. I then began creating the text objects and loading the fonts required.

```
116 void TextRenderer::CreateText(GameState gameState, string identifier, string text, glm::vec2 position, vec3 color, string fontName, float scale)
117 {
118     // Create text object and store its properties within the text hash table
119     auto textObject = new Text(text, position, color, scale);
120     textObject->mVertexArrayObject = this->mVertexArrayObject;
121     textObject->mVertexBufferObject = this->mVertexBufferObject;
122     textObject->mShader = this->mShader;
123     textObject->mCharacters = mFonts[fontName];
124     mDefaultTexts[gameState][identifier] = textObject;
125 }
126
127 void TextRenderer::DrawTexts(GameState gameState)
128 {
129     // Iterate through each text in text hash table and call its draw function
130     for (auto& [key, text] : mCurrentlyRenderedTexts[gameState])
131     {
132         text->Draw(); // Draw the text
133     }
134 }
135
136 void TextRenderer::LoadTexts(GameState gameState)
137 {
138     // Iterate through each text and copy it to the text hash table
139     for (auto& [key, text] : mDefaultTexts[gameState])
140     {
141         // Use the copy constructor to create a copy of the sprite
142         mCurrentlyRenderedTexts[gameState][key] = text;
143     }
144 }
```

src > Texts.cpp > ...

```cpp
#include "Game.hpp"
#include <format>

using std::format;

void Game::InitializeFonts()
{
    mTextRenderer.LoadFont("\\fonts\\OpenSans-ExtraBold.ttf",48,"default-48");
    mTextRenderer.LoadFont("\\fonts\\OpenSans-ExtraBold.ttf",24,"default-24");
    // std::cout<<"Initialized fonts"<<'\n';
}

void Game::InitializeTexts()
{
    // std::cout<<"Initializing texts!"<<'\n';
    mTextRenderer.mDefaultTexts.clear();

    LoadSettings();

    mTextRenderer.CreateText
    (
        GameState::SETTINGS,
        "scroll-speed-text",
        format("{}",mScrollSpeed),
        vec2(885.98,253.99),
        vec3(1.0f),
        "default-48",
        1.0f
    );

    mTextRenderer.CreateText
    (
        GameState::SETTINGS,
        "receptor-size-text",
        format("{}%",mReceptorSize),
        vec2(885.98,310.99),
        vec3(1.0f),
        "default-48",
        1.0f
    );

```

Mouse Class

The next section of development involved the creation of the mouse class. The mouse class will reuse the already existing elements of the adaptation and will be for allowing mouse input and user navigation via mouse as discussed throughout analysis and design. Another justification for making a mouse class is that I will require mouse input for allowing changes to settings user interface and for future system that will require mouse input such as the chart editor selection and the chart editor itself.

The mouse class will be much like the text renderer and sprite renderer in that it will also use a separate set of vertex array objects to render a texture that is separate from sprites and text. For this part of development, I mainly copied the already existing initialization code from the sprite renderer and copied the existing draw code from the sprite class and utilized them to create an independent class. The reason for doing this is because much of fundamental elements such a responsive user interface and text in my game engine, already use the same logic for their features to be integrated. Therefore, it will be more efficient and easier to reuse the existing features in my game to form a new similar system. In this case for the mouse class, the logic to be reused will be simply rendering a texture and updating its position. Both of the features have already been implemented previously in the sprite rendering system and therefore can be copied to be reused for the new mouse system.

This shows the overall benefits of writing my code in an object-oriented manner and further proves the previous points of writing my sprite renderer code will allow development to be more modular and reusable.

I first began by defining the mouse class in its header file and writing the initialization code for the mouse class. The code remained almost identical to the sprite renderer initialization code except the new set of variable names required for the mouse class and the new shader file to be loaded. The justification for adding an initialization procedure to the mouse is as described for the sprite renderer system; The mouse will need a separate set of vertex array objects and vertex buffer objects to render a different texture on top of every other sprite. In essence, the mouse class will be rendered completely separately.

```
void Mouse::InitializeMouse()
{
    GLuint vertexBufferObject;
    float vertices[] =
    {
        // Position       // Tex Coords
        0.0f, 1.0f,      0.0f, 1.0f,
        1.0f, 0.0f,      1.0f, 0.0f,
        0.0f, 0.0f,      0.0f, 0.0f,

        0.0f, 1.0f,      0.0f, 1.0f,
        1.0f, 1.0f,      1.0f, 1.0f,
        1.0f, 0.0f,      1.0f, 0.0f
    };

    // Set up 2D VAO/VBO
    glGenVertexArrays(1, &this->mVertexArrayObject);
    glGenBuffers(1, &vertexBufferObject);

    glBindVertexArray(this->mVertexArrayObject);
    glBindBuffer(GL_ARRAY_BUFFER, vertexBufferObject);
    glBufferData(GL_ARRAY_BUFFER, sizeof(vertices), vertices, GL_STATIC_DRAW);

    // Position attribute
    glEnableVertexAttribArray(0);
    glVertexAttribPointer(0, 2, GL_FLOAT, GL_TRUE, 4 * sizeof(float), (void*)0);
    // Texture coordinate attribute
    glEnableVertexAttribArray(1);
    glVertexAttribPointer(1, 2, GL_FLOAT, GL_TRUE, 4 * sizeof(float), (void*)(2 * sizeof(float)));

    // Unbind VAO and VBO
    glBindVertexArray(0);
    glBindBuffer(GL_ARRAY_BUFFER, 0);

    if (this->mVertexArrayObject == 0) {
    std::cerr << "VAO not initialized. Call InitializeMouse() first." << std::endl;
    return;
    }
```

```

    this->mShader = ResourceManager::GetShader("cursor");
    if (this->mShader.ID == 0) {
        std::cerr << "Failed to load shader: cursor" << std::endl;
        return;
    }

    this->mTexture = ResourceManager::GetTexture("cursor");
    if (mTexture.handle == 0) {
        std::cerr << "Failed to load texture: cursor" << std::endl;
        return;
    }
}
```

Afterwards, I then wrote out the code for drawing the mouse and updating its position, the code was also another copy of my already existing sprite system and did not involve a lot of new logic. This further justifies my previous points of writing the code in this style will allow the code to be more modular and reusable .

```
75  void Mouse::DrawMouse()
76  {
77      this->mShader.Use();
78      mat4 model = mat4(1.0f);
79
80      model = translate(model, vec3(mPosition, 0.0f));
81      model = glm::translate(model, vec3((mSize* mCurrentScale / 2.0f, 0.0f))); // Center sprite
82      model = glm::scale(model, vec3(mSize * mCurrentScale, 1.0f)); // Scale to match size
83      model = glm::translate(model, vec3((-mSize* mCurrentScale / 2.0f, 0.0f)));
84
85      this->mShader.SetMatrix4("model", model);
86      this->mShader.SetVector3f("color", mColor);
87
88      GLint uniformLocation = glGetUniformLocation(this->mShader.ID, "image");
89
90      if (uniformLocation == -1) {
91          std::cerr << "Uniform 'image' not found in shader" << std::endl;
92          return;
93      }
94
95      // Pass the sprite's texture handle to the shader
96      glUniformHandleui64ARB(uniformLocation, mTexture.handle);
97
98      // Draw the sprite
99      glBindVertexArray(this->mVertexArrayObject);
100     glDrawArrays(GL_TRIANGLE_STRIP, 0, 6);
101     glBindVertexArray(0);
102 }
```

```
104 vec2 Mouse::GetMouseCoordinate()
105 {
106     return mPosition;
107 }
108
109 vec2 Mouse::GetMouseSize()
110 {
111     return mSize;
112 }
113
```

```
68  void Mouse::Update(SDL_Event& event)
69  {
70      const float& xpos{ static_cast<float>(event.motion.x) };
71      const float& ypos{ static_cast<float>(event.motion.y) };
72      mPosition = vec2(xpos,ypos);
73  }
```

Mouse Collision

To continue development of my settings menu, I began to form a new mouse input system that checks collision of sprites. This involved the use of axis-aligned bounding box (AABB) collisions or in other terms the encompassing rectangle around a sprite. Since all sprite's textures are rendered on rectangles, this can allow the use of the sprite's rectangle position and size attributes to determine if both size and position are overlapping and thus if the any two sprites are colliding. My justification for using this method is that AABB is relatively easy to implement and more efficient than other methods that involve trying to form a hitbox for the sprites based their shape.

The code for doing this was relatively straightforward; it involved iterating through rendered sprites and checking if both sprites position and size attributes were greater than each other. In this case the sprite will always be in collision with the mouse. Once a collision with a sprite is found, it returns the sprite using the previously written GetSprite() function. This is the essence of AABB and the mouse input system. Furthermore, I included a vector to determine the list relevant sprite names that will be checked for collision based on the game state. This is because if the mouse is colliding the with many overlapping sprites, then multiples sprites can be returned simultaneously which can lead to incorrect sprite collision detection. My justification for doing this is also to prevent the incorrect sprites being clicked and returned which may cause memory issues.

```
Sprite* Game::CheckCollidingSprite(GameState gameState)
{
    // Define relevant sprites for each game state
    std::vector<std::string> relevantSprites;

    if (gameState == GameState::START_MENU)
    {
        relevantSprites = {"start-button", "exit-button"};
    }
    else if (gameState == GameState::MAIN_MENU)
    {
        relevantSprites = {"main-menu-back-button", "main-menu-chart-editor-button", "main-menu-chart-selection-button", "main-menu-settings-button"};
    }
    else if (gameState == GameState::SETTINGS)
    {
        relevantSprites = {"settings-increment-left-1", "settings-increment-left-2", "settings-increment-left-3", "settings-increment-right-1", "settings-increment-right
    }
```

```
197     // Check collision for relevant sprites
198     for (auto& [key, sprite] : mSpriteRenderer.mCurrentlyRenderedSprites[gameState])
199     {
200         if (std::find(relevantSprites.begin(), relevantSprites.end(), key) != relevantSprites.end())
201         {
202             bool CollisionX = (mMouse->GetMouseCoordinate().x + mMouse->GetMouseSize().x >= sprite->GetPosition().x) &&
203                               (sprite->GetPosition().x + sprite->GetSize().x >= mMouse->GetMouseCoordinate().x);
204
205             bool CollisionY = (mMouse->GetMouseCoordinate().y + mMouse->GetMouseSize().y >= sprite->GetPosition().y) &&
206                               (sprite->GetPosition().y + sprite->GetSize().y >= mMouse->GetMouseCoordinate().y);
207
208             if (CollisionX && CollisionY)
209             {
210                 return sprite; // Return the first colliding sprite
211             }
212         }
213     }
```

Development Continued

To continue development, I needed to consider the factor of text options being added to my menu system. This will particularly be useful for future systems such as the chart selection menu in which text will need to be changed to show the song selected. My overall justification for adding text objects to the menu system is to allow text objects to be animated and have highlight animations just like sprites. This is also to indicate to the user which menu option is to be selected as previously described.

Therefore, to add text to the menu system, I need to again copy the components of the existing sprite code within the menu system and replace them with the text objects instead. This involved creating a new set of procedures to increment and decrease the menu option, to allow the current menu option to be set to a text object and to have animations and highlight animations added to the menu text objects. Animations for text objects were achieved by previously defined utility functions within the text class. This also involved the creation of a new constructor for text objects.

```
12  Menu::Menu(vector<Text*> texts, bool wrapAround)
13      : mTexts(texts),
14        mWrapAround(wrapAround),
15        mTextMenuChoice(0),
16        mCurrentlySelectedTextChoice(mTexts[mTextMenuChoice])
17  {
18  }
```

```
// Increment menu choice for text objects
void Menu::IncrementTextMenuChoice()
{
    if (mWrapAround)
    {
        mTextMenuChoice = (mTextMenuChoice + 1) % mTexts.size();
    }
    else
    {
        if (mTextMenuChoice < mTexts.size() - 1)
        {
            ++mTextMenuChoice;
        }
    }
    mCurrentlySelectedTextChoice = mTexts[mTextMenuChoice];
}

// Allow vertical movement of the menu choice for text objects
void Menu::MoveTextMenuChoiceUp()
{
    int totalTextItems = mTexts.size(); // Get totalTextItems

    if (mWrapAround) // If menu wrap around is true
    {
        // Minus the length of the menu and the length of the menu columns
        mTextMenuChoice = (mTextMenuChoice - mHorizontalLength + totalTextItems) % totalTextItems
    }
    else
    {
        if (mTextMenuChoice >= mHorizontalLength)
        {
            mTextMenuChoice -= mHorizontalLength;
        }
    }

    mCurrentlySelectedTextChoice = mTexts[mMenuChoice];

}
```

```
// Allow vertical movement of the menu choice for text objects
void Menu::MoveTextMenuChoiceDown()
{

    int totalTextItems = mTexts.size();

    if (mWrapAround)
    {
        mTextMenuChoice = (mTextMenuChoice + mHorizontalLength) % totalTextItems;
    }
    else
    {
        if (mTextMenuChoice + mHorizontalLength < totalTextItems)
        {
            mTextMenuChoice += mHorizontalLength;
        }
    }

    mCurrentlySelectedTextChoice = mTexts[mTextMenuChoice];
}

// Increment menu choice for text objects
void Menu::DecrementTextMenuChoice()
{
    if (mWrapAround)
    {
        mTextMenuChoice = (mTextMenuChoice - 1 + mTexts.size()) % mTexts.size(); // Handle negative mod
    }
    else
    {
        if (mTextMenuChoice > 0)
        {
            --mTextMenuChoice;
        }
    }
    mCurrentlySelectedTextChoice = mTexts[mTextMenuChoice];
}
```

```
// Set menu option for text objects
Text* Menu::GetCurrentTextOption()
{
    return mCurrentlySelectedTextChoice;
}
```

Next, I began to write the logic for the settings file handling. This involved using the registry editor library and checking if the string pattern matched with the string pattern within the settings file. My justification for doing this is to ensure that settings can stored in a file and saved even after the user has closed the game. This prevents the user from constantly having to set their settings each time they open the game.

```cpp
void Game::LoadSettings()
{
    std::ifstream settingsFile(std::filesystem::current_path().string() + "\\settings.txt");
    if (!settingsFile.is_open()) {
        std::cerr << "Failed to open settings.txt" << std::endl;
        return;
    }

    std::string line;
    std::regex settingPattern(R"((\w+(?:\s\w+)*?)\s*:\s*(.*))");
    std::smatch match;

    while (std::getline(settingsFile, line)) {
        if (std::regex_match(line, match, settingPattern)) {
            const std::string& key = match[1];
            const std::string& value = match[2];

            if (key == "Scroll speed")
                mScrollSpeed = std::stoi(value);
            else if (key == "Receptor size")
                mReceptorSize = std::stoi(value);
            else if (key == "Scroll direction")
                mScrollDirection = value;
            else if (key == "Left keybind")
                mLeftKeybind = value;
            else if (key == "Down keybind")
                mDownKeybind = value;
            else if (key == "Up keybind")
                mUpKeybind = value;
            else if (key == "Right keybind")
                mRightKeybind = value;
        }
    }

    settingsFile.close();
}
```

```cpp
void Game::UpdateSettings()
{
    std::ostringstream updatedSettings;

    updatedSettings << "Scroll speed : " << mScrollSpeed << '\n';
    updatedSettings << "Receptor size : " << mReceptorSize << '\n';
    updatedSettings << "Scroll direction : " << mScrollDirection << '\n';
    updatedSettings << "Left keybind : " << mLeftKeybind << '\n';
    updatedSettings << "Down keybind : " << mDownKeybind << '\n';
    updatedSettings << "Up keybind : " << mUpKeybind << '\n';
    updatedSettings << "Right keybind : " << mRightKeybind << '\n';

    std::ofstream settingsFile(std::filesystem::current_path().string() + "\\settings.txt");
    if (!settingsFile.is_open()) {
        std::cerr << "Failed to open settings.txt for writing" << std::endl;
        return;
    }

    settingsFile << updatedSettings.str();
    settingsFile.close();
}
```

Once text objects have been added to the functionality of the menu, I began to write the actual menu logic for the settings menu. This involved adding an additional Boolean variable to determine whether the menu setting has been chosen. This is because there will be a difference between the highlighted menu option and the selected menu option that will be updated. After selection of the key bind settings menu option, the next key pressed will become their key bind for the main gameplay. The justification for having a key bind mode is to ensure that when the return key is pressed to select the menu option, the return key is not immediately set to a key bind. The code shown below only shows the settings menu section of the menu option. The rest of the menu logic that I wrote for the will be showed within the full source code appendix.

```
77      settingsMenu->SetHighlightAnimation(
78          [this,settingsMenu](){
79          settingsMenu->GetCurrentTextOption()->SetScale(true, 1.25,0.05);
80      });
81
82      settingsMenu->SetSelectionAnimation(
83          [this,settingsMenu](){
84          settingsMenu->GetCurrentTextOption()->SetColor(vec3(1.0f,1.0f,0.0f));
85      });
86
87      settingsMenu->SetUnselectionAnimation(
88          [this,settingsMenu](){
89          settingsMenu->GetCurrentTextOption()->SetColor(vec3(1.0f,1.0f,1.0f));
90      });
91
92      settingsMenu->SetUnhighlightAnimation(
93          [this,settingsMenu](){
94          settingsMenu->GetCurrentTextOption()->SetScale(true,1.0,0.1);
95          settingsMenu->GetCurrentTextOption()->SetColor(vec3(1.0f));
96      });
97
98  }
```

```
else if (mCurrentGameState == GameState::SETTINGS)
{
    Menu* settingsMenu = GetMenu(mCurrentGameState, "settings-menu");

    settingsMenu->UpdateCurrentTime();

    if(settingsMenu->CheckSelectionTime())
    {
        // Navigation between settings options
        if (event.key.keysym.sym == SDLK_UP && !mSelectedSetting)
        {
            settingsMenu->PlayUnhighlightAnimation();
            settingsMenu->DecrementTextMenuChoice();
            settingsMenu->PlayHighlightAnimation();
            settingsMenu->UpdateLastSelectedTime();
        }
        else if (event.key.keysym.sym == SDLK_DOWN && !mSelectedSetting)
        {
            settingsMenu->PlayUnhighlightAnimation();
            settingsMenu->IncrementTextMenuChoice();
            settingsMenu->PlayHighlightAnimation();
            settingsMenu->UpdateLastSelectedTime();
        }
        else if (event.key.keysym.sym == SDLK_RETURN)
        {
            if (!mSelectedSetting)
            {
                settingsMenu->PlaySelectionAnimation();
                mSelectedSetting = true;
            }
            else if (mKeybindMode)
            {
                // Re-enter into keybind mode only after exiting
                mKeybindMode = false;
                settingsMenu->GetCurrentMenuOption()->SetColor(vec3(1.0f));
            }
        }
```

```
267             if (currentText == GetText(mCurrentGameState, "scroll-speed-text"))
268             {
269                 if (event.key.keysym.sym == SDLK_RIGHT)
270                 {
271                     mScrollSpeed++;
272                     currentText->UpdateText(std::to_string(mScrollSpeed));
273                     UpdateSettings();
274                 }
275                 else if (event.key.keysym.sym == SDLK_LEFT)
276                 {
277                     mScrollSpeed--;
278                     currentText->UpdateText(std::to_string(mScrollSpeed));
279                     UpdateSettings();
280                 }
281             }
282             else if (currentText == GetText(mCurrentGameState, "receptor-size-text"))
283             {
284                 if (event.key.keysym.sym == SDLK_RIGHT)
285                 {
286                     mReceptorSize++;
287                     currentText->UpdateText(std::to_string(mReceptorSize) + "%");
288                     UpdateSettings();
289                 }
290                 else if (event.key.keysym.sym == SDLK_LEFT)
291                 {
292                     mReceptorSize--;
293                     currentText->UpdateText(std::to_string(mReceptorSize) + "%");
294                     UpdateSettings();
295                 }
296             }
297             else if(currentText == GetText(mCurrentGameState, "scroll-direction-text"))
298             {
299                 if (event.key.keysym.sym == SDLK_RIGHT || event.key.keysym.sym == SDLK_LEFT)
300                 {
301                     if (currentText->GetText() == "Down")
302                     {
303                         currentText->UpdateText("Up");
304                         mScrollDirection = "Up";
305                         UpdateSettings();
306                     }
307                     else if (currentText->GetText() == "Up")
308                     {
309                         currentText->UpdateText("Down");
310                         mScrollDirection = "Down";
311                         UpdateSettings();
312                     }
313                 }
314             }
```

```
315         else if (
316             currentText == GetText(mCurrentGameState, "left-keybind-text") ||
317             currentText == GetText(mCurrentGameState, "down-keybind-text") ||
318             currentText == GetText(mCurrentGameState, "up-keybind-text") ||
319             currentText == GetText(mCurrentGameState, "right-keybind-text"))
320         {
321             if (!mKeybindMode)
322             {
323                 mKeybindMode = true;
324                 settingsMenu->GetCurrentTextOption()->SetColor(vec3(1.0f,1.0f,0.0f)); // Change color to indicate keybind mode
325             }
326             else if (event.type == SDL_KEYDOWN) // Wait for the next key press
327             {
328                 const char* keyName = SDL_GetKeyName(event.key.keysym.sym);
329                 if (keyName && strlen(keyName) > 0) // Check if the key name is valid
330                 {
331                     std::string newKeybind = keyName;
332 
333                     if (currentText == GetText(mCurrentGameState, "left-keybind-text"))
334                         mLeftKeybind = newKeybind;
335                     else if (currentText == GetText(mCurrentGameState, "down-keybind-text"))
336                         mDownKeybind = newKeybind;
337                     else if (currentText == GetText(mCurrentGameState, "up-keybind-text"))
338                         mUpKeybind = newKeybind;
339                     else if (currentText == GetText(mCurrentGameState, "right-keybind-text"))
340                         mRightKeybind = newKeybind;
341 
342                     // Update the text and settings file
343                     currentText->UpdateText(newKeybind);
344                     UpdateSettings();
345 
346                     // Exit selection mode
347                     settingsMenu->GetCurrentTextOption()->SetColor(vec3(1.0f));
348                     mKeybindMode = false;
349                     mSelectedSetting = false;
350                 }
```

```
250             else if (event.key.keysym.sym == SDLK_ESCAPE)
251             {
252                 if (mSelectedSetting)
253                 {
254                     settingsMenu->PlayUnselectionAnimation();
255                     mSelectedSetting = false;
256                 }
257                 else
258                 {
259                     TransitionToGameState(GameState::MAIN_MENU);
260                 }
261             }
262         }
```

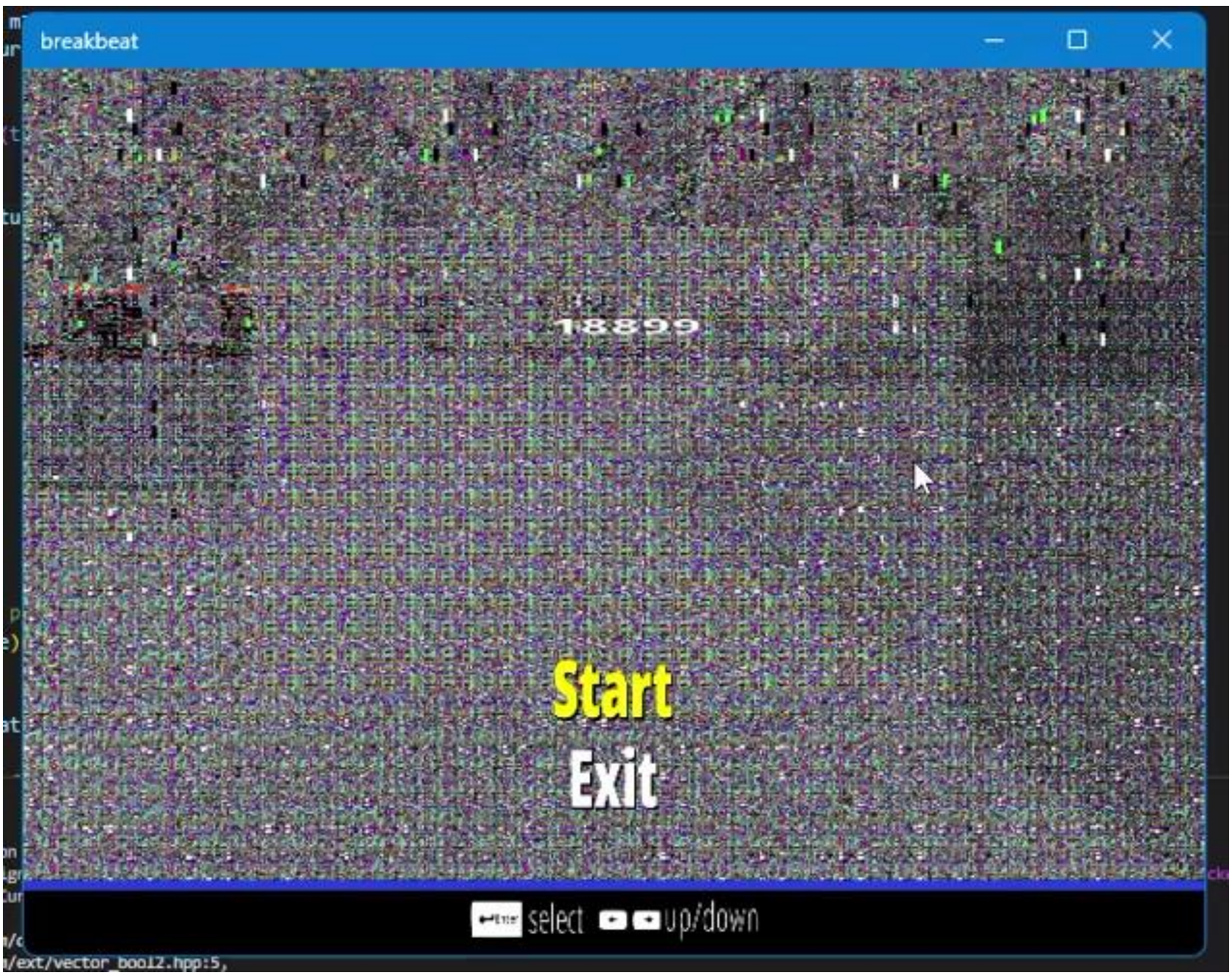
Afterwards, I then added the mouse collision code for the settings menu. The involved using the previously defined CheckCollidingSprite() procedure and UpdateSettings() procedure to dynamically update the rendered text and text file based on the mouse input. As mentioned above this is only the code for the settings menu. The rest of the code written for the main menu and previous sections will be shown within the appendix.

```
104         else if( mCurrentGameState == GameState::SETTINGS)
105         {
106             Menu* settingsMenu = GetMenu(mCurrentGameState, "settings-menu");
107 
108             if (mSelectedSetting)
109             {
110                 Text* currentText = settingsMenu->GetCurrentTextOption();
111 
112                 if (currentText == GetText(mCurrentGameState, "scroll-speed-text"))
113                 {
114                     if (collidingSprite == GetSprite(mCurrentGameState, "settings-increment-right-1"))
115                     {
116                         mScrollSpeed++;
117                         currentText->UpdateText(std::to_string(mScrollSpeed));
118                         UpdateSettings();
119                     }
120                     else if (collidingSprite == GetSprite(mCurrentGameState, "settings-increment-left-1"))
121                     {
122                         mScrollSpeed--;
123                         currentText->UpdateText(std::to_string(mScrollSpeed));
124                         UpdateSettings();
125                     }
126                 }
127                 else if (currentText == GetText(mCurrentGameState, "receptor-size-text"))
128                 {
129                     if (collidingSprite == GetSprite(mCurrentGameState, "settings-increment-right-2"))
130                     {
131                         mReceptorSize++;
132                         currentText->UpdateText(std::to_string(mReceptorSize) + "%");
133                         UpdateSettings();
134                     }
135                     else if (collidingSprite == GetSprite(mCurrentGameState,"settings-increment-left-2"))
136                     {
137                         mReceptorSize--;
138                         currentText->UpdateText(std::to_string(mReceptorSize) + "%");
139                         UpdateSettings();
140                     }
141                 }
142                 else if(currentText == GetText(mCurrentGameState, "scroll-direction-text"))
143                 {
144                     if (collidingSprite == GetSprite(mCurrentGameState, "settings-increment-right-3") ||
145                     collidingSprite == GetSprite(mCurrentGameState,"settings-increment-left-3"))
146                     {
147                         if (currentText->GetText() == "Down")
148                         {
149                             currentText->UpdateText("Up");
150                             mScrollDirection = "Up";
151                             UpdateSettings();
152                         }
153                         else if (currentText->GetText() == "Up")
154                         {
155                             currentText->UpdateText("Down");
156                             mScrollDirection = "Down";
157                             UpdateSettings();
158                         }
159                     }
160                 }
161             }
```

Testing

Error Encountered – Bindless Textures

To commence testing I ran the compiled program and immediately errors were prevalent. The games textures were not loading properly, there was visual artifacts and visible distortion upon the window. As well as that, after about 30 seconds of running my game, my laptop's GPU drivers would timeout, my screen would freeze and go black and before the application crashing totally. This was a major problem and was clearly to do the use incorrect usage of bindless textures. This is because bindless textures have direct access to GPU memory and therefore can also cause tremendous issues if the memory is not accessed properly. The footage of this error can be shown in the "Bindless Textures Error" video.



Reason For the Error

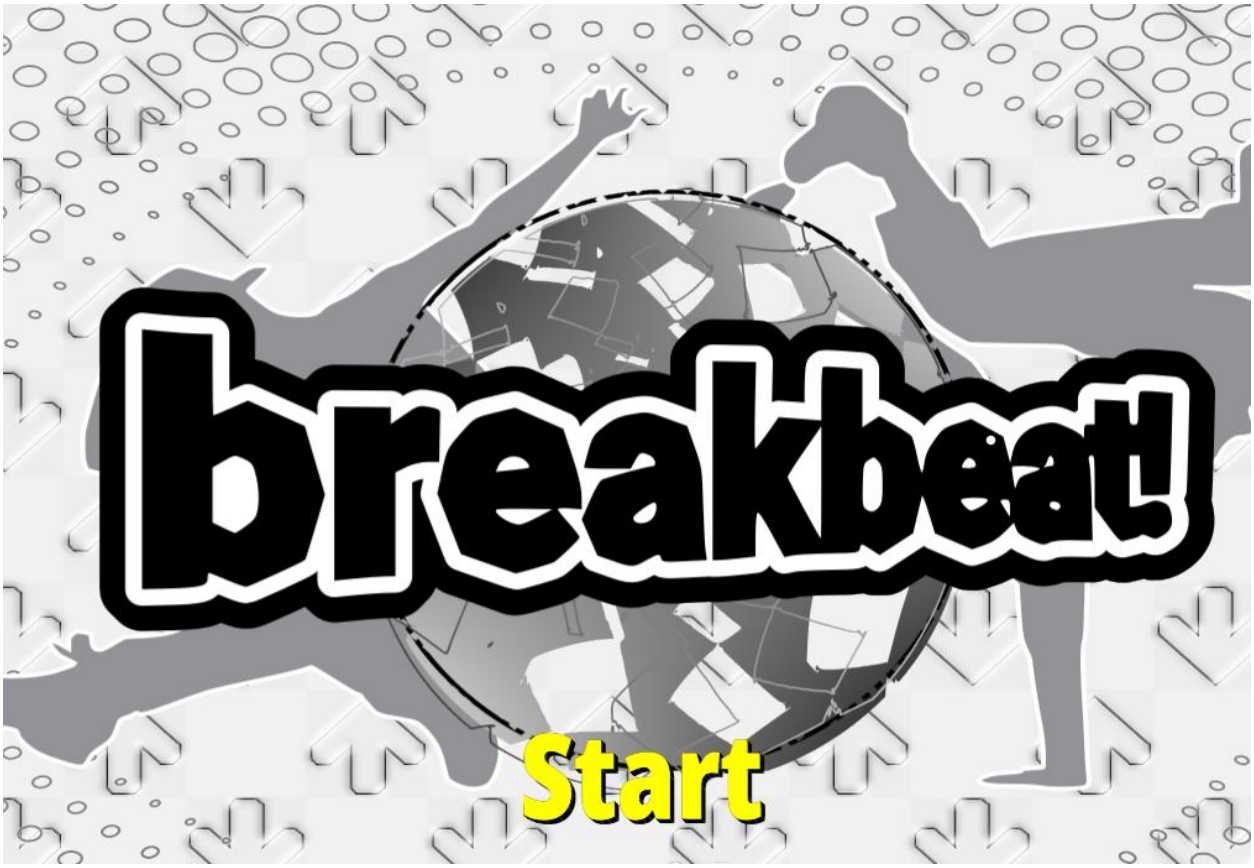
Upon spending about a day of searching for the cause of this error, I realised that the error laid within the texture destructor of my code. The issue was because my textures were prematurely being deleted before the being made into 64-bit unsigned integer reference handles. This meant the memory reference being accessed within the GPU was actually a reference to a random garbage value stored at that point in memory. This also explained why some textures were loading and some were not as it just so happened that the garbage value being stored in memory was the texture handle.

```
43  Texture::~Texture() {
44      if (glIsTexture(this->ID))
45          glDeleteTextures(1, &this->ID);
46      if (glIsTextureHandleResidentARB(this->handle))
47          glMakeTextureHandleNonResidentARB(this->handle);
48  }
```

Solution To the Error

Upon removing the texture destructor code from my code and running the program again, the program successfully loaded and was working as intended. This marked the passing of the first test case of the textures loading correctly.

The second test case was also passed as running the program showed the mouse being rendered on the screen. (The mouse is the white dot on the middle left of the photo below).



I then tested if navigating to the settings menu would load the settings menu interface and the settings menu interface successfully loaded as intended along with the settings text. This showed the text renderer system was working as intended.

I then tested if using the arrow keys to navigate the settings menu would work correctly and this was successful, I was able to change all scroll settings and key binds. Upon checking the settings file, the, the settings were successfully updated.

Upon pressing escape, I was able to successfully exit the settings currently selected menu option then exit the settings menu and terminate the program.

Chart Editor Selection

Test Plan

Test Plan	Expected Outcome
Running the compiled program	The program should successfully execute without any form errors relating to textures or low-level graphics implementation i.e. not immediately crashing, screen does not freeze or go black, and graphics drivers do not time out
Navigating to the “chart editor” user interface and pressing enter on the interface	The chart selection user interface should play the animation and successfully begin the transition into the chart selection

Transitioning to the chart editor selection user interface	As the game state is transitioning into the chart editor selection game state, the sprites should load successfully without
Pressing the left key whilst on the chart editor selection user interface	The text objects displaying the text for the song name and artist should “shift” up and the give the illusion of “scrolling up”
Pressing the right key whilst on the chart editor selection user interface	The text objects displaying the text for the song name and artist should “shift” down and give the illusion of “scrolling down”
Pressing the up key whilst on the chart editor selection	The text objects displaying the text for the difficulties should “shift” down and the giving the illusion of “scrolling down”
Pressing the down key whilst on the chart editor selection	The text objects displaying the text for the difficulties should “shift” up and the giving the illusion of “scrolling up”
Navigating using the mouse and proceeding to click on the create chart user interface/button	A file dialogue should allow the user to browse their files.
Choosing an audio file once on the file dialogue screen and pressing upload	The interface for the new chart screen should appear and their sprites in the background should darken as discussed in analysis. The text for the audio should show the path of the audio as text
Clicking on the different text objects on the new chart user interface	The user should be able to start type, and the text should update to the characters pressed
Beginning keyboard input and typing once clicked on the text box	The selected text box should match what the user is typing on their keyboard
Pressing enter/return once finished typing on the text box input	The text should no longer continue to log the user’s keystrokes and instead should remain as it is.
Clicking on the new image user interface and	Another file dialogue should appear giving the user the ability to browser through their files and search for an image file
Choosing an image file once on the file dialogue and pressing upload	The image text should show the image path as text
Clicking on the different text objects on the new difficulty user interface	The text objects should be updating their text as the user types of their keyboard
Clicking on the create button whilst on the new difficulty screen	The console should output a message stating a new chart file has been created and upon checking the charts directory there should be a newly created txt file

Clicking on the create button whilst on the new chart screen	The console should output a message stating a new chart file has been created and upon checking the charts directory there should be a newly created txt file
Clicking anywhere other than new chart and difficulty user interface	The user interface should immediately disappear, and the sprites should return to their original states from being darkened
Pressing the escape key	The new chart screen should fade out and the main menu sprites should transition and fade in as the game state changes.SS

Development

Before I begin development of the chart editor, I first must begin with the chart editor selection menu. For me to have any charts to edit, I first must have the ability to create a chart as per described in analysis. Therefore, I began development on the create chart button part of the chart editor menu.

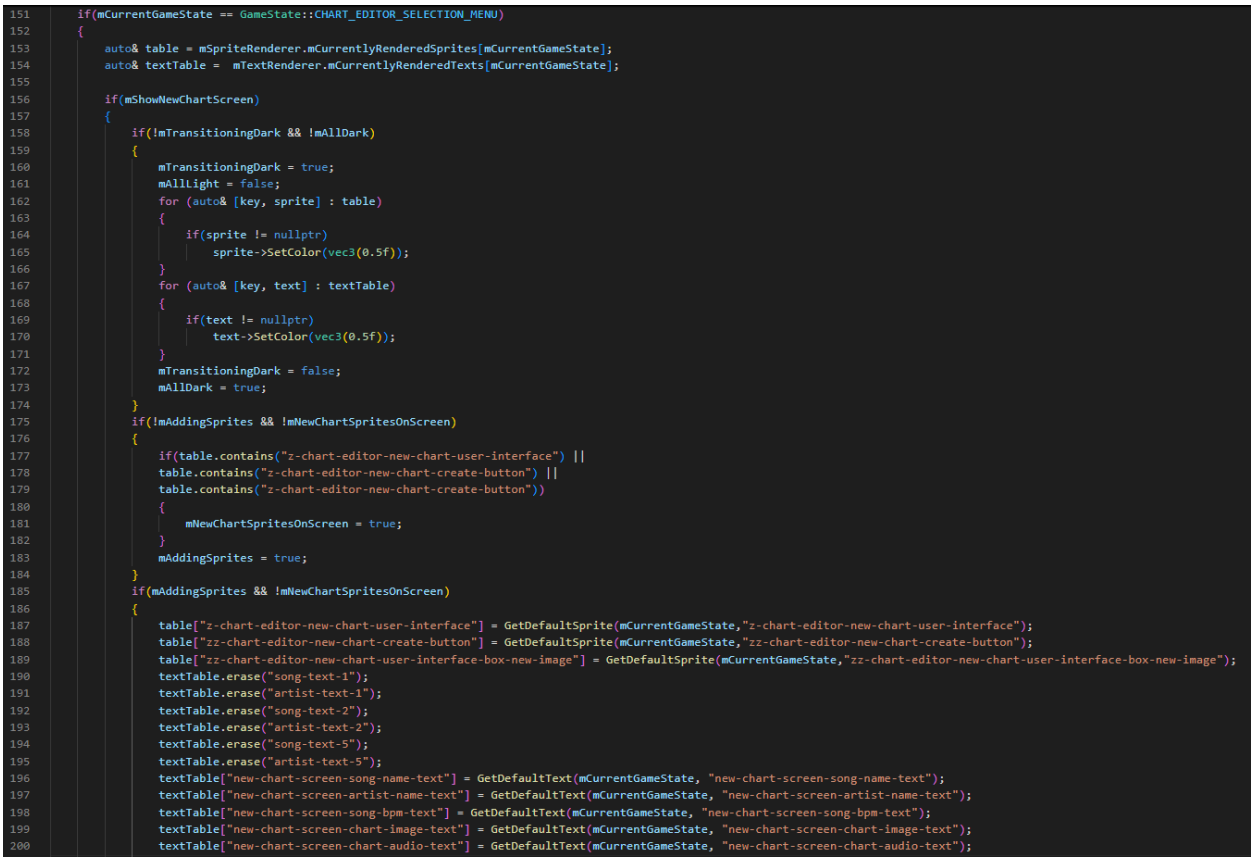
I first began by loading the textures and sprites for the chart editor selection menu. This included the button to create charts. However, the create button is different due to it only being accessible only if the create chart button is pressed via mouse input. My justification for making the create button mouse was described during analysis in which I stated that it will prevent the accidental creation of charts.

To do this, I first began writing code within the mouse input code to toggle the Boolean variable used to determine if the chart creation user interface will appear. This also involved the use of the tinyfilediaglogs file library to allow the user to upload an audio file. Furthermore, within this logic I implemented the case of whether the user does not upload a valid file path. In this case, the screen to show charts will not appear. My justification for doing this is to prevent the incorrect file paths being passed into the file creation of the chart.

```
175         else if (mCurrentGameState == GameState::CHART_EDITOR_SELECTION_MENU)
176         {
177             if (collidingSprite == GetSprite(mCurrentGameState, "chart-editor-new-chart-button") && !mShowNewChartScreen)
178             {
179                 // Open file explorer and store the selected path in mNewChartAudioPath
180                 const char* filterPatterns[] = {"*.mp3", "*.wav", "*.ogg"};
181                 const char* path = tinyfd_openFileDialog(
182                     "Select Audio File",  // Title of the dialog
183                     "",                   // Default path (empty for current directory)
184                     3,                    // Number of filter patterns
185                     filterPatterns,       // File type filters
186                     "Audio Files (*.mp3, *.wav, *.ogg)", // Description of filters
187                     0                     // Single selection (0: single file, 1: multiple files)
188                 );
189 
190                 if (path) // If a file was selected
191                     mNewChartAudioPath = string(path);
192                 else mNewChartAudioPath = ""; // Return an empty string if the user cancels
193 
194                 if (!mNewChartAudioPath.empty())
195                 {
196                     mShowNewChartScreen = true; // Show the new chart screen if a valid path is selected
197                 }
198                 else
199                 {
200                     mShowNewChartScreen = false; // Do not show the screen if no file is selected
201                 }
202             }
```

Afterwards, I then wrote the logic for the new chart screen to appear. Within this logic, I had to darken the sprites as mentioned in design for the chart editor. This involved simply using the built in SetDarken() procedure.

Furthermore, I had to use Boolean variables to determine whether the chart had to temporarily add and remove sprite and text objects and to add the letter z a few times into the identifiers of the sprites. This is because the nested map data structure within the SpriteRenderer's mCurrentlyRenderedSprites hash table is ordered and renders the sprite based on their name alphabetically.



```
151     if(mCurrentGameState == GameState::CHART_EDITOR_SELECTION_MENU)
152     {
153         auto& table = mSpriteRenderer.mCurrentlyRenderedSprites[mCurrentGameState];
154         auto& textTable =  mTextRenderer.mCurrentlyRenderedTexts[mCurrentGameState];
155 
156         if(mShowNewChartScreen)
157         {
158             if(!mTransitioningDark && !mAllDark)
159             {
160                 mTransitioningDark = true;
161                 mAllLight = false;
162                 for (auto& [key, sprite] : table)
163                 {
164                     if(sprite != nullptr)
165                         sprite->SetColor(vec3(0.5f));
166                 }
167                 for (auto& [key, text] : textTable)
168                 {
169                     if(text != nullptr)
170                         text->SetColor(vec3(0.5f));
171                 }
172                 mTransitioningDark = false;
173                 mAllDark = true;
174             }
175             if(!mAddingSprites && !mNewChartSpritesOnScreen)
176             {
177                 if(table.contains("z-chart-editor-new-chart-user-interface") ||
178                 table.contains("z-chart-editor-new-chart-create-button") ||
179                 table.contains("z-chart-editor-new-chart-create-button"))
180                 {
181                     mNewChartSpritesOnScreen = true;
182                 }
183                 mAddingSprites = true;
184             }
185             if(mAddingSprites && !mNewChartSpritesOnScreen)
186             {
187                 table["z-chart-editor-new-chart-user-interface"] = GetDefaultSprite(mCurrentGameState,"z-chart-editor-new-chart-user-interface");
188                 table["zz-chart-editor-new-chart-create-button"] = GetDefaultSprite(mCurrentGameState,"zz-chart-editor-new-chart-create-button");
189                 table["zz-chart-editor-new-chart-user-interface-box-new-image"] = GetDefaultSprite(mCurrentGameState,"zz-chart-editor-new-chart-user-interface-box-new-image");
190                 textTable.erase("song-text-1");
191                 textTable.erase("artist-text-1");
192                 textTable.erase("song-text-2");
193                 textTable.erase("artist-text-2");
194                 textTable.erase("song-text-5");
195                 textTable.erase("artist-text-5");
196                 textTable["new-chart-screen-song-name-text"] = GetDefaultText(mCurrentGameState, "new-chart-screen-song-name-text");
197                 textTable["new-chart-screen-artist-name-text"] = GetDefaultText(mCurrentGameState, "new-chart-screen-artist-name-text");
198                 textTable["new-chart-screen-song-bpm-text"] = GetDefaultText(mCurrentGameState, "new-chart-screen-song-bpm-text");
199                 textTable["new-chart-screen-chart-image-text"] = GetDefaultText(mCurrentGameState, "new-chart-screen-chart-image-text");
200                 textTable["new-chart-screen-chart-audio-text"] = GetDefaultText(mCurrentGameState, "new-chart-screen-chart-audio-text");
```

Afterwards, I then implemented the darkening of the sprites on screen for when the user clicks on the new chart button as described in the flowcharts of design. In this code I included checks to whether the sprite is a valid sprite to avoid read access violations that can potentially cause the game to crash. Furthermore, I made room for the new chart screen by erasing some the text on the chart user interface. This is to avoid the text overlapping with new chart screen.

```
else if (mShowNewDifficultyScreen)
{
    if (!mTransitioningDark && !mAllDark)
    {
        mTransitioningDark = true;
        mAllLight = false;
        for (auto& [key, sprite] : table)
        {
            if (sprite != nullptr)
                sprite->SetColor(vec3(0.5f));
        }
        for (auto& [key, text] : textTable)
        {
            if (text != nullptr && key != ("difficulty-select-box-text-1")
                && key != ("difficulty-select-box-text-2")
                && key != ("difficulty-select-box-text-3")
                && key != ("difficulty-select-box-text-4"))
                text->SetColor(vec3(0.5f));
        }
        mTransitioningDark = false;
        mAllDark = true;
    }
    if (!mAddingSprites && !mNewDifficultySpritesOnScreen)
    {
        if (table.contains("z-chart-editor-new-difficullty-user-interface-box") ||
            table.contains("zz-chart-editor-new-difficulty-user-interface-box-new-image") ||
            table.contains("zz-chart-editor-new-difficulty-create-button"))
        {
            mNewDifficultySpritesOnScreen = true;
        }
        mAddingSprites = true;
    }
    if (mAddingSprites && !mNewDifficultySpritesOnScreen)
    {
        table["z-chart-editor-new-difficullty-user-interface-box"] = GetDefaultSprite(mCurrentGameState, "z-chart-editor-new-difficullty-user-interface-box");
        table["zz-chart-editor-new-difficulty-user-interface-box-new-image"] = GetDefaultSprite(mCurrentGameState, "zz-chart-editor-new-difficulty-user-interface-box-n
        table["zz-chart-editor-new-difficulty-create-button"] = GetDefaultSprite(mCurrentGameState, "zz-chart-editor-new-difficulty-create-button");
        textTable.erase("song-text-3");
        textTable.erase("artist-text-3");
        textTable.erase("song-text-4");
        textTable.erase("artist-text-4");
        textTable.erase("song-text-5");
        textTable.erase("artist-text-5");
        textTable["new-difficulty-screen-difficulty-name-text"] = GetDefaultText(mCurrentGameState, "new-difficulty-screen-difficulty-name-text");
        textTable["new-difficulty-screen-difficulty-name-text-box"] = GetDefaultText(mCurrentGameState, "new-difficulty-screen-difficulty-name-text-box");
        textTable["new-difficulty-screen-chart-image-text"] = GetDefaultText(mCurrentGameState, "new-difficulty-screen-chart-image-text");
        mAddingSprites = false;
```

Afterwards, I wrote the procedures for updating the chart selection. The logic behind the procedure was rather simplified due to the use the built-in filesystem library. I first iterated through the charts directory and stored the name of each directory as a string inside a larger unified array. Afterwards I then used a secondary array to “preview” 7 charts at a time and update the text elements for the chart user interface based on the index of the preview point. This is due to only having 7 UI elements for the charts and therefor easier and more accessible for the user to navigate charts rather than all at once.

```
void Game::GetCurrentChartDirectories()
{
    // Iterate through the charts folder and store chart folder names
    const string chartDirectory = "charts";
    for (const auto& entry : std::filesystem::directory_iterator(chartDirectory))
    {
        if (entry.is_directory())
        {
            mAllCharts.push_back(entry.path().filename().string());
        }
    }
}
```

```
196  void Game::UpdateChartSelection()
197  {
198      unsigned totalCharts = mAllCharts.size();
199
200      // Ensure valid state when no charts are available
201      if (totalCharts == 0)
202      {
203          mCurrentlyPreviewedCharts.fill("");
204          for (unsigned i = 0; i < 7; ++i)
205          {
206              GetText(mCurrentGameState, "artist-text-" + to_string(i + 1))->UpdateText("NO SONG");
207              GetText(mCurrentGameState, "song-text-" + to_string(i + 1))->UpdateText("NO ARTIST");
208          }
209          return;
210      }
211
212      // Calculate the end index based on the start index
213      mChartPreviewEndIndex = (mChartPreviewStartIndex + 6) % totalCharts;
214
215      // Update `mCurrentlyPreviewedCharts` based on start and end indices
216      for (unsigned i = 0; i < 7; ++i)
217      {
218          unsigned currentIndex = (mChartPreviewStartIndex + i) % totalCharts;
219          mCurrentlyPreviewedCharts[i] = mAllCharts[currentIndex];
220      }
221
222      // Update the UI with artist and song names
223      for (unsigned i = 0; i < 7; ++i)
224      {
225          const string& chartName = mCurrentlyPreviewedCharts[i];
226          unsigned lastDelimiterPos = chartName.rfind('-'); // Find the last dash
227
228          if (lastDelimiterPos != string::npos)
229          {
230              string artistName = chartName.substr(0, lastDelimiterPos);
231              string songName = chartName.substr(lastDelimiterPos + 1);
232
233              // Update the respective text objects
234              GetText(mCurrentGameState, "artist-text-" + to_string(i + 1))->UpdateText(artistName);
235              GetText(mCurrentGameState, "song-text-" + to_string(i + 1))->UpdateText(songName);
236          }
237          else
238          {
239              // Clear the text for invalid chart name formats
240              GetText(mCurrentGameState, "artist-text-" + to_string(i + 1))->UpdateText("");
241              GetText(mCurrentGameState, "song-text-" + to_string(i + 1))->UpdateText("");
```

I then had to begin the logic of storing the chart difficulties as discussed during analysis and design for each chart within the chart difficulty user interface. Therefore, I reimplemented the same logic for retrieving charts however I iterated through the 4th directory within the currently previewed charts array and stored all ".txt" files. This is because all files chart within the chart directory will have the file extension ".txt". Furthermore, I iterated through the 4th directory within the charts array because its sprite user interface is highlighted to show that it's the selected song.

```
246  void Game::GetCurrentChartDifficulties()
247  {
248      // Get the currently selected chart directory
249      if (mCurrentlyPreviewedCharts.empty() || mCurrentlyPreviewedCharts[3].empty())
250      {
251          // Clear mAllCurrentChartDifficulties and UI if no valid chart
252          mAllCurrentChartDifficulties.clear();
253          for (unsigned i = 0; i < 4; ++i)
254          {
255              GetText(mCurrentGameState, "difficulty-select-box-text-" + to_string(i + 1))->UpdateText("NO DIFFICULTIES");
256          }
257          return;
258      }
259
260      string chartDirectory = "charts/" + mCurrentlyPreviewedCharts[3];
261      std::filesystem::path chartDirPath(chartDirectory);
262
263      // Ensure the directory exists
264      if (!std::filesystem::exists(chartDirPath) || !std::filesystem::is_directory(chartDirPath))
265      {
266          cerr << "Chart directory does not exist: " << chartDirectory << '\n';
267          return;
268      }
269
270      // Iterate through .txt files in the directory
271      mAllCurrentChartDifficulties.clear();
272
273      for (const auto& entry : std::filesystem::directory_iterator(chartDirPath))
274      {
275          if (entry.is_regular_file() && entry.path().extension() == ".txt")
276          {
277              string difficultyName = entry.path().stem().string(); // File name without extension
278              mAllCurrentChartDifficulties.push_back(difficultyName);
279          }
280      }
281  }
```

Once all of the difficulty files are loaded for the current chart. I then began writing a function to update the difficulty user interface for the chart difficulties. I first began by creating an array of size four that will again ”preview” the difficulties for the chart in chunks of four. Furthermore, I included the case of there being no difficulties within the difficulty array. My justification for doing this is to make the code more robust and the user interface clear and accessible.

```
284  void Game::UpdateCurrentChartDifficulties()
285  {
286      // Ensure there are difficulties to preview
287      if (mAllCurrentChartDifficulties.empty())
288      {
289          mCurrentlyPreviewedDifficulties.fill("");
290          for (unsigned i = 0; i < 4; ++i)
291          {
292              GetText(mCurrentGameState, "difficulty-select-box-text-" + to_string(i + 1))->UpdateText("NO DIFFICULTIES");
293          }
294          return;
295      }
296
297      // Update mCurrentlyPreviewedDifficulties based on circular queue logic
298      unsigned totalDifficulties = mAllCurrentChartDifficulties.size();
299      for (unsigned i = 0; i < 4; ++i)
300      {
301          unsigned currentIndex = (mChartDifficultyPreviewStartIndex + i) % totalDifficulties;
302          mCurrentlyPreviewedDifficulties[i] = mAllCurrentChartDifficulties[currentIndex];
303      }
```

Afterwards, I then continued the procedure by writing the code to open and read the current difficulty file and update the contents of the difficulty value on the difficulty user interface. This involved the use of the C++ regular expression (regex) library to perform pattern matching in within strings. My justification for using regex library is because the

library is prewritten which makes versatile and has also been tested. This to ensure the that the value can read from the file regardless of minor pattern differences E.g. there being 3 digits in the difficulty value in 2.

```
297     // Update mCurrentlyPreviewedDifficulties based on circular queue logic
298     unsigned totalDifficulties = mAllCurrentChartDifficulties.size();
299     for (unsigned i = 0; i < 4; ++i)
300     {
301         unsigned currentIndex = (mChartDifficultyPreviewStartIndex + i) % totalDifficulties;
302         mCurrentlyPreviewedDifficulties[i] = mAllCurrentChartDifficulties[currentIndex];
303     }
304
305
306     // Read difficulty values from the corresponding files and update UI
307     regex difficultyRegex(R"(Difficulty\s*:\s*([\d.,]+))"); // Regex to match difficulty value
308     for (unsigned i = 0; i < 4; ++i)
309     {
310         const string& difficultyName = mCurrentlyPreviewedDifficulties[i];
311         if (difficultyName.empty())
312         {
313             GetText(mCurrentGameState, "difficulty-select-box-text-" + to_string(i + 1))->UpdateText("NO DIFFICULTIES");
314             continue;
315         }
316
317         // Build the file path for the difficulty file
318         string fileName = "charts/" + mCurrentlyPreviewedCharts[3] + "/" + difficultyName + ".txt";
319
320         // Read the difficulty value from the file
321         ifstream difficultyFile(fileName);
322         if (!difficultyFile.is_open())
323         {
324             cerr << "Failed to open difficulty file: " << fileName << '\n';
325             GetText(mCurrentGameState, "difficulty-select-box-text-" + to_string(i + 1))->UpdateText(difficultyName + " : ERROR");
326             continue;
327         }
328
329         string line;
330         string difficultyValue = "N/A"; // Default if no difficulty line found
331         while (getline(difficultyFile, line))
332         {
333             smatch match;
334             if (regex_search(line, match, difficultyRegex) && match.size() == 2)
335             {
336                 difficultyValue = match[1].str();
337                 break; // Found the difficulty value, stop reading
338             }
339         }
340
341         difficultyFile.close();
342
343         // Update the text for the corresponding difficulty box
344         string displayText = difficultyName + " : " + difficultyValue;
345         GetText(mCurrentGameState, "difficulty-select-box-text-" + to_string(i + 1))->UpdateText(displayText);
346     }
347
348     mCurrentSongDifficulty = mCurrentlyPreviewedDifficulties[3];
349 }
```

Once I wrote the procedures for updating the chart selection, I then began to write the logic for the input required to update the user interface. This involved repeating the same logic as shown in the flowcharts as discussed in design and the logic within the main menu section. The logic first involved passing in the SDL_Event type to check for keypresses and it involved incrementing or decrementing the preview index based on whether left or right is pressed. As well as that it involved updating the index and then having it modulo by the size of the chart selection array. This is to ensure the that text display for the chart selection user interface maintains a circular queue logic, i.e. the text being previewed will" loop around" when scrolling through each text. My justification for having this circular queue logic is to ensure that user interface has no irregularities or problems in cases where there are empty directories.

```
351  void Game::HandleChartScrolling(SDL_Event& event)
352  {
353      if (event.type == SDL_KEYDOWN && !mNewChartSpritesOnScreen)
354      {
355          unsigned totalCharts = mAllCharts.size();
356
357          // Ensure there are charts to scroll through
358          if (totalCharts == 0)
359          {
360              return;
361          }
362
363          switch (event.key.keysym.sym)
364          {
365          case SDLK_RIGHT:
366              {
367                  // Increment start index and wrap around if necessary
368                  mChartPreviewStartIndex = (mChartPreviewStartIndex + 1) % totalCharts;
369
370                  // Update the currently previewed charts
371                  UpdateChartSelection();
372
373                  GetCurrentChartDifficulties();
374
375                  UpdateCurrentChartDifficulties();
376
377                  break;
378              }
```

The following code logic remained mostly the same except it contained the use of setting the index to the size of the chart array minus 1. My justification for doing this is the implementation of the “wrapping around” mechanism in the circular queue logic. As well as that it involved adding the transition into the main gameplay for later usage and the case

of when the user presses other keys

```
379         case SDLK_LEFT:
380             {
381                 // Decrement start index and wrap around if necessary
382                 if (mChartPreviewStartIndex == 0)
383                 {
384                     mChartPreviewStartIndex = totalCharts - 1;
385                 }
386                 else
387                 {
388                     mChartPreviewStartIndex = (mChartPreviewStartIndex - 1) % totalCharts;
389                 }
390 
391                 // Update the currently previewed charts
392                 UpdateChartSelection();
393 
394                 GetCurrentChartDifficulties();
395 
396                 break;
397             }
398         case SDLK_RETURN:
399             TransitionToGameState(GameState::MAIN_GAMEPLAY);
400             break;
401         default:
402             // Do nothing for other keys
403             return;
404         }
405     }
406 }
```

Afterwards I then transposed the logic required for the difficulty selection scrolling from the chart selection procedure. The logic was mainly the same except the use of a different index variable for array for the difficulty array.

```
408  void Game::HandleDifficultyScrolling(SDL_Event& event)
409  {
410      if (event.type != SDL_KEYDOWN)
411          return;
412
413      unsigned totalDifficulties = mAllCurrentChartDifficulties.size();
414      if (totalDifficulties == 0)
415          return;
416
417      switch (event.key.keysym.sym)
418      {
419      case SDLK_UP:
420          {
421              // Decrement the start index (circularly)
422              if (mChartDifficultyPreviewStartIndex == 0)
423                  mChartDifficultyPreviewStartIndex = totalDifficulties - 1;
424              else
425                  --mChartDifficultyPreviewStartIndex;
426
427              // Update the end index (circularly)
428              mChartDifficultyPreviewEndIndex =
429                  (mChartDifficultyPreviewStartIndex + 3) % totalDifficulties;
430
431              // Update the displayed difficulties
432              GetCurrentChartDifficulties();
433              UpdateCurrentChartDifficulties();
434              break;
435          }
436
437      case SDLK_DOWN:
438          {
439              // Increment the start index (circularly)
440              mChartDifficultyPreviewStartIndex =
441                  (mChartDifficultyPreviewStartIndex + 1) % totalDifficulties;
442
443              // Update the end index (circularly)
444              mChartDifficultyPreviewEndIndex =
445                  (mChartDifficultyPreviewStartIndex + 3) % totalDifficulties;
446
447              // Update the displayed difficulties;
448              GetCurrentChartDifficulties();
449              UpdateCurrentChartDifficulties();
450              break;
451          }
452
453      default:
454          break;
455      }
456  }
```

Once I had handled the chart selection logic, I began to write the library for the playing the audio for the selected song when selecting the song chart to edit. This process involved the use of the irrKlang audio library as well as the use of more file reading. The process first began by checking the chart arrays are empty. If they are empty then, the procedure will terminate. Afterwards, the procedure assimilates the directory for the current chart file using a combination of the chart name for the path and the difficulty name for the name of the chart file.

My justification for the procedure finding the path of the current chart is because it will access the path of the audio and image to within the song metadata within the file. This is

to allow the possibility for different chart files within the same song directory to have different image files for backgrounds (and potentially different audios for different difficulties).

```
472  void Game::PlayCurrentlySelectedChartAudio()
473  {
474      // Ensure there are charts and difficulties available
475      if (mCurrentlyPreviewedCharts.empty() || mCurrentlyPreviewedDifficulties.empty())
476      {
477          cerr << "No charts or difficulties available to preview audio." << '\n';
478          return;
479      }
480
481      // Get the chart folder name for the 4th index (0-based, index 3)
482      string chartFolderName = mCurrentlyPreviewedCharts[3]; // 4th chart in preview
483
484      // Get the selected difficulty file path
485      string selectedDifficulty = mCurrentlyPreviewedDifficulties[2];
486      string difficultyFilePath = "charts/" + chartFolderName + "/" + selectedDifficulty + ".txt";
487
488      // Open the difficulty file
489      ifstream difficultyFile(difficultyFilePath);
490      if (!difficultyFile.is_open())
491      {
492          cerr << "Failed to open difficulty file: " << difficultyFilePath << '\n';
493          return;
494      }
495
```

Afterwards, I then again used the C++ regex library for pattern matching in finding the audio path within the process of reading the chart file. This involved using the smatch data type to check if whether the match pattern matches the regular expression within the file. If this is true, then it will execute the regular expression on the string and return the secondary part of the string in the match which is the audio file. Another justification of the use of regex library is that it makes the code mode robust and overall, more maintainable.

Once the path to the audio file has been found it will then store the path within a variable for use in the loading of the audio using the audio library.

```
496         // Read the file line by line to find the "Audio : " line
497         string line;
498         string audioPath;
499         regex audioRegex(R"(Audio\s*:\s*(.+))");
500
501         while (getline(difficultyFile, line))
502         {
503             smatch match;
504             if (regex_match(line, match, audioRegex))
505             {
506                 if (match.size() == 2) // Ensure the match captures the path
507                 {
508                     audioPath = match[1].str();
509                     break;
510                 }
511             }
512         }
513
514         mCurrentChartAudioFile = audioPath;
515
516         difficultyFile.close();
```

The remaining part of the code involved retrieving the absolute path of the audio library relative to the user's system directory using the filesystem library. Furthermore, the code the use the irrKlang sound library to retrieve the duration of the audio file. My justification for retrieving the absolute path in this manner is to ensure that the audio file will be able to play regardless of where the user's puts the game folder. As well as that my justification for storing the song duration within a member variable is for the later use in main gameplay when showing the time elapsed for since the song audio started and for showing the song length in the chart selection menu.

```
517
518         // Ensure an audio path was found
519         if (audioPath.empty())
520         {
521             std::cerr << "No audio path found in difficulty file: " << difficultyFilePath << '\n';
522             return;
523         }
524
525         // Stop currently playing audio (if any) and play the new audio
526         if (mSoundEngine)
527         {
528             // Resolve the full path to the audio file
529             std::filesystem::path fullAudioPath = std::filesystem::current_path() / audioPath;
530
531             mSoundEngine->stopAllSounds();
532             mSoundEngine->play2D(fullAudioPath.string().c_str(), true);// Play the new audio
533             mCurrentSongDuration = mSoundEngine->getSoundSource(fullAudioPath.string().c_str())->getPlayLength();
534         }
535         else
536         {
537             std::cerr << "Sound engine is not initialized." << '\n';
538         }
539     }
```

I then began to finalize showing the new chart screen code. This involved adding logic to MouseInput.cpp file to remove the new chart screen sprites once the user clicked on

anywhere other than the new chart screen and then setting the removingSprites Boolean to true.

```
295         else if (mNewDifficultySpritesOnScreen)
296         {
297             if (collidingSprite != GetSprite(mCurrentGameState, "z-chart-editor-new-difficullty-user-interface-box") &&
298                 collidingSprite != GetSprite(mCurrentGameState, "zz-chart-editor-new-difficulty-create-button") &&
299                 collidingSprite != GetSprite(mCurrentGameState, "zz-chart-editor-new-difficulty-user-interface-box-new-image"))
300             {
301                 mShowNewDifficultyScreen = false;
302                 removingSprites = true;
303             }
```

I then set wrote the code for erasing the sprites of the screen once the removingSprites Boolean is set to true. Once the sprites were removed, I then set the mNewChartSpritesOnScreen Boolean to false. This is to prevent the rendering of the new charts until the new chart button is clicked and the Boolean is set to true again.

```
114     else if(removingSprites)
115     {
116         if(table.contains("z-chart-editor-new-chart-user-interface") ||
117             table.contains("zz-chart-editor-new-chart-create-button") ||
118             table.contains("zz-chart-editor-new-chart-user-interface-box-new-image") ||
119             table.contains("z-chart-editor-new-difficullty-user-interface-box") ||
120             table.contains("zz-chart-editor-new-difficulty-user-interface-box-new-image") ||
121             table.contains("zz-chart-editor-new-difficulty-create-button") )
122         {
123             table.erase("z-chart-editor-new-chart-user-interface");
124             table.erase("zz-chart-editor-new-chart-create-button");
125             table.erase("zz-chart-editor-new-chart-user-interface-box-new-image");
126 
127             textTable.erase("new-chart-screen-song-name-text");
128             textTable.erase("new-chart-screen-song-name-text-box");
129 
130             textTable.erase("new-chart-screen-artist-name-text");
131             textTable.erase("new-chart-screen-artist-name-text-box");
132 
133             textTable.erase("new-chart-screen-difficulty-name-text");
134             textTable.erase("new-chart-screen-difficulty-name-text-box");
135 
136             textTable.erase("new-chart-screen-song-bpm-text");
137             textTable.erase("new-chart-screen-song-bpm-text-box");
138 
139             textTable.erase("new-chart-screen-chart-image-text");
140             textTable.erase("new-chart-screen-chart-audio-text");
141 
142             mNewChartSpritesOnScreen = false;
```

I then wrote the remaining for restoring the original state of the chart editor selection menu once the sprites for adding the new chart user interface were removed. This involved restoring the state of the text objects back to their defaults. Immediately after this the texts objects would be updated to match the text of the currently selected chart via the already written GetCurrentChartDirectories() and UpdateChartSelection() procedures.

```
155             // Restore the standard chart UI elements
156             textTable["song-text-3"] = GetDefaultText(mCurrentGameState, "song-text-3");
157             textTable["artist-text-3"] = GetDefaultText(mCurrentGameState, "artist-text-3");
158             textTable["song-text-4"] = GetDefaultText(mCurrentGameState, "song-text-4");
159             textTable["artist-text-4"] = GetDefaultText(mCurrentGameState, "artist-text-4");
160             textTable["song-text-5"] = GetDefaultText(mCurrentGameState, "song-text-5");
161             textTable["artist-text-5"] = GetDefaultText(mCurrentGameState, "artist-text-5");
162
163
164             // Reload the charts from the directory
165             GetCurrentChartDirectories();
166
167             // Update the chart selection to reflect the current state
168             UpdateChartSelection();
169             removingSprites = true;
170         }
```

To finish this section of the code that will make the chart editor selection menu sprites return to their original state, I wrote the logic required for the sprites to return to their original color from being darkened. This involved iterating through each currently rendered sprite and then setting their color back to 1 using the written SetColor() procedure. However, in this code, I wrote logic to exclude making the text objects for showing the difficulty calculation for the chart's difficulty brighter. My justification for doing this is to ensure that the user will be able to see the text objects on the user interface. This is due to user interface having a similar contrast to white text and therefore if the text was white then the user would have trouble distinguishing the text.

```
171         if(!mTransitioningLight && !mAllLight)
172         {
173             mTransitioningLight = true;
174             for (auto& [key, sprite] : table)
175             {
176                 if(sprite != nullptr)
177                     sprite->SetColor(vec3(1.0f));
178             }
179             for (auto& [key, text] : mTextRenderer.mCurrentlyRenderedTexts[mCurrentGameState])
180             {
181                 if (text != nullptr && key != ("difficulty-select-box-text-1")
182                     && key != ("difficulty-select-box-text-2")
183                     && key != ("difficulty-select-box-text-3")
184                     && key != ("difficulty-select-box-text-4"))
185                     text->SetColor(vec3(1.0f));
186                 else
187                     text->SetColor(vec3(0.0f));
188             }
189             mTransitioningLight = false;
190             mAllLight = true;
191             mAllDark = false;
192         }
193     }
194 }
```

Afterwards I then began writing the logic required for adding a new difficulty via the new difficulty button user interface. This code would be triggered by the previously written code of when the user uses mouse input to trigger the mDifficultySpritesOnScreen Boolean. Once the Boolean is true, it will further check if any mouse collision is made outside the interface sprites and immediately imitate the process of removing the sprites.

Furthermore, the code then checks for mouse input on the text object for the chart difficulty name to allow the user to type input for the chart difficulty name.

Afterwards the code the sets the corresponding text object to the text object to edited. This means that each text object can be handle individually, instead having to write code for each section. My justification for doing this is to ensure that following procedures containing the use text objects can be reused in areas of my code without needing to rewrite it

```
295     else if (mNewDifficultySpritesOnScreen)
296     {
297         if (collidingSprite != GetSprite(mCurrentGameState, "z-chart-editor-new-difficullty-user-interface-box") &&
298             collidingSprite != GetSprite(mCurrentGameState, "zz-chart-editor-new-difficulty-create-button") &&
299             collidingSprite != GetSprite(mCurrentGameState, "zz-chart-editor-new-difficulty-user-interface-box-new-image"))
300         {
301             mShowNewDifficultyScreen = false;
302             removingSprites = true;
303         }
304         if (collidingText == GetText(mCurrentGameState, "new-difficulty-screen-difficulty-name-text"))
305         {
306             if (!mTypingMode)
307             {
308                 mCurrentTextBox = GetText(mCurrentGameState, "new-difficulty-screen-difficulty-name-text-box");
309                 mTypingMode = true;
310             }
311         }
```

However, for the user to have the ability to type anything, I needed to create a procedure for emulating the "typing" illusion on the text objects. This simply meant update the text object's text property whenever the user pressed a corresponding key. Therefore, I began writing the logic for handling keyboard input, this involved first checking if the user is in the game state in which they can type. I then began by writing the logic to check what key the user has pressed.

If the user pressed the backspace key, the program will remove the character from the current text object's character array and update it. Otherwise, when the user pressed the space bar, the program will append a space to the text object's text array.

```cpp
#include "Game.hpp"

void Game::HandleKeyboardInput(SDL_Event& event)
{
    if (mTypingMode)
    {
        if (event.type == SDL_KEYDOWN) // Wait for the next key press
        {
            SDL_Keycode keyCode = event.key.keysym.sym;

            // Define the maximum window size for the text box
            string text = mCurrentTextBox->GetText();

            // Handle Backspace key to delete the last character
            if (keyCode == SDLK_BACKSPACE)
            {
                if (!text.empty()) // Check if there's text to delete
                {
                    text.pop_back(); // Remove the last character
                    mCurrentTextBox->UpdateText(text);
                }
            }
            // Handle spaces
            else if (keyCode == SDLK_SPACE)
            {
                mCurrentTextBox->UpdateText(text += " ");
            }
```

I then continued the procedure by checking for all the other key inputs. This involved using if statements and switch/case chains to compare the key name of the character. Furthermore, the logic included the checking of whether the user is pressing the shift or caps lock key. If the user pressed the shift key, then the text will be updated based on what they key is.

```
28                  // Check if the key is a valid character (printable ASCII)
29                  else if ((keyCode >= 32 && keyCode <= 126))
30                  {
31                      const char* keyName = SDL_GetKeyName(keyCode);
32                      if (keyName && strlen(keyName) == 1) // Ensure it's a single character
33                      {
34                          char inputChar = keyName[0];
35
36                          // Check Shift state
37                          SDL_Keymod keyMod = SDL_GetModState();
38                          bool isShiftPressed = keyMod & KMOD_SHIFT;
39
40                          // Map shifted keys for special characters
41                          if (isShiftPressed)
42                          {
43                              switch (inputChar)
44                              {
45                                  case '1': inputChar = '!'; break;
46                                  case '2': inputChar = '@'; break;
47                                  case '3': inputChar = '#'; break;
48                                  case '4': inputChar = '$'; break;
49                                  case '5': inputChar = '%'; break;
50                                  case '6': inputChar = '^'; break;
51                                  case '7': inputChar = '&'; break;
52                                  case '8': inputChar = '*'; break;
53                                  case '9': inputChar = '('; break;
54                                  case '0': inputChar = ')'; break;
55                                  case '-': inputChar = '_'; break;
56                                  case '=': inputChar = '+'; break;
57                                  case '[': inputChar = '{'; break;
58                                  case ']': inputChar = '}'; break;
59                                  case '\\': inputChar = '|'; break;
60                                  case ';': inputChar = ':'; break;
61                                  case '\'': inputChar = '"'; break;
62                                  case ',': inputChar = '<'; break;
63                                  case '.': inputChar = '>'; break;
64                                  case '/': inputChar = '?'; break;
65                              }
66                          }
```

If the user pressed either the caps lock key or the shift key, then the program will convert the character input into the lower and uppercase characters accordingly. Finally, the use will be able to exit the typing state via pressing the return key.

```
67
68                          // Check Caps Lock and adjust alphabetic case
69                          bool isCapsLockOn = keyMod & KMOD_CAPS;
70                          if (inputChar >= 'A' && inputChar <= 'Z')
71                          {
72                              if (!(isCapsLockOn ^ isShiftPressed))
73                              {
74                                  inputChar = inputChar + ('a' - 'A'); // Convert to lowercase
75                              }
76                          }
77                          else if (inputChar >= 'a' && inputChar <= 'z')
78                          {
79                              if (isCapsLockOn ^ isShiftPressed)
80                              {
81                                  inputChar = inputChar - ('a' - 'A'); // Convert to uppercase
82                              }
83                          }
84
85                          mCurrentTextBox->UpdateText(text += inputChar);
86                      }
87                  }
88                  else if (keyCode == SDLK_RETURN) // Exit typing mode on Enter key
89                  {
90                      mTypingMode = false;
91                  }
92              }
93          }
94      }
95
```

Next, I then wrote the logic for allowing new image files to be uploaded and set as the background for the new difficulty. This involved writing to code to give the user the possibility of prompting a file dialogue to select an image for the new difficulty. My justification for this is to allow the possibility for different background images for different difficulty files to help distinguish between the different types of charts within the file. Furthermore, the code updated the image text object to display the path of the image. My justification for doing this is to ensure that the user does not need remember what image they have chosen based on details such as the image name when they are writing metadata for the new difficulty. This will make the creation more accessible and usable.

```
312     else if (collidingSprite == GetSprite(mCurrentGameState, "zz-chart-editor-new-difficulty-user-interface-box-new-image"))
313     {
314         // Open file dialog to select the image
315         const char* filterPatterns[] = { "*.png", "*.jpg", "*.jpeg", "*.bmp" };
316         const char* imagePath = tinyfd_openFileDialog(
317             "Select Image File",                // Title of the dialog
318             "",                                 // Default path
319             4,                                  // Number of filter patterns
320             filterPatterns,                     // Image file extensions
321             "Image Files (*.png, *.jpg, *.jpeg, *.bmp)", // Description of filters
322             0                                   // Single selection mode
323         );
324         if (imagePath) // If a file was selected
325         {
326             mNewChartImagePath = std::string(imagePath);
327             GetText(mCurrentGameState, "new-difficulty-screen-chart-image-text")->UpdateText("Image : " + mNewChartImagePath);
328         }
329         else
330         {
331             mNewChartImagePath = ""; // If no file was selected, leave it empty
332         }
333     }
```

I then began to write the logic for the creation of a new chart difficulty. This section of code checks for the mouse input on the create button for the new difficulty. If this is the case,

the code fetches the directory of the current chart and reads the chart file then copies some the data into the new chart file using the C++ regex library. This is done by storing the data within member variables to be used later.

```
334     else if (collidingSprite == GetSprite(mCurrentGameState, "zz-chart-editor-new-difficulty-create-button"))
335     {
336         // Get the directory of the currently selected chart
337         std::string chartDirectory = "charts\\" + mCurrentlyPreviewedCharts[3];
338         std::filesystem::path chartDirPath(chartDirectory);
339
340         // Ensure the directory exists
341         if (!std::filesystem::exists(chartDirPath) || !std::filesystem::is_directory(chartDirPath))
342         {
343             cerr << "Chart directory does not exist: " + chartDirectory + '\n';
344             return;
345         }
346
347         // Path to the chart difficulty file (assuming a consistent file naming convention)
348         std::string fileName = chartDirectory + "\\" + mCurrentlyPreviewedDifficulties[2] + ".txt";
349
350         // Open and read the chart difficulty file
351         std::ifstream difficultyFile(fileName);
352         if (!difficultyFile.is_open())
353         {
354             cerr << "Failed to open difficulty file: " + fileName + '\n';
355             return;
356         }
357
358         // Regex patterns for extracting chart details
359         std::regex songNameRegex(R"(Song Name\s*:\s*(.+))");
360         std::regex artistNameRegex(R"(Artist\s*:\s*(.+))");
361         std::regex bpmRegex(R"(BPM\s*:\s*(.+))");
362         std::regex audioPathRegex(R"(Audio\s*:\s*(.+))");
363         std::regex imagePathRegex(R"(Background Image\s*:\s*(.+))");
```

```
365         // Variables to store extracted data
366         std::string songName, artistName, bpm, audioPath, imagePath;
367
368         std::string line;
369         while (std::getline(difficultyFile, line))
370         {
371             std::smatch match;
372
373             if (std::regex_search(line, match, songNameRegex) && match.size() == 2)
374             {
375                 songName = match[1].str();
376             }
377             else if (std::regex_search(line, match, artistNameRegex) && match.size() == 2)
378             {
379                 artistName = match[1].str();
380             }
381             else if (std::regex_search(line, match, bpmRegex) && match.size() == 2)
382             {
383                 bpm = match[1].str();
384             }
385             else if (std::regex_search(line, match, audioPathRegex) && match.size() == 2)
386             {
387                 audioPath = match[1].str();
388             }
389             else if (std::regex_search(line, match, imagePathRegex) && match.size() == 2)
390             {
391                 imagePath = match[1].str();
392             }
393         }
394         difficultyFile.close();
395
396         // Get user input for new difficulty and image
397         std::string newDifficultyName = GetText(mCurrentGameState, "new-difficulty-screen-difficulty-name-text-box")->GetText();
398         std::string newImagePath = GetText(mCurrentGameState, "new-difficulty-screen-chart-image-text")->GetText().substr(8);
399
400         if (newDifficultyName.empty())
401         {
402         std::cerr << "New difficulty name cannot be empty" << '\n';
403             return;
404         }
```

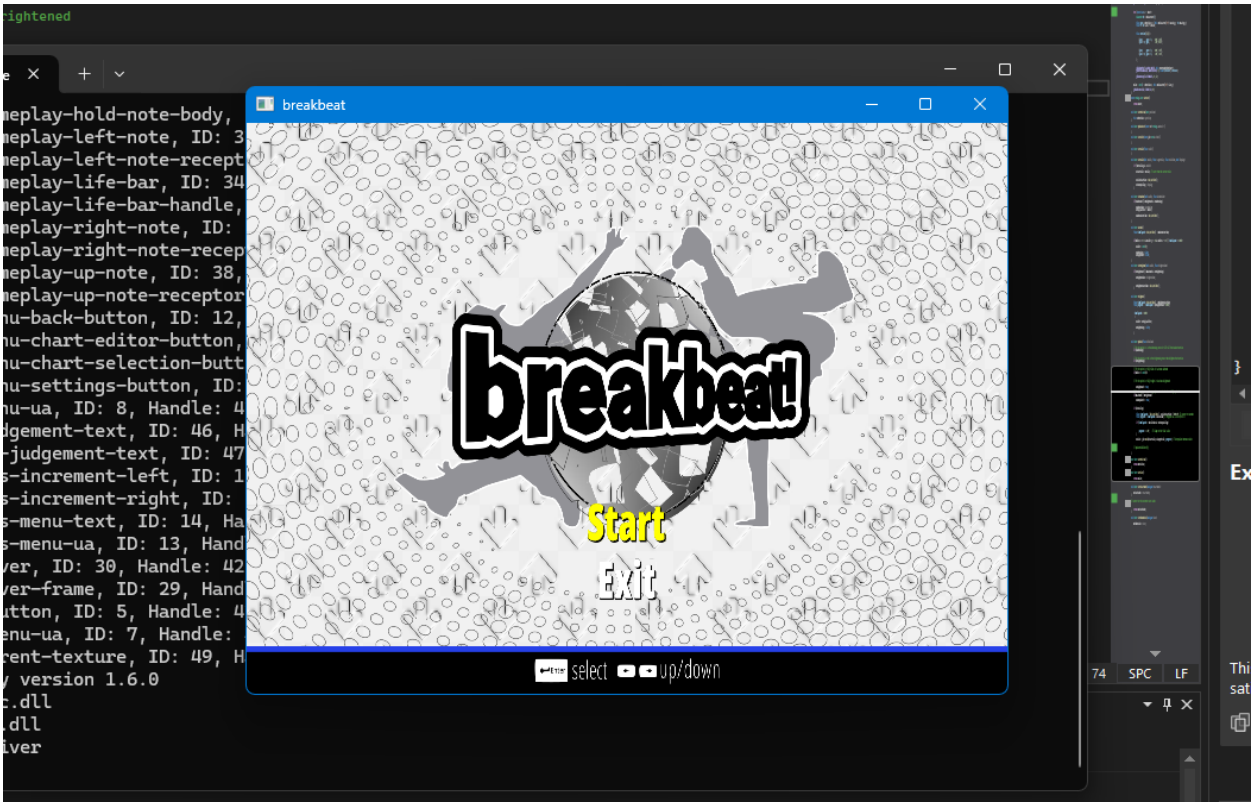
```
406                 // Set default values for the new chart
407                 mNewChartSongName = songName;
408                 mNewChartArtistName = artistName;
409                 mNewChartBPM = bpm;
410                 mNewChartAudioPath = audioPath;
411                 mNewChartImagePath = newImagePath.empty() ? imagePath : newImagePath;
412                 mNewChartDifficultyName = newDifficultyName;
```

To finish the chart selection menu, I wrote the procedure in which a new chart file is created. This procedure uses the default member values for the new chart creation that were previously described in the above procedures. Within these procedures I continually added test cases in which the user may be potentially adding in already existing chart directories. This is because the user may accidentally be uploading the same audio file for creating a new chart without realizing Furthermore my justification for doing this is to ensure my code is robust and maintainable for future processes of debugging and testing.

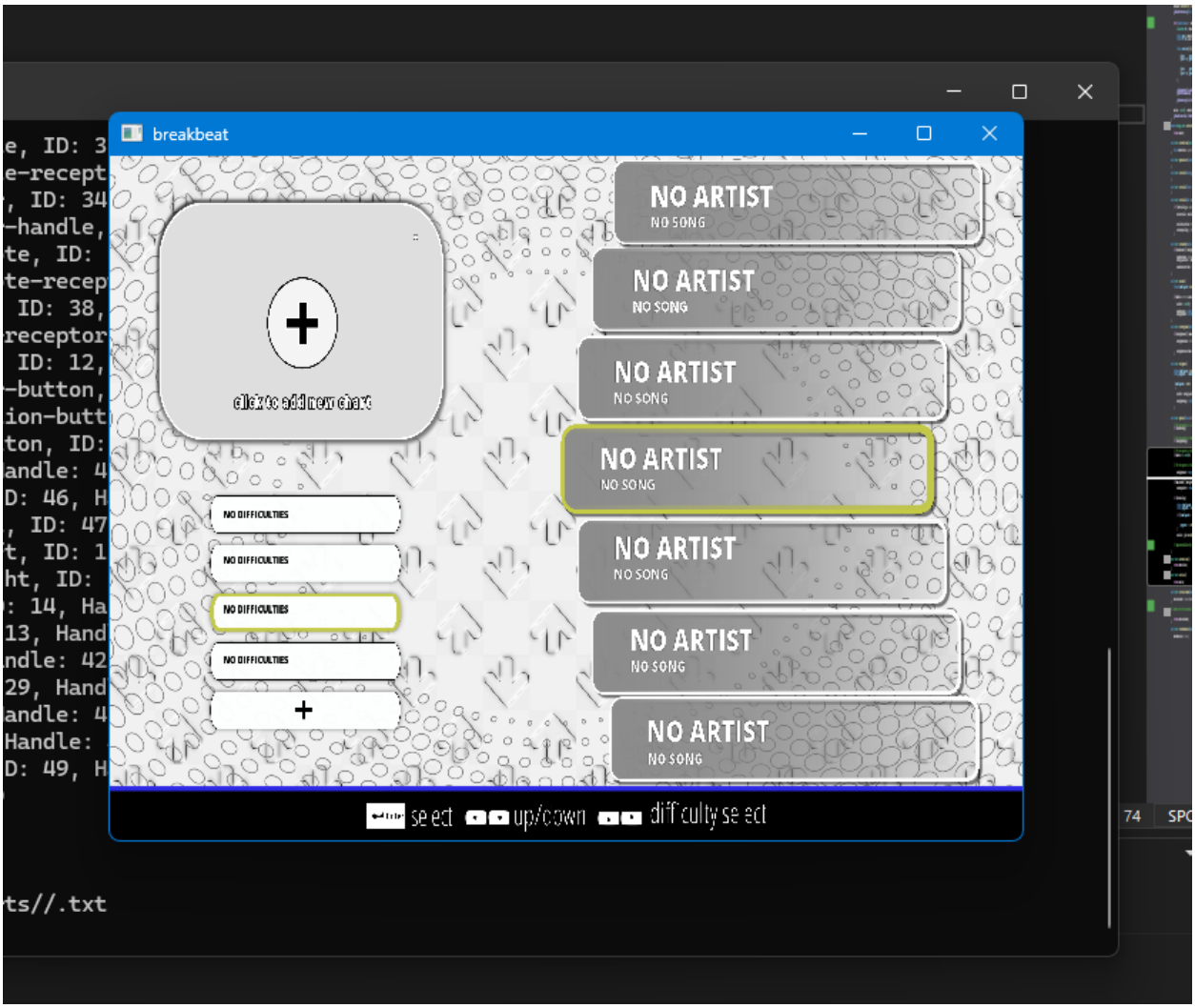
```cpp
void Game::CreateNewChart()
{
    // Step 1: Retrieve inputs from member variables
    string artistName = mNewChartArtistName;
    string songName = mNewChartSongName;
    string difficultyName = mNewChartDifficultyName;
    float difficulty = 0.00;
    string bpm = mNewChartBPM;
    string audioPath = mNewChartAudioPath;
    string imagePath = mNewChartImagePath;

    // Step 2: Create chart folder path
    string chartFolderName = artistName + "-" + songName;
    string chartFolderPath = "charts\\" + chartFolderName;

    try
    {
        // Step 3: Create directory
        if (!std::filesystem::exists("charts"))
        {
            std::filesystem::create_directory("charts");
        }
        std::filesystem::create_directory(chartFolderPath);

        // Step 4: Copy audio and image files into the folder
        std::string copiedAudioPath = chartFolderPath + "\\" + std::filesystem::path(audioPath).filename().string();
        std::string copiedImagePath = chartFolderPath + "\\" + std::filesystem::path(imagePath).filename().string();

        // Check if the audio file already exists in the directory
        if (!std::filesystem::exists(copiedAudioPath))
        {
            try
            {
                std::filesystem::copy_file(audioPath, copiedAudioPath, std::filesystem::copy_options::overwrite_existing);
            }
            catch (const std::filesystem::filesystem_error& e)
            {
                std::cerr << "Error copying audio file: " << e.what() << std::endl;
                // Handle the error if necessary (e.g., display a message or log it)
            }
        }
        else
        {
            std::cout << "Audio file already exists. Skipping copy: " << copiedAudioPath << std::endl;
        }

        // Check if the image file already exists in the directory
        if (!std::filesystem::exists(copiedImagePath))
        {
            try
            {
                std::filesystem::copy_file(imagePath, copiedImagePath, std::filesystem::copy_options::overwrite_existing);
            }
            catch (const std::filesystem::filesystem_error& e)
            {
                std::cerr << "Error copying image file: " << e.what() << std::endl;
                // Handle the error if necessary (e.g., display a message or log it)
            }
        }
        else
        {
            std::cout << "Image file already exists. Skipping copy: " << copiedImagePath << std::endl;
        }

        // Step 5: Create and write to the chart text file
        std::string chartFilePath = chartFolderPath + "\\" + difficultyName + ".txt";

        // Create and write to the chart filex
        std::ofstream chartFile(chartFilePath);

        if (chartFile.is_open())
        {
            chartFile << "Artist : " << artistName << "\n";
            chartFile << "Song Name : " << songName << "\n";
            chartFile << "Difficulty : " << difficulty << "\n";
            chartFile << "BPM : " << bpm << "\n";
            chartFile << "Background Image : " << copiedImagePath << "\n";
            chartFile << "Audio : " << copiedAudioPath << "\n";
            chartFile.close();
        }
        else
        {
            throw std::ios_base::failure("Failed to create chart text file.");
        }
```

Testing

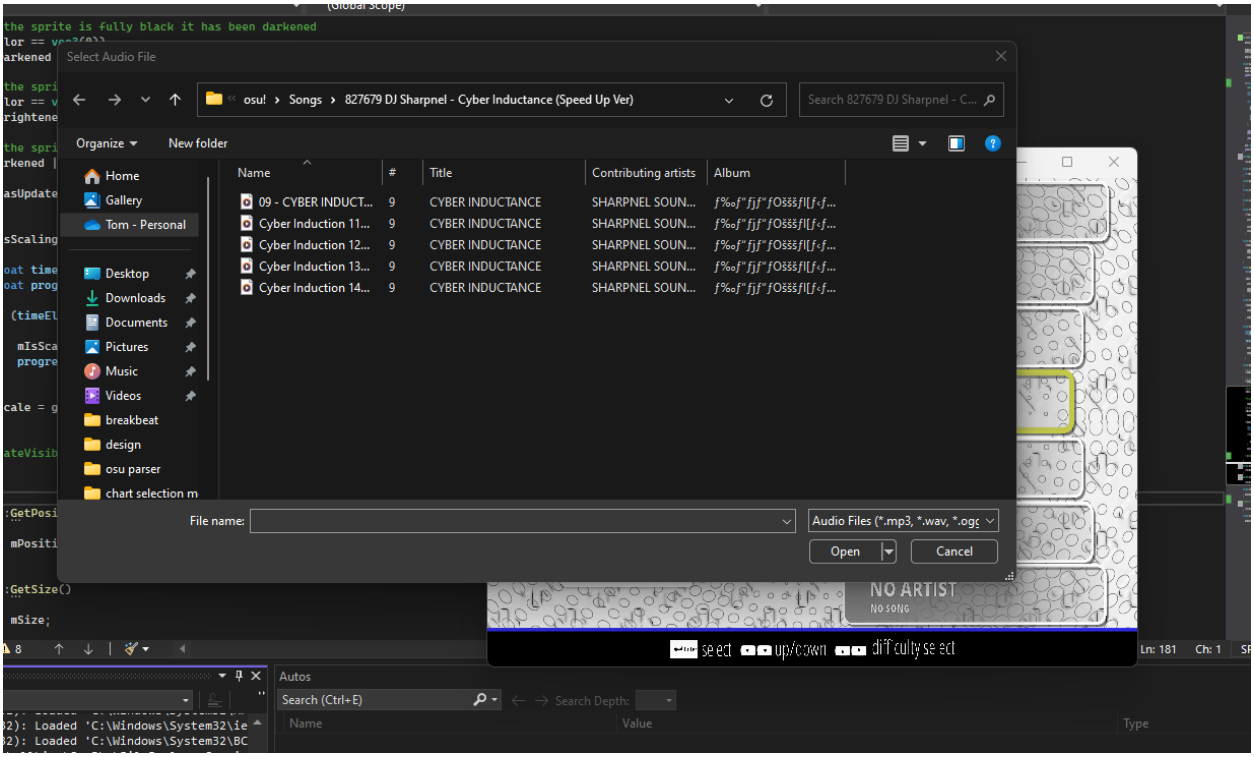
Upon running the compiled program, the program successfully ran without any form of errors in regards with the graphical function of the program. The sprites and the textures were rendered correctly and the start menu showed up correctly. My justification for these test cases is to ensure that there are no errors that regarded with low level specification such as memory usage on the graphics card, as shown in the previous section. Thus, this marked the completion of the first test case.



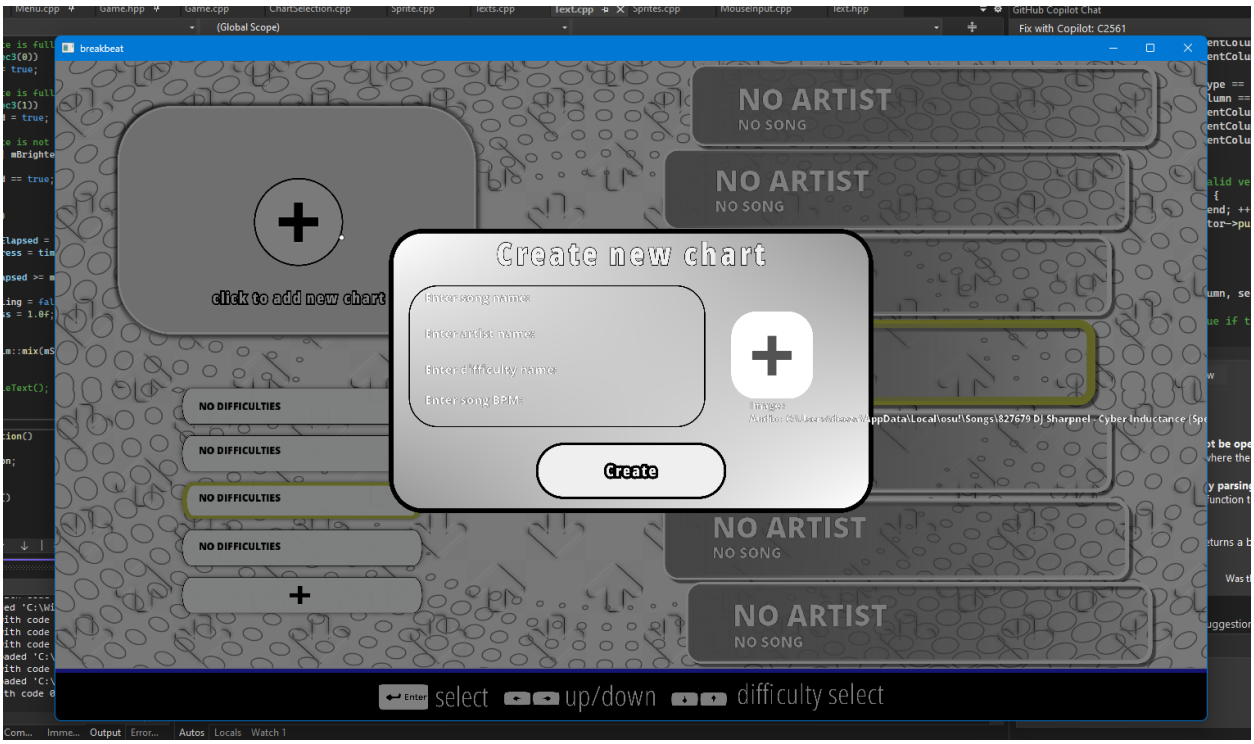
Afterwards, I was able to successfully navigate to the main menu and transition into the chart selection menu. This can be shown in the “Chart Selection Editor Test Video 1”. Upon pressing enter the menu selection animation played successfully and the transition begun into the chart editor selection menu. Upon transition into the chart selection editor menu, the sprites successfully loaded, and the user interface displayed correctly. This marked the meeting of the second and third test criteria.



Upon navigating using the mouse to hover and click over the new chart button user interface, I was immediately prompted with file dialogue and was able to successfully browse through my files and preview audio files. This marked the success of the sixth test case.

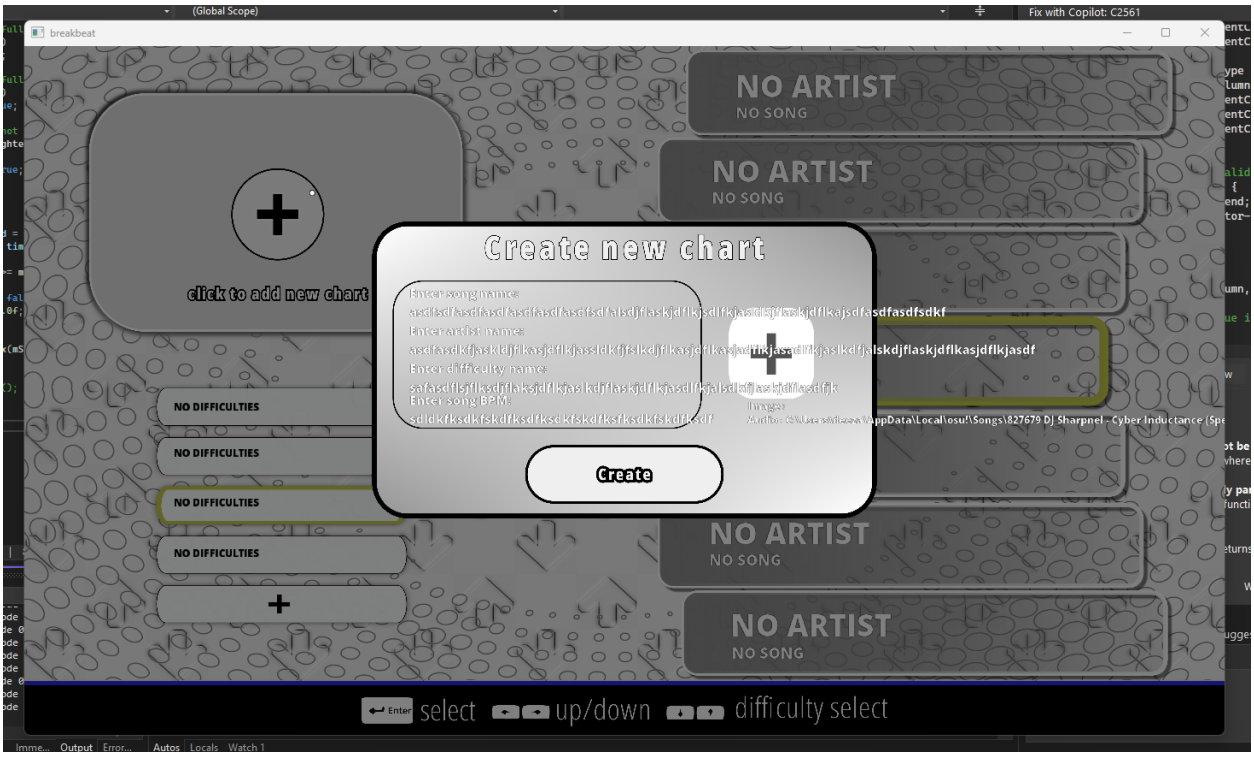


Upon browsing through my files and choosing an audio to upload, I was able to successfully upload a chart file and alongside the new chart screen user interface appeared.



Error Encountered – Text Overflow

However, upon inspecting the ability to type into the text boxes to enter the chart metadata, I realized that I was able to type beyond the user interface and thus the text would not be bound to the user interface. This can be seen in the “Text Overflow Error Video”



Reason For the Error

The reason for this error is due to the text class not having any form of bounds checking or “text box scrolling” mechanism to scroll through the text. A text box would have the ability to preview different parts of a long string of text, but the actual text itself remained the same. In my case my text box system had no way of previewing the different parts of text within the text box.

Solution To the Error

Therefore, a solution to this problem was to implement a text display system in which the text system displays the visible text based on the actual text. However, the starting point of the string for visible text is based on an index pointer to the actual text string.

```
17  class Text {
18  public:
19      string mText;
20      vec3 mColor;
21      vec2 mPosition;
22      vec2 mSize;
23      float mScale = 1.0f;
24      string mVisibleText;
25      unsigned mStartIndex=0;
26      unsigned mWindowSize=10;
27      map<char,Character> mCharacters;
```

This involved creating a small new procedure to display the visible text based on the actual text and then incrementing the starting index of the preview point of the visible text based on the length of the text.

```
197 void Text::UpdateVisibleText()
198 {
199     // Ensure the startIndex is within valid bounds
200     if (mText.size() <= mWindowSize)
201     {
202         mStartIndex = 0;
203     }
204     else if(mScrollableText)
205     {
206         float currentTicks = SDL_GetTicks();
207 
208         if (currentTicks - mLastScrollTime >= mScrollInterval)
209         {
210             if (mStartIndex >= mText.size() - mWindowSize)
211             {
212                 mStartIndex = 0; // Reset the text to loop
213             }
214             else
215             {
216                 ++mStartIndex; // Increment index normally
217             }
218 
219             mLastScrollTime = currentTicks;
220         }
221     } else
222     {
223         mStartIndex = mText.size() > mWindowSize ? mText.size()-mWindowSize : 0;
224     }
225 
226     mVisibleText = mText.substr(mStartIndex, mWindowSize);
227 }
```

```
116 void TextRenderer::CreateText(GameState gameState, string identifier, string text, glm::vec2 position, vec3 color, string fontName, float scale, unsigned windowSize, bool scrollableText)
117 {
118     // Create text object and store its properties within the text hash table
119     auto textObject = new Text(text, position, color, scale, windowSize, scrollableText);
120     textObject->mVertexArrayObject = this->mVertexArrayObject;
121     textObject->mVertexBufferObject = this->mVertexBufferObject;
122     textObject->mShader = this->mShader;
123     textObject->mCharacters = mFonts[fontName];
124     mDefaultTexts[gameState][identifier] = textObject;
125 }
```

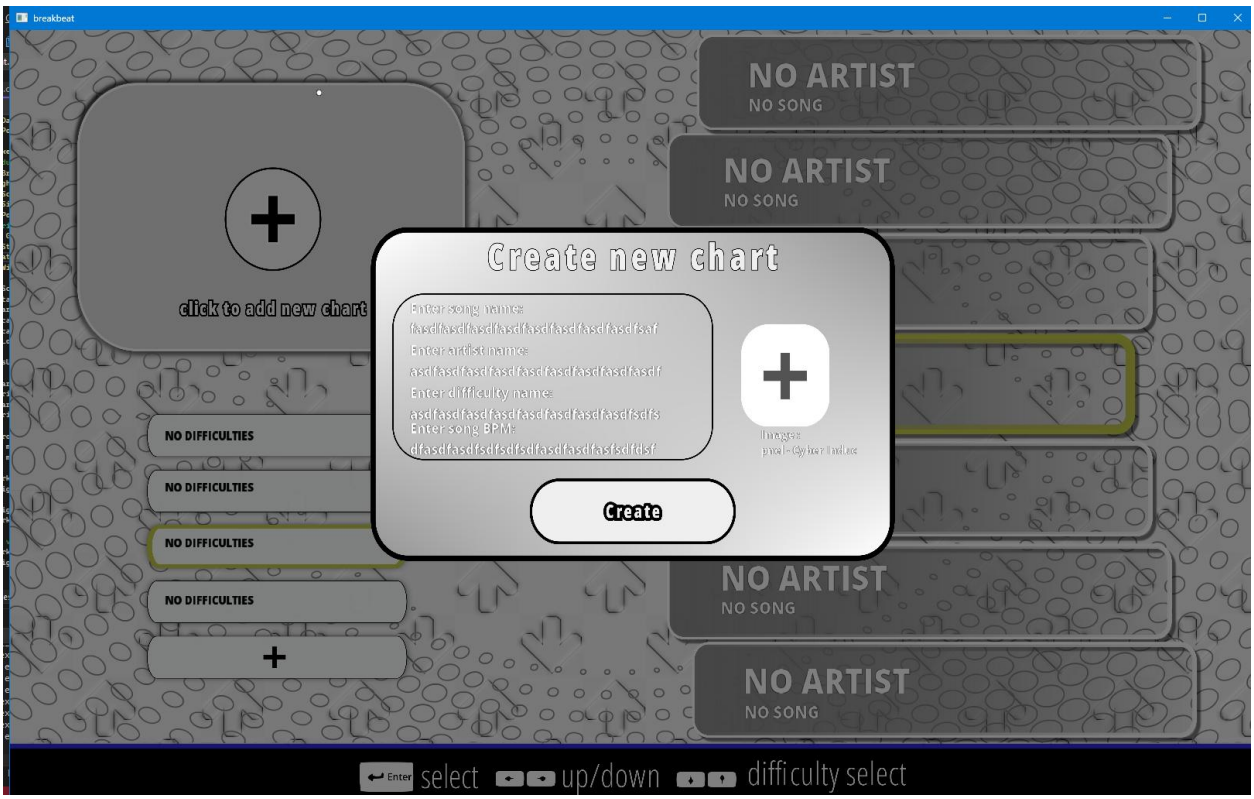
```
void Text::Draw()
{
    mShader.Use();
    mShader.SetVector3f("textColor", mColor);
    glActiveTexture(GL_TEXTURE0);
    glBindVertexArray(this->mVertexArrayObject);

    float x = mPosition.x;
    for (const char& c : mVisibleText)
    {
        Character ch = mCharacters[c];
        float xpos = x + ch.Bearing.x;
        float ypos = mPosition.y + (this->mCharacters['H'].Bearing.y - ch.Bearing.y);
        float w = ch.Size.x * mScale;
        float h = ch.Size.y * mScale;

        float vertices[6][4] =
        {
            {xpos,     ypos + h,    0.0f, 1.0f},
            {xpos + w, ypos,        1.0f, 0.0f},
            {xpos,     ypos,        0.0f, 0.0f},

            {xpos,     ypos + h,    0.0f, 1.0f},
            {xpos + w, ypos + h,    1.0f, 1.0f},
            {xpos + w, ypos,        1.0f, 0.0f}
        };

        glBindTexture(GL_TEXTURE_2D, ch.TextureID);
        glBindBuffer(GL_ARRAY_BUFFER, this->mVertexBufferObject);
        glBufferSubData(GL_ARRAY_BUFFER, 0, sizeof(vertices), vertices);
        glBindBuffer(GL_ARRAY_BUFFER, 0);
        glDrawArrays(GL_TRIANGLES, 0, 6);
        x += (ch.Advance >> 6) * mScale;
    }
    mSize = vec2(x - mPosition.x, this->mCharacters['H'].Size.y);
    glBindVertexArray(0);
    glBindTexture(GL_TEXTURE_2D, 0);
}
```

The text would start scrolling once at a certain number of characters, for example in the screenshot below, the text would start scrolling sideways once at 36 characters. My overall justification for this is to make the user interface more readable and maintainable

```
mTextRenderer.CreateText
(
    GameState::CHART_EDITOR_SELECTION_MENU,
    "new-chart-screen-difficulty-name-text-box",
    "",
    vec2(616.250, 528.75),
    vec3(1.0f),
    "default-20",
    1.0f,
    36,
    false
);
```

Testing Continued

Upon testing my new system, I was then able to successfully fix the text overflow, and they would begin to scroll even for text boxes and for static text boxes and so on. The results of this can be seen in the “Chart Editor Selection Text Overflow Solution Error” video.



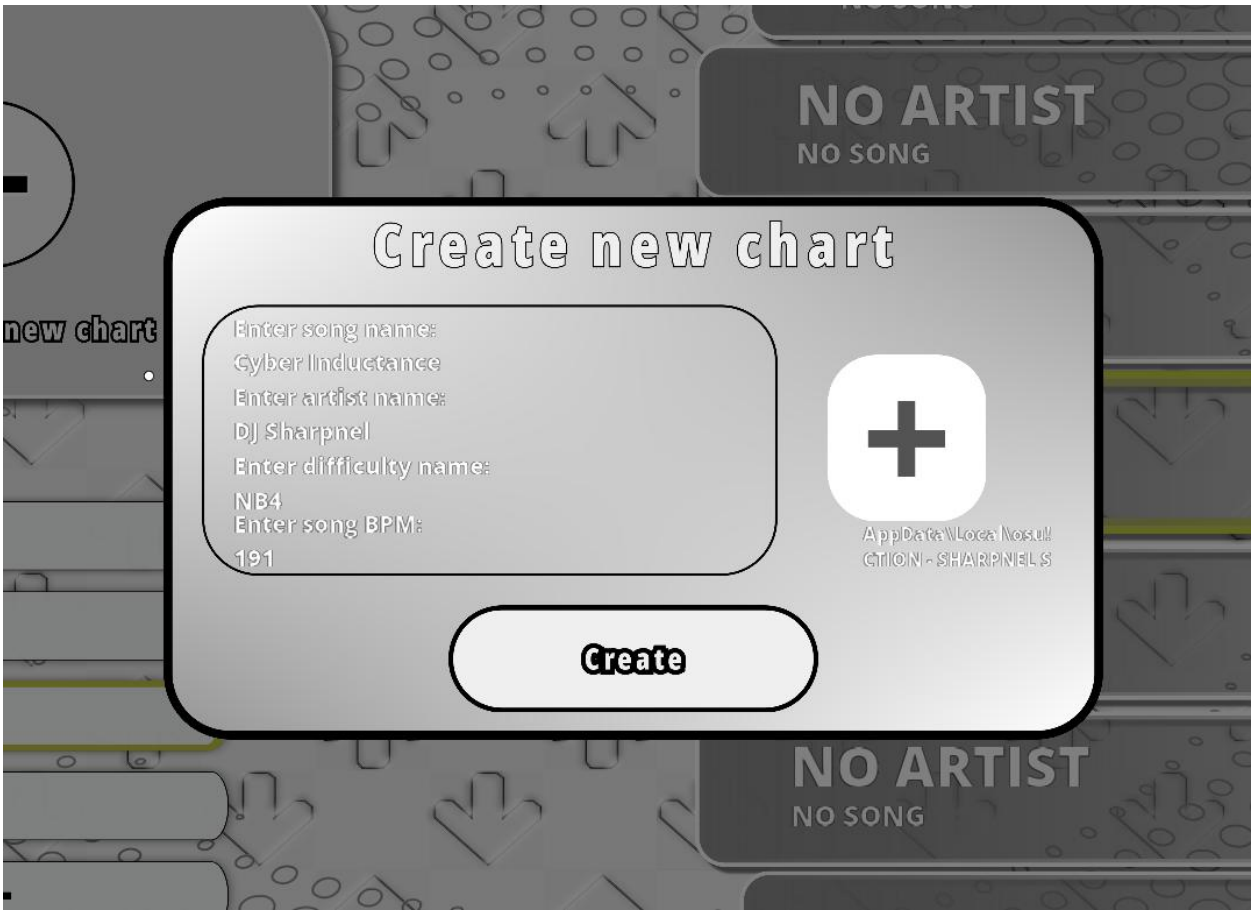
Afterwards, I then tested the ability to upload a chart and select an image file for it. Both worked successfully and both the path of the image file and the audio file showed properly. This can be seen in the “Chart Editor Selection Test Video 2” video.

I was then able to type into my keyboard and update the text user interface by typing. This marked the success of the typing test case.

After creating the chart file. I then decided to create another chart file to scroll through the selection menu and I was successful in doing so. This marked the meeting of the left and right test cases.

Finally, I was able to create a new difficulty using the new difficulty button and was then able to scroll through the chart selection menu with the newly created chart shown in the chart editor selection menu.

All the test case results can be shown to be successfully met in the “Chart Editor Selection Test Video 2”



Main Gameplay

Test Plan

Development

With the mechanics of chart editor selection menu successful, the logic will remain the same for the future development chart selection menu. With that in mind, I decided to begin the development of the main gameplay. My justification for starting development in this way is to prioritize the core aspect of my adaptation which is the main gameplay alongside the difficulty calculation system.

To begin with the main gameplay, I first began with the class diagram required for the use of the C/C++ struct data structure to place attributes required for the main gameplay logic. This will include the storage of the arrays and integral types to store data such as the hit timings for notes and the indexes for accessing each of the times within the array. Furthermore, this will involve the certain Boolean variables to detect whether user the has pressed a long note or a regular note.

The hitTime, releaseTime, longNoteHitTime floating point variables will be used to temporarily store the user's input time in the instance of time it is pressed. These variables will then be used to calculate the hit difference between a predetermined set of timings to produce a judgement of accuracy. The hit differences will then be stored in the corresponding hitDifference, longNoteHitDifference and releaseDifference.

The NoteColumn struct makes use of the C++ vector data structure to have dynamic arrays for the storage of the hit timings, hit timings for long notes and the release timings for long notes. These arrays will store the numerical value of the time that needs to be compared with the user's hitTIme

NoteColumn

float hitTime;
float hitDifference;

float longNoteHitTime;
float longNoteHitDifference;

float releaseTime;
float releaseDifference;

float noteTimeIndex;
float renderTimeIndex;

float longNoteTimeIndex;
float longNoteRenderTimeIndex;

float releaseTimeIndex;
float releaseTimeRenderTimeIndex;

vector<float> hitTimes;
vector<float> releaseTimes;
vector<float> longNoteHitTimes;

vector<float> renderTimes;
vector<float> releaseRenderTimes;
vector<float> longNoteRenderTimes;

bool isKeyHeld;

```
void Game::LoadBackgroundImage()
{
    // Convert file extension to lowercase for case-insensitive comparison
    std::string extension = fs::path(mCurrentChartImageFile).extension().string();
    std::transform(extension.begin(), extension.end(), extension.begin(), ::tolower);

    // Check if the file is PNG or JPG and enable alpha accordingly
    bool useAlpha = extension == ".png";

    Texture bgImage = ResourceManager::loadTextureFromFile((fs::current_path().string() + "\\" + mCurrentChartImageFile).c_str(), useAlpha);
    GetSprite(mCurrentGameState, "main-gameplay-background-image")->SetTexture(bgImage);
}
```

```
void Game::InitializeMainGameplay()
{
    // Load the background image
    // Stop all the sounds (assuming transitioning from the chart selection)
    mSoundEngine->stopAllSounds();
    LoadBackgroundImage();
    GetSprite(mCurrentGameState, "main-gameplay-judgement-text")->SetTexture(ResourceManager::GetTexture("nil"));
    // Determine the size of the receptor based on the user's receptor size preference
    vec2 receptorSize = vec2(192.000 * (mReceptorSize / 100.0f), 180 * (mReceptorSize / 100.0f));

    // Ensure each sprite is fully brightened
    for (auto& [key, sprite] : mSpriteRenderer.mCurrentlyRenderedSprites[mCurrentGameState])
    {
        if (sprite != nullptr)
            sprite->SetColor(vec3(1));
    }

    // Determine the size of the left most receptor
    GetSprite(mCurrentGameState, "main-gameplay-left-note-receptor")->SetSize(receptorSize);
    // Offset the position of the receptor to the right based on the size of the receptor
    GetSprite(mCurrentGameState, "main-gameplay-left-note-receptor")->SetPosition(vec2(581.936, ((mReceptorSize / 100.0f) * -180) + 1080));
    GetSprite(mCurrentGameState, "z-main-gameplay-down-note-receptor")->SetSize(receptorSize);
    // OFfset the position of the rectpor to right plus an additional extra offest based on the spacing between each receptor
    GetSprite(mCurrentGameState, "z-main-gameplay-down-note-receptor")->SetPosition(vec2(((mReceptorSize / 100.0f) - 1) * 183.487 + 765.423, ((mReceptorSize / 100) * -180) + 1080));
    GetSprite(mCurrentGameState, "main-gameplay-up-note-receptor")->SetSize(receptorSize);
    GetSprite(mCurrentGameState, "main-gameplay-up-note-receptor")->SetPosition(vec2(((mReceptorSize / 100.0f) - 1) * 366.974 + 949.361, ((mReceptorSize / 100) * -180) + 1080));
    GetSprite(mCurrentGameState, "main-gameplay-right-note-receptor")->SetSize(receptorSize);
    GetSprite(mCurrentGameState, "main-gameplay-right-note-receptor")->SetPosition(vec2(((mReceptorSize / 100.0f) - 1) * 550.461 + 1138.061, ((mReceptorSize / 100) * -180) + 1080));
    // Multipy the size of the notefield based on the size of the receptors
    GetSprite(mCurrentGameState, "main-gameplay-gameplay-vsrg-column")->SetSize(vec2(755.998 * (mReceptorSize / 100.0f), 1080));
    // Set the position of the life bar to the right based on the size of the notefield
    GetSprite(mCurrentGameState, "main-gameplay-life-bar-handle")->SetPosition(vec2(575.321 + GetSprite(mCurrentGameState, "main-gameplay-gameplay-vsrg-column")->GetSize().x, 477.547));
    GetSprite(mCurrentGameState, "main-gameplay-life-bar")->SetSize(vec2(17.158, -594.710));
    GetSprite(mCurrentGameState, "main-gameplay-life-bar")->SetPosition(vec2(579.121 + GetSprite(mCurrentGameState, "main-gameplay-gameplay-vsrg-column")->GetSize().x, 1080));
    // Set the position of the judgement text to the center of the notefield.
    GetSprite(mCurrentGameState, "main-gameplay-judgement-text")->SetPosition(vec2(579.121 + (GetSprite(mCurrentGameState, "main-gameplay-gameplay-vsrg-column")->GetSize().x / 2.0f) -
        (GetSprite(mCurrentGameState, "main-gameplay-judgement-text")->GetSize().x / 2), 481.407));
    // Set the position song name and time elapsed text to the either side of the top of the notefield
    GetText(mCurrentGameState, "gameplay-song-name")->UpdateText(mCurrentSongName + format("[{}]", mCurrentSongDifficulty));
    GetText(mCurrentGameState, "gameplay-time-elapsed")->SetPosition(vec2(579.121 + GetSprite(mCurrentGameState, "main-gameplay-gameplay-vsrg-column")->GetSize().x * .70, 8.904));
```

```
// Data structure to store the regular note hit times, long note times and release times
struct NoteData {
    vector<int> hitTimes;
    vector<int> longNoteTimes;
    vector<int> releaseTimes;
};

// Function to extract the hit times from an .osu file
void ParseOsuFile(const string& osuFilePath, const string& outputFilePath) {
    // Input stream into the the file to be read
    ifstream osuFile(osuFilePath);
    // Output stream into file to be written to (creates a new file)
    ofstream outputFile(outputFilePath);
    // String data structure to temporarily store a line within the file as a string
    string line;
    // This is where the parsing happens
    // The regular expression pattern to determine the data to extract from the osu hit objects
    regex hitObjectRegex(R"((\d+),(\d+),(\d+),(\d+),(\d+),?([^,]*))");
    // Storing and returning the data when a match for the regular expression is found.
    smatch match;

    // Store the NoteData in a hash table with the column number as the index
    map<int, NoteData> noteColumns;
    const int numColumns = 4;
    // Osu's column numbers are based on the 512 divided by the number of columns
    int columnWidth = 512 / numColumns;
```

```
    // Read the .osu file line by line
    while (getline(osuFile, line)) {
        // Check if the line meets the regular expression pattern
        if (regex_match(line, match, hitObjectRegex)) {
            // The value of the note column
            int x = stoi(match[1].str());
            // The hit timing
            int time = stoi(match[3].str());
            // The type of note it is i.e is it a long note
            int type = stoi(match[4].str());

            // The column index
            int columnIndex = x / columnWidth;

            if (type & 1) { // Normal hit object
                noteColumns[columnIndex].hitTimes.push_back(time);
            }
            if (type & 128) { // Long note
                int releaseTime = stoi(match[6].str());
                noteColumns[columnIndex].longNoteTimes.push_back(time);
                noteColumns[columnIndex].releaseTimes.push_back(releaseTime);
            }
        }
    }

    // Output the data based on the note column
    for (const auto& [columnIndex, data] : noteColumns) {
        outputFile << columnIndex + 1 << " Column Hit Times:" << '\n';
        for (const int& time : data.hitTimes) {
            outputFile << time << "," << '\n';
        }
        outputFile << '\n';

        outputFile << columnIndex + 1 << " Column Long Note Times:" << '\n';
        for (const int& time : data.longNoteTimes) {
            outputFile << time << "," << '\n';
        }
        outputFile << '\n';

        outputFile << columnIndex + 1 << " Column Release Times:" << '\n';
        for (const int& time : data.releaseTimes) {
            outputFile << time << "," << '\n';
        }
        outputFile << '\n';
    }
}
```

```cpp
int main() {
    string osuFilePath = "Kurokotei - Galaxy Collapse (Mat) [Cataclysmic Hypernova].osu";
    string outputFilePath = "Kurokotei - Galaxy Collapse (Mat) [Cataclysmic Hypernova].osu.txt";
    ParseOsuFile(osuFilePath, outputFilePath);
    cout << "Parsing complete. Output saved to " << outputFilePath << '\n';
    return 0;
}
```

```cpp
bool Game::ParseDifficultyFile()
{
    // Open difficulty file
    ifstream difficultyFile(fs::current_path().string() + "\\" + mCurrentChartFile);
    if (!difficultyFile.is_open()) {
        cerr << "Error: Unable to open difficulty file: " << mCurrentChartFile << '\n';
        return false;
    }

    // Store the line of the file being as a string object
    string line;
    // The regular expression for the column and the column type
    regex columnRegex(R"(^(\d+)\s+Column\s+(Hit|Long Note|Release)\s+Times:)");
    // The timing regular expression
    regex numberRegex(R"(\d+)");

    int currentColumn = -1; // Tracks which column is being processed
    string currentType; // Tracks the type: "Hit", "Long Note", "Release"
```

```
35      while (getline(difficultyFile, line)) {
36          smatch match;
37
38          // Check if the line starts a new note type (Hit, Long Note, Release) for a column
39          if (regex_search(line, match, columnRegex)) {
40              currentColumn = stoi(match[1]); // Extract the column number (0, 1, 2, 3)
41              currentType = match[2]; // "Hit", "Long Note", "Release"
42              continue; // Move to the next line
43          }
44
45          // If we are processing a known column, extract times
46          if (currentColumn >= 1 && currentColumn <= 4) {
47              std::sregex_iterator it(line.begin(), line.end(), numberRegex);
48              std::sregex_iterator end;
49
50              vector<float>* targetVector = nullptr;
51
52              // Determine which vector to update based on type
53              if (currentType == "Hit") {
54                  if (currentColumn == 1) targetVector = &firstColumn.hitTimes;
55                  else if (currentColumn == 2) targetVector = &secondColumn.hitTimes;
56                  else if (currentColumn == 3) targetVector = &thirdColumn.hitTimes;
57                  else if (currentColumn == 4) targetVector = &fourthColumn.hitTimes;
58              }
59              else if (currentType == "Long Note") {
60                  if (currentColumn == 1) targetVector = &firstColumn.longNoteHitTimes;
61                  else if (currentColumn == 2) targetVector = &secondColumn.longNoteHitTimes;
62                  else if (currentColumn == 3) targetVector = &thirdColumn.longNoteHitTimes;
63                  else if (currentColumn == 4) targetVector = &fourthColumn.longNoteHitTimes;
64              }
65              else if (currentType == "Release") {
66                  if (currentColumn == 1) targetVector = &firstColumn.releaseTimes;
67                  else if (currentColumn == 2) targetVector = &secondColumn.releaseTimes;
68                  else if (currentColumn == 3) targetVector = &thirdColumn.releaseTimes;
69                  else if (currentColumn == 4) targetVector = &fourthColumn.releaseTimes;
70              }
71
72              // If we have a valid vector, populate it with parsed times
73              if (targetVector) {
74                  for (; it != end; ++it) {
75                      targetVector->push_back(std::stof(it->str()));
76                  }
77              }
```

```
80      mNoteColumns = { firstColumn,secondColumn,thirdColumn,fourthColumn };
81      difficultyFile.close();
82      return true;
```

```
Artist : DJ Sharpnel
Song Name : Cyber Inductance (Speed Up Ver.)
Difficulty : 25.01
BPM : 191
Background Image : charts\DJ Sharpnel-Cyber Inductance (Speed Up Ver.)\Bg-Cyber.png
Audio : charts\DJ Sharpnel-Cyber Inductance (Speed Up Ver.)\09 - CYBER INDUCTION - SHARPNEL SOUNDS.mp3

1 Column Hit Times:
1728,
1963,
2356,
2591,
2984,
3141,
3455,
3612,
4084,
4319,
4476,
4712,
4869,
5262,
5497,
5890,
6126,
6283,
6597,
6832,
6989,
7382,
7775,
8010,
8325,
8639,
9267,
9424,
9738,
```

```cpp
void Game::PreloadAudio()
{
    // Load the sound into memory
    mSongSource = mSoundEngine->addSoundSourceFromFile(
        mCurrentChartAudioFile.c_str(),
        ESM_AUTO_DETECT, // Automatically detects format
        true             // Forces the sound to stay in memory
    );

    mSongSource = mSoundEngine->getSoundSource(mCurrentChartAudioFile.c_str());
    mCurrentSongDuration = mSongSource->getPlayLength();
}
```

```
269  void Game::RenderNote(NoteColumn& noteColumn)
270  {
271      float totalDistance = 1080; // Adjusted for note height and receptor size
272      if (noteColumn.renderTimeIndex < noteColumn.renderTimes.size() &&
273              mTimeElapsed >= (noteColumn.renderTimes[noteColumn.renderTimeIndex]))
274      {
275          mSpriteRenderer.CreateNote(
276              mCurrentGameState,
277              noteColumn.noteName + '-' + to_string(noteColumn.renderTimeIndex),
278              ResourceManager::GetTexture(noteColumn.noteName),
279              vec2(noteColumn.xPos, -180 * (mReceptorSize / 100)), // Start position at the top
280              vec2(192.000 * (mReceptorSize / 100.0f), 180 * (mReceptorSize / 100.0f)),
281              0.0f,
282              vec3(1.0f),
283              ResourceManager::GetShader("default"),
284              false
285          );
286          noteColumn.renderTimeIndex++;
287      }
```

```
289      // Check if it's time to create the long note head (hit time)
290      if (noteColumn.longNoteRenderTimeIndex < noteColumn.longNoteHitTimes.size() &&
291              mTimeElapsed >= (noteColumn.longNoteHitTimes[noteColumn.longNoteRenderTimeIndex] - mTimeTakenToFall)) {
292          mSpriteRenderer.CreateNote(
293              mCurrentGameState,
294              noteColumn.noteName + "-long-note-head-" + to_string(noteColumn.longNoteRenderTimeIndex),
295              ResourceManager::GetTexture(noteColumn.noteName),
296              vec2(noteColumn.xPos, -180 * (mReceptorSize / 100)), // Start position at the top
297              vec2(192.000 * (mReceptorSize / 100.0f), 180 * (mReceptorSize / 100.0f)),
298              0.0f,
299              vec3(1.0f),
300              ResourceManager::GetShader("default"),
301              false
302          );
303
304          float timeDifference = noteColumn.releaseTimes[noteColumn.releaseTimeRenderTimeIndex] -
305              noteColumn.longNoteHitTimes[noteColumn.longNoteRenderTimeIndex];
306
307          // Avoid negative or zero body size for very short long notes
308          float bodyHeight = abs(timeDifference * mScrollSpeed / 1000.0f);
309
310          mSpriteRenderer.CreateNote(
311              GameState::MAIN_GAMEPLAY,
312              noteColumn.noteName + "-long-note-body-" + to_string(noteColumn.longNoteRenderTimeIndex),
313              ResourceManager::GetTexture("main-gameplay-hold-note-body"),
314              vec2(noteColumn.xPos, -90 * (mReceptorSize / 100)), // Start position at the top
315              vec2(192 * (mReceptorSize / 100), -bodyHeight),
316              0.0f,
317              vec3(1.0f),
318              ResourceManager::GetShader("default"),
319              false
320          );
321
322          noteColumn.releaseTimeRenderTimeIndex++;
323          noteColumn.longNoteRenderTimeIndex++;
324      }
325  }
```

```cpp
void Game::UpdateLongNoteState(NoteColumn& noteColumn) {
    if (noteColumn.isKeyHeld) {

        // Access the note body from the renderer
        auto& note = mSpriteRenderer.mNoteBuffer[mCurrentGameState][
            noteColumn.noteName + "-long-note-body-" + to_string(noteColumn.releaseTimeIndex)
        ];

        if (note != nullptr)
        {
            if (note->GetPosition().y >= 1080 - 90 * (mReceptorSize / 100))
                note->mIsMoving = false;
            note->SetPosition(vec2(noteColumn.xPos, 1080 - 90 * (mReceptorSize / 100)));
        }

        float elapsedTimeSinceHit = mTimeElapsed - noteColumn.shrinkStartTime;
        float shrinkProgress = elapsedTimeSinceHit / noteColumn.shrinkDuration;

        shrinkProgress = std::clamp(shrinkProgress, 0.0f, 1.0f);

        // Calculate new height based on shrink progress
        float originalHeight = noteColumn.longNoteOriginalHeight;
        float newHeight = originalHeight * (1.0f - shrinkProgress);

        // Update the size of the note body
        if(note!=nullptr)
            note->SetSize(vec2(note->GetSize().x, newHeight));
    }
}
```

```cpp
void Game::HandleLongNoteRelease(NoteColumn& noteColumn)
{
    if (noteColumn.isKeyHeld && noteColumn.releaseTimeIndex < noteColumn.releaseTimes.size())
    {
        noteColumn.isKeyHeld = false;
        noteColumn.releaseTime = mTimeElapsed;
        noteColumn.releaseDifference = abs(noteColumn.releaseTime -
            noteColumn.releaseTimes[noteColumn.releaseTimeIndex]);
        mSpriteRenderer.mNoteBuffer[mCurrentGameState].erase(
            noteColumn.noteName + "-long-note-body-" + to_string(noteColumn.releaseTimeIndex));
        if ( noteColumn.releaseDifference <= 180)
        {
            noteColumn.releaseTimeIndex++;
            mMaximumPossibleScore += 320;
            UpdateScoreRelease(noteColumn);
        }
    }
}
```

```cpp
void Game::RegisterHit(NoteColumn& noteColumn)
{
    if (noteColumn.noteTimeIndex < noteColumn.hitTimes.size())
    {
        noteColumn.hitTime = mTimeElapsed;
        noteColumn.hitDifference = abs(noteColumn.hitTime - noteColumn.hitTimes[noteColumn.noteTimeIndex]);
        if (noteColumn.hitDifference <= 180)
        {
            noteColumn.hitNote = true;
            mSpriteRenderer.mNoteBuffer[mCurrentGameState].erase(noteColumn.noteName + '-' + to_string(noteColumn.noteTimeIndex));
            if (noteColumn.noteTimeIndex < noteColumn.hitTimes.size())
                noteColumn.noteTimeIndex++;
            mMaximumPossibleScore += 320;
            UpdateScore(noteColumn);
        }
    }
```

```
343     if (noteColumn.longNoteTimeIndex < noteColumn.longNoteHitTimes.size())
344     {
345         noteColumn.hitTime= mTimeElapsed;
346         noteColumn.hitDifference = noteColumn.hitTime- noteColumn.longNoteHitTimes[noteColumn.longNoteTimeIndex];
347 
348         if (noteColumn.hitDifference <= 180 && noteColumn.hitDifference > -45) { // Within hit window
349 
350             // Stop the long note movement
351             mSpriteRenderer.mNoteBuffer[mCurrentGameState].erase(
352                 noteColumn.noteName + "-long-note-head-" + to_string(noteColumn.longNoteTimeIndex)
353             );
354 
355             // Start shrinking logic
356             auto& note = mSpriteRenderer.mNoteBuffer[mCurrentGameState][
357                 noteColumn.noteName + "-long-note-body-" + to_string(noteColumn.releaseTimeIndex)
358             ];
359 
360             if (note == nullptr)
361                 noteColumn.releaseTimeIndex -= 1;
362 
363             if (note != nullptr)
364             {
365                 noteColumn.longNoteOriginalHeight = note->GetSize().y;
366                 noteColumn.shrinkDuration = noteColumn.releaseTimes[noteColumn.releaseTimeIndex] -
367                     noteColumn.longNoteHitTimes[noteColumn.longNoteTimeIndex];
368                 noteColumn.shrinkStartTime = mTimeElapsed;
369                 noteColumn.longNoteTimeIndex++;
370                 mMaximumPossibleScore += 320;
371                 noteColumn.isKeyHeld = true; // Indicates the note is being held
372             }
373 
374             UpdateScoreLongNotes(noteColumn);
375         }
376     }
377 }
```

```
410     void Game::HandleHitRegistration(SDL_Event& event)
411     {
412         if (SDL_GetKeyName(event.key.keysym.sym) == mLeftKeybind &&!event.key.repeat)
413         {
414             RegisterHit(mNoteColumns[0]);
415         }
416         if (SDL_GetKeyName(event.key.keysym.sym) == mDownKeybind && !event.key.repeat)
417         {
418             RegisterHit(mNoteColumns[1]);
419         }
420         if (SDL_GetKeyName(event.key.keysym.sym) == mUpKeybind && !event.key.repeat)
421         {
422             RegisterHit(mNoteColumns[2]);
423         }
424         if (SDL_GetKeyName(event.key.keysym.sym) == mRightKeybind && !event.key.repeat)
425         {
426             RegisterHit(mNoteColumns[3]);
427         }
428     }
429
430     void Game::HandleHitRelease(SDL_Event& event)
431     {
432         if (SDL_GetKeyName(event.key.keysym.sym) == mLeftKeybind)
433         {
434             HandleLongNoteRelease(mNoteColumns[0]);
435         }
436         if (SDL_GetKeyName(event.key.keysym.sym) == mDownKeybind)
437         {
438             HandleLongNoteRelease(mNoteColumns[1]);
439         }
440         if (SDL_GetKeyName(event.key.keysym.sym) == mUpKeybind)
441         {
442             HandleLongNoteRelease(mNoteColumns[2]);
443         }
444         if (SDL_GetKeyName(event.key.keysym.sym) == mRightKeybind)
445         {
446             HandleLongNoteRelease(mNoteColumns[3]);
447         }
448     }
```

```
469     void Game::HandleMissedHitRegistration()
470     {
471         for (auto& noteColumn : mNoteColumns)
472         {
473             if (noteColumn.noteTimeIndex < noteColumn.hitTimes.size() &&
474                 mTimeElapsed > noteColumn.hitTimes[noteColumn.noteTimeIndex] + 180)
475             {
476                 noteColumn.noteTimeIndex++;
477                 mMaximumPossibleScore += 320;
478                 mMissCount++;
479                 mHealth -= 10;
480                 GetSprite(mCurrentGameState, "main-gameplay-judgement-text")->SetTexture(ResourceManager::GetTexture("miss-judgement-text"));
481             }
482             if (noteColumn.longNoteTimeIndex < noteColumn.longNoteHitTimes.size() &&
483                 mTimeElapsed > noteColumn.longNoteHitTimes[noteColumn.longNoteTimeIndex] + 180)
484             {
485                 noteColumn.longNoteTimeIndex++;
486                 mMaximumPossibleScore += 320;
487                 mMissCount++;
488                 mHealth -= 10;
489                 GetSprite(mCurrentGameState, "main-gameplay-judgement-text")->SetTexture(ResourceManager::GetTexture("miss-judgement-text"));
490             }
491             if (noteColumn.releaseTimeIndex < noteColumn.releaseTimes.size() &&
492                 mTimeElapsed > noteColumn.releaseTimes[noteColumn.releaseTimeIndex] + 180)
493             {
494                 mSpriteRenderer.mNoteBuffer[mCurrentGameState].erase(
495                     noteColumn.noteName + "-long-note-body-" + to_string(noteColumn.releaseTimeIndex));
496                 noteColumn.releaseTimeIndex++;
497                 mMaximumPossibleScore += 320;
498                 mMissCount++;
499                 mHealth -= 10;
500                 GetSprite(mCurrentGameState, "main-gameplay-judgement-text")->SetTexture(ResourceManager::GetTexture("miss-judgement-text"));
501             }
502         }
503     }
```

```cpp
void Game::UpdateGameplayStatisticsText()
{
    mAccuracy = (mScore == 0.0f && mMaximumPossibleScore == 0.0f) ? 0 : mScore / mMaximumPossibleScore * 100;
    GetText(mCurrentGameState, "gameplay-score")->UpdateText("Score : "+format("{}",mScore));
    GetText(mCurrentGameState, "gameplay-accuracy")->UpdateText("Accuracy : "+format("{:.2f}", mAccuracy));
    GetText(mCurrentGameState, "gameplay-flawless-count")->UpdateText("Flawless : "+format("{}",mFlawlessCount));
    GetText(mCurrentGameState, "gameplay-perfect-count")->UpdateText("Perfect  : "+format("{}",mPerfectCount));
    GetText(mCurrentGameState, "gameplay-great-count")->UpdateText("Great : "+format("{}",mGreatCount));
    GetText(mCurrentGameState, "gameplay-good-count")->UpdateText("Good : "+format("{}",mGoodCount));
    GetText(mCurrentGameState, "gameplay-bad-count")->UpdateText("Bad : "+format("{}",mBadCount));
    GetText(mCurrentGameState, "gameplay-miss-count")->UpdateText("Miss : "+format("{}",mMissCount));

    if(mAccuracy == 100)
        GetText(mCurrentGameState, "gameplay-current-grade")->UpdateText("Current Grade : SS");
    else if (mAccuracy > 95)
        GetText(mCurrentGameState, "gameplay-current-grade")->UpdateText("Current Grade : S");
    else if (mAccuracy <= 95 && mAccuracy > 90)
        GetText(mCurrentGameState, "gameplay-current-grade")->UpdateText("Current Grade : A");
    else if (mAccuracy <= 90 && mAccuracy > 80)
        GetText(mCurrentGameState, "gameplay-current-grade")->UpdateText("Current Grade : B");
    else if (mAccuracy <= 80 && mAccuracy > 70)
        GetText(mCurrentGameState, "gameplay-current-grade")->UpdateText("Current Grade : C");
    else
        GetText(mCurrentGameState, "gameplay-current-grade")->UpdateText("Current Grade : D");
}
```

```cpp
void Game::UpdateScoreRelease(NoteColumn& noteColumn)
{
    if (noteColumn.releaseDifference >= 0 && noteColumn.releaseDifference <= 18.9) {
        GetSprite(mCurrentGameState, "main-gameplay-judgement-text")
            ->SetTexture(ResourceManager::GetTexture("flawless-judgement-text"));
        mScore += 320;
        mHealth += mHealth < 100 ? 10 * 320 / 320 : 0;
        mFlawlessCount++;
    }
    else if (noteColumn.releaseDifference > 18.9 && noteColumn.releaseDifference <= 37.8) {
        GetSprite(mCurrentGameState, "main-gameplay-judgement-text")
            ->SetTexture(ResourceManager::GetTexture("perfect-judgement-text"));
        mScore += 300;
        mHealth += mHealth < 100 ? 10 * 300 / 320 : 0;
        mPerfectCount++;
    }
    else if (noteColumn.releaseDifference > 37.8 && noteColumn.releaseDifference <= 75.6) {
        GetSprite(mCurrentGameState, "main-gameplay-judgement-text")
            ->SetTexture(ResourceManager::GetTexture("great-judgement-text"));
        mScore += 200;
        mHealth += mHealth < 100 ? 10 * 200 / 320 : 0;
        mGreatCount++;
    }
    else if (noteColumn.releaseDifference > 75.6 && noteColumn.releaseDifference <= 113.4) {
        GetSprite(mCurrentGameState, "main-gameplay-judgement-text")
            ->SetTexture(ResourceManager::GetTexture("good-judgement-text"));
        mScore += 100;
        mHealth += mHealth < 100 ? 10 * 100 / 200 : 0;
        mGoodCount++;
    }
    else {
        GetSprite(mCurrentGameState, "main-gameplay-judgement-text")
            ->SetTexture(ResourceManager::GetTexture("bad-judgement-text"));
        mScore += 50;
        mHealth += mHealth < 100 ? 10 * 50 / 320 : 0;
        mBadCount++;
    }
}
```

```cpp
void Game::UpdateHealthBar()
{
    mHealth = std::clamp(mHealth, 0, 100);
    GetSprite(mCurrentGameState, "main-gameplay-life-bar")->SetSize(vec2(17.158, -594.710 * (mHealth / 100.0f)));
}
```

```cpp
void Game::UpdateGameplayState()
{
    // Handle all necessary game logic updates
    RenderNotes();
    UpdateGameplayStatisticsText();
    HandleMissedHitRegistration();
    UpdateHealthBar();
    for (auto& noteColumn : mNoteColumns)
    {
        UpdateLongNoteState(noteColumn);
    }
}

void Game::HandleMainGameplay()
{
    if (!mSongPlaying)
    {
        StartSongWithAdjustedTiming();
        // Ensure notes are synced before song starts
    }
    if(mSongPlaying)
    {
        // Update the time elapsed
        mTimeElapsed = SDL_GetTicks() - mSongStartTime;

        // Update the UI text to display minutes and seconds
        int minutes = static_cast<int>(mTimeElapsed / 60000); // Convert ms to minutes
        int seconds = (static_cast<int>(mTimeElapsed / 1000)) % 60; // Convert ms to seconds (mod 60)
        GetText(mCurrentGameState, "gameplay-time-elapsed")
            ->UpdateText("Time Elapsed : " + to_string(minutes) + ":" + (seconds < 10 ? "0" : "") + to_string(seconds));

        // Update the gameplay state (handles rendering, hit registration, etc.)
        UpdateGameplayState();
    }
}
```

```cpp
            if (mCurrentGameState == GameState::MAIN_GAMEPLAY)
            {
                if (mSongPlaying)
                {
                    HandleHitRegistration(event);
                }

                // Handle exiting gameplay when Escape is pressed
                if (event.key.keysym.sym == SDLK_ESCAPE)
                {
                    mSoundEngine->stopAllSounds(); // Stop any playing sounds
                    // Transition back to Chart Selection
                    TransitionToGameState(GameState::CHART_SELECTION_MENU);
                }
            }
```

Testing

Chart Selection

Test Plan

Development

```
45      if (mCurrentGameState == GameState::CHART_SELECTION_MENU)
46      {
47          HandleChartScrolling(event);
48          HandleDifficultyScrolling(event);
49
50          if (event.key.keysym.sym == SDLK_ESCAPE)
51          {
52              TransitionToGameState(GameState::MAIN_MENU);
53              mSoundEngine->stopAllSounds();
54          }
55      }
```

```
579 void Game::UpdateChartSelectionImage()
580 {
581     mCurrentChartFile = "charts/" + mCurrentlyPreviewedCharts[3] + "/" + mCurrentlyPreviewedDifficulties[2] + ".txt";
582     mCurrentSongName = mCurrentlyPreviewedCharts[3];
583     std::ifstream difficultyFile(mCurrentChartFile);
584
585     std::string line;
586     std::string imagePath;
587     std::regex imageRegex(R"(Background Image\s*:\s*(.+))");
588     std::regex bPMRegex(R"(BPM\s*:\s*(.+))");
589
590     while (std::getline(difficultyFile, line))
591     {
592         std::smatch match;
593         if (std::regex_match(line, match, imageRegex))
594         {
595             if (match.size() == 2) // Ensure the match captures the path
596             {
597                 mCurrentChartImageFile = match[1].str();
598             }
599         }
600         if (std::regex_match(line, match, bPMRegex))
601         {
602             if (match.size() == 2) // Ensure the match captures the path
603             {
604                 mCurrentSongBPM = match[1].str();
605             }
606         }
607     }
608
609     GetText(mCurrentGameState, "song-bpm-text")->UpdateText("BPM : "+mCurrentSongBPM);
610     int minutes = static_cast<int>(mCurrentSongDuration / 60000); // Convert ms to minutes
611     int seconds = (static_cast<int>(mCurrentSongDuration / 1000)) % 60; // Convert ms to seconds (mod 60)
612     GetText(mCurrentGameState, "song-length-text")
613         ->UpdateText("Length : " + to_string(minutes) + ":" + (seconds < 10 ? "0" : "") + to_string(seconds));
614
615     // Convert file extension to lowercase for case-insensitive comparison
616     std::string extension = fs::path(mCurrentChartImageFile).extension().string();
617     std::transform(extension.begin(), extension.end(), extension.begin(), ::tolower);
618
619     // Check if the file is PNG or JPG and enable alpha accordingly
620     bool useAlpha = extension == ".png";
621
622     Texture bgImage = ResourceManager::loadTextureFromFile((fs::current_path().string() + "\\" + mCurrentChartImageFile).c_str(), useAlpha);
623     GetSprite(mCurrentGameState, "song-cover")->SetTexture(bgImage);
624 }
```

```
572     if (mCurrentGameState == GameState::CHART_SELECTION_MENU)
573         UpdateChartSelectionImage();
574
575     mCurrentChartFile = "charts/" + mCurrentlyPreviewedCharts[3] + "/" + mCurrentlyPreviewedDifficulties[2] + ".txt";
```

```
429         if (mCurrentGameState == GameState::CHART_SELECTION_MENU)
430         {
431             mGameActive = false;
432             mSongPlaying = false;
```

```
434             firstColumn = {
435             0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
436             "main-gameplay-left-note",
437             581.936, 0,
438             {},
439             {},
440             {},
441             {}, {}, {}, false, false, false, 0, 0 };
442             secondColumn = {
443             0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
444             "main-gameplay-down-note",
445             765.423, 0,
446             {},
447             {},
448             {},
449             {}, {}, {}, false, false, false, 0, 0 };
450             thirdColumn = {
451             0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
452             "main-gameplay-up-note",
453             949.361, 0,
454             {},
455             {},
456             {},
457             {}, {}, {},false, false, false, 0, 0 };
458             fourthColumn = {
459             0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
460             "main-gameplay-right-note",
461             1138.061, 0,
462             {},
463             {},
464             {},
465             {}, {}, {},false, false, false, 0, 0 };
```

```
467                 // Clear note columns
468                 for (auto& noteColumn : mNoteColumns)
469                 {
470                     noteColumn = {};
471                 }
472
473                 mSongStartTime = 0;
474                 mTimeElapsed = 0;
475                 mDelayStartTime = 0;
476                 mNegativeOffset = 0;
477                 mTimeTakenToFall = 0;
478
479                 mAccuracy = 0;
480                 mScore = 0;
481                 mMaximumPossibleScore = 0;
482                 mHealth = 100;
483
484                 mFlawlessCount = 0;
485                 mPerfectCount = 0;
486                 mGreatCount = 0;
487                 mGoodCount = 0;
488                 mBadCount = 0;
489                 mMissCount = 0;
490
491                 mWaitingForGameplayStart = true;
492                 mGameplayStartTime = SDL_GetTicks();
493             }
494         }
495     }
```

Testing

Grade Screen

Test Plan

Development

Testing

Chart Editor

Test Plan

Development

Testing

Evaluation

Testing to inform evaluation

- Put success critias

Useability Testing

- Put feedback for user

Robustness Testing

User Feedback

Summary

Appendix